

Complex Systems Modelling Platform

CSMP++

User Guide

copyright (c) S.K. Matthäi, last revised, Octobre 2019.

Table of Contents

TABLE OF CONTENTS.....	II
1. WHAT IS CSMP++ ?	1
<i>Design Philosophy.....</i>	<i>4</i>
<i>Design History.....</i>	<i>5</i>
<i>Getting Started.....</i>	<i>7</i>
2. FROM A REAL WORLD SYSTEM TO A CSMP++ MODEL.....	9
<i>Data Types: Scalar, Vector, Tensor, Array, FlaggedArray.. -Variable.....</i>	<i>20</i>
3. COMPUTATION OF A STEADY-STATE FLUID PRESSURE DISTRIBUTION	21
4. A TOUR OF CSMP++	31
<i>Region.....</i>	<i>31</i>
<i>Boundaries.....</i>	<i>34</i>
<i>Split Boundaries</i>	<i>37</i>
<i>ErrorHandler.....</i>	<i>38</i>
<i>PropertyHandle</i>	<i>40</i>
<i>Interrelations</i>	<i>41</i>
<i>Operands</i>	<i>45</i>
<i>Visitors.....</i>	<i>46</i>
<i>PDE Integrators</i>	<i>47</i>
<i>PDE Operators.....</i>	<i>50</i>
5. DEFINING & SOLVING FINITE-ELEMENT EQUATIONS.....	53
<i>Example: Steady-state fluid pressure</i>	<i>55</i>
<i>Transient Fluid Pressure.....</i>	<i>59</i>
<i>Buoyancy-Driven Flow.....</i>	<i>60</i>
<i>Lumping.....</i>	<i>61</i>
<i>LinearElasticity.....</i>	<i>61</i>
<i>References Cited.....</i>	<i>64</i>
6. INITIAL AND ESSENTIAL CONDITIONS	65
6. INPUT TO & OUTPUT FROM CSMP++.....	70
<i>Model Configuration from File</i>	<i>70</i>
<i>Regular Meshes from Pixel Data.....</i>	<i>72</i>
1. Drawing the Geologic Cross Section	73
2. Importing the Section into an Image Editing Program.....	73
3. Color Coding the Image	74
4. Output of the section as color-coded text file.....	75
<i>Output To Paraview, Visit, or MayaVi via VTK Interface</i>	<i>76</i>
<i>Output via the JPEG Interface</i>	<i>76</i>
<i>CSMP++ UoM GUI and Model Configurator.....</i>	<i>77</i>
A tour of the functionality of the GUI.....	77
Interactive model configuration	84
Visualisation of Simulation Results	86
Time Driven Simulation (TDS) visualization	86
Event (DES) visualization	87
Specific visualisation examples.....	87
Interfacing of CSMP++ with conventional reservoir simulation tools	90
7. COMPREHENSIVE SIMULATION EXAMPLE: CARBONATE DISSOLUTION.....	99
0. <i>Input Geometry.....</i>	<i>99</i>
1. <i>Physical Variables.....</i>	<i>99</i>
2. <i>Material Properties, Initial values, and Boundary Conditions</i>	<i>99</i>
3. <i>Interrelations</i>	<i>100</i>
4. <i>The calculation</i>	<i>101</i>
5. <i>The Regions</i>	<i>101</i>
6. <i>The Visitors.....</i>	<i>101</i>
7. <i>FiniteDifferenceGrids.....</i>	<i>101</i>
8. <i>Speciation Calculations.....</i>	<i>102</i>
9. <i>Initial Conditions.....</i>	<i>103</i>
10. <i>The Overall Model.....</i>	<i>104</i>
8. THE CSMP++ LIBRARIES.....	105
<i>/source/includes/integration/pde_integrators Library.....</i>	<i>109</i>

<i>/source/includes/math/ Library</i>	110
<i>/source/include/discretisation/ Library</i>	111
<i>/source /includes/Analysis/ Library</i>	112
<i>/source/includes/eos/ (Equation of State) Library</i>	113
<i>/source /includes/integration/finite_elements/ Library</i>	117
The Linear Triangular Element: LinearTriangle	117
The LinearTetrahedron	119
The IsoparametricQuadraticTriangle with 6 Nodes	122
The IsoparametricBarycentricQuadraticTriangle with 7 Nodes	127
The IsoparametricQuadraticTetrahedron with 10 Nodes	130
<i>/source/includes/integration/finite_volumes/ Library</i>	133
<i>/source_code/includes/property_manipulation/ Library</i>	136
<i>/source_code/hydrothermal/ Library</i>	140
<i>/source_code/interfaces/ Library</i>	150
<i>/source_code/interrelations/ Library</i>	Error! Bookmark not defined.
<i>/source_code/main_library/ Library</i>	151
<i>/source_code/pde_operators/ Library</i>	153
PDE Operators for Analytical Integration	156
PDE Operators for Numerical Integration	158
<i>/source_code/solver/ Library</i>	162
<i>/source_code/triangular_finite_volumes/ Library</i>	Error! Bookmark not defined.
<i>/source_code/unit_test/ Library</i>	163
<i>/source_code/visitors/ Library</i>	I
BoundaryTransferVisitor (2D linear triangles only)	144
InterfaceVisitor (2D linear triangles only)	144
AlterationVisitor	145
H2OPropertiesVisitor	145
HydroFractureVisitor	146
TransportVisitor (2D only)	148
VelocityAndVolumeFluxVisitor	149
APPENDIX	I
<i>A: The Distribution</i>	<i>I</i>
<i>B: Installation</i>	<i>2</i>
CSMP++ Projects in CodeWarrior	Error! Bookmark not defined.
PC Windows 2000 / XP	2
Linux	2
SGI / Irix 6.5 or Later	Error! Bookmark not defined.
CSMP++ GLOSSARY	4

1. What is CSMP++ ?

The Complex Systems Modelling Platform (CSMP++) is an application programmer interface (API) invented by S. K. Matthäi in 1995 for the simulation of geological processes and their interactions. CSMP++ is a cross-platform toolkit for the PC, Macintosh, or Linux-Unix-Irix hardware environments (PC, SGI, Digital Alpha, Sun). The current version CSMP++ reflects the ANSI / ISO C++ standard (July 1998) and compiles with the latest C++ compilers capable of compiling the test program suite of the Blitz++ computational library by Todd Verhuizen.

CSMP++ is a finite element - finite volume computational environment, but some finite-difference functionality is implemented as well. CSMP++ makes extensive use of the C++ Standard Template Library (STL) and the solution of large ($>1000 \times 1000$) (sparse) systems of algebraic equations is accomplished with the algebraic multigrid solver SAMG (e.g., Stüben 2002). Thus, matrix-inversion time relates to matrix size in a $n \log n$ fashion (n = rows = columns) as opposed to n^2 for the more typically used (and also available) conjugate gradient and direct methods. Given enough computer (RAM) memory, this makes it possible to apply CSMP++ to very large models ($>10^7$ degrees of freedom).

CSMP++ implements line, surface and volume based computations. For simple 2D color-coded bitmaps, an integrated mesh generator discretizes input geometry as regular triangle meshes. Planar, straight line graphs (PSLG) of polygons, lines, and regional property information can be discretized using J. Setchuk's constrained, conforming Delauney triangulation algorithm. Volumetric element meshes with attached material properties can be input to CSMP++ as ANSYS Tetra and hybrid element meshes, or via GoCad's Tsurf and Tsolid files. In any of these formats, finite elements can be tagged to specify which geological objects they belong to. CSMP++ will automatically identify element neighbors and outer model boundaries.

Simulated component processes of complex natural systems may be restricted to specific regions of a model. This permits to benefit from the fact that certain subsurface processes affect rock units only above a certain temperature or pressure, relative permeabilities only need to be calculated where multiple fluid phases are present, and thermal volume strain must only be calculated where the rock is cooled or heated. To provide such flexibility, CSMP++ allows restriction of computations to groups of finite elements name *regions*. A **Region** is a mesh segment (1D), area (2D) or volume (3D) of finite elements that belong together on the basis of their spatial coordinates, material properties, property ranges or combinations thereof. This domain decomposition capability allows for spatially variable timestepping (certain regions can be solved more often than others), distributed computations (parallel or vector processing) or efficient swapping of restricted memory. CSMP++ is parallelized using MPI and the vectorized version of the AMG solver SAMGp.

Any CSMP++ region can also be monitored and-or output individually for visualization. In 3D, this allows to greatly reduce the number of finite elements that must be rendered and helps to selectively visualize features of interest. In 2D models,

computed variables can also be output onto regular grids for faster rendering and animation.

Input-output parameter-range checking for all CSMP++ modules simulating process interactions ascertains the correctness of complex calculations and, in combination with a C++ exception handling scheme, permits the precise location of range violations, as well as occasional error recovery at runtime. To manage the spatial and dynamic complexity of geological systems, abstract data types are used in the implementation of geometric search capabilities and boolean operations on the discretized model geometry are available (union, intersection, difference etc.). In practice, this functionality assumes the same importance as the solution of differential equations.

CSMP++ has been designed with geological applications in mind. Material properties can vary over many orders of magnitude in time and in space and simulations with a high spatial resolution such as for models with cm-wide discrete fractures in km-thick rock packages are feasible. Various provisions have been made to solve linear and non-linear equations, including equations of state and other pressure and temperature-dependent physical variables. To do this, partial differential equations are integrated in a cell-by-cell fashion by **PDE_Integrator** objects. To integrate pre- or user-defined partial differential equation in space and in time, specific operators provide integral forms of differential operators and associate them with physical variables.

Geological applications necessitate the solution of weakly to fully coupled equations, as well as the post-processing of physical variables derived from the (primary) dependent variables in the global system of algebraic equations (for instance, the Darcy velocity from pressure gradients, permeability and dynamic viscosity). These secondary physical variables track constitutive relationships and / or component processes in the geological system such as change of permeability during sediment compaction. CSMP++ encapsulates such interactions in **Visitor** objects. They can compute a range of variable values at once as is done by the `H2OPropertiesVisitor` for the PT-dependent properties of water. To honor range of validity constraints, user-defined visitors have to perform a range checks on their input variables. By contrast, their output variables are range-checked automatically using bounds defined in a specific input file.

To directly modify, combine, or calculate distributed physical variables, one can create or associate variables with **PropertyHandle** objects. They permit to add, subtract, and multiply operands, creating temporary variables at runtime, and modifying properties of model sub-regions identified by **Region**, **Boundary** or **SplitBoundary** objects, selectively.

Every CSMP++ model also contains a **PropertyDatabase** managing the physical variables for a specific simulation. For each variable, it keeps a **Parameter** record of the variable type (scalar, vector, tensor, array) and placement (node, element etc.). This database usually is initialized from a text file called “*-variables.txt”. Alternatively, variables can be defined and added to the database at runtime. Importantly, the storage for each variable is allocated dynamically and in association with the storage sites (nodes, elements etc.). This is essential because it allows the model to shrink during potential remeshing in a simulation.

CSMP++ represents an API (application programmer interface). This implies that users must write their own main programs. To learn how to accomplish this, the user will start by customizing examples or existing applications according to his / her needs. A wide range of examples is provided with CSMP++, including:

- Steady-state or transient topography-, pressure- and-or buoyancy driven Darcian fluid flows, including fluid sources and sinks and a stream function solution to visualize flow lines in 2D.
- Diffusion and advection as well as dispersion of heat, solutes or reactants with the flow.
- Rate-controlled gradient reactions along the fluid flow path, such as calcite dissolution and precipitation.
- Two-phase flow of slightly compressible wetting and non-wetting phases using the Linear, Brooks-Corey or Van Genuchten displacement model.
- Two-phase fluid flow of brine and steam in intrusive-related hydrothermal settings.
- Linear-elastic deformation of a heterogeneous rock mass.
- etc.
- Combinations of the aforementioned processes plus a variety of interdependencies among physical variables.

CSMP++ is organized into a series of libraries with different themes like computational geometry or equations of state. The centerpiece of it is the main library that contains the base classes from which user-defined types are derived. Everybody is invited to add to this framework in order to create the capabilities regarding the simulation of natural systems.

CSMP++ also provides utility classes for geometric manipulations of models and data (for instance polygons, triangles, line segments and points, DXF file exchange etc.), for the pre-processing of input data and for the communication with other computational tools. A number of interface modules connect CSMP++ to the ANSYS mesh generator, the Visualization Toolkit (VTK, [kitware.com](http://www.kitware.com)), GoCad, Tekplot and other programs.

DESIGN PHILOSOPHY

CSMP++ adheres to an object-oriented design with the goal to disassemble the complexity of geological and other processes into components that are small enough that we can understand and express them mathematically, verify their accurate model behaviour, and ascertain that the interactions of such modular components occur within parameter value ranges for which their correct behavior has been verified. While we can understand component processes in isolation, without simulation, we cannot foresee how these will interact or what system behavior they may produce, especially when material properties have values which vary over many orders of magnitude. To facilitate *the simulation of such complex interactions and the resulting overall system behaviour is the design objective of CSMP++*. Years of development following this objective have made CSMP++ a tool for simulation of complex earth-, engineering-, and other systems that lends itself to:

- Investigate how processes that are well understood in isolation will interact,
- Carry out sensitivity analyses of how system behavior changes in response to critical variables,
- Investigate processes that cannot be observed directly because they occur over very long spans of time or at a great depth,
- Interpret and understand the meaning of observations and measurements on complex natural systems,
- Evaluate proposed cause-and-effect chains in complex geological systems,
- Determine which data should be collected and where in order to constrain specific processes,
- Forecast how man-made changes to natural systems are likely to affect their future behavior.
- Answer the typical What-If? questions of engineering.

DESIGN HISTORY

CSMP++ was invented and implemented by Stephan Matthäi using an object-oriented design and the programming language C++. The prototype was created in collaboration with Stephen Roberts (ANU, Australia) and applied in subsequently published research conducted at Stanford University, CA (USA). Bruce Eckel (Mindware.com) made very valuable suggestions that greatly improved the first incarnation of CSMP++ that became CSMP++ in 2008, after a complete rewrite.

Between September 1996 and December 1999, Stephan completely rewrote and extended CSMP++ while being a member of Chris Heinrich's Fluids and Ore Deposits Group at the ETH Zürich, Switzerland. This time period constitutes CSMP++'s main implementation phase and resulted in the CSMP++ beta version 4.0. In January 2001, Stephan left the ETH to become part of the faculty of Imperial College, London (UK). Since then he has made CSMP++ version 4.0 fully compliant with the C++ ANSI-ISO standard and improved its interface with the commercial finite-element mesh and fracture-geometry generators ICEM Tetra (ICEM Technologies, Inc.) and FRED (Golder Associates, Intl.).

In collaboration with Chris Pain and Matthew Eaton at Imperial, Stephan added an implicit FEM-FV-based multi-phase transport module to CSMP++. In 2003, together with Andrey Mezentsev and Ulrike Wolf, Mike Hohmeyer (ICEM Technologies, Berkeley) this scheme was generalized to hybride meshes of tetrahedral, prism, pyramid and hexahedral finite elements.

In the time period 1999-2003, Thomas Driesner and Sebastian Geiger at the ETH contributed an H_2O -NaCl equation of state and a finite-volume-based IMPES multi-phase fluid-flow simulation module for triangular finite elements, respectively.

At Imperial, Andrey Mezentsev implemented a number of additional finite-element types and corresponding finite-volume stencils. This work was conducted in Phase 1 of the industry funded research project "Improved Simulation of Fractured Reservoirs".

In 2005, Adriana Paluszny refactored Mezentsev's implementation of the general finite-voume framework in CSMP++ and added fracture-propagation and other mechanical capabilities in the years to come, identifying a need to depart from a centralized variable and mesh storage towards a more flexible framework. At the same time, Dim Coumou parallelized CSMP++ using MPI and the SAMGp multigrid solver.

Following his departure from Imperial College, SKM rewrote the kernel of CSMP++, changing from array to single-object mesh and variable storage. Together with Philipp Lang, a code-wide application of policy-based class design was used to simplify user interfaces and make the code base more compact and maintable. This effort was accompanied by the creation of a rigorous framework for unit testing and code verification, implementation via a SVN – buildbot framework by Julian Mindel. The development of CSMP++ was discontinued and the new code CSMP++ was adopted as the platform for all new developments at the Garmisch workshop in 2011.

Further significant development took place at the Montanuniversitaet Leoben (2008-2014) and The University of Melbourne (2015 – present).

It follows that CSMP++ now is developed by an international group of developers, whose objective it is to design and implement all future additions to CSMP++ as compatible extensions of the basic CSMP++ libraries. The rationale for this is to achieve synergies that will arise when independent researchers and developers can share future developments. The primary goal of such developments is to make the package as widely applicable and accessible for research and teaching in the international academic community. Currently, CSMP++ serves primarily as a research tool for the numerical investigation of reactive (multi-phase) fluid-flow and heat transport processes in porous media with an emphasis on fractured rock. However, we have also already applied CSMP++ to problems of commercial importance. The main current interest comes from the resource exploration, production and management sector of primary industries, but application to geothermal energy extraction, CO₂ geo-sequestration, and environmental management, have already been developed.

GETTING STARTED

The CSMP++ User Guide would be esoteric without programming examples but there are plenty. Please compile, run, and modify the tutorials and example files in the subdirectory `/example_suite` of the `/trunk/source_code` directory to get familiar with CSMP++'s functionality. This documentation is no substitute for this activity but it provides the necessary theoretical background to understand the exercises with the code. There is also a detailed DOxygen-formatted documentation (nightly built) of CSMP's functions and classes accessible through CSMP's webpage (thmc-project.unileoben.ac.at). In the document in hand, a top-down review of CSMP is organized as follows:

Chapter 1: (this chapter): Introduction and CSMP++ design history

Chapter 2: A single-phase flow simulation example.

Chapter 3: A steady-state fluid pressure computation example illustrating how a CSMP++ simulation is structured.

Chapter 4: Tour of CSMP++; description of high-level classes.

Chapter 5: Introduction to finite-element computations and partial differential operators.

Chapter 6: Assignment of initial conditions to a model of transient flow.

Chapter 7: Building your own two-dimensional finite-element model, simulating it and exporting results to text files, JPEG images and the VTK visualization tools.

Chapter 8: A summary of all steps involved in running transient CSMP++ simulations: workflow is illustrated with a reactive-transport simulation of dissolution-precipitation of carbonate in a fractured limestone.

Chapter 9: Library documentation of the classes and underpinning principles of the code base covered in the preceding chapters.

Appendix. Class listings for the CSMP++ library components in Appendix A, installation advice and internet sites for support in Appendix B; final glossary of terms for newbies.

The documentation of the CSMP++ API is structured into four segments, the User Guide (this document), the Developer's Guide, the Interface Guide, and the DOxygen Reference Manual (online). While the Users Guide only documents the high-level classes and their interdependencies and is intended for a front-to-back read of all new users and developers, the online documentation covers the entire code base.

The Interface Guide deals with the pre- and post-processing of models for meshing and visualization. It briefly introduces third party tools, both freeware and commercial ones. The Developer's Guide describes the CSMP++ API in depth, making recommendations how customized applications are built using it as a basis. Many

modern C++ programming tools now integrate class browsers which allow a user to quickly examine class hierarchies and public class interfaces. These will ease the transition to CSMP++.

The objects themselves are documented in the CSMP++ Reference Manual which should always be open in a Browser window while code is being developed.

The following second chapter describes the high-level object hierarchy of CSMP++ and explains the conventions adhered to in writing this documentation. Before you start to read please note:

1. The usage of the C++ language imposes its terminology on CSMP++. One should be familiar with object-oriented programming including concepts like *class* and *subclass*, *interface*, *method*, *inheritance*, *polymorphism*, *object instantiation*, *templates*, *operator overloading* etc. to make effective use of this document. Consult a C++ book and the *UML Reference Manual* (Rumbaugh et al. 1999, Addison & Wesley) to learn about OOP basics. Use the CSMP++ Glossary following the Appendix in this document to familiarize yourself with the most commonly used expressions.
2. For people used to procedural programming, perhaps the most important principle to appreciate in C++ apart from user-defined classes combining operations with associated data, is that such objects have a lifecycle. It begins with their construction also called instantiation, when a named “instance” of the object is created. The construction is performed by a constructor function that initializes the object’s internal data and storage. The lifecycle of an object ends with its destruction, when the memory that was occupied by the object is returned back to the operating system.
3. The diagrams used in this documentation are all drawn using the “unified modeling language” (UML), including “class”, “sequence”, “interaction”, and “collaboration” diagrams. The overview chapter of the aforementioned UML manual is highly recommended reading.
4. In class listings, combined capitalized nouns illustrate the convention for class names in CSMP++: *In class or method names, all words are spelled out and chained together. Where two capital letters would meet, these are separated by an underscore*. Function names, object instances, and variable names are written in small print. Classes that do not adhere to this naming convention, usually do not belong to CSMP++ (and the ‘csmp’ namespace).
5. General C++ style and design guidelines that are adhered to in CSMP are described in Sutter and Alexandrescu (2004, C++ Prof. Series, Addison Wesley).
6. In the text, class names are written in boldface when they are first introduced and subsequently typed in courier font.

2. From a Real World System to a CSMP++ Model

Numerical simulation aims at understanding the expression of a physical process or process interactions in a natural system too complicated to be described by analytical mathematical formulae. The goal is to find the spatially variable value of a dependent variable – like temperature or fluid pressure – in the entire region of interest. Typically, the value of this variable is already known in a few specific locations. This data is incorporated into the model as boundary and initial conditions. Spatially variable material properties determined in the field are prescribed to the computer model. This, however, leads to a range of simplifications relative to the real-world scenario and process. Modelling also requires a bit of housekeeping for the variables and results, and, most importantly, a computational scheme that accomplishes the calculations with a desired accuracy. Numerical simulation also necessitates monitoring of potential errors and it should implement appropriate recovery mechanisms. At the very least, CSMP++ the software must tell the user if and where something went wrong so that he or she can properly diagnose such errors.

The choice of the numerical method leads to additional requirements:

1. *Discretization* of the real-world system that is continuous to the atomic scale means that – given runtime and memory limitations – it can only be represented at a few selected points that link up into computational cells. The number of these points, excluding those where the property that shall be computed is known already, defines the degrees of freedom (DOF) or unknowns of the problem of interest.
2. A *set of physical variables* is selected to capture the behavior of interest in the real-world system. This also requires a mapping of material properties onto the discretization, for example, the assignment of permeability values to each mesh cell.
3. Spatial and temporal *coupling mechanisms* are needed to relate dependent variables with one another. These typically are basic laws of physics expressed by partial differential equations (PDEs) that apply to the entire or parts of the problem domain and are represented in their finite-element form by the *PDE_Integrator*. Numerical element-by-element integration of the PDEs over the model domain leads to linear algebraic equations the solution of which gives the result of interest.
4. *Interrelations* among physical variables that apply locally and without a need for global coupling mechanism(s), can be expressed in the smallest units of the discretization, the finite elements. For example, the hydraulic conductivity discretized on the mesh cell can be calculated from its permeability and a temperature and pressure dependent fluid viscosity that is discretized on the element nodes.
5. *Visitors* allow for a change in reference frame (Eulerian = attached to the material, Lagrangian = attached to the flowing fluid) when sampling model behaviour.
6. *Essential conditions* and *initial values* of the dependent physical variables must be prescribed to the model as a prerequisite for a unique solution and because a transient model requires an initial state. The term *essential condition* is preferred over *boundary condition* because corresponding assignments may also be made somewhere inside the model domain.
7. *Input* and *output* methods which transform the results of a computation into a human-readable format such as graphics, charts etc.

The CSMP++ code is built in modules organized into hierarchical layers implementing these features and capabilities. The remainder of this chapter introduces the top-level objects of this hierarchy.

A simple box-shaped geologic model (Fig. 1) illustrates the basic terminology and object relationships in CSMP++. Terminology is introduced via the description of the unifying modeling process.

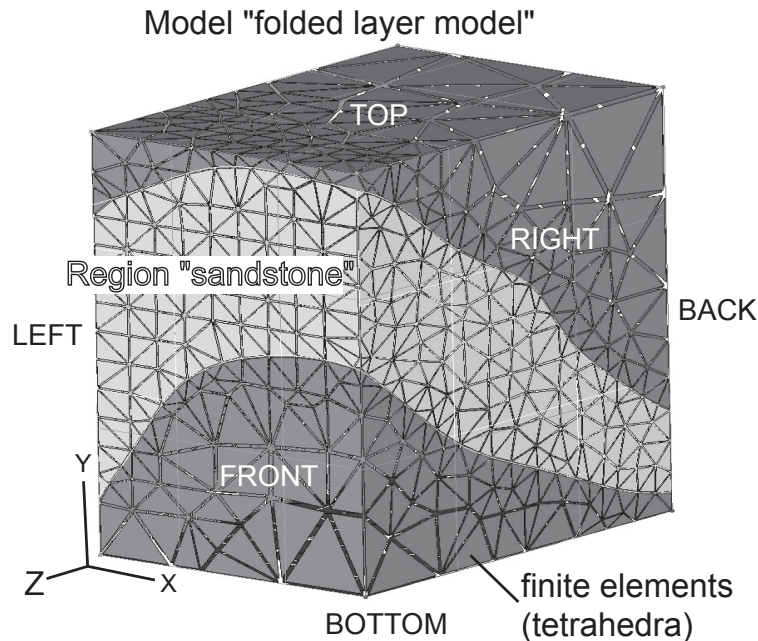


Fig. 1: Box-shaped CSMP++ model of a folded rock sequence illustrating how CSMP++ class names relate to model entities: The computational model (=geometry + connectivity + data) is stored by the **Model** object “folded layer model”. It is discretized by a finite-element mesh to which “permeability” and “porosity” values are assigned as material properties. Selected sub-regions of the model are identified by **Region** objects. Each region has a unique name given by a string. The Region object “sandstone” is contiguous but this is not a requirement for regions. Its distinction as a named region could, for instance, be used to monitor the fluid flux in this unit. The names “LEFT, BOTTOM, RIGHT” and “TOP”, distinguish straight model boundaries that are parallel to the coordinate axes. Boundaries do not have to be flat. If so they are flagged IRREGULAR.

The Model connects, manages, and stores a hierarchy of class objects that represent the finite element mesh and the associated material properties and physical variables (Fig. 2). Their specific functionality relates to the finite element method as will be explained in a later section.

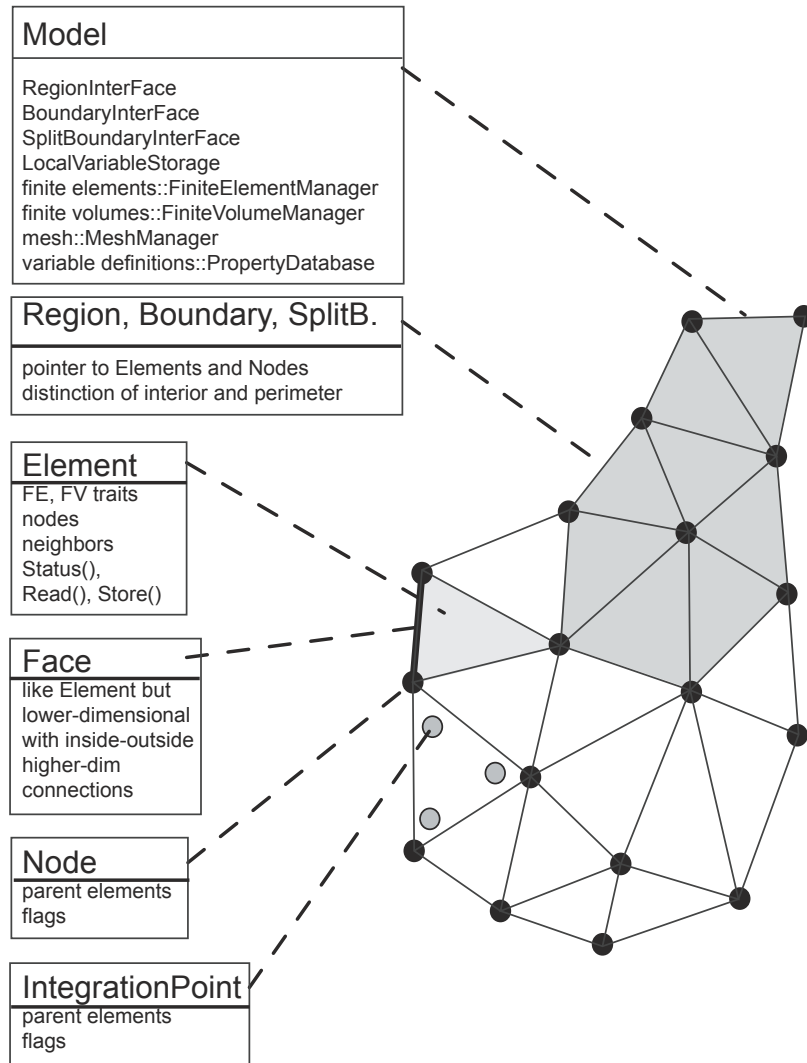


Fig. 2: Class hierarchy that captures the discretization and connectivity of a CSMP++ finite element mesh. NB: Apart from the top down connections there are also flat and bottom up connections. Thus, nodes know their parent elements and elements know their neighbors.

The CSMP++ main_library implements the aforementioned modelling essentials as follows:

1. The class **Model** combines objects with complementary functionality to encapsulate the computer model and associated variables. The mesh with associated properties can be written to a CSMP binary file:
 - In the live Model the finite-element mesh is handled by the class **MeshManager**. The finite-element mesh connectivity imposes the class hierarchy Model > Region > Element > Face > InterFace > Node & integration points, see Figure 2.
 - The class **FiniteElementManager** associates Elements, Faces and InterFaces with finite-element capabilities. This achieved via the “Bridge” design pattern in which these objects are associated dynamically with the respective finite element types contained in the manager. A similar facility exists for finite-volume capabilities in the form of the **FiniteVolumeStencilManager**.

- A **PropertyDatabase** with **Parameter** records is used to generate access keys to *physical variables* (enum **csmp::INDEX** and class **csmp::Index**). Global and local handles to distributed variables exist in the form of **Operand** and **PropertyHandle** classes.
 - *Material properties* as well as *physical variables* are stored in the **LocalVariableStorage**. This storage is distributed (on the Nodes, Elements ...Model) and custom-built at runtime using information from the PropertyDatabase and the **MeshManager**.
 - **Region** objects are also stored and managed by the Model. It treats them as named entities managed by the **RegionInterface**. A distinction is made between unique (=space exclusive) regions and general ones that can be overlapping. Regions know their boundary elements, their faces, and nodes. Elements that have at least one face at a boundary are boundary elements. Regions can be queried for their volume, surface area and so forth. Each model has a master region “Model” that contains all its elements.
 - **Boundary** objects are lower dimensional regions that consist of Face objects. Any visitation, PDE solve or post-processing that can be performed on a Region can also be performed on boundaries. Boundary Face objects know the higher dimensional elements on either side to which they connect. This allows for specific boundary processing.
 - **SplitBoundary** objects are (and are created from) Boundary instances. They are different because they contain multiple yet space-coincident nodes that support decoupling and jump discontinuities in the values of node variables.
2. *Computations* are performed with three classes of objects listed in an order of increasing complexity:
 - **Interrelation** objects perform computations that lead to a single result variable. *Example:* Computation of the interstitial flow velocity from the Darcy velocity and the porosity of the elements.
 - **Visitor** objects are for more complex but still group-by-group, element-by-element, face-by-face etc. computations. *Example:* AlterationVisitor computes chemical reactions while looping over the elements.
 - **PDE_Integrator** objects assemble finite-element and finite-volume integrals into a global sparse matrix and essential conditions into a right-hand vector. These integrals are defined in MathOperator subclasses. The solution of the matrix equations is possible with a variety of numerical methods made available as subclasses of the **Solver** class. *Example:* Solution of the Laplace equation with Fraunhofer’s SAMG solver.
 3. *Input and output* interfaces for the import and export of data to and from computations are contained in the CSMP++ interface library. They provide the glue necessary to use CSMP++ in conjunction with other software packages for CAD, meshing, analysis and visualization. *Example:* 2D results can be written directly to JPEG images. This is accomplished by interpolation to a **FiniteDifferenceGrid** and the association of variable values with a **ColorPalette**.

4. Users can also resort to CSMP++'s internal tools to analyze computation results:
Example: the **StatisticalAnalyzer** creates flow velocity histograms for each model subregion using user-defined velocity bins.

Figure 3 shows the high-level objects that a typical user will interact with in a simulation. Additional classes come into play for the input and output of the model and associated variables.

To extend, while reusing the functionality of this class hierarchy, a developer will create new subclasses inherited from the `PDE_Integrator`, `Visitor`, and `Interrelation` base classes. This is alluded to in Figure 3 by the display of a few subclasses that are already part of the CSMP++ API library (e.g., `HydraulicHead` interrelation, `AlterationVisitor` visitor). Such classes are documented in the library chapters of this User Guide.

The placement of variables on the mesh (see Fig. 2) is defined by the enumeration `PLACEMENT` (NB: enumerations are a variable type in C++):

```
enum PLACEMENT { NODE,
FACE,
FACE_INTEGRATION_POINT,
FACE_SECTOR_INTEGRATION_POINT,
FACE_FACET_INTEGRATION_POINT,
INTER_FACE,
INTER_FACE_INTEGRATION_POINT,
INTER_FACE_SECTOR_INTEGRATION_POINT,
INTER_FACE_FACET_INTEGRATION_POINT,
ELEMENT,
ELEMENT_INTEGRATION_POINT,
SECTOR_INTEGRATION_POINT,
FACET_INTEGRATION_POINT,
REGION,
BOUNDARY,
SPLIT_BOUNDARY,
MODEL,
```

Element properties permit the representation of material interfaces across which material properties vary discontinuously, i.e. from element to element (Fig. 4).

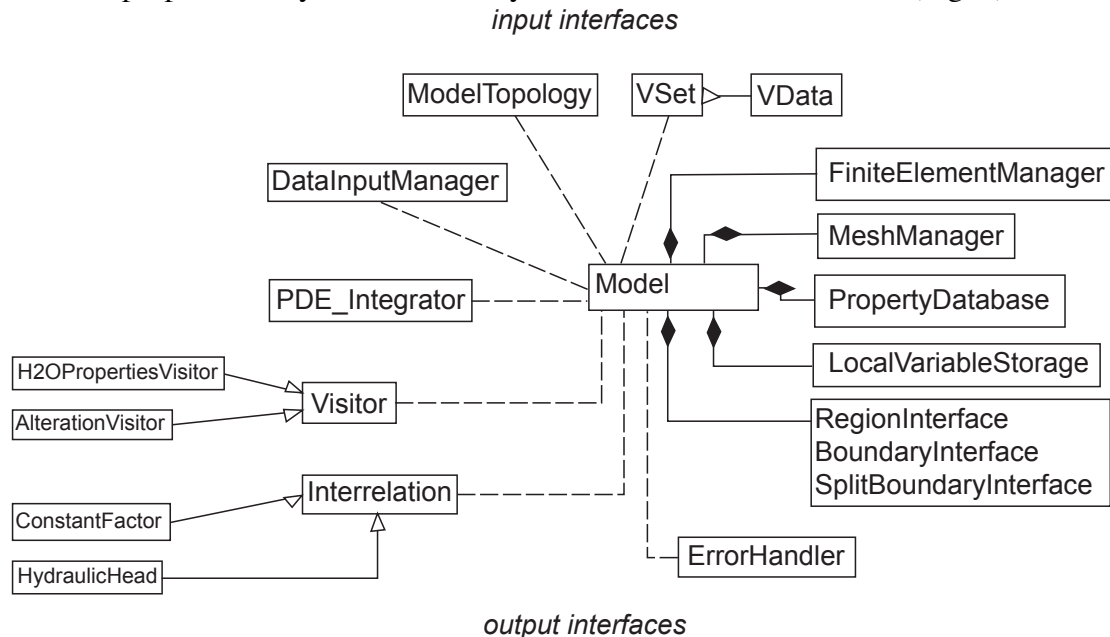


Fig. 3: Basic high-level base class relationships from a viewpoint of the **Model** object. The Model permits Visitors, Interrelations, and `PDE_Integrators` to act on its data. With the help of the **PropertyDatabase** it performs range checks and controls read and write privileges for physical

variables. If range violations or other problems are detected, the **ErrorHandler** is notified. It will report errors in a platform-dependent way using the `std::exception` hierarchy and log them to file. Each executable needs a `main ( )` function written by the application programmer unless the QT graphical user interface (GUI) is employed. Results can be visualized with various complementary graphics tools like **Paraview**, **Matlab** etc..

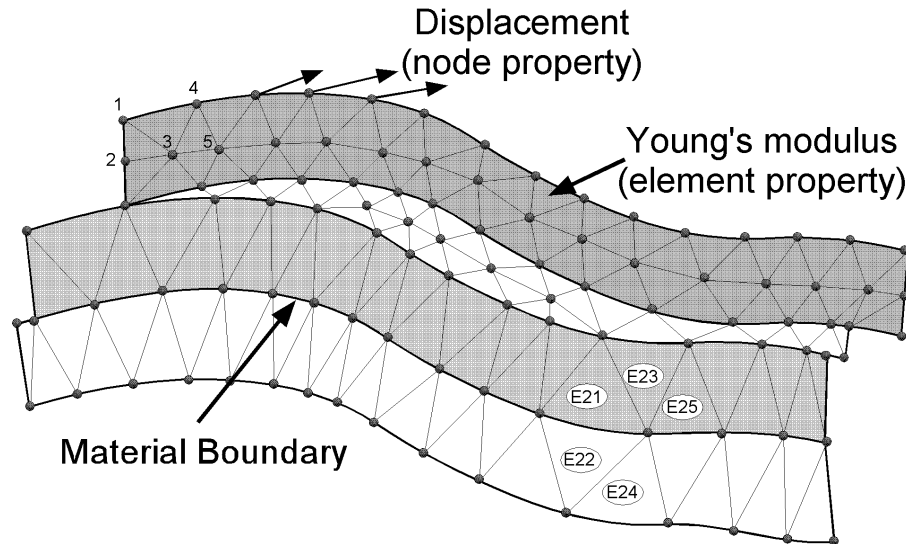


fig.3-mesh-explanation

Fig. 4: Two-dimensional triangular element discretization of folded layers illustrating node- and element numbering and the placement of node and element variables. In this example from a mechanical model, grey-shading indicates domains with elevated Young's modulus.

Each **Node** in the finite-element mesh can be assigned a unique unsigned integer value to create a mapping to a solution matrix or vector. Such indices can be set and retrieved via the `Idx ( )` member functions.

To facilitate finite-element computations, the spatially distributed physical variables are associated with constraint flags that tell a **PDE_Integrator** which values shall serve as essential conditions and at which points modifiable variable values lie (so called degrees of freedom = **DOF**). These flags are 1-byte enumerations. Supported flag values are:

```
enum VARIABLE_FLAG { PLAIN,           modifyable, dependent or indep. var.
                     INIT_GUESS,      convergence oriented
                     INIT_COND,       physically meaningful initial condition
                     DIRICH,          Dirichlet boundary condition
                     NEUMANN,         Neumann boundary condition
                     CONSTANT_FLUX,   for prescribed flux
                     ROBIN,           combination of derivative and value
                     PERIODIC,        linked via ID to opposite side of model
                     FIELD_DATA,      (Geological) field data for comparison
                     ANY };           unspecified default discriminator
```

The default value of a **VARIABLE_FLAG** flag is **ANY**.

Note that at the same node, fluid pressure may be flagged as **DIRICH** while temperature is flagged **PLAIN**. For this reason **VARIABLE_FLAG** flags are associated with variables and not with nodes or elements etc.. Note also that CSMP's high-level functions and methods (**PDE_Operator**, **Visitors**, **Interrelations** etc.) modify variable values only when the flag of the output variable matches that of the spatially distributed variable;

else no alteration occurs. Thus, to assign a new value, `val`, while retaining the distributed variable's original flag one writes:

```
object.Store( propertyKey, makeScalar(object.Status(propertyKey), val) );
```

Flags are enforced, for instance, when one assigns conditions explicitly, for instance by using the overloaded `Model` method `AssignBoundaryConditions()`. This operation will overwrite variables irrespective of their flags.

As already shown in Figure 1, the discretized area or volume of the real world system called the model domain is represented by a `Model` delimited by **Boundary** objects with unique names. For a box-shaped model these are predefined by capitalized tags. These C++ enumeration variables are declared as follows:

```
enum SG_BOUNDARY { NOT, IRREGULAR = IRREGULAR_OUTSIDE,
                    TOP      = TOP_OUTSIDE,
                    BOTTOM   = BOTTOM_OUTSIDE,
                    LEFT     = LEFT_OUTSIDE,
                    RIGHT    = RIGHT_OUTSIDE,
                    FRONT    = FRONT_OUTSIDE,
                    BACK     = BACK_OUTSIDE,
                    CNR1     = CNR_MIN,
                    CNR2     = CNR_MIN_MAXX,
                    CNR3     = CNR_MAX_MAXX,
                    CNR4     = CNR_MAX_MINXZ,
                    CNR5     = CNR_MIN_MAXZ,
                    CNR6     = CNR_MIN_MAXXZ,
                    CNR7     = CNR_MAX,
                    CNR8     = CNR_MAX_MAXZ,
                    EDGE1    = BACK_BOTTOM,
                    EDGE2    = BACK_RIGHT,
                    EDGE3    = BACK_TOP,
                    EDGE4    = BACK_LEFT,
                    EDGE5    = BOTTOM_LEFT,
                    EDGE6    = BOTTOM_RIGHT,
                    EDGE7    = TOP_RIGHT,
                    EDGE8    = TOP_LEFT,
                    EDGE9    = FRONT_BOTTOM,
                    EDGE10   = FRONT_RIGHT,
                    EDGE11   = FRONT_TOP,
                    EDGE12   = FRONT_LEFT };
```

Individual values are defined by integer constants using `#define` directives (see `CSMP++_definitions.h` header file). The flags starting with `CNR_` refer to the corners of box-shaped models. Position qualifiers follow, making reference to the coordinate axes of the model. The corner that is the farthest removed from the origin of the model is `CNR_MAX_MAXXZ`, i.e. `CNR8` at max *x*, max *y*, and max *z*.

The boundaries to which these enumeration strings correspond in a box-shaped model are shown in the following figure:

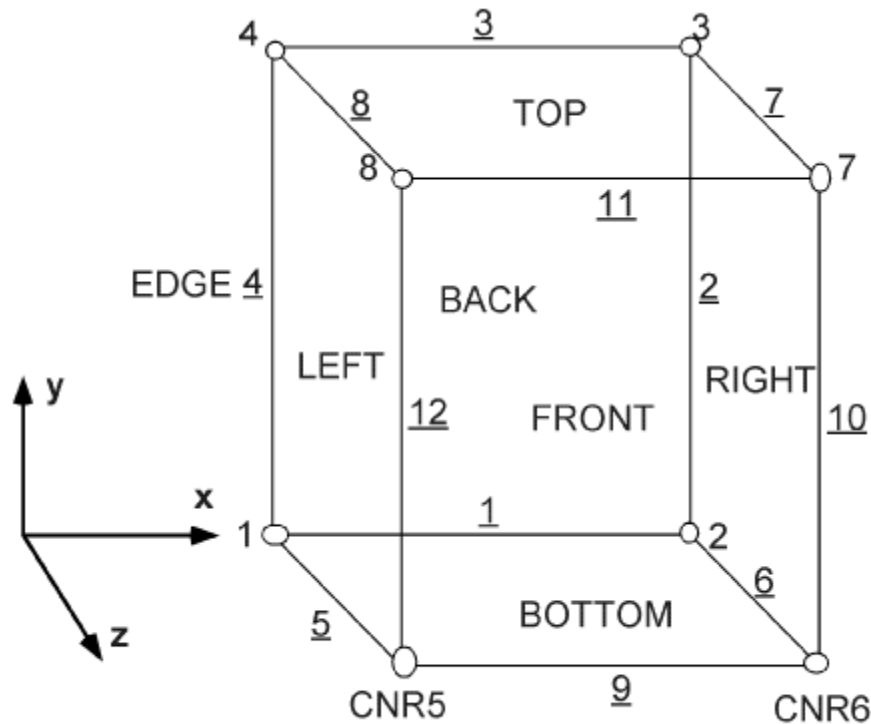


Fig. 5: Boundary flags of a box-shaped CSMP++ model.

Figure 5 shows that the **boundary flags** also comprise labels for 4 corners in two-dimensional models or 8 corners and 12 boundary edges in three-dimensional models (Fig. 5). Two-dimensional models have the coordinate axis X and Y equivalent to the BACK side of three-dimensional models. The Y-coordinate points upwards. If the original model does not comply with this convention, it should be rotated before initialization involving gravity forces (CSMP's gravity vector points in the negative Y-direction).

For box-shaped models, boundary flags are assigned with the Model's methods `AssignConstraint()`, `AssignBoundaryValues()`, `AssignBoundaryFlags()`, `AssignEssentialValuesToRegion()`. All of these methods take a `BOX_BOUNDARY` enumeration as an argument. The numbering of the model corners is counterclockwise following the right-hand coordinate rule and starting from the lower right ($xyz=0$). If one tries to generate a model with irregularly shaped boundaries using default (box) constructors, boundary assignments attempted by CSMP++ will prompt warnings.

For irregularly shaped models, boundaries must be tagged either as `IRREGULAR` or created from lower-dimensional regions with a unique name. This is done outside of CSMP++ in the CAD system used to create the model geometry. CSMP tracks the CAD entities concatenating names on the basis of the volumetric regions that are joint at the boundary (example: `SANDSTONE_SHALE`). Outer model boundaries will contain the string "Boundary."

Having established the variable placement, flags, and tags necessary for CSMP++-based finite-element computations, what is missing, is the description of- and the management of the physical variables themselves. This is the subject of the next subsection.

MANAGEMENT OF PHYSICAL VARIABLES

In CSMP, the management of physical variables begins with the generation of a text file by the user. This “*-variables.txt” file may just be a spreadsheet output as tab-delimited ASCII text. Columns are separated by tabs so that variable names can contain white space, like “fluid pressure”. This implies, that there must be no leading or trailing whitespace in any of the files columns. The name of the text file is communicated to the **PropertyDatabase** that lives inside of the **Model** via its constructor, for instance:

```
const string  model_name("one_sphere_0.45_tetra");
ANSYS_Model3D model( model_name.c_str(), "example25.txt");

printModelDimensions( model, true );
```

The initialization is performed before the **Model** is instantiated. The file’s default name is ‘CSMP++_variables’. If the user does not specify any variables file, the **PropertyDatabase** will attempt to read a binary file with this name when it is constructed.

The variables text file enlists the specifications of each variable used in the computation. These are (see example line below):

1. The name represented by a text string, e.g., “fluid pressure”,
2. The notation that relates the name to the symbols used in the underlying mathematical equations, e.g., P,
3. The SI unit of the variable (unit conversion utilities are available through the interface of the PropertyDatabase), e.g., Pa
4. An integer or string that identifies the enum VARIABLE_TYPE for the variable (scalar (=1), vector (=2), tensor (=3), >3 for any array where the number signifies its size. These types are represented by the classes ScalarVariable, VectorVariable, TensorVariable, Array and FlaggedArray, see next subsection. The strings (**SCALAR**, **VECTOR**, **TENSOR**, **ARRAY**, **FLAGGEDARRAY**) are preferred to the integer codes. Flagged arrays can only be specified as **FLAGGEDARRAY#** where the number indicates its size.
5. A physical meaningful value range for the variable, typically specified in scientific notation; this range should be reasonably specific for the simulation of interest because it will be used to check calculation results before they are written back to CSMP’s distributed storage,
6. The variable placement in the finite-element mesh as identified by the enum PLACEMENT (see above).
7. Optional auxiliary information that relates the variable to a potential written documentation of the simulation. The headline of the variables file gives the column titles and determines how many columns are being read.

For example, an entry for the scalar node variable “fluid pressure” may look like this:

name	MParameter	unit	index	min.	max.	place	usage
fluid pressure	P	Pa	1	0.00E+00	1.00E+10	node	input

Definitions of physical variables occupy a single line each. The *min. max.* value used to indicate the value range of each variable is very important because it gives CSMP++ the possibility to check whether a calculation went wrong at its source, ascertaining that calculation errors are caught where they arise instead of being propagated. Beyond this, it is the responsibility of the developer of new modules to define ranges for input values of these so that constitutive relations or EOS are not applied outside of their range of validity. Such checks have to be performed inside the new Interrelations or Visitor classes. For classes that constitute the CSMP++ API library, legitimate input ranges are spelled out in the DOxygen documentation.

Integration point variables have a special role: There is no one-to-one correspondence to a class for them, but they either belong to finite elements or finite volumes. How many of these exist has to be queried from instances of these entities, for example:

```
const size_t ip = element_ptr->IntegrationPoints();
```

This code will return how many integration points the element contains that the pointer points to.

In finite-element computations, dependent variables usually live on the nodes. Integration point variables (***_INTEGRATION_POINT**) come into play, for instance, if variables vary discontinuously from element to element, like the tensor variables “stress” or “strain.” In this case, values obtained by post-processing are only exact at the Gauss points and storing them on the nodes would average values in a physically meaningless way. As another example, a fluid-flow velocity vector variable stored at a material interface node would be problematic because the interface refracts the flow direction.

The specifications of physical variables are stored as **Parameter** records in the PropertyDatabase inside the Model object. When the PropertyDatabase is first initialized all variable names are listed on screen in alphabetical order. The database is then written to the binary file “CSMP++_variables.dat” that is subsequently used when loading models in CSMP binary format. A simulation can also be conducted creating all variables at runtime and without storing any of them to file. This is done by calling a constructor of the PropertyHandle class, see examples.

At runtime, the main role of the PropertyDatabase is to return variable keys (instances of the **csmp::Index** struct) needed to access variable flags and values:

```
csmp::Index p_key = model.Database().StorageKey("fluid pressure");
...
Region<DIM>& rref=model.Region("folded layer");
for ( auto it=rref.NodesBegin(); it!=InteriorNodes(); ++it)
{
    // retrieving a pointer to the current node
    double64 p_value = (*it)->Read( p_key );
}
```

the PropertyDatabase can also be queried for specific variable (Parameter) records or simply to find out whether a variable is already defined as is shown in this code example:

```
// write the specifications of an existing variable to screen
```

```
Model.Database().FlushToScreen("fluid pressure");

// test whether 'temperature' is defined
if ( model.Database().IsDefined("fluid pressure") )
    // we can use it...
```

The variable keys (**Index** instances) are used inside the Model to efficiently write and retrieve variable values (Fig. 6). **ScalarVariable** - **FlaggedArray** objects are inlined and stored inside of **LocalVariableStorage** objects that are data members of the Node – Model. Due to differences in their placement and type the memory required to store them differs. The corresponding amount of used RAM can grow or shrink at runtime when variables are created or deleted inside of CSMP modules. The local storage provides property-access via its Status(), Read(), Store() interfaces:

```
// accessing variables
VARIABLE_FLAG condition = node_ptr->Status( P_key );
double64 v_min(1.0e-7), v_max(1.0);
bool in_correct_range = element_ptr->IsWithinRange( v_key, vmin, vmax );
// initializing tensor variable stored on element integration point variable
TensorVariable<2> ts(ANY,1.,0.,0.,1.);
element_ptr->Store( ip, ts );
// starting at the end of a previous run
model time = model.Read( global time key );
```

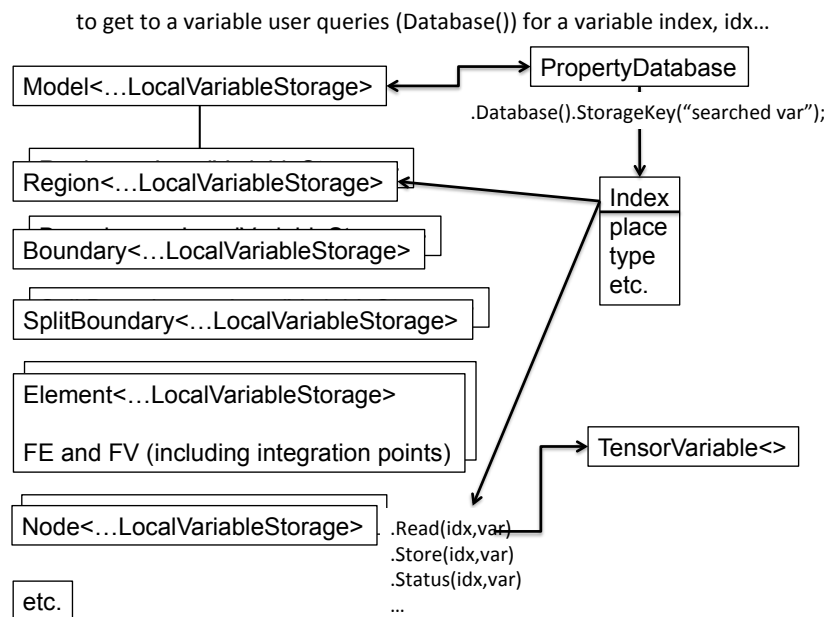


Fig. 6: Steps involved in accessing physical variables stored in the Model, using string identifiers such as “permeability” and interfaces of the MemoryManager class. The comparably slow operation of finding a variable in the database is carried out only once in each call to a Model, Visitor, or Interrelation object. The MeshManager inside the Model grants access to the finite elements, constraint points, and nodes constituting the model.

There are three ways to access and modify variables:

1. By name, e.g., “fluid pressure”
2. By `CSMP++_INDEX`, a structure with the members `placement`, `type`, and `property index` (Fig. 6, and example above).
3. Via `Operand` or `CSMP++_Operand` class objects which one can associate with an existing variable, or use to define or remove a variable at runtime.

The first way is used in all the high-level interfaces of the Model, PDE_Integrator, Visitor, Interrelation and other class objects with which a user will interact. Behind the scenes, the property database issues a CSMP++_INDEX for each variable that has been queried to rapidly carry out 'Read()' and 'Store()' operations. The third variable access strategy is described in the section on CSMP++_Operands.

DATA TYPES: SCALAR, VECTOR, TENSOR, ARRAY, FLAGGEDARRAY.. -VARIABLE

Each physical variable registered by the PropertyDatabase is stored in inline format in the model in an STL vector of a suitable type (values as double64, flags as one-byte enums). Implemented variable classes are **ScalarVariable**, **VectorVariable**, **TensorVariable**, **ArrayVariable** and **FlaggedArrayVariable**. Their default flag value is ANY. Corresponding enumerations used for their initialization are contained in the C++ header file "CSMP_global_enumerations.h". Here, the enumeration VARIABLE_TYPE is defined as.

```
enum VARIABLE_TYPE { SCALAR=1, VECTOR=2, TENSOR=3, ARRAY=4, FLAGGEDARRAY=5 };
```

To insure portability of the CSMP++ code between platforms with a different numerical precision, there is a stand-alone program 'CSMP_number_types_main.cpp' whose execution establishes the implementation- and machine-dependent precision of C++ inbuilt-variable types on the target platform. With the target compiler and settings, compile this program into an executable and run it. The output is the header file 'CSMP++_number_types.h' whose automatic inclusion when the CSMP++ library is built establishes prescribed-size cross-platform compatible types. The default floating point type used by CSMP++ is double64 and has 8-byte precision. CSMP++ initializes each floating point value to a default value of NaN (not-a-number, see IEEE conventions). This approach highlights any uninitialized variable value as calculations involving it will also return NaN. Such values can also be tested for by the macro `is_nan()`.

C arrays and STL random access containers like the `vector` and `deque` templates use integer types to access individual elements:

```
// create an instance of a std::vector with 200 elements
vector<double64> v(200,13.3);
cout << v[150] << endl;
```

The range of these integers depends on machine architecture. CSMP++ uses the unsigned integral type `size_t` in stead of the container-specific `::size_type` to access data in these containers. It is normally typedef'ed as an unsigned 32-bit integer. Note that to avoid implicit type conversions the size of `size_t` must be consistent with that of the STL `size_type` parameter.

`ScalarVariable`, `VectorVariable`, and `TensorVariable` are partially specialized template classes that associate `VARIABLE_TYPE` double64 with `VARIABLE_FLAG` enums (see above). Operations involving them and the inbuilt types have been optimized and overloaded operators allow the user to write mathematical expressions without a need of the usual low level expansions. For example, matrix multiplication is carried out

automatically (for the following vector and tensor variables of a three dimensional model:

```
TensorVariable<3> A, B;
VectorVariable<3> n, u;
...//do some initializations of 3 x 3 matrices A and B as well as vector n;
TensorVariable<3> C = A * B;
u = C * n;
```

The dimensions of vector and tensor variables must always be equivalent to the dimensionality of the model. A `TensorVariable<2>` will contain a 2 x 2 matrix of `ScalarVariables`.

The public interfaces of the basic data type classes are documented in detail in the DOxygen online reference.

As the name indicates, the **ArrayVariable** variable stores a linear vector of doubles, the size of which is chosen by the user. While there is just a single flag for this type, in the `FlaggedArray` there are as many flags (of `VARIABLE` type) as there are floating point numbers.

3. Computation of a Steady-State Fluid Pressure Distribution

The previous chapter explained how discretized model geometry with associated material properties and physical variables is stored and managed by the `Model` class and associated CSMP++ modules. How this `Model` object interacts with other CSMP++ objects is illustrated in this chapter using an example program that calculates a steady-state fluid pressure distribution in an aquifer (Fig. 7). The steady-state fluid-pressure distribution that ensues as a consequence of spatial permeability variations and fixed boundary pressures (Dirichlet boundary conditions) is governed by the partial differential equation

$$Q = -\frac{\mathbf{k}}{\mu} \nabla^2 p \dots\dots\dots(1)$$

p , $\mathbf{k}$, μ , and Q are the fluid pressure, the permeability, the dynamic viscosity of the fluid, and a fluid source term, respectively. Once this pressure equation has been solved, the piecewise constant Darcy velocity, $\mathbf{q}$, discretized on the element is obtained by post-processing of the piecewise constant fluid-pressure gradients. For this calculation the hydraulic conductivity, $\mathbf{K}$ ($\mathbf{K} = k/\mu$) is introduced:

$$\mathbf{q}^{(e)} = -\mathbf{K}^{(e)} \nabla p \dots\dots\dots(2)$$

The solution to the pressure equation is computed via a piece-wise integration over the finite elements of the finite-element form of equation 1, yielding the incremental change in fluid pressure over the domain of interest. Resulting volume integral terms for each finite element are assembled into a matrix equation of the form:

$$[K]\{p\} = \{q\} \dots\dots\dots(3)$$

The left-hand side of this system of algebraic equations contains element contributions to the global conductance matrix K ; p is the solution vector which will hold the nodal fluid-pressure values, and the right-hand vector q stores the fluid source terms and boundary conditions. This global system of linear equations has a size determined by its degrees of freedom (DOF). The DOF count is equivalent to the number of nodes at which fluid pressure must be computed. This number is equal to the total number of nodes in the model minus the Dirichlet nodes at which the fluid pressure is prescribed. Dirichlet nodes impose fixed coefficients on the other algebraic equations constituting the solution matrix.

The element contributions to the global system of linear algebraic equations consist of element conductance matrices $K^{(e)}$ which are added to the empty global matrix K (equation 3). For (two-dimensional) linear triangular finite-elements, element conductance matrices have the form:

$$K^{(e)} = \iint_A \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_1}{\partial y} \\ \frac{\partial N_2}{\partial x} & \frac{\partial N_2}{\partial y} \\ \frac{\partial N_3}{\partial x} & \frac{\partial N_3}{\partial y} \end{bmatrix} \begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix} \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_2}{\partial x} & \frac{\partial N_3}{\partial x} \\ \frac{\partial N_1}{\partial y} & \frac{\partial N_2}{\partial y} & \frac{\partial N_3}{\partial y} \end{bmatrix} dx dy \dots\dots\dots(4)$$

where $N_{1,3}$ are three element interpolation functions (one for each node), used to approximate the fluid-pressure variation over the element. These functions are listed in the documentation of the finite element library. A is the area of the element, and $K_{x,y}$ is the directional hydraulic conductivity. $K_{x,y}$ is constant within each element but is allowed to vary from element to element. In the example, $K_{x,y}$ is isotropic such that $K_x = K_y$.

The global conductance matrix K is accumulated and solved by passing a `PDE_Integrator` object to the model. The integral in equation (4) is generated in matrix form by the finite-element PDE operator **Integral_dNT_op_dN_dV**. It is a subclass of the generic differential operator **MathOperatorLHS**. Mathematical operator names follow special naming conventions that were introduced because of a need for brevity (see Table 1). These names contain placeholders for physical variables. In this example, `op` is used to represent the hydraulic conductivity and the `N` functions model the fluid pressure. They appear twice, first as a placeholder for the element weighting functions and, second, for the element interpolation functions. This is the case because in the Galerkin finite-element formulation used in this example, weighting and interpolation functions are the same.

Notation	Spelled Out	Explanation
Integral	analytical integral	exists only for straight-sided finite elements
NumInt	numerical integral	for arbitrarily shaped elements with integration points
N	FEM test functions	finite-element method, also called basis or interpolation functions
M	FV test functions	finite volume interpolation functions, typically piece-wise constant

T	transposed	used in conjunction with interpolation function vector or derivate matrix
dV	volume integration	normally by finite-element method
dS	surface integration	normally by finite-volume = control volume method
op	operand	normally the material property variable like permeability
lhs	left-hand side	contribution goes into global solution matrix
rhs	right-hand side	contribution goes into global right-hand vector
2	exponent	dN2 is the second derivative of the N functions
B	dN expanded	used if solution variable is a vector as opposed to a scalar
P	N expanded	solution variable is a vector property like displacement

Table 1.: Naming conventions used for PDE operators that represent the finite element of finite volume integrals over partial differential equations (MathOperatorLHS and MathOperatorRHS objects).

Element contributions to the right-hand vector have the form:

$$Q^{(e)} = s^{(e)} \int_A N dA \xrightarrow{\text{integration}} \frac{s^{(e)} A^{(e)}}{3} \begin{Bmatrix} N_1 \\ N_2 \\ N_3 \end{Bmatrix}$$

In this example, $s^{(e)}$ is the fluid source term per unit area ($\text{m}^3 \text{m}^{-2} \text{s}^{-1}$), which could be envisaged as the rainfall on the land surface represented by the element or as the amount of fluid produced from a well. The factor 3 appears since the source term is distributed over the 3 nodes of each triangular element. $Q^{(e)}$ terms are accumulated by the **Integral_NT_op_N_dV** derived from the differential operator **MathOperatorRHS** base class.

```
model.Apply( evolve_pressure );
```

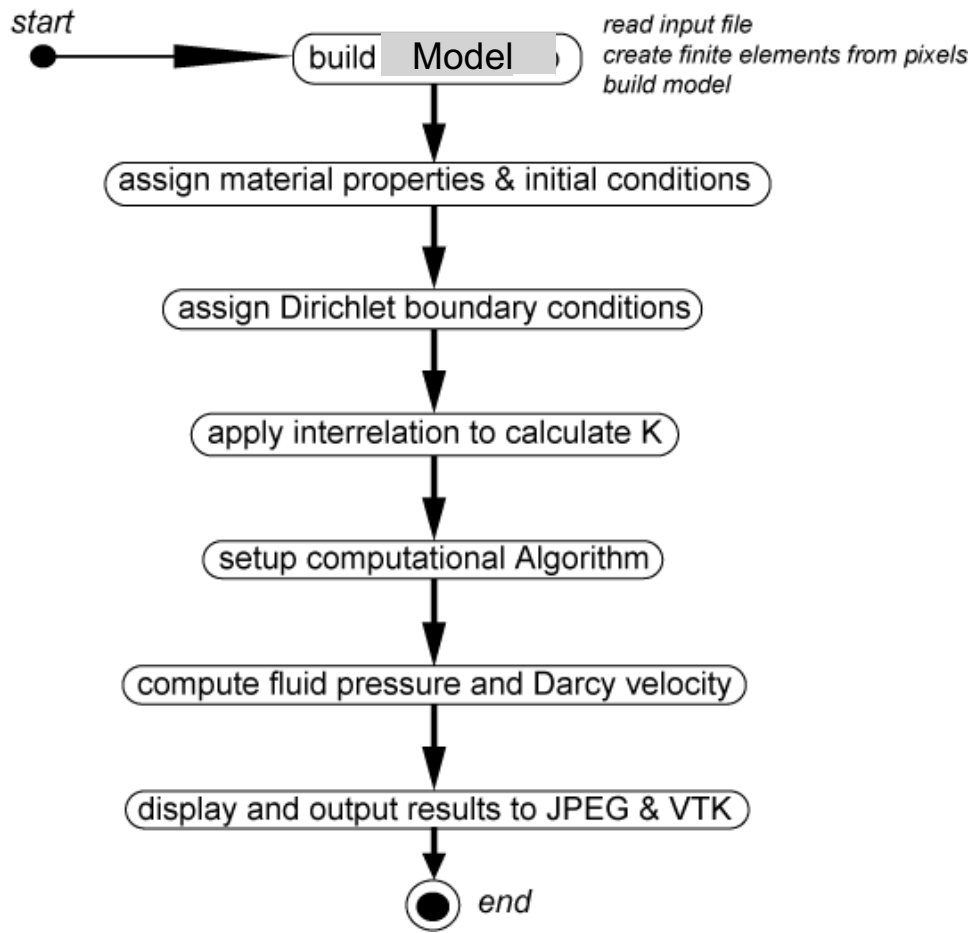


Fig. 7.: UML activity diagram for the simulation example1: calculation of a steady-state fluid pressure distribution.

In this simulation example, an adaptively refined finite-element mesh created by the program Triangle is used. It represents a rectangular domain (see CSMP++ Model Building Guide to learn about mesh creation). When the PDE_Integrator is applied to the Model it calculates the Darcy flow velocity from the computed fluid pressure gradients and the hydraulic conductivities of the elements (see equation 2). This is done by the post-processing operator **VelocityAndVolumeFlux**. In the example, the results of the computation are output as JPEG images (Fig. 6), text files, VTK files, and GoCad formatted TSurface files.

```

// inclusion of class declarations necessary for this simulation
#include "CSMP++_definitions.h"

// example_main file
#include "CSMP++_Examples.h"

// main library
#include "Model.h"
#include "ErrorHandler.h"
#include "CSMP_highLevelUtilities.h"

// pde operator library
#include "Integral_NT_op_N_dV.h"
#include "Integral_dNT_op_dN_dV.h"
#include "VelocityAndVolumeFlux.h"

// solver library
#include "SAMG_Solver.h"

```

```

// interrelations library
#include "ConductivityFromPerm.h"

// interface library
#include "Triangulator.h"
#include "CSMP++_JPEG_Interface.h"
#include "CSMP++_VTK_Interface.h"

using namespace std;
using namespace CSMP++;

// *****
// (1) 2D steady-state calculation of fluid pressure distribution using triangular
//     element meshes generated by CSMP++'s Triangulator from color-coded bitmaps.
//     Linear finite element interpolation is used and Dirichlet essential
//     conditions are applied.
//
//     Use 'granite_model1' as input geometry (needs 100MB of RAM memory)
//
//     Use example_viewer.tcl or MayaVi to visualize vtk output
//
// *****
namespace CSMP++ {

int steady_state_pressure()
{
    // 0. Setting global variables
    // -----
    // singletons (exist only once so that only a reference can be obtained)
    ErrorHandler& csmpp_error ( ErrorHandler::Instance() );

    double64&    global_time( ModelTime::Instance().modelTime );
    global_time = 0.;

    const string global_physvar_textfile("example1.txt");

    // 1. A Model named "example1" is built
    //     from a mesh based on a color coded image
    // -----
    Triangulator          mesh_triangulator;
    Model<CSMP++_float, 2> example1( mesh_triangulator );
    const PropertyDatabase& p_ref = example1.Database();

    // 2. Assignment of material properties & initial conditions
    // -----
    example1.InputUniformScalarValue( "porosity",          0.1 );
    example1.InputUniformScalarValue( "fluid volume source", 0.0 );
    example1.InputUniformScalarValue( "fluid pressure",     0.0 );

    // 3. Assignment of Dirichlet fluid pressure conditions
    //     at top and bottom boundary of the model
    // -----
    example1.AssignBoundaryValues( TOP,    "fluid pressure", DIRICH, 1.0e+05, 1.0e+05 );
    example1.AssignBoundaryValues( BOTTOM, "fluid pressure", DIRICH, 1.0e+07, 1.0e+07 );

    // 4. Calculation of hydr. conductivity from permeability using Interrelation subclass
    // -----
    CSMP++_float fluid_viscosity(1.0e-03);
    ConductivityFromPerm<double64> conductivity( p_ref, fluid_viscosity );
    example1.Pass( conductivity );
    printRangeOfVariable( example1, "conductivity" );

    // 5. Building PDE_Integrator "total_pressure" with the PDE operator subclasses
    //     A post processing operation is added to compute the fluid velocities and
    //     fluid fluxes (by setting the boolean variable in the constructor of the post-
    //     processor to true, the element velocities are averaged to the nodes)
    // -----
    SAMG_Solver solver;
    PDE_Integrator<2U, Region> total_pressure;
    total_pressure.SetSolver(solver);

    // conductance matrix [K] on the left-hand side
    Integral_dNT_op_dN_dV<double64> conductance( p_ref, "conductivity",

```

```

                                "fluid pressure", "fluid pressure" );
// source vector {Q} on the right-hand side
Integral_NT_op_N_dV<double64>    source( p_ref, "fluid volume source",
                                "fluid pressure" );
// post-processing operation to compute flow velocities
VelocityAndVolumeFlux<double64>    velocity( example1, "conductivity",
                                "porosity", "fluid pressure", true );

// add PDE operators to FE PDE_Integrator
total_pressure.Add( &conductance );
total_pressure.Add( &source );
// add post-process to the FE PDE_Integrator
total_pressure.AddPostProcess( &velocity );

// Solve [K]{p} = {Q} by passing the PDE_Integrator to the model
example1.Apply( total_pressure );

// 6. Screen ranges of computed variables
// -----
printRangeOfVariable( example1, "fluid pressure" );
printRangeOfVariable( example1, "velocity" );
printRangeOfVariable( example1, "pore velocity" );
printRangeOfVariable( example1, "volume flux" );
printRangeOfVariable( example1, "nodal volume flux" );

// 7. Output results to VTK and JPG files
// -----
CSMP++_VTK_Interface<CSMP++_float,2>    vtk_output;

// VTK output
vtk_output.OutputDataToVTK( example1, "fluid-pressure", "fluid pressure",    0 );
vtk_output.OutputDataToVTK( example1, "velocity", "velocity",    0 );
vtk_output.OutputDataToVTK( example1, "nvelocity", "nodal velocity",    0 );
vtk_output.OutputDataToVTK( example1, "volume-flux", "volume flux",    0 );
vtk_output.OutputDataToVTK( example1, "nvolume-flux", "nodal volume flux", 0 );

// JPG output (only scalar variables can be visualized)
CSMP++_JPEG_Interface                jpg_output;

jpg_output.OutputDataToJPG( example1, "fluid-pressure", "fluid pressure",    0 );
jpg_output.OutputDataToJPG( example1, "volume-flux", "volume flux",    0 );

// 8. Form a Region named "granite" in order to access
//    intrusion in the center of the model.
//    Region granite is build from finite elements in
//    the permeability range between 1.0e-21 to 1.0e-18 m2
// -----
cout << "\nFORMING A GROUP CALLED 'granite'" << endl;
example1.FormAndAddGroup( "granite", "permeability", 1.0e-21, 1.0e-18 );

// add fluid volume source term to region granite and repeat computation
const CSMP++_float source_term(1.0e-13); m3 m-2 s-1
example1.ChangePropertyInGroupToWhere( "granite",
                                "fluid volume source", source_term, PLAIN );
example1.ChangePropertyInGroupToWhere( "granite",
                                "fluid volume source", source_term, BOUNDARY );

// fixing pressure at perimeter of group
example1.ChangePropertyStatusInGroupToWhere( "granite",
                                "fluid pressure", DIRICH, BOUNDARY );

// restricting PDE_Integrator to group
total_pressure.RestrictApplicationTo("granite");

// recomputing fluid pressure in the granite taking into account the source term
example1.Pass( total_pressure );

printRangeOfVariable( example1, "granite", "fluid pressure" );
printRangeOfVariable( example1, "granite", "velocity" );
printRangeOfVariable( example1, "granite", "pore velocity" );
printRangeOfVariable( example1, "granite", "volume flux" );
printRangeOfVariable( example1, "granite", "nodal volume flux" );

jpg_output.OutputDataToJPG( example1, "fluid-pressure", "fluid pressure", 1 );
jpg_output.OutputDataToJPG( example1, "volume-flux", "volume flux", 1 );

```

```

// output of the group "granite"
vtk_output.OutputDataToVTK( example1, "granite",
                             "fluid-pressure", "fluid pressure", 1 );
vtk_output.OutputDataToVTK( example1, "granite", "velocity", "velocity", 1 );

cout <<"\nmain: That's it..."<< endl;

return 0;

} // end example1 function
} // end namespace CSMP++

```

The example source code displays several key features, which are germane to any C++ object-oriented program:

- Comment lines start with a double slash (*//*).
- Functions typically have a return value which is output when the instructions in the body of the function enclosed in “{ }” have been executed.
- For all objects which are used in the main program, corresponding header files must be included into the source code file. This is done at the very beginning, before the function which uses these object begins. These header files contain necessary object *declarations*. The actual functionality of the objects is *defined* (implemented) inside the binary library files.
- Each object must be constructed by a *constructor* call before it can be used (for example the CSMP++_VTK_Interface object in program segment 5).
- Constructors without arguments are called *default constructors*, see for example the construction of the jpg_output object.
- Object construction implies that an *instance* of a C++ class is created. This instance has a unique name independent of the class name. Therefore, the construction process is also called *instantiation*.
- Before global objects or variables can be used in a function, C++ requires these to be declared using an *extern* statement. This statement just tells the compiler that there is global instance of a class x with the instance name y. Again, the class must be declared in a corresponding header file.
- Once an object has been instantiated, it can be manipulated by calling its *interfaces*. Interfaces are functions which only work in conjunction with that object. In C++, this is signified by the *scope operator* “.”. Hence, the statement `dog.bark(loud)`, tells the instance `dog` of a class representing dogs to invoke its *member function* `bark()`. The *function argument* `loud` could be double giving a value in decibels.
- Objects may also be linked via pointers. If so, connections may be changed at runtime. The syntax “->” is used instead of the scope operator, i.e. `dog->bark(loud)` w.r.t. example above.
- Member functions are normally called *methods* to distinguish them from stand-alone functions.
- There are no destructor calls in `main()`. Unless an object is created dynamically (using the `new` operator, see creation of SAMG_Solver in example above) its destructor is called automatically at the end of the main program. Dynamic objects are destructed by calling the `delete()` function.

In the example program, the blocks 1 to 10 have the following roles:

1. The (Model) model is constructed from an image with color-coded pixels defining its geometry. The mesh is created by CSMP++'s Triangulator mesher. This tool is described in more detail in the CSMP++ Model Building Guide. The user is then prompted to specify the physical dimensions of the model.
2. The input data describe the geometry of the model and the material property “permeability” whose local values are determined by the the pixel colors. Initial values for the fluid source term etc. must still be assigned to the mesh. By default, all variables specified in the “...variables.txt” file are initialized to a status of `PLAIN` and a value of `NAN` (not a number)*. One can immediately tell if one forgot to initialize a variable, because the result of calculations involving such a variable will also be `NAN`.
3. Fixed fluid-pressure values are assigned as Dirichlet boundary conditions to the right and left model boundaries.
4. The interrelation `ConductivityFromPerm` is instantiated and applied to the model (calling the `Pass()` method of the `Model` class), to calculate the hydraulic conductivity from the permeability and the fluid viscosity of water at room temperature.
5. The PDE (equation 1) is assembled and solved using an algebraic multigrid solver with the `PDE_Integrator` instance `total_pressure`. This includes post-processing of the flow velocities by the `VelocityAndVolumeFlux` class and the velocity magnitude is stored for later display as the scalar variable “volume flux”. To be able to visualize the volume flux with the visualization toolkit, this property is extrapolated to the nodes and stored in the node variable “nodal volume flux”.
6. The result variables are output to screen to examine their minimum and maximum values.
7. The computed “fluid pressure” and “volume flux” are output as JPEG images which associate colors with variable values and to VTK files for the MayaVi visualization tool. The latter files store the actual variable values and, in contrast to the JPEG format, the vector quantity (Darcy) “velocity” can be visualized.
8. Finite elements with a permeability between 10^{-21} and 10^{-18} m² are included in a group called “granite”. A fluid volume source term is assigned to the interior and the marginal elements of this “granite” group and the variable fluid pressure is flagged Dirichlet at its boundary. Now the algorithm is restricted to apply only to this subregion of the model and fluid pressure is recomputed using the fluid pressure values at the group's perimeter as Dirichlet boundary conditions. The results from this computation are output selectively for the granite group.

* `NAN` can be tested for using the standard C++ function `isnan()` but its initialization varies from compiler to compiler.

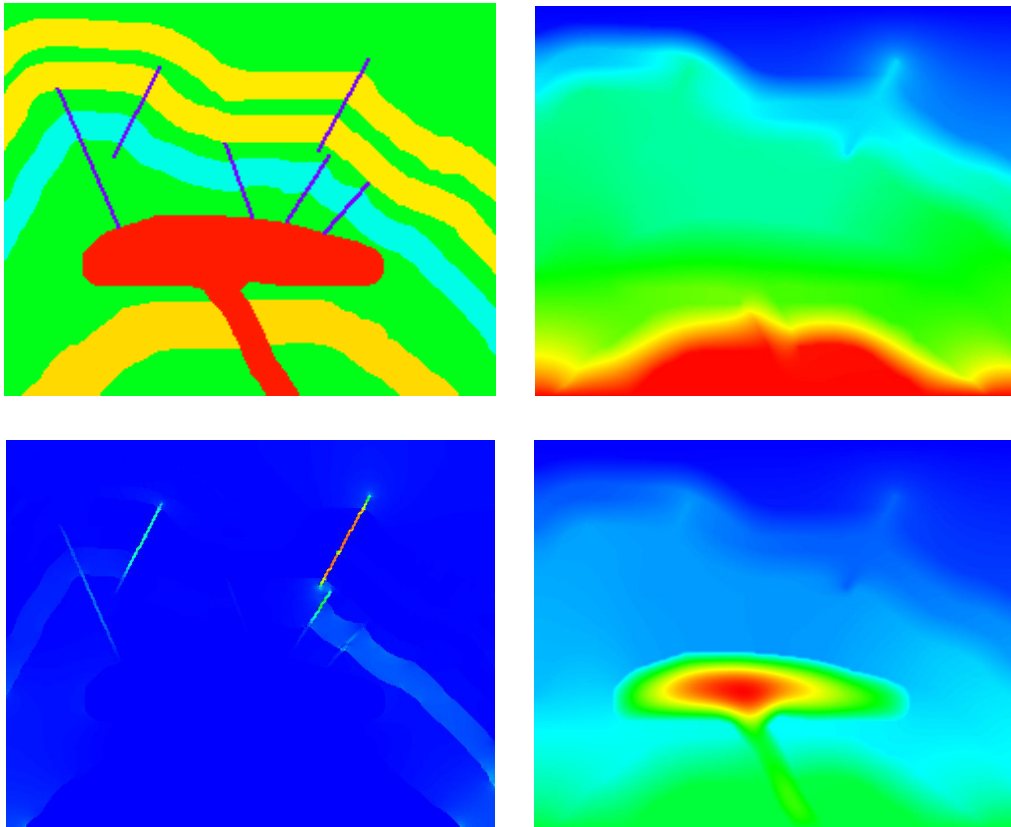


Fig 8: Cross-sectional model of a granite intrusive body in a folded and faulted rock sequence (file: granite_model1). The colors in the upper left image correspond to the $\log_{10}$ of permeability. Upper right: fluid pressure distribution ensuing from fixed upper and lower boundary pressures. Rainbow shading of variables (Red colours represent high and blue colours low values). Lower left: volume flux is highest in the fault that breaches the (yellow) low permeability layers. Lower right: fluid pressure after a region-restricted computation in which a fluid source term was assigned to the (red) granite intrusion.

Summary. A CSMP model (Model) contains the finite-element mesh (MeshManager), a property database enlisting the physical variables in use (PropertyDatabase), an object that keeps track of the current finite element types (FiniteElementManager) as well as containers of the model regions and boundaries (RegonInterface etc.). Properties are discretized on such regions, their nodes and elements and their integration points. They are stored locally in LocalVariableStorage objects. Geological entities in the model are discerned as Region objects (Region->ModelSubmain).

Data access and modification occurs through the public methods of the Model (Fig. 6). Key steps are that the property database is queried for the specifications of each variable identified by a string (for instance "permeability"). The database then returns a struct `csmp::Index` that uniquely identifies the variable. This index is used for Read() / Store() / Status() operations overloaded to retrieve Scalar-, Vector-, Tensor and Array variables. Within the LocalVariable storage, the variables live in STL vectors whose size varies with variable placement. Typically, a triangular/tetrahedral mesh will contain many more elements than nodes. Memory to store variables is allocated upon demand according to the specifications in the property database that is usually initialized before a model is build using the specifcatons of variables in the variables text input file. Even if it is not used in a calculation, every variable in the database requires storage.

Computations are carried out with PDE_Integrator that performs the assembly of partial differential equations and integrates them with Solver classes. The Model::Apply() interface applies PDE_Integrators and Interrelations to itself. Interrelations compute local relationships between variables. Any PDE_Integrator, visitor or interrelation object can be selectively applied to model subregions or boundaries.

4. A Tour of CSMP++

The following subsections provide a first look at the high-level CSMP++ classes which a user or developer is likely to interact with. For documentation of their individual methods (=member functions), consult the CSMP++ DOxygen documentation.

REGION

The aforementioned Region class implements selective access to user-named subregions of a model including itself via a region named “Model”.

Region objects are accessed by name and their elements via iterators.

```
Region<3>& target = model.Region("upper sandstone");
for ( auto it=target.ElementsBegin(); it!=target.ElementsEnd(); it++ )
    (*it)->Out(); // writes state of element to console
```

Each region object consists of vectors of pointers to its Element and Node objects. Each of these vectors is sorted into two ranges. The first range contains the interior- and the second range the perimeter elements of the region. The same applies for the node vector. Perimeter elements and nodes make up the external boundary of the region (Fig. 9). Importantly, elements are considered perimeter elements only when they have at least one face (edge in 2d) on the region boundary.

Being able to distinguish the INTERIOR from the PERIMETER of a COMPLETE region via enums is useful because this permits quick determination of respective properties and the selective search for and assignment of property values. Important: Region objects have their own property value storage (LocalVariableStorage). This is used, for instance, to associate constitutive relationships with model regions to which they apply. Also uniform property values can be stored there, rather than on each individual element of the region.

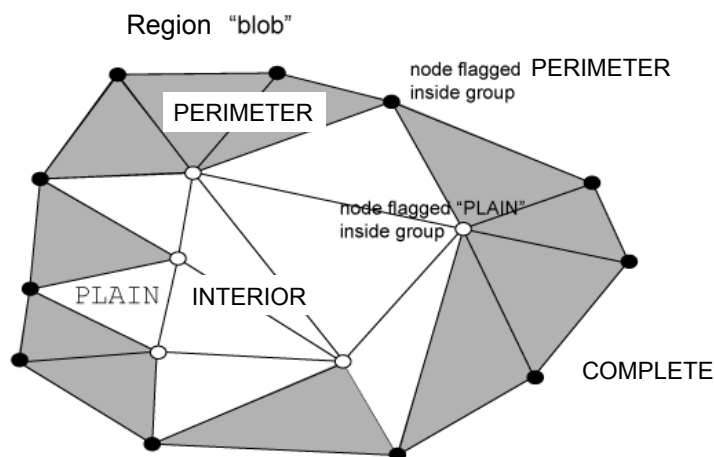


Fig. 9: Region flags distinguishing different parts of the region. Elements that classify as located at the boundary=PERIMETER are shaded in gray. They must at least share one face on the boundary.

For each perimeter element, there is a record of the face(s) that is(are) located at the region boundary. It is stored in separate boundary face vector to be used in operations targeted on the faces of a region perimeter.

Regions do not have to represent contiguous model domains. They can also contain numerous disconnected entities with similar properties; for example, all the fractures in a block of rock, see Figure 10. Such discontinuous regions can be partitioned into contiguous subregions with Model methods like:

```
PartitionRegionIntoContiguousSubRegions( const char* );
```

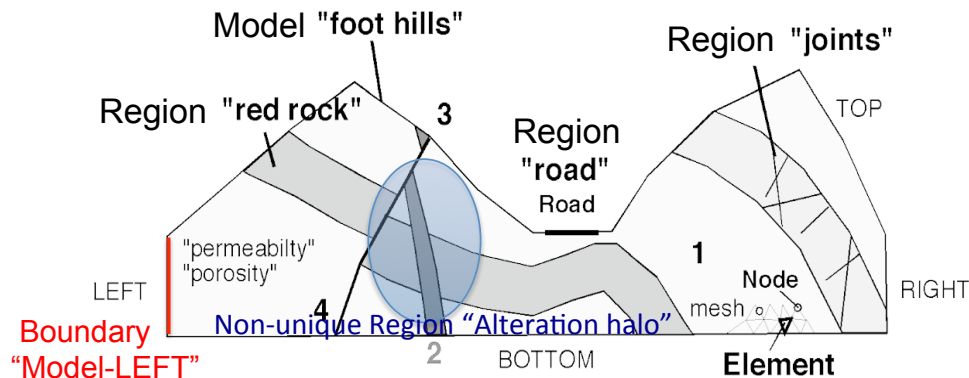


Fig. 10: Region types in a CSMP++ model.

This method creates numbered subregions that are all contiguous.

Regions can be created flexibly on the basis of user-defined selection criteria. The smallest possible region contains at least one Element object. Thus, a region cannot consist only of nodes or integration points.

Normally, a region consists of equi-dimensional elements. There are special cases, however. For example, a volumetric region that contains surface mesh that sticks out of its boundary represents such a case. If so, all these exterior surface elements and their nodes are treated as perimeter entities.

When a geomodel is input into CSMP++ its subdomains become non-overlapping = unique regions of the Model. However, the user also has the option to build regions on the fly using a wide variety of construction criteria. The most widely used ways to build regions are:

- From a geometrical entity pre-defined in a CAD input model; this information is communicated to CSMP++ via a ModelTopology object automatically when the Model is constructed. To do it manually use `FormRegionsFrom()`.
- From elements within a certain property range; see example in the previous chapter (call `FormAndAddRegion()` or `RegionsFromPropertyValues()` to create multiple regions),
- From a combination of property values specified in a PropertyConstraints object, see example below,

- As a rectangular / box shaped region with specific coordinates (use `FormAndAddRectangularRegion()`). This region will only have smooth boundaries if the mesh has appropriate element shapes.
- From a list of elements specified by the user (use `FormFromElements()`).
- By copying or carrying out Boolean operations on pre-existing regions: union, difference, intersection etc.

The use of a `PropertyConstraint` object delivers the highest degree of flexibility for region selection. This is illustrated in the following example where a region is build from elements that have a permeability below 10^{-18}m^2 and are subjected to a pressure and temperature above 100MPa and 400°C, respectively:

```
PropertyConstraints constraints("permeability", 1e-21, 1e-18 );
constraints.AddConstraint( "fluid pressure", 1e8, 1e7 );
constraints.AddConstraint( "temperature",      800., 450. );
// relaxing the inclusion criterion from all to at least one node
constraints.SatisfyConstraintsForAtLeastOneNode( true );

model.FormAndAddRegion( "element group", constraints );
```

Due to the differences among the region creation methods and apart from region contiguity, two types of regions will result from creation: 1) “unique” ones without any overlap that will, in most cases, cover the entire model volume in terms of subvolumes like in a CAD model; 2) “non-unique” ones that may overlap with one-another as in the case above where the region “element group” was created using the `PropertyConstraints` object. CSMP keeps such regions in two separate containers allowing its users to access them selectively. This is important, for instance to avoid that the overlap of non-unique regions may create unexpected behaviour during REGION property value assignment.¹

Lower-dimensional regions can be unique although they share their nodes with higher dimensional ones.

The region “Model” always exists, but will be contained in the unique regions container if there is no other unique region. If region “Model” is not alone, however, it will overlap any other unique region. In this case it is therefore stored in the non-unique regions container.

Starting with any type of region, additional ones can be created using *boolean algebraic operations*. This capability includes region *intersection*, *difference*, *symmetric difference* and *union*. Using this capability, one could, for instance, find a subregion of a shaly layer which is faulted:

```
// form new group from the intersection of a shale and a fault group
intersection( "shale", "diffuse faults", "faulted shale" );
```

Regions are ideally suited for selective monitoring of variables over the course of a simulation. This is achieved with the `RegionMonitor` class:

```
string file_name("monitored-regions.txt");
```

¹ Since property values can be placed on the regions an awkward situation arises when regions overlap. Then, dependent on which region is queried, reading the same property may return a different value.

```

list<string> integral_properties;
integral_properties.push_back("saturation oil");

list<string> range_properties;
range_properties.push_back("fluid pressure");
range_properties.push_back("saturation oil");

RegionMonitor<2> monitor( model, integral_properties,
                        range_properties );

// monitor all regions at this particular time level
monitor.ScalarPropertyRanges( model, global_time );
monitor.ScalarPropertyIntegrals( model, global_time );

// writing the current level of chronologically ordered data to file
monitor.Out( file_name.c_str() );

```

As already shown in the steady-state fluid pressure example in the previous chapter, computations can be restricted to regions. Operations that can be carried out on regions include calculations of single variables (via Interrelation objects), multiple variables and constitutive relationships (via Visitor objects), and the integration of partial differential equations (via PDE_Integrator objects). The interface of the Region class also permits the modification of distributed variables and their flags. This rich functionality of regions is further illustrated in the example application **Region_Example** (Region_Example.h).

A special flavor of the Region class is the **NimbleRegion**. This light-weight region class can be constructed more rapidly and is optimized for shape modification over time. It also supports the IntegrateOver() interface of the PDE_Integrator class.

BOUNDARIES

Model boundaries can be regarded as special regions that have a lower spatial dimension than the model as a whole. This comprises the side boundaries of the model, material interface in a volumetric model represented by surface elements, and or wells in three-dimensional models represented by a chain of line elements. Computations involving boundaries are possible. When a mesh is imported into CSMP++ from another application, CSMP++ has to be told which of the lower dimensional elements that this mesh contains should be treated as boundaries. This decision has to be made by the user or conveyed via region names that contain the string BOUNDARY. When prompted this way, CSMP++ will convert a corresponding Region object into Boundary object, deleting the original region.

A **Boundary** is defined as a (dimension – 1) Region, sharing much of its functionality with the latter through the common base class **ModelSubDomain** (look at this base class for key Region or Boundary functionality). Thus, boundary objects also contain sorted ranges of elements and nodes and have a perimeter, which is an edge curve in 3D models, but reduces to the two end points of a line boundary in a two-dimensional model.

The other important difference between Boundary and Region is that it consists of Face objects as opposed to elements. FACE is a different variable placement than ELEMENT and element variables therefor are only available from the higher-dimensional neighbor Element objects of the Face objects. By contrast to the Element,

however, the Face knows its higher-dimensional neighbors and neighboring Element objects are readily accessed (`Element<dim>* Parent( INTERFACE_SIDE ) const`).

When a boundary is located on the outside of a model, each face only has a single higher dimensional neighbor element. This always is the INSIDE neighbor located on the opposite side of the direction to which the normal of the Face points. This way everything is consistent: the Face normal and the outward pointing normals of the Element, which is covered by the Face.

For box-shaped models introduced earlier (see `Box.h`), Boundary objects can be constructed, extending the functionality of the `BOX_BOUNDARY` flags (Fig. 5). By contrast with flagged-only box models, one with Boundary objects will contain boundaries labeled `TOP, LEFT...EDGE#` so that computations can be performed on them. Configuration and boundary condition assignment to these models is explained in a later section on the **InputDataManager** class.

As discussed earlier, CSMP++ has a range of input interfaces which allow the construction of a `csmp::Model` from an imported mesh, including region and boundary objects. The convention adopted in this context is that, beyond box-shaped models, during model construction, any (model dimension-1) region whose name contains the string “BOUNDARY” will be automatically converted into a Boundary object. In the “establish boundary” process, per default, the original Region object and all its elements will be removed. The Element objects get replaced by new (dimension-1) **Face** objects with the aforementioned extra functionality: they have a unit normal that points outside of the higher-dimensional Region that they surround. They also have connectors to the higher-dimensional “parent” Element objects on either side (method `Face::Parent(INTERFACE)`). However, when a Boundary is located on the perimeter of the “Model” region, the unit normals of its Face objects point into open space and the Face objects only have INSIDE parent elements.

Note that `BOUNDARY` and `FACE` constitute unique variable placements. If variables are placed there, storage for them is allocated only where boundary objects exist and their names ought be accordingly specific; for instance “basal heat flow” prescribed in W/m^2 . If the computation of boundary variable values involves variables placed on the Element, these are readily fetched from the parent elements of the faces via the aforementioned pointers:

```
csmp::Index element_property=model.Database().StorageKey("permeability");
for ( auto it=boundary.ElementsBegin(); it!= boundary.ElementsEnd(); it++ )
{
    double k = (*it)->Parent(INSIDE)->Read( element_property );
    ...
}
```

Note that the Face class is not inherited from the Element. In stead, both share `FiniteElementPolicy`, which delivers their commonalities. This class design with switchable policies supports *static polymorphism* and has the advantage that there is no runtime nor memory overhead by contrast to the use of dynamic polymorphism (virtual function, vtables, etc.).

Creation of boundaries. The CSMP API currently supports three ways to create Boundary objects, implemented in the class **BoundaryInterface** accessible through the public interface of the Model object:

1. Around volumetric regions:

- `bool AddFaces( const char* region );`
- `InsertBoundary( BOX_BOUNDARY boxBoundary, const char* region );`

2. From (dimension-1) regions:

`CreateInternalBoundaryFrom();`

3. Between volumetric regions:

- `bool InsertBoundary( const char* region1, const char* region2 );`

Naming conventions for boundaries. Apart from the predefined names for the boundaries of box-shaped models (LEFT, RIGHT...), boundary names always contain the string “BOUNDARY.” Unique names for internal boundaries (located between volumetric regions) are achieved by concatenating this keyword with the names of the adjacent volumetric regions; for instance,

`“BOUNDARY_SAND_SHALE”`

The order of these region names matters: the first region (SAND) marks the inside of the boundary; the second (SHALE) refers to the outside, i.e. the side that the Face normal vectors all point toward. Such directions are imposed by the meshing tools honouring the model’s original topology.

When internal boundaries are created from (dimension – 1) regions, the name of these parent regions is added to the boundary name. Extending the example from above and assuming that the parent region was called “FAULT”, this yields the name

`“FAULT_BOUNDARY_SAND_SHALE”`

Such nametags are useful because they support a treatment of interface processes which is dependent on which rocktypes are juxtaposed against one another. The creation of boundaries from surface meshes that intersect multiple unique model regions therefore automatically triggers the creation of multiple boundary objects – as many as regions are juxtaposed against one-another, while the name prefix (FAULT) ascertains that the relationship to the former parent region from the (geo)model is recorded.

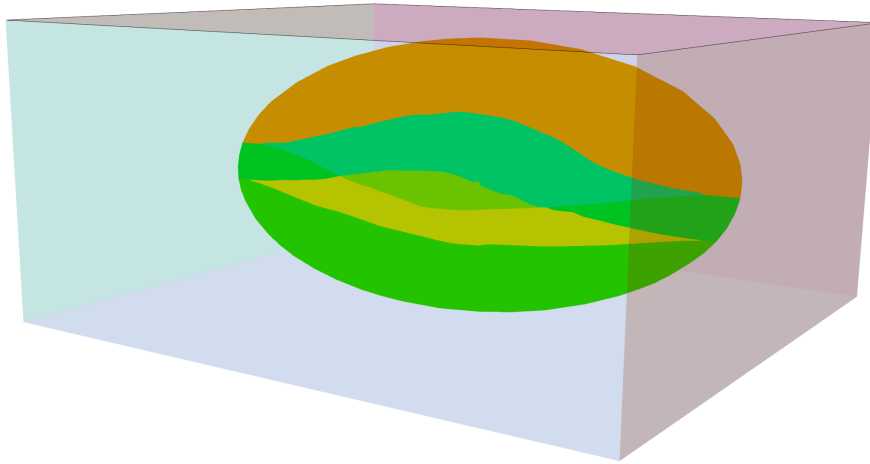


Fig. 11: Fault surface partitioned into Boundary patches on the basis of juxtaposition relationships. Original surface region 'NORMAL_FAULT' intersected the 3 different layers `LAYER_BOTTOM`, `RESERVOIR_LAYER`, `LAYER_TOP` and was partitioned into `NORMAL_FAULT_BOUNDARY0_LAYER_RESERVOIR_LAYER_TOP`, `NORMAL_FAULT_BOUNDARY1_LAYER_RESERVOIR_INTERSECTION`, `NORMAL_FAULT_BOUNDARY2_LAYER_BOTTOM_INTERSECTION`, `NORMAL_FAULT_BOUNDARY3_LAYER_BOTTOM_LAYER_TOP`, `NORMAL_FAULT_BOUNDARY4_LAYER_BOTTOM_LAYER_RESERVOIR`, `NORMAL_FAULT_BOUNDARY5_LAYER_TOP_INTERSECTION`. The label `INTERSECTION` refers to patches where the same rocktype is present on either side of the fault. This code example is from 'BoundaryInterface_Test'.

SPLIT BOUNDARIES

In some cases the introduction of Boundary/Face objects is not sufficient to achieve computational goals. For instance when jump discontinuities in the computed variables are expected to arise at material interfaces (e.g., Nick and Matthai, 2011, VZJ). In this case, node-matched mesh patches can be imported or a contiguous mesh can be split (=“exploded” into separate mesh objects) in a 2-step procedure: 1) creation of boundary objects from lower dimensional regions at where the splitting shall occur; 2) converting existing Boundary- into SplitBoundary objects. Mesh splitting is achieved by replacing the lower-dimensional Face objects that made up the boundary by so-called **InterFace** objects with additional nodes. Each Face node is replaced by as many InterFace nodes as materials come together at this point.

The new mesh contains many additional nodes = degrees of freedom, but remains node matched, at least initially. Like the Face, the InterFace knows its higher-dimensional neighbor Element objects – exactly 2 – one on each side. Its inside is identified as opposite to the outward pointing normal.

InterFace objects are the building blocks of **SplitBoundary** objects. It follows that these can exist only on the inside of a model and always have higher-dimensional neighbors on their inside and outside.

If a node-matched set of mesh patches is supplied to CSMP++, SplitBoundary objects can be created to connect them by calling

```
bool DetectAndCreateSplitBoundaries();
```

This is a costly process, but may be justifiable when the model is subsequently saved to a CSMP native binary file for further use, which is always desirable.

There is only one method to create `SplitBoundary` objects within CSMP++:

```
bool CreateFrom(Boundary);
```

`SplitBoundary` creation transforms an existing model so that it will have a range of properties that are very different from the original model with `Boundary` objects only:

- Split boundaries decouple computational domains from one another; if nothing else is done, either side becomes a “natural boundary.”
- If split boundaries completely surround a part of a model, this part becomes a stand-alone mesh that needs extra essential conditions as integration constants for the PDE that will be solved.
- New nodes with property storage will be created and property values will be mapped to them from the original nodes

By contrast with `Boundary`, `SplitBoundary` objects can only be created in the interior of a model. Notwithstanding, one might use similar functionality to couple separate Model instances together.

ERRORHANDLER

The CSMP++ API contains the Singleton (class that can only have a single instance) **ErrorHandler** that ascertains that error messages are reported in a consistent way and appropriately for the current computational platform. The easiest way ErrorHandler can be used to track and report a notable incident is:

```
ErrorHandler& incident( ErrorHandler::Instance() );  
incident.notice( WARNING, incident_source, message );
```

The first line of code provides a local reference to the global singleton initialized elsewhere. Here `incident_source` refers to the `class::method` name from where the incident originated. The message describes what happened.

ErrorHandler handles how incidents are reported and processed by CSMP++ and does so in a platform specific way. Since it takes control of the entire program it can only exist once and has therefore been implemented as singleton. Any CSMP++ program contains one instance of the ErrorHandler, which also contains the licensing mechanism.

Normally, the error handler only prints the messages to a log file or the console / screen using either buffered (`cout`) or unbuffered (`cerr`) streams (Tip: make console memory big enough to scroll up to see the entire output of a simulation to see when and where WARNING or ERROR messages started to occur). The behaviour of the ErrorHandler depends on whether a program is run in DEBUG or RELEASE mode. In DEBUG mode, the program halts allowing the user to intervene manually. To recover from problems the user has to implement exception handling facilities that are part of CSMP-

based applications. CSMP++ only contains rudimentary exception handling, complementing the ErrorHandler mechanism (see `csmp::Exception`).

The enumeration `CSMP_MESSAGE` is defined in “Exception.h” and is used by the error handler to categorize what happened and to control how it is reported and handled on the current computer platform.

```
enum CSMP_MESSAGE { INFO=0, EXCEPTION=-1, WARNING=-2, ERROR=-3,
                   FATAL_ERROR=-4 };
```

Important here is that when the ErrorHandler issues `INFO` and `WARNING` messages, these will not interrupt program execution. A maximum number of `ERRORS` is permitted to occur before the handler terminates a simulation. By contrast, a `FATAL_ERROR` will lead to the immediate termination of a program.

The ErrorHandler logs information as strings that are printed to 'cerr' and to the text file 'Runtime.log'. To be able to trace a message to its source, a string that consists of the class and method name from which the message originated is supplied as first message argument. In case of an overloaded method, additional information in braces should be included into the output to identify the called method uniquely, for example:

```
Error: Region::FormAndAddGroup(from PropertyConstraints): Supplied
element list is empty.
```

Messages logged to file, will be preceded automatically by a time string. This tells the user when the incident prompting the message occurred. Since messages may also contain numerical data in the description of the error message the ErrorHandler interface is overloaded to accept `std::string` arguments. To handle top level (C++) exceptions the ErrorHandler employs the standard exception base classes `exception`, `logic_error`, and `runtime_error`. For CSMP++-specific error types instances of the class `csmp::Exception` can be *thrown*. A goal achieved with this exception handling is to store the state of a run just before it got aborted due to a `FATAL_ERROR`. This permits the restart of a simulation at a later time, for diagnosis and elimination of errors. To save a computation to a binary file, the model is first exported into a `csmp::VSet` container and then stored permanently in a binary file. To make use of these capabilities, a program segment that may throw exceptions has to be surrounded by try and catch statements, for example:

```
try {
    cout << "\nmain: Advecting non-wetting phase..." << endl;
    // here goes the code that does this
}
catch(Exception& csmp_excep ) {
    cout << "\nCSMP++ Exception: Raised by: " << csmp_excep.What() << endl;
    VSet<3> crashed_model;
    cout << "\nmain: Saving current model to CSMP++_VSet... " << endl;
    model.OutputTo( crashed_model );
    const csmp::Index time_key(model.Database().StorageKey("time"));
    crashed_model.OutputTo( model.Name(), model.Read( time_key ) );
    // any other code here...
    return 1;
}
```

Alternatively, the high-level output of a complete Model object to binary file may be preferred:

```
model.OutputToBinaryFile( model.Name() );
```

ErrorHandler is only used inside CSMP++ specific larger classes like Model. Small generic single-purpose classes report errors directly to the 'cerr' stream to prevent a loss of generality by making them dependent on the error handler. Naturally, classes do throw standard exceptions.

PROPERTYHANDLE

The class **PropertyHandle** was implemented to write top-level mathematical expressions involving distributed variables. Due to the inelegance of the current implementation it is likely to be refactored in the near future. Lambda functions already provide a viable alternative.

PropertyHandle objects associate themselves with physical variables registered in the PropertyDatabase or prompt the latter to create a new variable at runtime according to specifications supplied via custom constructor. The new variable has local scope and will be destroyed when the destructor of the handle is called. PropertyHandle currently includes C++11 standard mathematical functions. In the following example, a PropertyHandle is used to calculate the natural logarithm of the variable “porosity” times a multiplication factor of 0.9. Afterwards the new variable "log porosity" is output to a text file. Note that in the example, the handle is created dynamically such that the corresponding variable can be removed at a user-defined time:

```
// existing variables
PropertyHandle pf( current_Model, "fluid pressure"); // node scalar
PropertyHandle phi( current_Model, "porosity", SCALAR, ELEMENT );

// new variable only needed for logarithmic output
PropertyHandle* log_phi = new PropertyHandle( current_Model,
                                              "log porosity",
                                              SCALAR, ELEMENT );

// assign the porosity to the new variable
double64 factor(23.4);
*log_phi = 0.0;
*log_phi += phi * factor; // a+=b is C++ shorthand for a=a+b

// logarithmitize values
log_phi->Log();
log_phi->Out( logarithmitized_porosity );

// delete the temporary variable
delete log_phi;
```

It is also possible, to write expressions involving variables with different placements, which involves interpolation or extrapolations, supported by the finite element basis functions. However, this functionality is still in its infancy.

In summary, PropertyHandle provides direct access to- and mathematical manipulation options for distributed physical variables. This facilitates a math software-like notebook style of setting up calculations.

INTERRELATIONS

As seen in the steady-state fluid-pressure example, CSMP++ calculations, which can be carried out on an element-by-element basis because they do not involve the inversion of a global solution matrix, are done by subclasses inherited from the Interrelation base class. When writing the source code for such subclasses, one associates required global physical variable(s) with operand class objects and writes intended mathematical expressions inside the virtual `Calculate()` method of the new subclass. A decisive rule for interrelation objects is that *a single interrelation can only alter the value of a single physical variable*. This restriction allows to execute interrelations in a sequence that takes mutual interdependencies into account. While the output from interrelations is checked against the property database, the input is not. It is the responsibility of the interrelation implementer to perform such range checks to ascertain that the intended calculations are only performed for input values for which the interrelation has been tested.

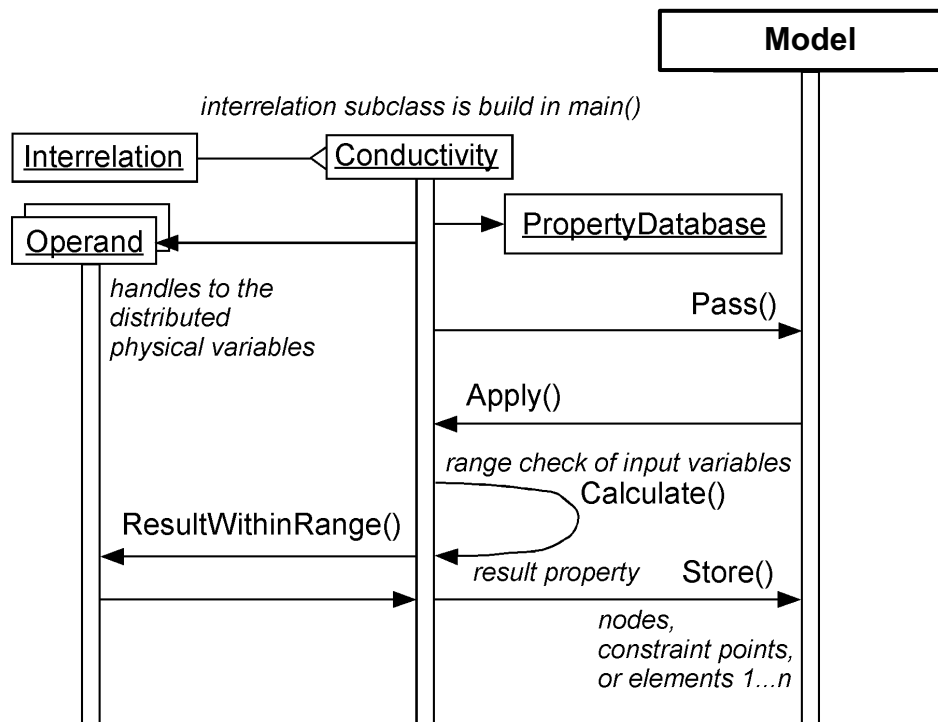


Fig. 10: Sequence diagram illustrating the use of an interrelation subclass (e.g., Conductivity) in a CSMP++ simulation.

As an example of a calculation, which could be carried out using an Interrelation object, the calculation of the element property “hydraulic conductivity” by dividing permeability by fluid viscosity is presented in Figure 10. In the `main()` programme an instance of this interrelation would be constructed and used in the following way.

```
#include "ConductivityFromPerm.h"

const PropertyDatabase& pref(model.Database());
const double          viscosity(1.6e-3);

ConductivityFromPerm< 3U> K( pref, viscosity );

K.RestrictApplicationTo("saturated zone");

Model.Pass( K );
```

The code for this interrelation is in the header file of the class 'ConductivityFromPerm' and is listed below. To apply this interrelation to a model, one simply creates an instance of ConductivityFromPerm and supplies it as argument to the Pass() interface of the Model.

```
#ifndef CONDUCTIVITY_FROM_PERM_H
#define CONDUCTIVITY_FROM_PERM_H

#include "CSMP_definitions.h"
#include "Interrelation.h"

namespace csmp {
```

The interrelation ConductivityFromPerm is inherited from the base class Interrelation:

```
template<size_t dim>
class ConductivityFromPerm : public Interrelation<dim> {
    Operand< dim>& k;
    Operand< dim>& K;
    double64      viscosity;
public:
    ConductivityFromPerm( const PropertyDatabase& p, double visc );
    ~ConductivityFromPerm() {};
    void Calculate();
};
```

When the interrelation ConductivityFromPerm is constructed, the Operands, k and K are associated with the target distributed variables “permeability” and “conductivity”, respectively. The viscosity is initialized to a fixed value. Also the VARIABLE_FLAG flag that permits the output of the result to the Model is set (INIT_COND) and the result property is identified as the “conductivity”.

```
// constructor
template<size_t dim>
ConductivityFromPerm<dim>::ConductivityFromPerm( const
                                                    PropertyDatabase& p,
                                                    double64 visc )
    : Interrelation<dim>(p),
      k( Interrelation<dim>::GlobalProperty("permeability") ),
      K( Interrelation<dim>::GlobalProperty("conductivity") ),
      viscosity(visc)
{
    Interrelation<dim>::Name(" ConductivityFromPerm");
    // setting up the variables for the formula
    Interrelation<dim>::OutputCondition( K, INIT_COND );
    Interrelation<dim>::ResultProperty("conductivity");

    // test whether the input viscosity is within a physical meaningful
    // range
    ErrorHandler& csmp_error(ErrorHandler::Instance());
    if ( viscosity < 1.0e-5 or viscosity > 1e6 )
        error.notice( ERROR,
                      "ConductivityFromPerm<dim>(constructor)",
```

```

        "input fluid viscosity out of range");
    }

```

The calculation that is done when the interrelation ConductivityFromPerm is passed to the Model is defined in the Calculate() method:

```

template<size_t dim>
void ConductivityFromPerm<dim>::Calculate()
{
    // range testing of input against PropertyDatabase
    k.IsWithinRange();

    // calculation
    K = 1.0 / viscosity;
    K *= k;

    // testing
    K.PrintValue(); // turn off once tested

} // end specification of interrelation
} // end namespace CSMP++

#endif

```

The source code listing illustrates that a new Interrelation subclass can be created in a few simple steps:

1. Test the formula that describes the interdependencies of the physical variables of interest. This may involve graphing and designing this Interrelation as a formula in Matlab or Mathematica or doing a curve fit through some experimental data. One must also define a value range for each of the involved input variables as the range in which the mathematical formulation produces meaningful results. Ideally, this range should be at least as wide as the value range specified in the property-database text file from which the CSMP++ program initializes the property storage. The new formulae must be tested for all possible input parameter combinations within the prescribed ranges before it is used inside CSMP++ computations: Using the global instance `skm_err` of the `SkmErrorHandler`, range violations can be reported as was described earlier. This approach is the *only way of ascertaining that a program produces correct results once a model becomes complex* and contains several interrelations.

2. Define a notation and make each symbol a reference to an Operand (`Operand&`). These references will be private data members of the new interrelation subclass declared by public derivation from the Interrelation class. In the example above, `k` and `K` are made to refer to the distributed variables “permeability” and “conductivity”, respectively.

3. Register the Operand references in the constructor of the derived Interrelation subclass. In C++ references must be initialized in the constructor. First, one constructs the base class, then the base class method `GlobalProperty()` is used to associate the Operands with the distributed variables. Finally, one defines which of these variables will store the result of the calculation using the base class method `DefineResultProperty()`. One must also define an output condition flag (`OutputCondition()`) for the result property. The use of this flag prevents Interrelations from overwriting variable values with a different flag.

4. Override the pure virtual member function `Calculate()` of the `Interrelation` base class. The user defined new method `Calculate()` will contain the mathematical formulation of the interrelation written as equation, using the Operand reference notation and involving `*` `/` `+` `-` `<` `>` `<=` `>=` and other mathematical symbols which are interpreted as overloaded operators of the `Operand` class. If many temporary variables are needed they should be defined as private data of the `Interrelation` subclass to prevent their destruction when the `Calculate()` method goes out of scope. In the example above, the `viscosity` is such a private variable.

To assign the value of a distributed variable represented by an `Operand` to a scalar, vector or tensor variable, the `AssignTo()` method of the `Operand` should be used:

```
TensorVariable<CSMP++_float,3> perm;
k.AssignTo( perm );
```

Note that if one writes something like

```
K += b + c - d * 24.5;
```

this will automatically create four temporary `Operands` (why ? - do C++ exercise). If speed is critical because interrelations are applied again and again in a transient calculation, avoid the creation of temporaries by converting such code as follows:

```
K -= d * 24.5;
K += b;
K += c;
```

The product of *d* and the decimal number will be a floating point number; this temporary is not costly, as compared with a new `Operand`.

5. Apply the new `Interrelation` by passing it to the `Model` (`Pass()`). This will invoke the `Interrelation` method `Apply()`, which computes the formula and performs a range check on the result values.

6. By default, the new interrelation is applied to the entire model. If the new interrelation shall be applied only to a group of elements the method `RestrictApplicationTo()` facilitates this.

Calculations one can perform with `Interrelation` subclasses are limited to result variables that are on the same or a higher level in the `Model` mesh hierarchy as the input variables. For instance, one can calculate an element property from node- or integration point properties but not the other way around because this would require information from more than one finite element. If one calculates an element property from node property values, the interrelation algorithm will use a *nodal average* rather than interpolated values within `Calculate()`. If this is too limiting, one may consider to do the calculation via a `Visitor` object (see next section). As another alternative one can extrapolate element properties to the nodes with:

```
void Model<dim>::ExtrapolateElementPropertyToNodeProperty(
    const char* eprop, const char* nprop, bool by_distance=true );
```

Do not forget to create the necessary node variable first. Also, there are different ways of weighting the contributions from elements surrounding the node in this calculation:

Weighting by the inverse distance of the element barycenter to the node is the default. Weighting by element area (`by_distance=false`) gives different, visually less convincing results.

As described earlier CSMP++ supports that variables are placed on Region, Boundary and SplitBoundary objects. To benefit from the economy of such placements, calculations involving corresponding Operand objects (either results or input variables), must be applied in a Region by Region etc. sequence so that required variable values are read only once rather than fetching values again and again as nodes or elements are processed. Currently, the Interrelation scheme is the only one in the API library which fully implements this framework, although there are some visitors which make use of region, boundary or splitboundary properties.

OPERANDS

As shown above, Operand objects support calculations involving variables with a different type or placement inside of Interrelation classes.

It is a yet-to-be-realised design objective of CSMP++ to support the writing of mathematical expressions involving physical variables with different placements and types so that they read like standard math with a concise notation. Since C++ meta-template programming permits compile-time assembly of such expressions this should be possible and has already been demonstrated elsewhere by domain-specific languages (DSL). Such a design could also implement, compile time error checking, and the detection of run-time range violations incurred in resulting calculations, including the detection of results that are mathematically correct, but not physically meaningful. This functionality is still missing from the CSMP++ API.

A small subset of such capabilities are made available through Operand objects used by the Interrelation class hierarchy. Apart from facilitating to write mathematical expressions which are applied to the entire Model as is done using PropertyHandle, the Operand class meets the need to use distributed instances of properties in calculations inside code modules, making distributed variables with a different placement interoperable. The current implementation requires the user to (1) define a mathematical notation for the variables used in mathematical expressions, and (2) associate `csmp::Index` objects with the physical variable using this notation, see example below. This way the PropertyDatabase needs to be queried only once when an Operand is initialised. Finally (3), temporary storage is needed to hold the result value of a calculation for range-checking before it is written back to the Model.

These capabilities are implemented in the Operand class, which is a simple composite holding a `std::string` with its name, a `csmp::Index` for the represented variable, any one of the basic CSMP++ variables, their physically meaningful range, and an output condition flag (`VARIABLE_FLAG`), which identifies distributed property values, which the variable is permitted to overwrite if the calculation yields a result that is within the permitted range.

The Operand has the overloaded operators `+`, `-`, `*`, `/`, `+=`, `-=`, `*=`, `/=` such that Operand objects can interact with each other, double64 data, and CSMP++ basic variables. In C++ the combined assignment operators are shorthands for expressions like:

`a = a * b;` is equivalent to `a *= b; // C++`

One should not write expressions like $A = C$ because this means, following the logic of the C++ assignment operator, that the Operand A now is an exact copy of the variable B . Thus, if Operand A represents a property stored in the Model, the corresponding variable will no longer exist following this assignment.

It is also important to remember that scalar-, vector-, and tensor variables differ from their mathematical counterparts because they contain `STATUS` flags which specify how their entries are treated during the assembly of finite-element equations. This requires rules for the assignment of flags to the result of Operand-based calculations. For example, the scalar variable expression

`A = C + B;`

needs to resolve the flag of the temporary variable that is created before the assignment is made. The rule adopted here is that a temporary Operand like that created by addition will always be flagged like the result Operand.

If one compares an Operand with a `VARIABLE_FLAG` enumeration, the comparison will be carried out using the flag(s) of the underlying variable. Thus, if a vector variable in a 2D calculation has two different flags (for instance, a `DIRICH` flag for the x displacement and a `PLAIN` flag for the y displacement), any comparison will evaluate as false. In this case, one should use the method `VectorVariable::Flag( int i )` to carry out a more specific comparison. Any comparisons with uninitialized Operands or Operands that store NaN (not a number) values will evaluate to false.

By default, Operands provide storage for a single variable value of arbitrary type. However, Operands can only be created if the variable that they associate with already exists in the `PropertyDatabase`. This differs from `PropertyHandle` objects, which can spawn the generation of a new variable in the `PropertyDatabase`.

VISITORS

The visitor scheme, as described in Gamma et al. (Reusable object-oriented software patterns), provides access to the public interfaces of classes inside of a class hierarchy. It thus permits the implementation of new functionality without modification of existing classes. In the visitor pattern, each target class has an interface `Accept( visitor& )`. This interface calls the visitor's member function `Visit(target_class*)` giving the visitor access to the public interface of the target class. With this simple technique a wide variety of operations are instrumentalized in CSMP++ (Fig. 11). All Visitor subclasses derived from the **Visitor** base class have access to the public methods of the visited classes and can differentiate the treatment of sub-targets in the Model hierarchy. This is achieved by overloading the method `Visit()` for regions, boundaries, elements, faces, interfaces, integration points and nodes, i.e. `Visit( Node* )`, `Visit( Region* )`. Like interrelations and

PDE_Integrators, visitors can be restricted to regions or boundaries. Since visitors have access to the functionality of elements, the visitation path can also employ their connectivity, moving from an element to its neighbor. Any kind of distributed data can be retrieved from the finite-element mesh during its visitation. A visitation starts with the submission of a specific visitor to a target object through the `Accept( Visitor& )` method (Fig. 11), defined for all classes in the Model hierarchy. As a source code example of an inherited visitor, the **ID_Visitor** collects ID numbers of target objects. Its declaration is in “ID_Visitor.h” (including inline functions that show how the visitor accumulates the desired data).

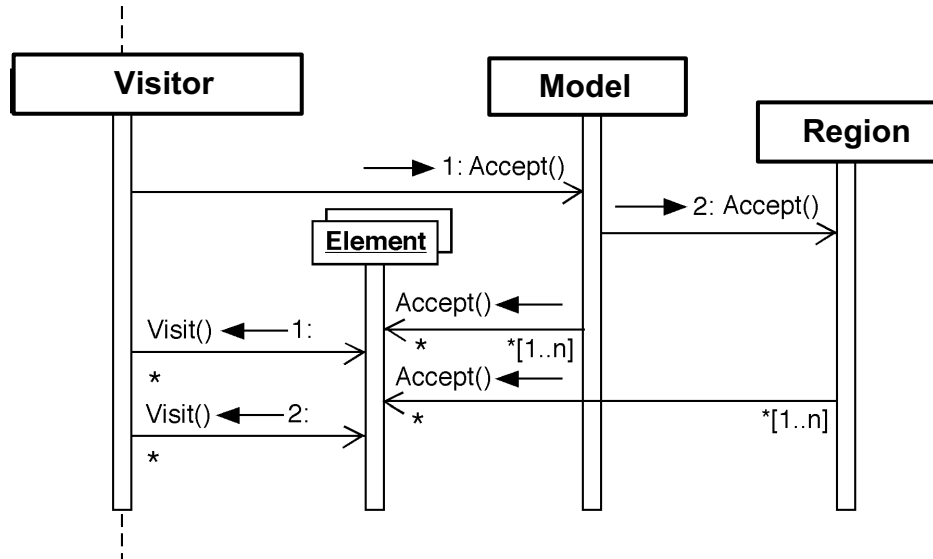


Fig. 11: Application of visitor object to the Model hierarchy of objects.

The application of a visitor involves recursive dispatching by target objects: When the visitor is submitted to the Model (`model.Accept(vtor)`) it determines whether to pass it on to a Region or directly to the collection of elements. Subsequently, either the Model or the Region, allows the visitor element by element access to its members. The visitor may also choose to travel directly from an element to its neighbor, tracking a fluid flow path or conducting a mesh sweep such as a “flood fill” (protected method `Model::FloodFill()`). To control the entry and the target level of a visitation of the model class hierarchy, the user defines the `APPLICATION_LEVEL` (start of the top-down visitation) and the `APPLICATION_TARGET` (the lowest level member of the class hierarchy that shall be visited. Defaults are `MODEL` and `ELEMENT`, respectively).

PDE_INTEGRATORS

PDE_Integrators, in CSMP++ terminology, were designed for calculations that require the inversion of a global solution matrix A . One or several second-order partial differential equations are discretized as a set of linear algebraic equations and solved in the form $Ax = r$, where x is the solution vector and r is the right-hand vector. A , x , and r are implemented as **SparseMatrix** and double precision STL vector templates (see `applied_math` library).

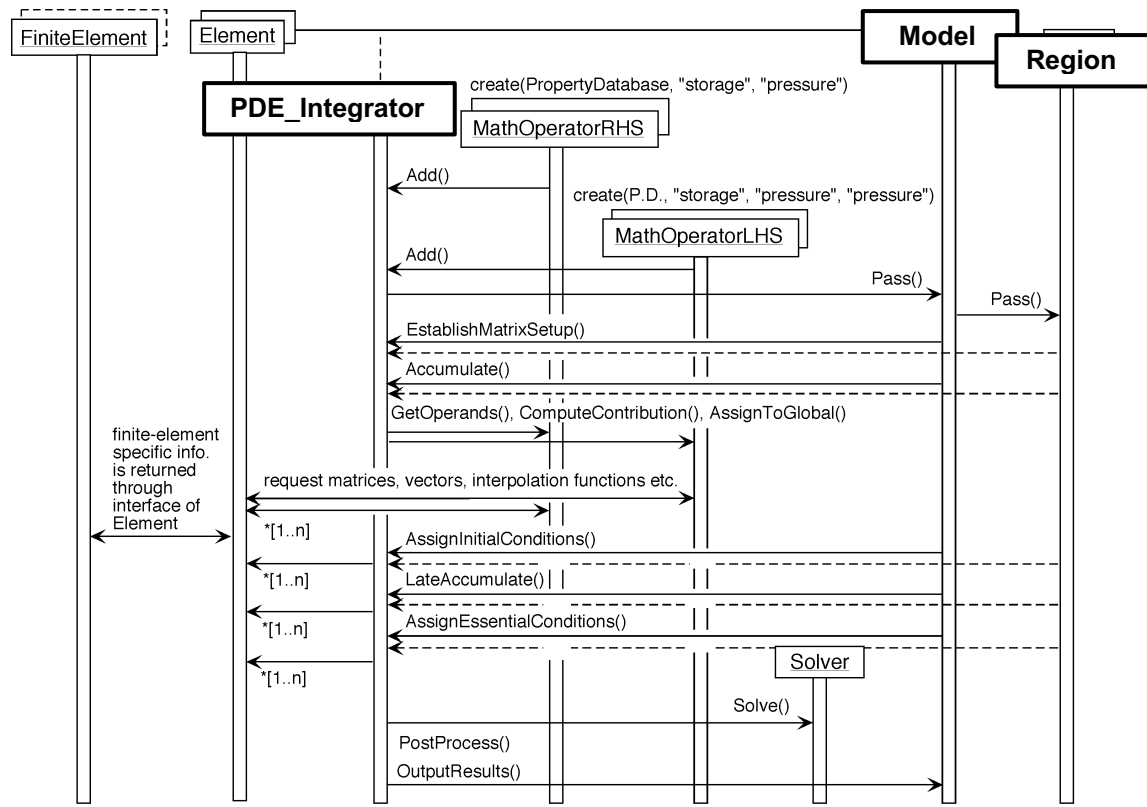


Fig. 12: Sequence diagram for the application of a PDE_Integrator to the Model. Key steps are (1) construction of MathOperator objects added to the integrator to compose the integral form of the PDE which shall be solved; (2) application of PDE_Integrator to Model, and (3) solution of the global sparse matrix, A , by a Solver object. (4) writing the results back from the solution vector to the model while performing range checks uses the ranges stored in the PropertyDatabase.

The implementation utilises several design patterns (Bridge, Strategy etc.) to make the solving of partial differential equations on global and local domains relatively simple. The non-iterative solution process is implemented in the **PDE_Integrator** base class utilising **MathOperatorLHS** and **MathOperatorRHS** base classes, in the composition of differential equations. This process is the same for any type of finite element(s) used in the integration. The user has to pick the appropriate order of finite-element interpolation to obtain the desired continuity and accuracy of the solution.

The solution of the matrix equation is carried out by subclasses of the Solver base class. This polymorphism is implemented in the frame of the strategy design pattern offering the user a variety of solution methods (Fig. 13).

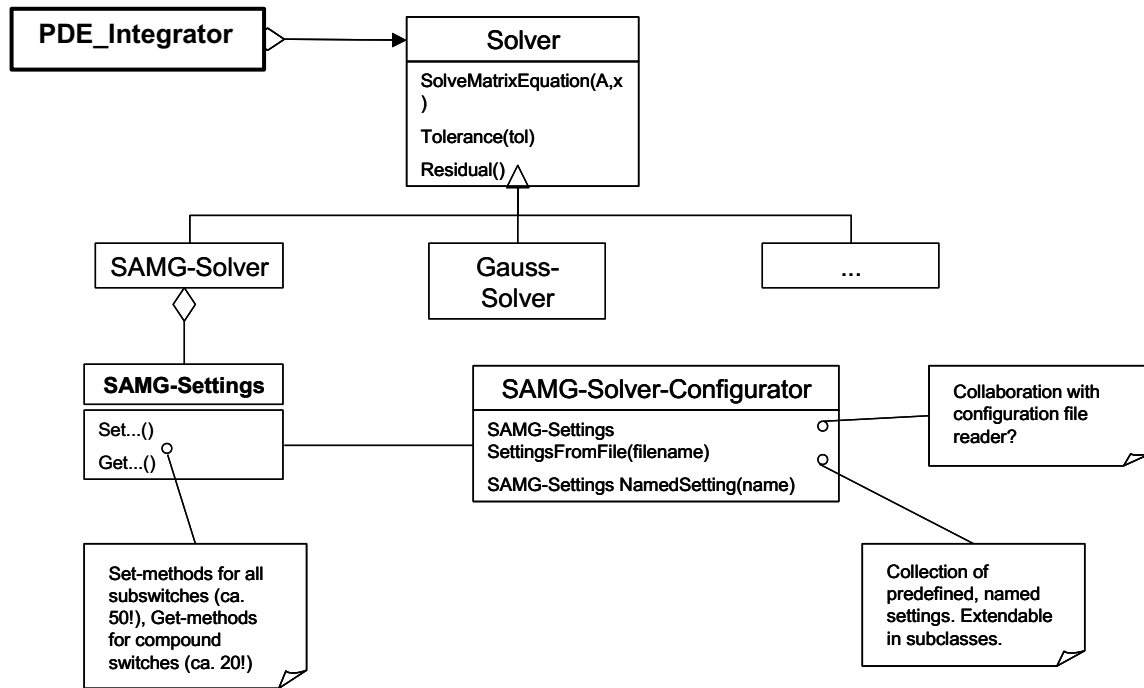


Fig. 13: Strategy pattern employed to interface different solvers with the `PDE_Integrator`. Illustration for `SAMG_Solver` class, which receives its settings from the `SAMG_Settings` object, passed by reference to the constructor.

One can choose among algebraic multigrid (commercial `SAMG_Solver`, see Ruge and Stüben, 1987 and Stüben, 2000), conjugate gradient (`CG_Solver`) and Gaussian elimination (`Gauss_Solver`) schemes. These solver objects are configured with subclasses of the Settings base class (`SAMG_Settings` etc.). Dependent on the type of computation, solver parameters may have to be set manually, see documentation of third party tools in corresponding directory of API library. As an example, the following code configures the `SAMG_Solver` for the computation of displacements, see `LinearElasticity` example:

```

SAMG_Settings settings;

// sort rhs vector so that [x1, y1, x2, y2, ..., xn, yn]
settings.Set_napproach(2);

PDE_Integrator<3,Region> deformation;

deformation.SetSolver(new SAMG_Solver(&settings));

```

Detailed descriptions of the SAMG solver parameters are given also in their user manual (SCAI Fraunhofer) and the HTML Reference manual in the documentation of the Solver class.

Apart from integration in space, PDEs may have to be integrated in time. Explicit, implicit, and semi-implicit schemes can be configured according to needs. The following 1D example computes the evolution of fluid pressure with time using a fully implicit (Backward-Euler) finite-difference time-stepping scheme and a lumped formulation for the capacitance (mass) matrices on the left and righthand sides:

```

// PDE_Integrator computes: ([C] + dt[K]){p}t+dt = [C]{p}t + dt {Q}t+dt

```

```
// -----
PDE_Integrator<1U,Region> transient_pressure;
transient_pressure.SetSolver(new Gauss_Solver());
PropertyDatabase& p_ref = model.Database();

// [K] conductance matrix
Integral_dNT_op_dN_dV<1>
conductancel( p_ref, "conductivity", "fluid pressure", "fluid pressure" );
conductancel.MultiplyWithTimeIncrement();

// [C] capacitance matrix lefthandside
Integral_NT_lhsop_N_dV<1>
capacitance_lhs( p_ref, "storativity", "fluid pressure", "fluid pressure" );
// make lefthandside matrix equivalent to right-hand vector
capacitance_lhs.LumpedFormulation(true);

// [C] capacitance vector righthandside
Integral_NT_op_N_dV<1>
capacitance_rhs( p_ref, "storativity", "fluid pressure" );
capacitance_rhs.LumpedFormulation(true);

// [Q] fluid volume source
Integral_NT_op_N_dV<1>
source1( p_ref, "fluid volume source", "fluid pressure" );
source1.MultiplyWithTimeIncrement(true);
// add to right-hand side after its multiplication with the current pressure
source1.AddAccumulateLater();
source1.LumpedFormulation(true);

transient_pressure.Add( &conductancel );
transient_pressure.Add( &capacitance_lhs );
transient_pressure.Add( &capacitance_rhs );
transient_pressure.Add( &source1 );

...

while (global_time < max_time && timestep <=100 ) {
    global_time += time_increment;
    transient_pressure.TimeIncrement( time_increment );
    model.Pass( transient_pressure );
    printRangeOfVariable( "fluid pressure" );
    time_increment *= 1.2;
    timestep++;
}
```

To use a PDE_Integrator in a transient (timestepping) calculation one has to communicate the TimeIncrement() to it as shown in the code example above.

Apart from the PDE_Integrator base class, derived Picard-, Newton- and Upwind-PDE_Integrator classes are available in the /algorithm library. As the names suggest, these facilitate iterative solution procedures for non-linear PDEs. The PDE_Integrator classes also supports the solution of coupled PDEs, i.e. equations with several dependent variables. In this case, cross terms (“Jacobians”) are needed to create the coupling between dependent variables. If the coupling is non-linear such equations must be solved iteratively (see the advanced examples 13 and 14).

PDE OPERATORS

For the standard type of finite element integrals, suitable PDE operators have been inherited from the MathOperatorLHS and MathOperatorRHS base classes, supporting the PDE_Integrator design pattern. The syntax of the method arguments (see constructors in the examples above) prescribes the position of integral coefficients in the integral:

```
MathOperatorLHS( const PropertyDatabase<dim>&,
                 const char* oper,
                 const char* basic,
                 const char* test );
```

The 'oper' typically is a material property matrix Operand and 'test' stands for test functions (=element interpolation functions) or weighting function Operand objects. An Operand will never be an output variable. Since element-interpolation functions are used to approximate the value of the dependent variable, the physical variable that one puts as a placeholder of 'test' will be the result and output property of the finite-element computation. In the code library the following standard and many other differential operators are available:

MathOperator subclass	Placement in Gx=rh	Description
Integral_NT_lhsop_N_dV	left-hand vector operator	mass matrix, volume integral
Integral_NT_op_N_dV	right-hand vector operator	mass matrix, volume integral
Integral_dNT_op_dN_dV	left-hand	conductance, volume integral
Integral_dNT_rhsop_dN_dV	right-hand	conductance, volume integral
Integral_NT_op_dNi_dV	right-hand	volume integral in direction i
Integral_dNT_op_dN_NT_v_dN_dV	right-hand	advection - dispersion
Integral_NT_op_dS	right-hand	surface integral
Integral_op_NT_dN_orthogonal_dV	left-hand	streamfunction operator
Integral_dNT_rhsop_dN2_dV	right-hand	Laplacian volume integral
<i>the same operators as above but for equations with vector type solution variables, e.g.,</i>		
Integral_BT_op_B_dV	left-hand	stiffness, volume integral etc.
<i>the same operators but using numerical integration in conjunction with isoparametric FEs, e.g.,</i>		
NumIntegral_BT_op_B_dV	left-hand	stiffness, volume integral etc.

Inside of the PDE_Integrator, the PDE operators are stored in std::maps. Each integral contributes to the equation to be solved and contains information on the place that the Operand has in the equation. Operands are shared among equations. The PDE_Integrator object encapsulates the whole process of (cf., Fig. 12):

1. Definition of a finite-element equation
2. Collection of distributed variable values from the Model object
3. Calculation of the finite-element matrices (contributions)
4. Assembly of the solution matrix
5. Solution of the linear algebraic system
6. Transfer of results back to Model after range checking of dependent variables.

PDE_Integrator are applied by passing them to the Model:

```
model.Apply( evolve_pressure );
```

or passing the model or a region thereof to them:

```
pressure_integrator.IntegrateOver( model.Region("fracture set 1") );
```

Inside the Model, the public methods of the PDE_Integrator class are called in the execution sequence shown in Figure 11. Only in time-dependent calculations, initial

conditions are assembled, usually into the right-hand vector. By default, PDE_Integrators objects compute a steady state.

For large problems or transient calculations it is a good practice to re-use PDE_Integrator objects to minimize memory allocation for global matrices. Note that if one re-uses a steady-state PDE_Integrator in a Newton-Raphson iteration, global matrices will be reset automatically.

5. Defining & Solving Finite-Element Equations

When the finite-element method (FEM, e.g., Huyakorn and Pinder, 1983; Istock 1989; Zienkiewicz and Taylor, 1994; Bathe, 1996) is used to solve a partial differential equation (PDE) that describes the distribution of a *field variable* (e.g., fluid pressure, solute concentration) across a domain of interest, this domain is discretized into a finite number of chunks, each of which is represented by a single *finite element*. All finite elements together form the *finite element mesh*. To see how a dependent variable varies in space and time as a function of material properties and independent / constitutive variables, one needs to integrate the PDE, obtaining the variable distribution. In the FEM, field variables are integrated element by element such that the incremental change across the mesh yields the variation across the model. As there is no exact solution to the PDE of interest, one integrates a piece-wise continuous approximation of which one hopes that it satisfies the PDE. For this approach to be successful, the approximating function must have a range of properties, which are discussed in detail in the CSMP++ Developer Guide. An FEM user needs to know that the functions must be differentiable to at least one order higher than the order of the derivative of the solution which shall be exact. This requirement reflects a side effect of the finite-element discretization: dependent on the order of the approximating functions, discontinuities will exist across element boundaries of the function derivative. In the presence of material property variations in the computational domain one cannot find the derivative of a solution at a node (the derivatives computed for each of the elements which share the node will differ from one another). Implications of this for calculating fluxes, stresses and strains are discussed in the Developer Guide and in the description of the Finite Volume library. In CSMP's hybrid FE-FV discretization, the FVM represents a complementary numeric method, synergizing with the FEM.

This section explains the basics of the Galerkin finite-element method for a linear triangular element implemented in the `LinearTriangle` class in CSMP++, comprising matrices and vectors. The finite-element forms of second order partial derivatives are discussed approximating the physics of fluid flow through porous media and stress and strain in structural mechanics, respectively, in the following example.

A relatively small set of PDE operators representing volume, surface and time integrals over the change in the dependent variable, suffices to compose finite-element simulations of fluid flow, heat flow, and linear elasticity. The finite-element forms of such common PDE operators have a standard notation in the finite-element literature that is reviewed below. Subclasses of the `MathOperatorLHS` and `MathOperatorRHS` constitute the PDE operator library to be used together with the `PDE_Integrator` and derived classes.

Notation	PDE Operator	Application
basic operators		
A		area of 2D element, volume in 3D
M		material property matrix
D		elasticity matrix

J		Jacobian matrix
N		interpolation functions (i.f.)
DN, dN		i.f. derivative matrix (N_x, N_y)
B		i.f. deriv.m. 2dof per node (dim x n)
r		scalar property value
composite operators		
[K]	$A B^T D B$	stiffness matrix
[C]	$A D N^T K D N$	conductance matrix
[S]	$A/3 N r N$	lumped capacitance matrix
{Q}		source-sink terms
[V]		advection-dispersion matrix
{u}		flux, displacement

Table 2: Notation used in the description of finite-element method.

In the following, these operators are explained for a simple two-dimensional steady-state fluid pressure computation that was already discussed in Chapter 2. N_i are the element interpolation functions which are used to model variations of the dependent variable, ϕ , inside of each element, such that

$$f(x, y) = \sum_{i=1}^n N_i(x, y) \phi_i$$

$f(x, y)$ is the approximated value of at the point x, y inside of the element, calculated as the sum of the products of ϕ values at the nodes with the corresponding interpolation functions for these nodes. Index n refers to the number of nodes, and basal indices denote node numbers local to the element. Since the discussion focuses on individual finite-elements, the (e) superscript is omitted. It usually distinguishes element- from global matrices.

A , the area of a two-dimensional element is equivalent to the space integral over the interpolation functions:

$$A = \iint N dx dy$$

DN is the matrix of the first spatial derivatives of the element interpolation functions and has the general form (in 2 dimensions):

$$[DN] = \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial x} & \dots & \frac{\partial N_n}{\partial x} \\ \frac{\partial N_1}{\partial y} & \dots & \frac{\partial N_n}{\partial y} \end{bmatrix}}_{2 \times n}, \quad [DN]^T = \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_1}{\partial y} \\ \vdots & \vdots \\ \frac{\partial N_n}{\partial x} & \frac{\partial N_n}{\partial y} \end{bmatrix}}_{n \times 2}$$

MTRL is a symmetrical material property matrix with a size of space dimensions squared. It can be used to represent scalar, vector, or tensor properties. For example, the anisotropic hydraulic conductivity, K , in a two-dimensional element is expressed as:

$$[MTRL] = \begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix}$$

$\mathbf{J}$ is the Jacobian matrix for the translation of the local coordinate system $(\varepsilon, \eta, \zeta)$ into global coordinates (x, y, z) . Inside of the source code of CSMP++, local coordinates are represented by the variables r , s , and t because the Greek letters do not form part of the ASCII character set that is available for C++ programming.

$$[J] = \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial \varepsilon} & \dots & \frac{\partial N_n}{\partial \varepsilon} \\ \frac{\partial N_1}{\partial \eta} & \dots & \frac{\partial N_n}{\partial \eta} \\ \frac{\partial N_1}{\partial \zeta} & \dots & \frac{\partial N_n}{\partial \zeta} \end{bmatrix}}_{3 \times n} \underbrace{\begin{bmatrix} x_1 & y_1 & z_1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix}}_{n \times 3}$$

Matrix contributions calculated for a straight-sided reference element are translated to the global coordinate system using the inverse of the Jacobian, the transposed inverse of $\mathbf{J}$, and $\mathbf{J}$'s determinant. In CSMP++, this procedure is applied in all higher-order elements. This is further discussed in the Developer Guide.

$\mathbf{N}$ is a 1 by n matrix (vector) with element interpolation function values at point x , y :

$$[N] = [N_1 \quad \dots \quad N_n]$$

Test function products of the form $N^T N$ are required when properties are integrated over an element.

EXAMPLE: STEADY-STATE FLUID PRESSURE

The variation of fluid pressure in a model that was computed by the program described in Chapter 2, can be regarded as the sum of local fluid pressure variations in the x, y, z directions and their perturbations by local fluid sources and sinks:

$$\frac{\partial}{\partial x} \left(K_x \frac{\partial p}{\partial x} \right) + \frac{\partial}{\partial y} \left(K_y \frac{\partial p}{\partial y} \right) + \frac{\partial}{\partial z} \left(K_z \frac{\partial p}{\partial z} \right) = Q$$

If one works with hydraulic conductivities, K , treated as constant over the finite elements (ignoring fluid viscosity variations), the steady-state pressure equation simplifies to the *elliptic* PDE

$$K^{(e)}_x \frac{\partial^2 p}{\partial x^2} + K^{(e)}_y \frac{\partial^2 p}{\partial y^2} + K^{(e)}_z \frac{\partial^2 p}{\partial z^2} = -Q^{(e)}$$

also known as Laplace equation and expressed more concisely using the *nabla* operator

$$-K \nabla^2 p = Q$$

A global matrix expression is assembled as the sum of the element contributions:

$$[K]\{p\} = \{Q\}$$

where K is a global matrix of the combined element conductances, p , is the solution vector which will hold the nodal values of the dependent variable “fluid pressure”, and Q is an elemental fluid source or sink. The exact form of the conductance matrix is dependent on the dimensionality of the model and the type of finite element, which is used in the computation. When global element coordinates are used, the operator **Integral_dNT_op_dN_dV**, i.e., the two-dimensional conductance matrix has the form

$$\begin{aligned} \underbrace{K^{(e)}}_{n \times n} &= \iint_A \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_1}{\partial y} \\ \vdots & \vdots \\ \frac{\partial N_n}{\partial x} & \frac{\partial N_n}{\partial y} \end{bmatrix}}_{n \times 2} \underbrace{\begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix}}_{2 \times 2} \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial x} & \dots & \frac{\partial N_n}{\partial x} \\ \frac{\partial N_1}{\partial y} & \dots & \frac{\partial N_n}{\partial y} \end{bmatrix}}_{2 \times n} dx dy \\ &= \frac{1}{2A} \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_1}{\partial y} \\ \vdots & \vdots \\ \frac{\partial N_n}{\partial x} & \frac{\partial N_n}{\partial y} \end{bmatrix} \underbrace{\begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix}}_{2 \times 2} \frac{1}{2A} \begin{bmatrix} \frac{\partial N_1}{\partial x} & \dots & \frac{\partial N_n}{\partial x} \\ \frac{\partial N_1}{\partial y} & \dots & \frac{\partial N_n}{\partial y} \end{bmatrix} \iint_A dx dy \end{aligned}$$

This finite-element integral is solved by (associative but not commutative) matrix multiplication. This integral is just equal to the area, A , of the finite element. Thus,

$$K^{(e)} = \frac{1}{2A} \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_1}{\partial y} \\ \vdots & \vdots \\ \frac{\partial N_n}{\partial x} & \frac{\partial N_n}{\partial y} \end{bmatrix} \begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix} \begin{bmatrix} \frac{\partial N_1}{\partial x} & \dots & \frac{\partial N_n}{\partial x} \\ \frac{\partial N_1}{\partial y} & \dots & \frac{\partial N_n}{\partial y} \end{bmatrix} A$$

For straight-sided triangular elements there is a simple formula for the integration of element matrices taking into account the order of the element interpolation functions:

$$\int_{A^{(e)}} (N_1)^a (N_2)^b (N_3)^c dA = \frac{a!b!c!}{(a+b+c+2)!} 2A$$

a, b, c are exponents of the interpolation functions, and the factor 2 captures the spatial dimension of the finite element.

If local coordinates are used, which allows to represent curved element boundaries of higher-order elements in physical (global) space, these are specified as constants for a reference element and transformed into global coordinates by multiplication with the inverse of the Jacobian matrix and its determinant.

Using this Jacobian transformation, the conductance matrix for a two-dimensional element with local coordinates becomes:

$$\underbrace{K^{(e)}}_{n \times n} = \int_{-1}^1 \int_{-1}^1 \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial \varepsilon} & \frac{\partial N_1}{\partial \eta} \\ \vdots & \vdots \\ \frac{\partial N_n}{\partial \varepsilon} & \frac{\partial N_n}{\partial \eta} \end{bmatrix}}_{n \times 2} \underbrace{\begin{bmatrix} J^{-1} \end{bmatrix}^T}_{2 \times 2} \underbrace{\begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix}}_{2 \times 2} \underbrace{\begin{bmatrix} J^{-1} \end{bmatrix}}_{2 \times 2} \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial \varepsilon} & \dots & \frac{\partial N_n}{\partial \varepsilon} \\ \frac{\partial N_1}{\partial \eta} & \dots & \frac{\partial N_n}{\partial \eta} \end{bmatrix}}_{2 \times n} |J| d\varepsilon d\eta$$

This integral **NumIntegral_dNT_op_dN_dV** is solved by numerical integration:

$$\underbrace{K^{(e)}}_{n \times n} = \sum_{i=1}^{N_\varepsilon} \sum_{j=1}^{N_\eta} W_i(\varepsilon_i) W_j(\eta_j) \left\{ \begin{array}{l} \underbrace{\begin{bmatrix} \frac{\partial N_1(\varepsilon_i, \eta_j)}{\partial \varepsilon} & \frac{\partial N_1(\varepsilon_i, \eta_j)}{\partial \eta} \\ \vdots & \vdots \\ \frac{\partial N_n(\varepsilon_i, \eta_j)}{\partial \varepsilon} & \frac{\partial N_n(\varepsilon_i, \eta_j)}{\partial \eta} \end{bmatrix}}_{n \times 2} \underbrace{\begin{bmatrix} J(\varepsilon_i, \eta_j)^{-1} \end{bmatrix}^T}_{2 \times 2} \underbrace{\begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix}}_{2 \times 2} \underbrace{\begin{bmatrix} J(\varepsilon_i, \eta_j)^{-1} \end{bmatrix}}_{2 \times 2} \times \\ \underbrace{\begin{bmatrix} \frac{\partial N_1(\varepsilon_i, \eta_j)}{\partial \varepsilon} & \dots & \frac{\partial N_1(\varepsilon_i, \eta_j)}{\partial \eta} \\ \frac{\partial N_n(\varepsilon_i, \eta_j)}{\partial \varepsilon} & \dots & \frac{\partial N_n(\varepsilon_i, \eta_j)}{\partial \eta} \end{bmatrix}}_{2 \times n} |J(\varepsilon_i, \eta_j)| \end{array} \right\}$$

This calculation amounts to a summation of element matrices that arise at each integration point. However, at each integration point, the interpolation function derivatives and $\mathbf{J}$, $\mathbf{J}^{-1}$ and $\det \mathbf{J}$ must be newly evaluated (although for the simplex elements line, triangle, and tetrahedron the Jacobian matrix is constant). $\mathbf{W}_{ij}$ represents weighting function values for each of the n integration points inside each element. They ascertain that the n contributions add up to the length, area or volume of the element in physical space when a field variable with a value of one is integrated. The location of integration points inside the reference element is chosen according to theoretical considerations, normally Gaussian quadrature rules.

In summary, interpolation-function partial derivatives at each integration point and with respect to the local coordinate system, are multiplied with the inverse of the Jacobian matrix at this integration point, and then summed, performing the numerical integration. All this happens behind the scenes, when isoparametric finite elements are chosen for a CSMP++ model (this is the default for poly-element type meshes).

Each of the integrals defined above has a corresponding PDE operator that is derived either from `MathOperatorLHS` or `MathOperatorRHS`, see above. To compose the partial differential equation using an `PDE_Integrator` object, references to these math

operators are passed to the PDE_Integrator before it is applied on the Model object, see the code in the examples:

```
// 2. Steady-state fluid pressure computation [K]{p} = {Q}
// -----
PDE_Integrator<2> steady_state_pressure;
steady_state_pressure.SetSolver(new SAMG_Solver());

Integral_dNT_op_dN_dV<2>
conductance( p_ref, "conductivity", "fluid pressure", "fluid pressure" );
Integral_NT_op_N_dV<2>
source( p_ref, "fluid volume source", "fluid pressure" );
VelocityAndVolumeFlux<2>
postpro( model, "conductivity", "porosity", "fluid pressure", true );

steady_state_pressure.Add( &conductance );
steady_state_pressure.Add( &source );
steady_state_pressure.AddPostProcess( &postpro );

model.Apply( steady_state_pressure );
```

Note that this code and the math operator objects are identical for two and three dimensional models and different types finite-element types. The MathOperatorLHS subclass `Integral_dNT_op_dN_dV` represents the exact volume integral over the derivatives of the element interpolation functions modeling the fluid-pressure distribution described by the PDE based on Darcy's Law: The MathOperatorRHS subclass `Integral_NT_op_N_dV` represents a fluid source term per specific volume that needs to be integrated over each element. It enters the right-hand side of the matrix equation.

When an `PDE_Integrator` is passed to a `Model` or `Region` object it accumulates the contributions from the `Elements` it loops over, calling the math-operator subclasses sequentially (Fig. 16, see also Fig. 11). The MathOperator methods

```
void AddAccumulate();
void MultiplyAccumulate();
void AddAccumulateLater();
void MultiplyWithTimeIncrement( bool multiply );
```

allow the user to specify the type of accumulation and when this is done within the global matrix and vector assembly process. Each PDE operator executes its methods

```
virtual void GetOperands();
virtual void ComputeContribution();
AssignToGlobal();
```

on each element that participates in the accumulation. The information that is specific to the type of finite-element that the element represents, is provided by the corresponding subclass of the `FiniteElement`. This FEM functionality is made available through the interface of the `Element` class through Policy-Based Class Design (cf., Alexandrescu). For this purpose, the `Element` class inherits the `FiniteElementPolicy` class, which contains a pointer to the `FiniteElement`. This is essentially the Bridge design pattern (Fig. 17).

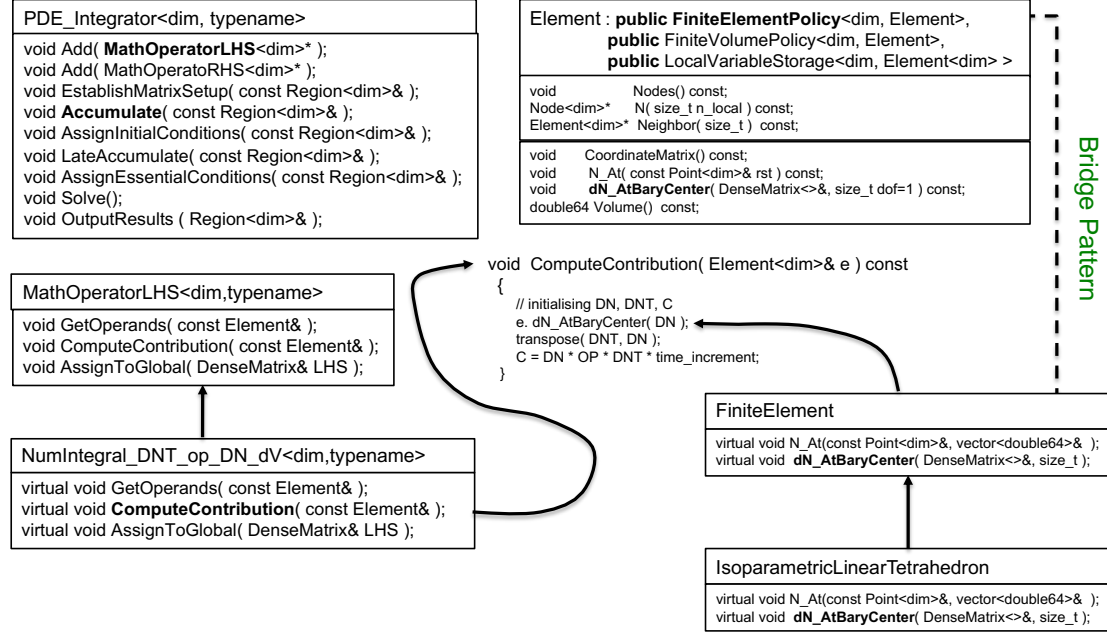


Fig. 16: Policy-based class design of the Element class with Bridge design pattern connecting it to the FiniteElement which is polymorphic, allowing its behaviour to vary with finite element type. Although not shown here, for simplicity, the Element also contains the other policies: FiniteVolumePolicy and LocalVariableStorage policy.

TRANSIENT FLUID PRESSURE

The transient fluid pressure distribution in a fluid-saturated domain is described by:

$$\frac{\partial}{\partial x} \left(K_x \frac{\partial p}{\partial x} \right) + \frac{\partial}{\partial y} \left(K_y \frac{\partial p}{\partial y} \right) + \frac{\partial}{\partial z} \left(K_z \frac{\partial p}{\partial z} \right) - \frac{\partial Q}{\partial t} = S \frac{\partial p}{\partial t}$$

This equation is integrated incrementally breaking total time up into steps. Time-stepping is usually implemented by a fully-implicit (Backward Euler) finite-difference scheme:

$$([S] + \Delta t[K])\{p\}_{t+1} = [S]\{p\}_t + \Delta t\{Q\}_{t+1}$$

As already discussed above, S and K are the storage capacity and conductance matrices, and p and Q are the fluid pressure (dependent variable) and fluid production/consumption vectors, respectively. The fluid-pressure distribution at the next timestep $t+1$ is computed from the present fluid-pressure distribution at t , and potential pressure or flux changes at Dirichlet or Neumann nodes, respectively. Fluid sinks or source rates are also taken into account.

The storage capacity matrix represented by Integral_NT_op_N_dV and Integral_NT_rhsop_N_dV has the form:

$$\underbrace{S^{(e)}}_{n \times n} = \iint_{x \ y} \begin{bmatrix} N_1 \\ \vdots \\ N_n \end{bmatrix} S^{(e)} [N_1 \ \cdots \ N_n] dA$$

If this integral is solved by **numerical integration**, the storage capacity matrix is computed as:

$$\underbrace{S^{(e)}}_{n \times n} = \sum_{i=1}^{N_\varepsilon} \sum_{j=1}^{N_\eta} W_i(\varepsilon_i) W_j(\eta_j) \left\{ \underbrace{\begin{bmatrix} N_1(\varepsilon_i, \eta_j) \\ \vdots \\ N_n(\varepsilon_i, \eta_j) \end{bmatrix}}_{n \times 1} S^{(e)} \underbrace{\begin{bmatrix} N_1(\varepsilon_i, \eta_j) & \cdots & N_n(\varepsilon_i, \eta_j) \end{bmatrix}}_{1 \times n} J(\varepsilon_i, \eta_j) \right\}$$

BUOYANCY-DRIVEN FLOW

The buoyancy force F_b which acts on a volume of fluid, V with a density ρ , diverging from a reference density ρ_0 , is

$$F_b = V(\rho - \rho_0)g$$

$$\Delta p^{(e)} = A \int_{z_1}^{z_2} \frac{\partial \rho}{\partial z} dz$$

where g is the acceleration of gravity (9.81 m s^{-2}). The corresponding vertical “fluid pressure” contribution of the element is equivalent to the integral over the changing fluid density. The buoyancy-related volumetric flow in the element, q , due to this gradient is calculated from Darcy’s law:

$$q^{(e)} = \frac{(\rho / \rho_0) k \Delta P}{\mu}$$

Since the hydraulic conductivity refers to a volume flux not a mass flux the latter is reduced as a greater flow volume passes through the rock when the fluid density is less than the reference density. The effect of buoyancy forces on the flow, requires the accumulation of an `Integral_NT_op_dNi_dV` operator to represent the vertical pressure gradient term in the equation. This term is non-zero only where fluid density deviates from the reference density and is accumulated into the right-hand side of the matrix equation.

$$\begin{aligned} G_{lumped}^{(e)} &= \int_A \begin{bmatrix} \frac{\partial N_1}{\partial z} \\ \vdots \\ \frac{\partial N_n}{\partial z} \end{bmatrix} K_y (\rho - \rho_0) g [N_1 \quad \cdots \quad N_n] dA \\ &= \frac{Kg}{3} \begin{Bmatrix} \rho_1 - \rho_0 \\ \vdots \\ \rho_n - \rho_0 \end{Bmatrix} A \end{aligned}$$

We recommend against the accumulation of the full density term because this would skew the distribution of the residual across the model.

LUMPING

The diagonalization of element matrices is called “lumping”. In this process, the “off-diagonal” elements of a matrix are set to zero by multiplying these terms into the diagonal. This is equivalent to defining the interpolation functions differently such that

$$N_i N_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

where n is the number of nodes in the element. In a “consistent” matrix for a two-dimensional triangular element (left matrix below), the area factor A , is divided by 12. This is required to obtain an exact integral over the interpolation functions according to the integration formula presented earlier.

$$\frac{A}{12} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \rightarrow \frac{A}{3} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

In the “lumped” matrix on the right, lumping has been achieved by multiplying the identity matrix with a third of the area. This denominator is equivalent to the number of nodes = interpolation functions used. The determinant remains the same ($4A/12$ vs. $A/3$). Lumping may also be achieved by finding the Eigenvalues of a matrix. In many cases, this is the most physically meaningful lumping procedure. Relative to the consistent formulation, lumping can introduce errors since off-diagonal matrix terms are effectively ignored. Lumping must therefore be used with caution, especially for linear elasticity problems where off-diagonal terms describe shear stresses etc.

LINEAR ELASTICITY

The elastic deformation of a discretized rock volume in response to (1) boundary displacements, (2) point forces applied to the nodes, (3) stresses applied at the boundaries, (4) body forces, and (5) volume changes (thermal expansion, contraction, or compaction), can be simulated with a specially configured PDE_Integrator (example12.cpp). Also quadratic finite elements are used, this computation is only first-order accurate. This means that while the displacements in the model are computed fairly realistically (however only if variations in the material properties are small), the strains and stresses computed in post-processing operations, are somewhat noisy. Example 12 solves the matrix equation:

$$[K]\{u\} = \{F\}$$

where K is the global stiffness matrix, u is the nodal displacement vector and F represents boundary conditions in the form of displacements, stresses, body forces, and volumetric source or sink terms. Element contributions to the stiffness matrix have the form:

$$[t A [B]^T [D][B]]\{u\} = \{F\}$$

t , is a model thickness multiplier, A is the area of the element, and B is the interpolation-function derivative matrix (since a Galerkin method is used, the test functions are also used as weighting functions). D is the material property matrix for a plane stress computation (stresses act only in the model plane):

$$[D] = \left(\frac{E}{1 + \mu} \right) \begin{bmatrix} 1 & \mu & 0 \\ \mu & 1 & 0 \\ 0 & 0 & \frac{1 - \mu}{2} \end{bmatrix}$$

In D , E refers to *Young's modulus* and μ to *Poisson's ratio*. After the displacements have been computed, the strain that is constant in each element is calculated as:

$$\{\varepsilon\} = \{B\}\{U\}$$

From the strain, the stress is calculated as:

$$\{\sigma\} = [D]\{\varepsilon\}$$

$$\begin{bmatrix} \sigma_{11} \\ \sigma_{22} \\ \sigma_{33} \\ \sigma_{23} \\ \sigma_{13} \\ \sigma_{12} \end{bmatrix} = \frac{E}{(1 + \nu)(1 - 2\nu)} \begin{bmatrix} 1 - \nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1 - \nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1 - \nu & 0 & 0 & 0 \\ 0 & 0 & 0 & (1 - 2\nu)/2 & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 - 2\nu)/2 & 0 \\ 0 & 0 & 0 & 0 & 0 & (1 - 2\nu)/2 \end{bmatrix} \begin{bmatrix} \varepsilon_{11} \\ \varepsilon_{22} \\ \varepsilon_{33} \\ 2\varepsilon_{23} \\ 2\varepsilon_{13} \\ 2\varepsilon_{12} \end{bmatrix}$$

The right-hand side vector $\{F\}$ of the global matrix equation can include either one or a combination of fixed displacements, body forces, stresses, or volume strains as essential conditions.

Volume strain can be incorporated into $\{F\}$, by defining fictitious point forces at the nodes of elements undergoing the volume change. The resulting element contribution to $\{F\}$ is:

$$\{F^{(e)}\} = tA[B]^T[D]\{\varepsilon_0\}$$

The actual volume change vector ε_0 , has no shear components, and its form, in the case of thermal expansion is:

$$\{\varepsilon_0\} = \alpha\Delta T \begin{Bmatrix} 1 \\ 1 \\ 0 \end{Bmatrix}$$

where α is the thermal expansivity and T is the temperature. The resulting vector has six components since each node has two degrees of freedom, determining its displacement in two dimensions. The directional components of **point forces** are assigned individually to desired nodes of the finite element. In the equation below, f_x and f_y are point forces which act on node 1 of the element in horizontal and in vertical direction, respectively.

$$\{F\} = \{F\} + \begin{Bmatrix} f_x \\ f_y \\ 0 \\ 0 \\ 0 \\ 0 \end{Bmatrix}$$

Fixed displacements are added to the matrix equation after all other right-hand terms have been applied to $\{F\}$. They are inserted as Dirichlet boundary conditions without reducing the size of the global solution matrix. This procedure is documented in the CSMP++3D Reference guide for the method `AssignEssentialConditions()` of the `CSMP++_PDE_Integrator`.

Stresses. Stresses acting on element faces at model boundaries are applied as distributed nodal forces acting on the boundary nodes of the element. This requires knowledge of the length of each boundary segment (area of face), L , and the x , y components of stress (force per unit area). Since the stress is distributed on 2 nodes, a and b , the area-thickness product (tL) is divided by 2.

$$\{F\} = \{F\} + \frac{tL}{2} \begin{Bmatrix} \sigma_x \\ \sigma_y \\ \sigma_x \\ \sigma_y \\ 0 \\ 0 \end{Bmatrix}$$

Body forces are accumulated into $\{F\}$, by distributing the total force among the nodes of an element:

$$\{F\} = \{F\} + (tA/3) \begin{Bmatrix} x1 \\ y1 \\ x2 \\ y2 \\ x3 \\ y3 \end{Bmatrix}$$

where x and y are the directional components of the body force per unit volume and the indices denote nodes 1 to 3 of the triangular element (again t refers to the element thickness and A to its area).

REFERENCES CITED

- Bathe, K.J., 1996, Finite Element Procedures, Prentice Hall, 1037 p.
- Istock, J., 1989, Groundwater Modeling by the Finite Element Method, AGU Water Resources Monograph, Vol. 13, 495 p.
- Huyakorn, P.S. and Pinder, G.F., 1983, Computational Methods on Subsurface Flow, Academic Press, 473 p.
- Liu, Y. and Vinokur, M, 1997, Exact Integrations of Polynomials and Symmetric Quadrature Formulas over Arbitrary Polyhedral Grids, NASA Technical Memorandum 112202
- Schwarz, H.R., 1991, Methoden der Finiten Elemente, Teubner Studien Bücher Mathematik, 435 p.
- Weatherburn, C.E., 1927, Differential Geometry of Three Dimensions. Cambridge University Press
- Zienkiewicz, O.C. and Taylor, R.L., 1994, The Finite Element Method Volume 1, McGraw-Hill, 648 p.

5. Initial and Essential Conditions

The boundary of a Model or Region subregion thereof, is defined by those finite-elements which share at least one face (edge in 2D) with this boundary. To such elements, their faces, or nodes a range of essential conditions can be applied.

The simplest way of constraining a model is to fix values of dependent variables at boundary nodes or some nodes inside. These “first order” constraints which must be satisfied by the solution are called “*Dirichlet*” conditions and are incorporated into the resulting system of linear algebraic equations by simple substitution, to the effect that the number of degrees of freedom is reduced. Since nodal values can be fixed inside of a model one speaks of “*essential*” rather than boundary conditions. Dirichlet conditions can be assigned with a variety of Model methods (for details see HTML documentation of the Model class):

```
void AssignBoundaryFlags( SG_BOUNDARY, const char*, VARIABLE_FLAG );
void AssignBoundaryValues( SG_BOUNDARY, const char*, VARIABLE_FLAG,
                           double, double );

void AssignBoundaryValues( SG_BOUNDARY, const char*,
                           const VectorVariable&,
                           const VectorVariable& );

void AssignConstraint( const char*, long, const ScalarVariable& );
void AssignConstraint( const char*, long, const VectorVariable& );
void AssignConstraint( const char*, long, const TensorVariable& );

void AddBoundaryNormalFluxes( SG_BOUNDARY, const char*,
                              double, double ); // influxes are positive

bool AssignBoundaryValues( SG_BOUNDARY, const char*, VARIABLE_FLAG,
                           double, double, double, double );

bool AssignBoundaryValues( SG_BOUNDARY, const char*,
                           const VectorVariable&, const VectorVariable&,
                           const VectorVariable&, const VectorVariable& );

void ChangePropertyStatusInGroupToWhere( const char*, const char*,
                                          VARIABLE_FLAG, VARIABLE_FLAG );
void ChangePropertyStatusInGroupToWhere( const char*, const char*,
                                          VARIABLE_FLAG, double, double );
```

One must set a *Dirichlet* value at least for one Node, otherwise there will be an infinite number of correct solutions to the integration problem and the solver will produce an arbitrary result. The Dirichlet value has the same role as the integration constant, C , picked up in the integration of a standard indefinite integral of a function,

$$\int f'(x) dx = f(x) + C$$

This is the reason for why such conditions are essential.

To assign constraints like a fixed flow rates across interfaces or fixed chemical source rates etc. a special approach is needed, since these involve first or higher-order spatial derivatives of the dependent variable. First, one projects the gradients onto the normal of the boundary and ends up with a scalar quantity. A little digression is warranted here

in order to explain the meaning of this type of condition in an integral formulation:

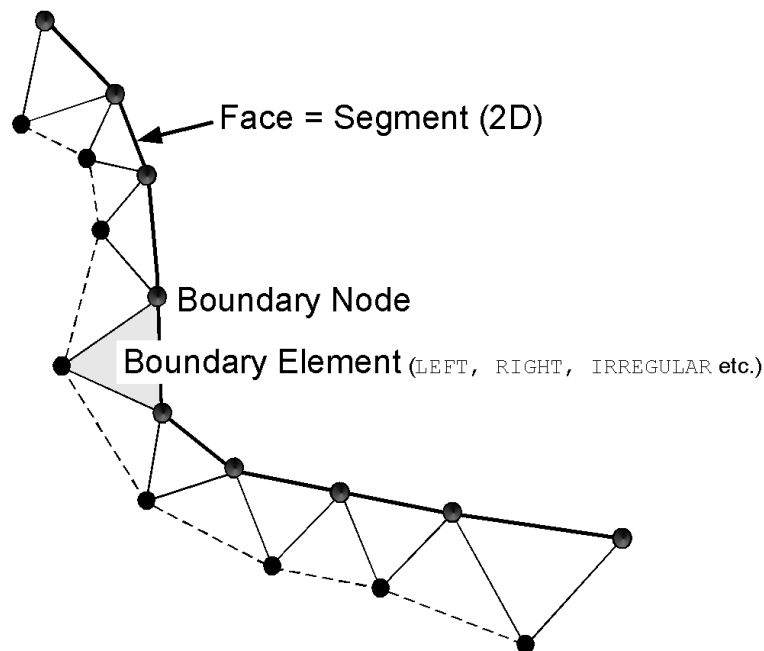


Fig. 15: Boundary flagging in a CSMP++ model. Only the elements which share a face with the model boundary are considered as boundary elements.

Think for a moment about the difference between *specific properties* such as the density of inhabitants per km^2 , and *overall properties*, such as the total number of inhabitants of a village. A finite element represents a certain volume of rock with *specific* properties defined per unit volume such as the heat capacity or a fluid volume source term ($\text{m}^3 \text{ m}^{-3} \text{ s}^{-1}$). Specific quantities are integrated over the element volume by the FEM. One calculates the specific value of quartz which is precipitated per unit volume of rock due to gradient reactions (assuming a quartz-saturated fluid) by multiplying the fluid mass flux with time and the solubility-gradient in the direction of flow. However, if one is interested in the total amount of quartz precipitated within an element, this quantity must be multiplied with the element's volume. This amounts to an integration. Similarly, essential conditions can be applied to CSMP++ models by specifying specific quantities or total values. Taking the specific approach has the advantage that one does not have to perform the integrations *a priori*.

Neumann conditions. The area of the face of an element (Fig. 15) comes into play when essential conditions are distributed as sources or sinks to node or element variables. To apply such NEUMANN boundary conditions (also called *non-essential* conditions because they are not critical), one has three alternatives:

- (1) One does nothing. This implies that the derivative of the dependent variable across the boundary has to be zero. This Neumann condition is called *natural* boundary condition and is equivalent to a *no-flow* condition in a flow model (fluid will flow down a pressure gradient, i.e. parallel and not perpendicular to this kind of a boundary).
- (2) One can distribute the specific face-normal quantities as point sources to the boundary nodes of the boundary element via a **PointSource_rhsop** PDE operator. However, this necessitates prior integration of these quantities and, because the

boundary face area is rarely unity is a fairly intricate calculation. For example, for a boundary flux, q , per unit area, the nodal contribution to a linear triangular element and the righthand side of the global equation has the form

$$\{F\} = \{F\} + \frac{tL}{N_b} \begin{Bmatrix} q \\ q \\ 0 \end{Bmatrix}$$

where t is the element thickness, L is the boundary segment length (facial area), A is the element area (volume in 3D), and N_b is the number of nodes at the boundary. This type of boundary conditions is also assigned with `AssignBoundaryValues()` and requires a separate variable to hold the prescribed boundary in- or outfluxes specified as boundary-normal components. These are added in the global matrix accumulation procedure using the differential operators as follows:

```
// 2D triangular finite elements only
model.AssignBoundaryValues( BOTTOM, "influx", NEUMANN, 1.0, 1.0 );
model.AddBoundaryNormalFluxes( BOTTOM, "heat flow", 1.0, 1.0e-12 );
```

The caveat of this method is that it only works for triangular finite elements in 2D.

(3) Alternatively, the boundary normal flux can be assigned as an *face source term* at the boundary of the mesh. This term is integrated with a suitable PDE operator, such as **Integral_NT_op_N_dS** which performs a surface integration of face variables where they have been flagged `NEUMANN`.

If the model uses a combination of finite elements and finite volumes, the most accurate way of assigning boundary fluxes is via the finite volume technology:

```
FiniteVolumeTransport* advector(0);

advactor = new FiniteVolumeTransport( /* args */ );

model.AssignBoundaryValues( LEFT, "nodal source", 1., 1., 1., 1. );
advactor->AssignScalarBoundaryValues( model, LEFT, "nodal source" );

...

// when the fluid pressure algorithm is defined include the following
PointSource_rhsop<CSMP++_float,3>
nodal_source( p_ref, "nodal fluid source", "fluid pressure" );

fluid_pressure_algorithm.Add( &nodal_source );
```

Initial conditions such as a fluid pressure distribution or initial stresses at the onset of a transient computation have to be specified by the user. CSMP++ initializes any physical variable specified in the variables “*.txt” file with a default value of NAN (not a number) and a `VARIABLE_FLAG` flag of `PLAIN`. With these settings, one cannot compute anything, because the NaN value will be the result, highlighting any not initialized variables as was the design intention.

To initialize physical variables or material properties one must set them in the configuration file or write a statements like

```
model.InputPropertyValue( "fluid pressure", makeScalar(ANY,0.) );

VectorVariable<1> vc(PLAIN,2.);
model.InputPropertyValue( "velocity", vc );
```

The condition flags of initial values, can be changed in Groups or the entire Model, using the methods:

```
VARIABLE_FLAG global_to(PLAIN), group_flag(BOUNDARY);

model.Region("region").ChangePropertyStatus(
    "fluid pressure", global_to, group_flag );

model.ChangeConditionFromTo("fluid pressure", DIRICH, PLAIN );
```

This example will remove Dirichlet boundary conditions assigned to the distributed variable “*fluid pressure*”.

Box-shaped or rectangular models. CSMP++ allows to assign essential conditions anywhere within a Model model. However, this process is greatly simplified for rectangular or box-shaped models essential conditions using the overloaded method `AssignBoundaryValues()`.

```
model.InputBoundaryValue( LEFT, "fluid pressure", makeScalar(DIRICH, 1e7 );
```

In this example, scalar Dirichlet boundary conditions are assigned to the variable “fluid pressure”.

Other interfaces for box-shaped geometries exist in Model:

```
/// use vector of variable flags to change the status of the target variable at the given BOX_BOUNDARY
void InputBoundaryFlags( BOX_BOUNDARY, const char*, const vector<VARIABLE_FLAG>& );

/// 4 values (2 in 2D) are linearly interpolated across the square (line) boundary of the box-shaped model
void InterpolateBoundaryValues( BOX_BOUNDARY, const char*, const
vector<VectorVariable<dim> >& );

/// 4 values (2 in 2D) are linearly interpolated across the square (line) boundary of the box-shaped model
void InterpolateBoundaryValues( BOX_BOUNDARY, const char*, const
vector<ScalarVariable>& );
```

In two-dimensional models, 2 input values specify a linear property gradient along the model boundaries, in 3D, four values permit the prescription of bilinear variations to the side boundaries of a box-shaped model. The floating-point values supplied as method arguments specify the variable values that shall be fixed.

Thus, values of the Node variable “fluid pressure” may be varied linearly from 10^7 to 0 between the first and the last Node situated on the left model boundary.

A few important conventions apply with regard to boundary labeling and condition assignment for box-shaped models (see Box.h):

1. A righthand-rule coordinate system is used always (note that this is the same as in most engineering software tools, but different from geomodelling (UTM)

coordinates where the Y axis points downward such that these are lefthanded). By contrast, in SMP models, the TOP boundary is the one with the highest y-coordinate values since Y points upward reflecting altitude.

2. In 3D, X points to the right (east), Y to the top (towards the sky), and Z to the front (south), defining a right-hand coordinate system.
3. In 3D, all faces of the model are numbered counterclockwise, looking down the Z-axis from the front into the model.
4. These faces are labeled BOTTOM, LEFT, FRONT, RIGHT, BACK and TOP.
5. Face BOTTOM has the corners CNR1, CNR5, CNR6, CNR2.
6. Face LEFT has the corners CNR1, CNR4, CNR8, CNR5.
7. Face FRONT has the corners CNR5, CNR6, CNR7, CNR8.
8. Face RIGHT has the corners CNR2, CNR6, CNR7, CNR3.
9. Face BACK has the corners CNR1, CNR2, CNR3, CNR4.
10. Face TOP has the corners CNR3, CNR7, CNR8, CNR4.
11. Edges 1-12, 3D models only (see Fig. 5).

Corner locations and boundary gradients are only meaningful if the model is rectangular.

If the side boundaries of a model are irregular or are tilted more than 45° relative to the coordinate axes, CSMP++ will wrongly identify boundaries if one attempts to treat such models as box-shaped.

Irregular shaped boundaries are best identified and flagged in the input programs with which the (geo)models are generated. This way the user has more flexibility and control, creating named boundary entities supported by the graphics of the program in which the geometry is build. Essential conditions are then assigned with (in this example 'land surface' was defined as an entity in the CAD program, retained in the mesh and then output to CSMP):

```
const ScalarVariable pf_b(DIRICH,1.0e6);
model.Boundary("land surface 1").InputPropertyValue( "pressure", pfb );
```

The following code snippet, illustrates how one can quickly examine the boundaries that are contained in a model:

```
for ( map<string, csmp::Boundary<3U> >::iterator
      git = model.BoundariesBegin(); git !=model.BoundariesEnd(); git++ )
{
    cout <<"\n\tBoundary: "<< (*git).first.first << (*git).first.second;
    cout.flush();
}
```

6. Input to & Output from CSMP++

This chapter describes how geomodels, imported into CSMP, can be configured with ASCII configuration files and it introduces a very simple way to build rectangular two-dimensional models from bitmap images color-coded for permeability with arbitrary drawing programs. More sophisticated model building methods which require third-party software tools are documented in the *CSMP_InterfaceGuide*. It is a stand-alone manual and contains step-by-step instructions for model building with Rhinoceros (McNeel & Associates), and meshing with the TETRA & PRISM family of meshing tools from ICEM CFD Engineering (now part of ANSYS). There are also a number of brief HowTo files that have been created for this purpose in the doc/ directory. The *CSMP_InterfaceGuide* also describes how two-dimensional, adaptively refined models are constructed from line drawings using the 2D meshing tool *Triangle* and it introduces the now deprecated `GeometryPreprocessor` that can still be furnished upon request. A further section is dedicated to model building with *GoCAD* and *SKUA* (its commercial pendant). However, Rhinoceros and ANSYS TETRA are the key SOFTWARE tools currently relied on in CSMP++'s modeling workflow. They are recommended for state-of-the-art modeling.

CSMP++ now also has a GUI for model building and configuration. It is introduced at the end of this section.

We strongly recommend to test newly created CSMP++ simulation programs with simplified and coarsely meshed models first, before building and using memory-intensive elaborate models. This is especially true for transient simulations. There are several classes such as **MeshDiagnostics** designed to help the user to test whether a model is fit for purpose before a simulation is started.

We also strongly recommend that any model that was generated with a third-party application is converted into a CSMP native model and saved to binary file before it is loaded into a specific application. This way the potential creation of any missing mesh connectivity and consistency checking is carried out only once and the loading time is reduced considerably. The binary file format for CSMP models has been dramatically improved and variables can be written or restored from it selectively. Also, the binary file can now be transferred across different platforms and file systems including OSX, Windows and Linux.

CSMP binary files can also be built and read with the GUI allowing a further visual QC of the models.

MODEL CONFIGURATION FROM FILE

It is not practical to recompile programs everytime input variables have to be changed are the timing when output is written shall be changed. For this reason, CSMP++ simulations can be configured from file using the **CSMP++_InputDataManager** class:

```
CSMP++_InputDataManager<3>  model_configuration;
ComputationalSettings         run_settings;

const bool groups_from_perm(false);
const bool set_prop_defaults(true);
const bool set_prop_of_groups(true);
```

```

const bool set_essential_conds(true);
const bool set_group_flags(false);

model_configuration.ConfigureFromFile( model,
                                     file_name,
                                     groups_from_perm,
                                     et_prop_defaults,
                                     set_prop_of_groups,
                                     set_essential_conds,
                                     set_group_flags,
                                     run_settings );

```

Typically, simulation programme are configured with two text files which select the target regions of the mesh and initialize the model with material properties, initial, and essential conditions. The names of these files must include the name of the model and the extensions:

```

-regions.txt
-configuration.txt

```

The regions file is needed only if a three-dimensional mesh from ICEM's tetra mesher is used as an input. Mesh files generated by TETRA are recognized by their file extensions (*.asc) and (*.dat) and contain subregion information and mesh-connectivity data, respectively. Following the headline and a property specification line, the regions file lists a selection of model subregions which shall be used inside the simulation, e.g.:

```

'ICEMexample-regions.txt' created by SKM 2/3/01
no properties
FRACTURE1
FRACTURE2
CARBONATE_MATRIX
SANDSTONE_MATRIX

```

All other discretized regions in the (.asc) input file are ignored. The user has to make sure that the model is still contiguous and retains a box shape if respective methods of the CSMP++_DataInputManager shall be called. The “-regions.txt” file can also be used to assign initial property values to the selected regions (see documentation of interface library).

The “-configuration.txt” file has the following format (listing all possible data blocks). Comments are preceded with a pound (#) sign:

```

Reservoir simulation testcase (1): reservoir with fractures 30/5/2001 # file header

# 1. subregions of model identified from the permeability values assigned to
# individual finite elements by the meshing tool
# (NB: This datablock does NOT assign properties to elements but it will create
# Region objects with the names 'joints' and 'oil pool' which contain elements
# within the specified permeability ranges)
# This data block is not needed in ICEM models because model subregions are
# already identified by their family name in the CAD model.
# -----
joints          permeability  1.0e-13 1.0e-1
oil pool        permeability  1.0e-14 1.0e-14

# 2. default material properties and initial variable values
porosity                0.3
total system compressibility 1.0e-9
pore radius              1.0e-4
pore throat radius       0.25e-4
fluid volume source      0.0
capillary pressure oil    0.0

```

```

density oil                800.0
density water              1000.0
viscosity oil              1.0e-4
viscosity water            1.0e-4
saturation oil             0.0
saturation water           1.0
fluid pressure             1.0

# 3. properties which shall be assigned to model subregions =Groups
#   subregion properties can be assigned to the subregion boundary, interior,
#   or the complete subregion
# -----
joints      complete      porosity      1.0
joints      complete      pore radius   1.0e-3 # joint aperture
joints      complete      pore throat radius 1.0e-3
oil pool    complete      permeability 1.0e-13
oil pool    complete      saturation oil 0.9
oil pool    complete      saturation water 0.1 # well parameters

# 4. essential conditions which shall be assigned to model boundaries
# -----
# here as many values are given as there are corner points that shall be interpolated
# between (2 in 2D models with line boundaries and 4 in 3D models with rectangular
# boundary surfaces
TOP    Dirichlet      fluid pressure      1.0E+07 1.0E+07
LEFT   Dirichlet      fluid pressure      1.0E+06 1.0E+06
RIGHT  Dirichlet      fluid pressure      0.0E+00 0.0E+00
TOP    Dirichlet      saturation oil       0.0      0.0
TOP    Dirichlet      saturation water     1.0      1.0
BOTTOM Dirichlet      saturation oil       0.0      0.0
BOTTOM Dirichlet      saturation water     1.0      1.0
RIGHT  Dirichlet      saturation water     1.0      1.0
RIGHT  Dirichlet      saturation oil       0.0      0.0
LEFT   Dirichlet      saturation water     1.0      1.0
LEFT   Dirichlet      saturation oil       0.0      0.0

# 5. (optional) essential condition flags for model subregions
# -----
joints      complete      capillary pressure oil      Dirichlet
joints      interior      saturation oil                Dirichlet
joints      interior      saturation water              Dirichlet

```

Datablocks (including the file header) must be separated by an empty line. A *problem that users frequently have with the configuration files relates to the use of tab and blank characters*: Since, region and variable names are allowed to contain white space “ ”, columns must be separated by tab characters (this feature exists as an output option for Excel spreadsheets which can be used to generate the configuration files). Also, end of line characters must not be preceded by whitespace.

When the configuration file is read by the `DataInputManager` it checks variables against the `PropertyDatabase` and reports range violations or if a variable does not exist. If a range is exceeded, the variable value will be set to the nearest range bound specified in the `global_variables_text` file. If the variables are vectors or tensors, a corresponding number of values and flags need to be set. This number depends on the dimensionality of the model (for details see the documentation of the `CSMP++_DataInputManager` in the HTML reference guide).

REGULAR MESHES FROM PIXEL DATA

A fast and simple way to create rectangular two-dimensional models for CSMP++ is to use 8-bit-color-coded bitmap images to resolve geologic features in models. These representations are converted by a triangular finite-element-mesh generator (`Triangulator` class) into evenly refined meshes. Input geometries are drawn either

directly in “pixel paint” programs like *GIMP* or *Scion Image* or with standard drawing programs like *Freehand*, *Illustrator*, *Canvas* and so forth. In the drawings, structures and horizons are color-coded for permeability and regions of interest for the modeling are clipped. *Scion Image* (formerly NIH Image) permits the output of selected image regions as 256-color pixel dumps in text file format. These ASCII files can be read by CSMP++. On unix / linux systems other drawing programs are available in the public domain. Below, the regular-mesh-input generation procedure is described in detail for the freeware software tool *Scion Image* and tips are given for drawing and manipulating geometries.

The following steps are required for the production of a color-coded text input file:

1. Draw the Geologic cross section
2. Import the section into *Scion Image*
3. Color-code the section using a 256-value (8-bit) rainbow scheme
4. Output the section as an ASCII text file

Step 1 can be accomplished with any drawing program that supports bitmap output.

1. Drawing the Geologic Cross Section

As the first step towards the two-dimensional CSMP++ model, the geologic cross-section needs to be prepared in which each geological unit or structure has a unique color of the rainbow color scheme (In *Scion Image*: Options -> Color Tables -> Rainbow). The horizons must be filled with a unique color and must not be framed with black lines as these would be interpreted as separate low-permeability units (see below, and corresponding HowTo documents). Care has to be taken to avoid blank areas between the horizons. The most convenient way of drawing a horizon with polygon tools is to use the same line color as the fill color of the polygon. Smoothing of the lines is recommended. However one needs to check afterwards with the magnification tool whether the horizons still overlap.

The generated drawing can be labeled further as a useful documentation of the model (publishing, presentation etc.). Since one may want to modify the section in the future or even build an adaptively refined version of it using procedure (2) from above, it is advisable to keep a copy in the format of the original drawing program before proceeding with the next steps.

2. Importing the Section into an Image Editing Program

Most image editing programs can import bitmap, PICT- and TIFF files. If you did prepare the model in another drawing program, save a copy of your model file (with a new name) as TIFF file. Now open this file with the image editing program like NIH image or GIMP and choose a Rainbow color palette. Once the file appears on the screen, use the selection tool (stippled quadrangle in the upper right of the tools menu)

to extract the region of the model (geologic cross section), which you intend to input into your CSMP++ model (or use the whole section). If the section is too large to fit on the screen use "Scale to fit window" from the Options menu before using the selection tool. One can also scale drawings with NIH Image. Since the model is build from pixel data, the screen size of the geological cross-section controls model resolution. For test runs, the model should not be larger than 100 by 100 pixels. This will lead to a computation size of about 30-50 MB RAM in CSMP++ depending on how many physical variables the model has. A 1000 x 1000 image will translate into a 2000 x 2000 triangle mesh (4 millions elements), requiring about 1.5-2.3 GB RAM (Fig. 13).

3. Color Coding the Image

The constructor for regular-gridded CSMP++ models (`Model::Model(Triangulator& )`) prompts the user for the name of the input text file and expects the input of property values of choice in the form of color-coded pixels. Use the paint bucket and pen tools to change colors in the image according to the 256-rainbow color vs. property value scheme. With this scheme, permeability values are assigned ranging from 0.1 nano-Darcy (10^{-21} m^2) to 1.0 Darcy (10^{-12} m^2):

10^{-21}	255	BLACK
10^{-20}	0	WHITE
10^{-19}	236-254	RED
10^{-18}	216-236	ORANGE
10^{-17}	206-215	PEACH
10^{-16}	183-205	YELLOW
10^{-15}	126-182	GREEN
10^{-14}	75-125	AQUAMARINE
10^{-13}	45-74	BLUE
10^{-12}	1-44	AZURE

The desired integer color values (second column) are set by calibrating the pipette on the color bar in *Scion Image*. The color value chosen with the pipette tool appears in the "Info" window in the lower left hand corner underneath the color bar. Values with decimal places which may be output when *Scion Image* tracks color saturation (which is optional) are not permitted. One can vary values later in CSMP++ once a mesh has been created and Region objects can be build for each uniformly colored region.

Ideally the extracted region should now be complete for output as text-formatted CSMP++ input file. This is achieved using "File -> Export -> text" from the file menu. Before you output the file, test whether the color values are unique. The cross hair tool (+ symbol) will indicate the 256-color value when pointed at a horizon (Again, the tool should not indicate decimal color values, otherwise the text file cannot be interpreted by CSMP++). With the magnifying tool individual pixels of the drawing can be checked which normally are only poorly visible in the default view. This is necessary when converting pictures from drawing-program formats to TIFF since minor changes may have occurred in the image. Units must not have black outlines (color value 255) ! - If one finds any errors, the color picker (pipette tool) and pencil tools can be used to replace erroneous colors. In many cases, one can also changes whole areas with the paint-bucket tool (the color for this tool is also chosen with the pipette; choose it either from the image or from the palette= color bar). Like in *Photoshop* or *GIMP*, the paint

bucket will color a mono-chromatic region bounded by other colors. Such regions will be filled-in completely with the new color.

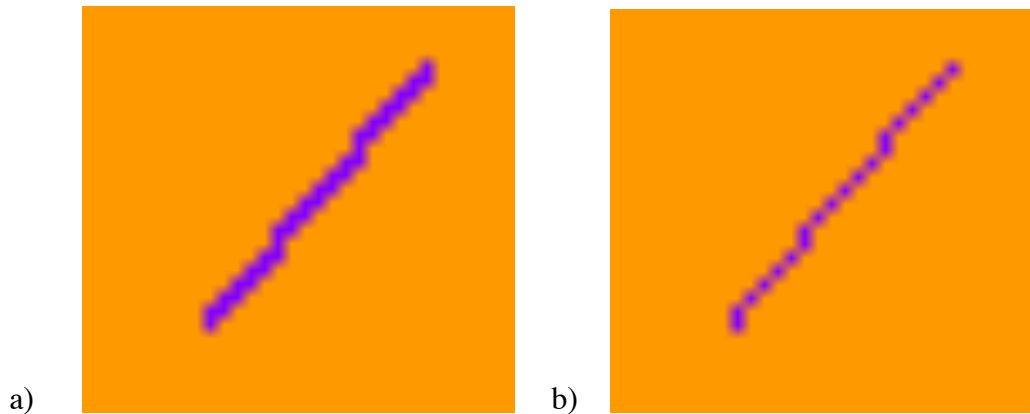


Figure 14: Correct (a) and incorrect (b) representation of a fracture in a 2D pixel input image for the Triangulator mesher inside of CSMP++.

One should give individual structures or units enough thickness to achieve contiguity in the finite-element models (Fig. 14). In a steady-state model, oblique fractures must be interconnected pixel chains to be represented as continuous high-permeability pathways. In models with fluid sources and sinks (e.g., wells), these units must be at least 3 elements across to get meaningful computation results for linear finite elements.

4. Output of the section as color-coded text file

If the image satisfies the listed guidelines it can be exported as pixel dump in text format: Choose "Export" from the File menu and then click the text box. Keep a TIFF version of the geometry because it may need to be modified later. We recommend against using selected region output ("Export Selection as...text") as input files for CSMP++ since region size almost always differs between manual selections or they are slightly offset relative to one-another. This can render work useless if one later wants to compare simulation results. Instead, one extracts only one selection from the original geometry file and make copies with alternate color coding. This assures that all output files have the same size and the same reference frame.

When the Model constructor reads the input file, it asks the user to specify the actual dimensions of the model. This is necessary because the pixel-data lack this information. In order to get optimal finite-element shapes, one should keep the aspect ratio, when specifying model dimensions.

A Model model is build from the ASCII text file in the following code segment:

```
Triangulator  linear_triangle_mesher;
Model<2>      model( linear_triangle_mesher );
```

This specific Model constructor will prompt the user to enter the name of the input file and to specify the actual dimensions of the model as this information is not conveyed by the image.

OUTPUT TO PARAVIEW, VISIT, OR MAYAVI VIA VTK INTERFACE

An effective way to visualize CSMP++ simulation results with the popular visualization tool PARAVIEW (paraview.org, see brief manual in doc/) is facilitated via the **VTK_Interface** **VTU_Interface** (xml) to the visualization toolkit (VTK) supported by kitware.com. VTK is a very powerful C++ library of visualization algorithms and tools for structured and unstructured data, albeit with somewhat limited interactivity. Normally one will use VTK with the Paraview GUI, but one can also write visualization scripts in Tcl, Python, Java or C++. In addition to the impressive sample suite, which comes with VTK, a suite of CSMP++ specific scripts demonstrate many features relevant to CSMP++-related visualization issues. It is provided with the CSMP++ Distributed Example set.

As the perhaps most severe limitation as seen from a CSMP++ perspective, the VTK / VTU file formats do not support region tags. However, regions and boundaries can be output separately and then visualized together in one Paraview session.

The use of the interface to VTK is simple. The interface object is instantiated and its interfaces are called as is shown in the code example below for an instance of a Model named “box” and for a Region identified by the string “fault”:

```
CSMP++_VTK_Interface<3>  vtk_output;

//                               S.G.  filename      variable name  timestep
vtk_output.OutputDataToVTK( box, "nodal_test",      "nodal_test", 0 );
vtk_output.OutputDataToVTK( box, "fluid-pressure",  "fluid pressure", 0 );

//                               S.G.  groupname  filename  variable name  timestep
vtk_output.OutputDataToVTK( box,"fault", "nodal_test", "nodal_test", timestep );
```

OUTPUT VIA THE JPEG INTERFACE

For convenience and to save space when monitoring runs, scalar variables from two-dimensional models can be output directly to JPEG image files. There are two classes for this purpose: **CSMP++_JPEG_Interface** and **CSMP++_JPEG_GroupInterface**. As the names suggest, the first outputs entire two-dimensional models and the second outputs subregions identified as groups. Output is generated as follows:

```
CSMP++_JPEG_Interface  jpg_output;
//                               file name      property name
jpg_output.OutputDataToJPG( model, "fluid-pressure", "fluid pressure" 0 );
```

In this operation, CSMP++ maps the variable values to a regular grid corresponding to the raster image and it associates a greyscale- or rainbow color scheme with these data. By default, the actual value range determines the color-mapping. Red (black) generally marks the highest values, and blue (white) marks the lowest.

Alternatively, a fixed value range can be set by the user. This is useful when images are to be strung together as animations.

A caveat of the JPG interface lies in the number of pixels of the auxiliary grid. Pixel size is automatically set to 1.6 of the inner radius of the smallest finite element in the mesh. If a model locally contains very small elements, the memory required for the grid will be excessive causing CSMP++ to terminate with an error message.

CSMP++ UoM GUI AND MODEL CONFIGURATOR

Using QT Creator, CUDA and SKUA, Giorgio Urso and Junchul Kim have created a new GUI and interactive model configurator for CSMP++.

The GUI is a multi-platform and data visualization tool that uses a CUDA-enabled graphics processing unit (GPU) for efficient manipulation of extremely huge CSMP++ 2D/3D models. To achieve platform independence, the debugger was developed in Qt using OpenGL, which supports visualization of meshes, regions, variables and their properties as well as model configurations for simulations.

The GUI makes use of CUDA and NVidia's massively parallel GPUs, delivering graphical output orders of magnitude faster than our standard public-domain visualisation tool Paraview (paraview.org) based on the Visualisation Toolkit (VTK), dependent on what graphics hardware is used (we worked with the M6000 graphics card, vintage 2016). More importantly, the GUI also supports the visualisation of the dual finite element – finite volume mesh, and scalar and vector variables with all possible placements therein, which was decisive for the development of CSMP.

The GUI, has its own help menu and video tutorials stored in the electronic repository.

In summary, the CSMP++ GUI supports visualization of any (geo)model, its mesh and variables discretised thereon, as well as simulations, handling the complexity of asynchronous forward simulations. It also provides interactive manipulations by supporting selection and display for model subregions. For example (Figure 0.1), shows a visualization of the 'Otway' CO₂ geo-storage corner-point grid used by the Australian CO₂CRC for plume spreading simulations after conversion into a CSMP++ FE-FV model, including regions, and distributed properties. The display is interactive: regions can be visualised and manipulated separately.

A tour of the functionality of the GUI

Once loaded into the GUI, the user can select a specific region of the model to be activated, and show or hide it by checking or unchecking boxes. The user can also show or hide the properties associated with the selected region. The GUI displays all variables with different placements and types for FEM-FVM discretised processes. As shown in Figure 0.1, GUI is comprised of menus, toolbars, docking tools, and 3D main view.

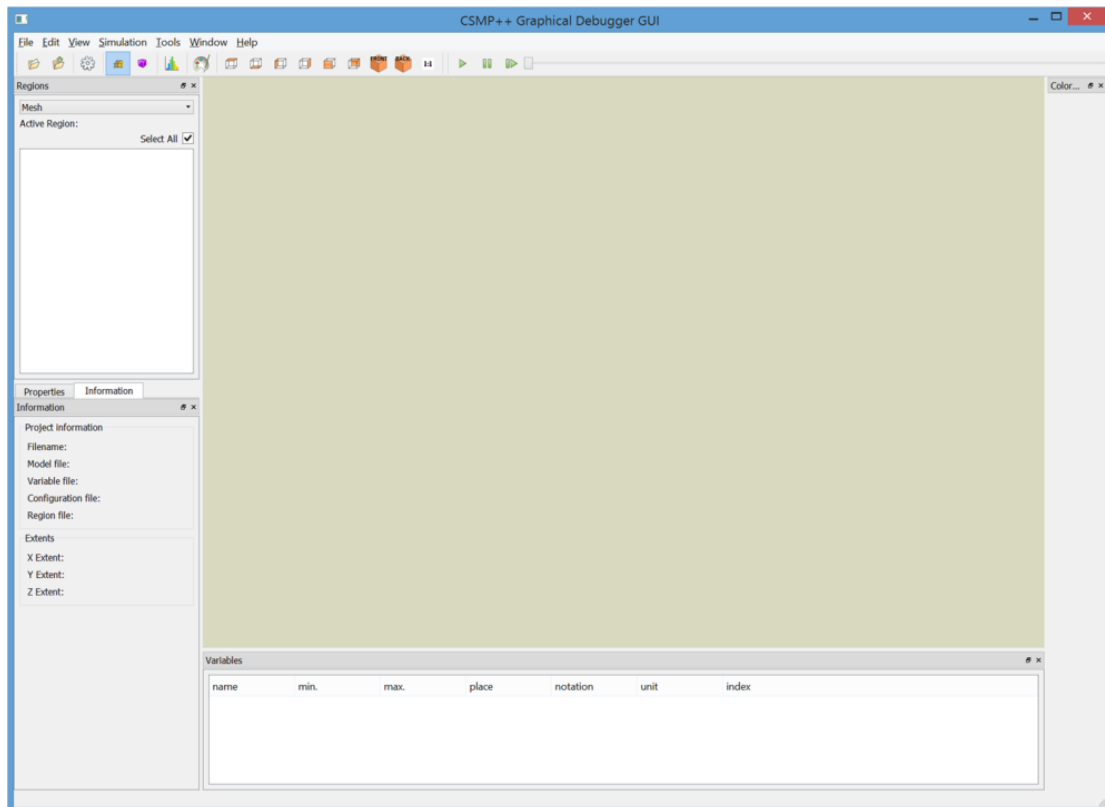


Figure 0.1 CSMP++ GUI window.

Menus & Toolbars (

Figure 0.2), include a standard set of functions to:

- Open CSMP++ native format (*.vset) files
- Import ANSYS file (*.asc) with its relevant files
- Save the loaded model as the CSMP++ native format (*.vset)
- Manage the model configuration and variables
- Change color ramp of the legend
- Manipulate view modes
- Control a model simulation



Figure 0.2 CSMP++ GUI – toolbars.

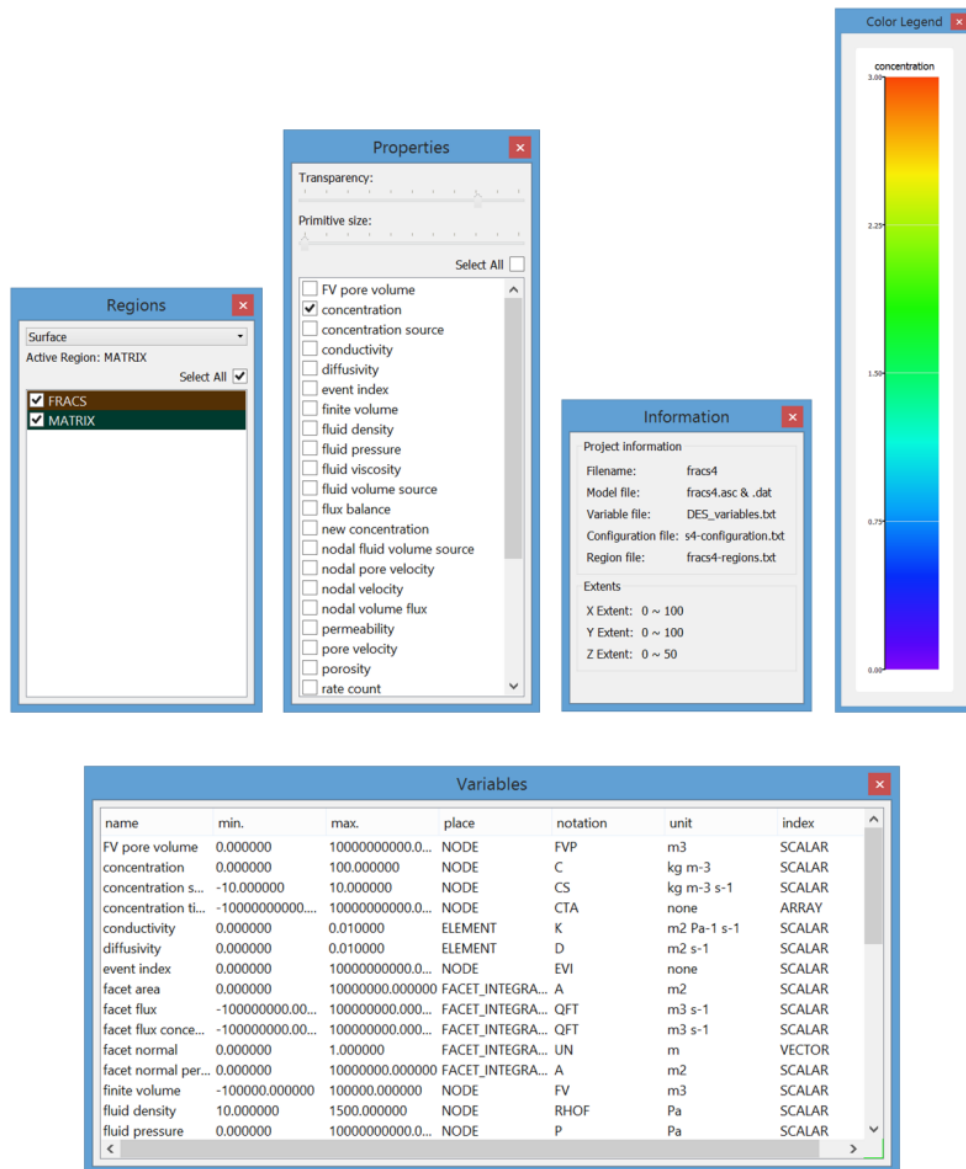


Figure 0.3 CSMP++ GUI – docking tools.

Docking Tools (

Figure 0.3): a useful set of tools to manipulate:

- Regions: checkable list of regions
- Properties: checkable list of properties, and manipulation of their opaque display and primitive size
- Information: details about model inputs
- Variables: details about model variables
- Color legend: legend display for multiple variables
- Mesh, regions and numerical values of properties.

(A model is loaded and visualized with an initial display, colored by regions in the 'Regions' tool and colored by multiple variables in the 'Color Legend' tool.)

- 3D Main View (Figure 0.4): 3D visualization window to display

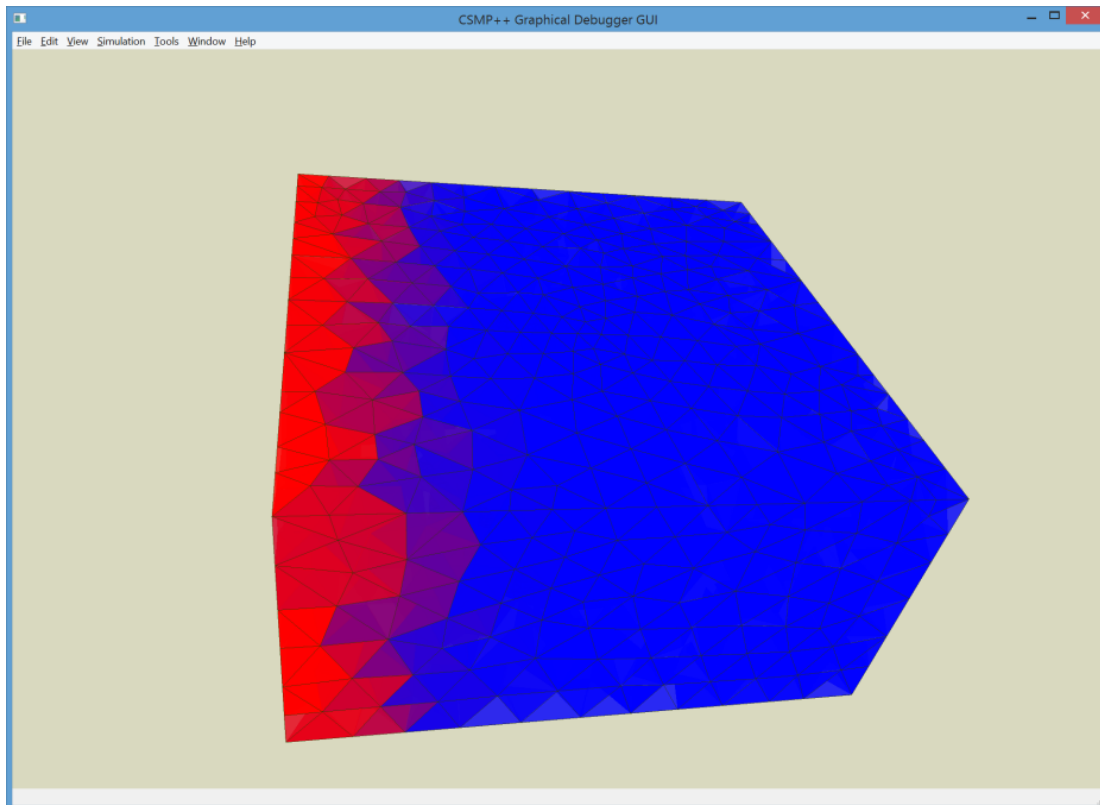


Figure 0.4 CSMP++ GUI – 3D main view.

Opening / Saving models

To open models, the GUI supports CSMP++ native format (Figure 0.5) as well as ANSYS format (from the import menu as shown in Figure 0.6). Rather than going back to ANSYS, the loaded models are stored in CSMP++ native format. To open a CSMP++ native format binary file from the GUI, use the following steps.

- Click ‘Open’ button in the toolbar for CSMP++ native models (default) and select the file. For example, the input file ‘HeuristicModel1fine.vset’ as file type ‘Native format (*.vset)’.

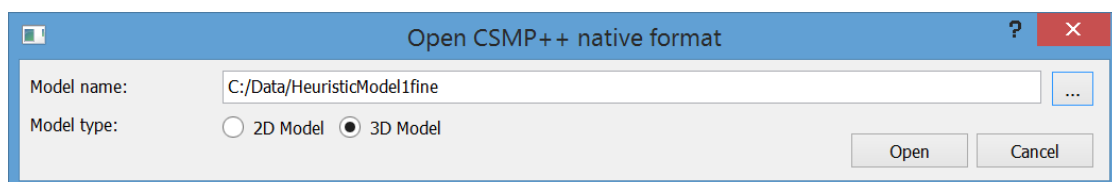


Figure 0.5 CSMP++ GUI – Dialog to open a CSMP++ native file.

- Or use the ‘Import menu’ for ANSYS models.

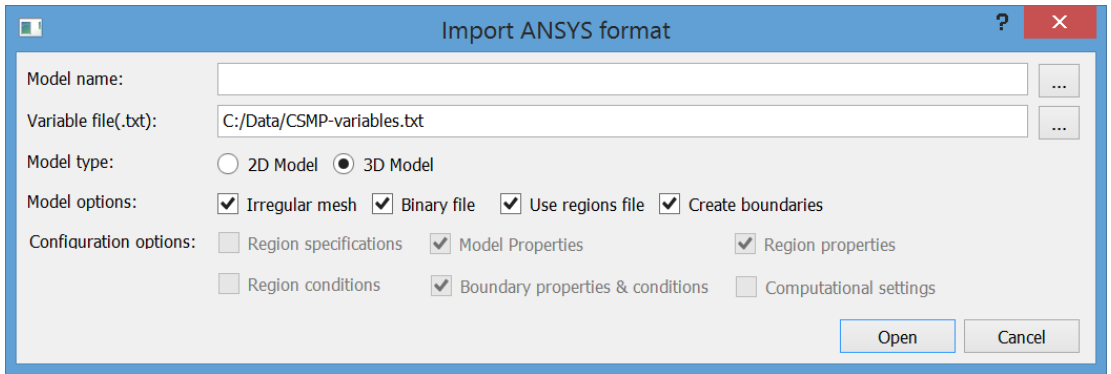


Figure 0.6 CSMP++ GUI – Dialog to import an ANSYS file.

Then, check a model type (dimension) of the input data (for example, 3D Model), and then click ‘Open’ button. In the open dialog, the default model type is 3D Model. The model will be displayed as shown in

Figure 0.7 and

Figure 0.8.

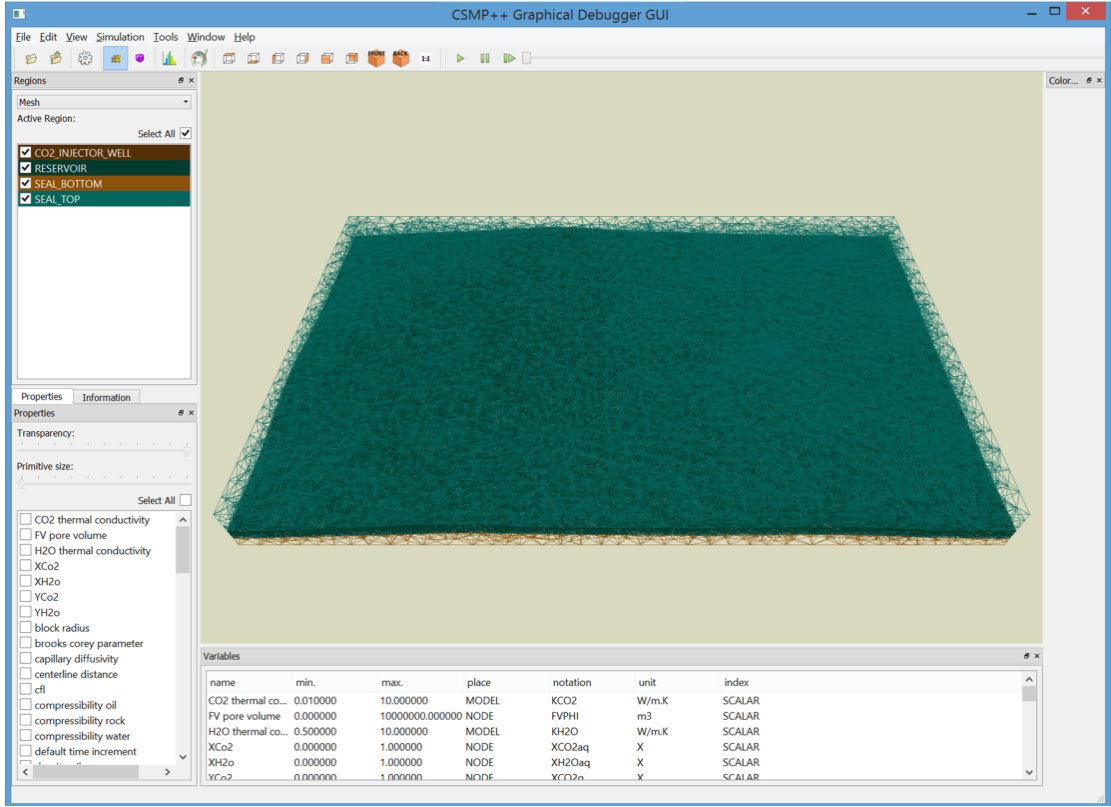


Figure 0.7 Opening CSMP++ native ‘HeuristicModel1fine.vset’ and auxiliary file set that stores regions, boundaries, split boundaries and variable sets.

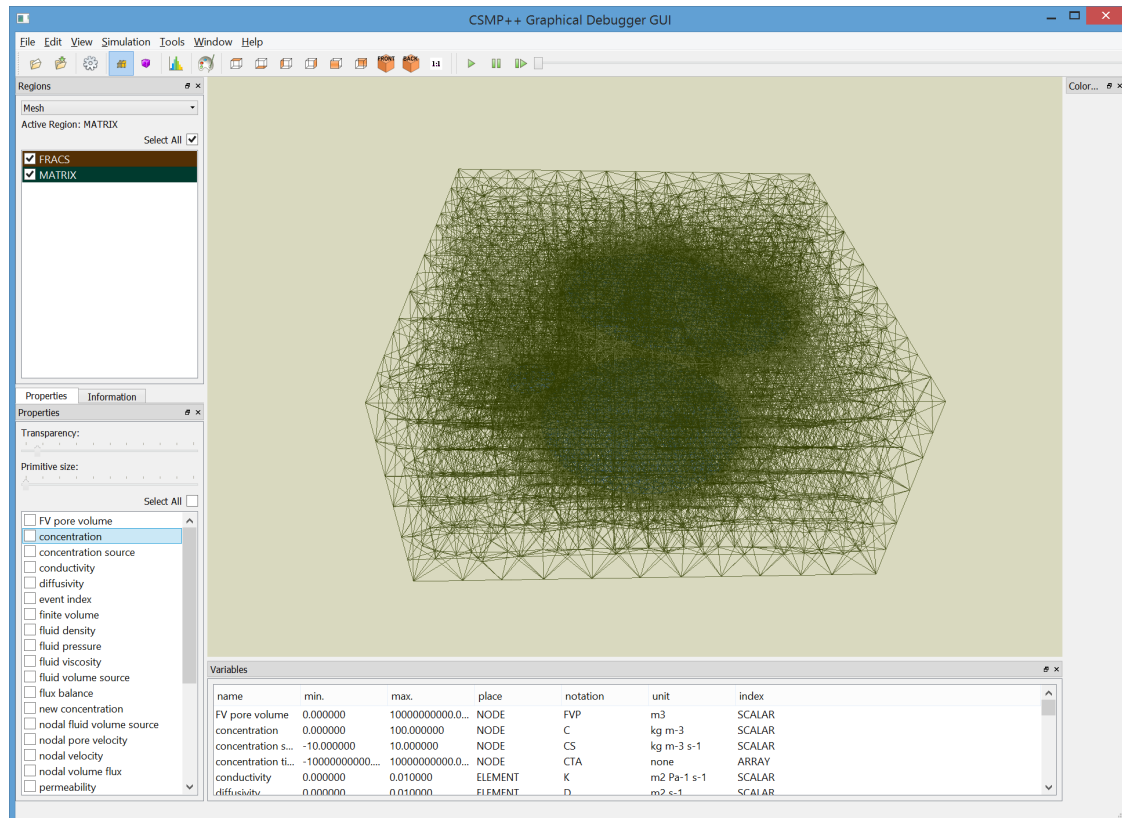


Figure 0.8 Example of importing ANSYS model ‘fracs4.asc, fracs4.dat.’

Display & Rendering of Model

The user can show or hide each property by checking it. The selected properties of the active region will be displayed dynamically in the legend as shown in Figure 0.9 and Figure 0.10.

The basic display operations such as zoom-in and zoom-out are as follows:

- Zoom in and out with mouse wheel scrolling
- Rotate with mouse left button
- Move with mouse right button
- Display with different views in the toolbars
- Transparency and primitive size adjustment in the Properties tool.

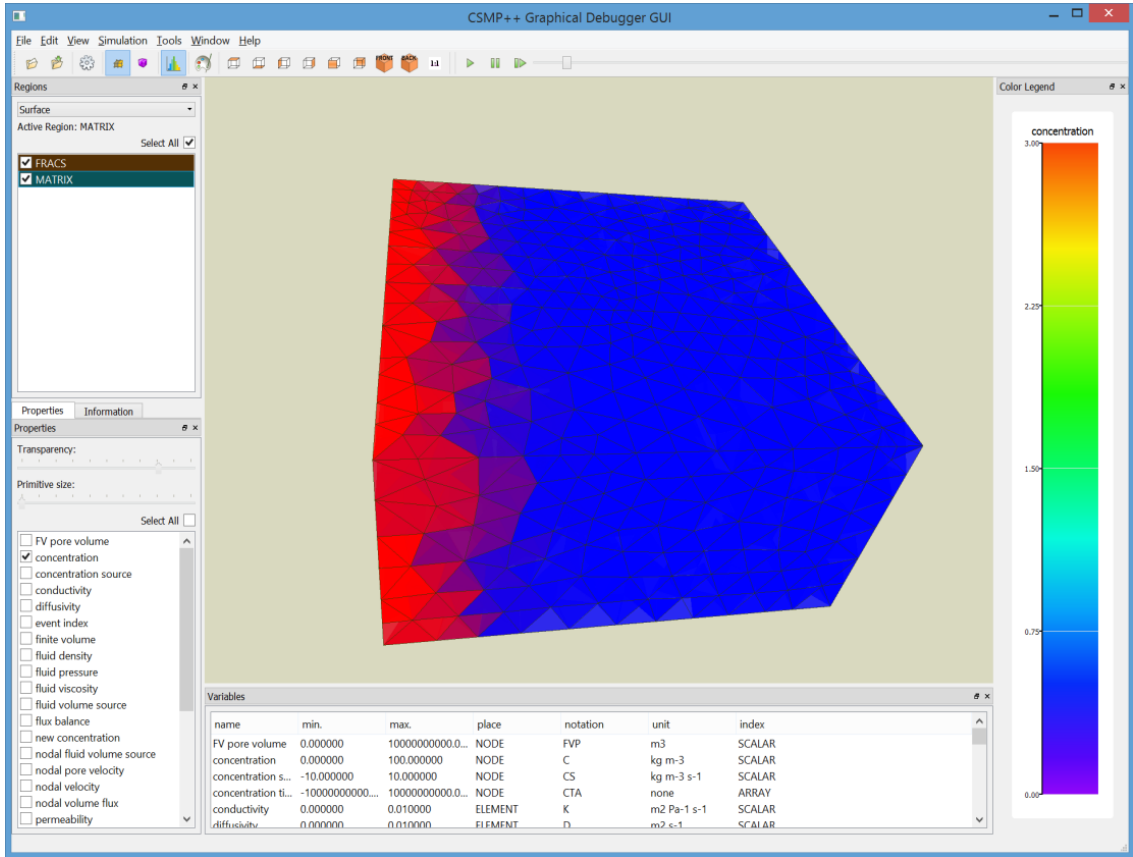


Figure 0.9 Example of displaying the model using the surface type visualisation.

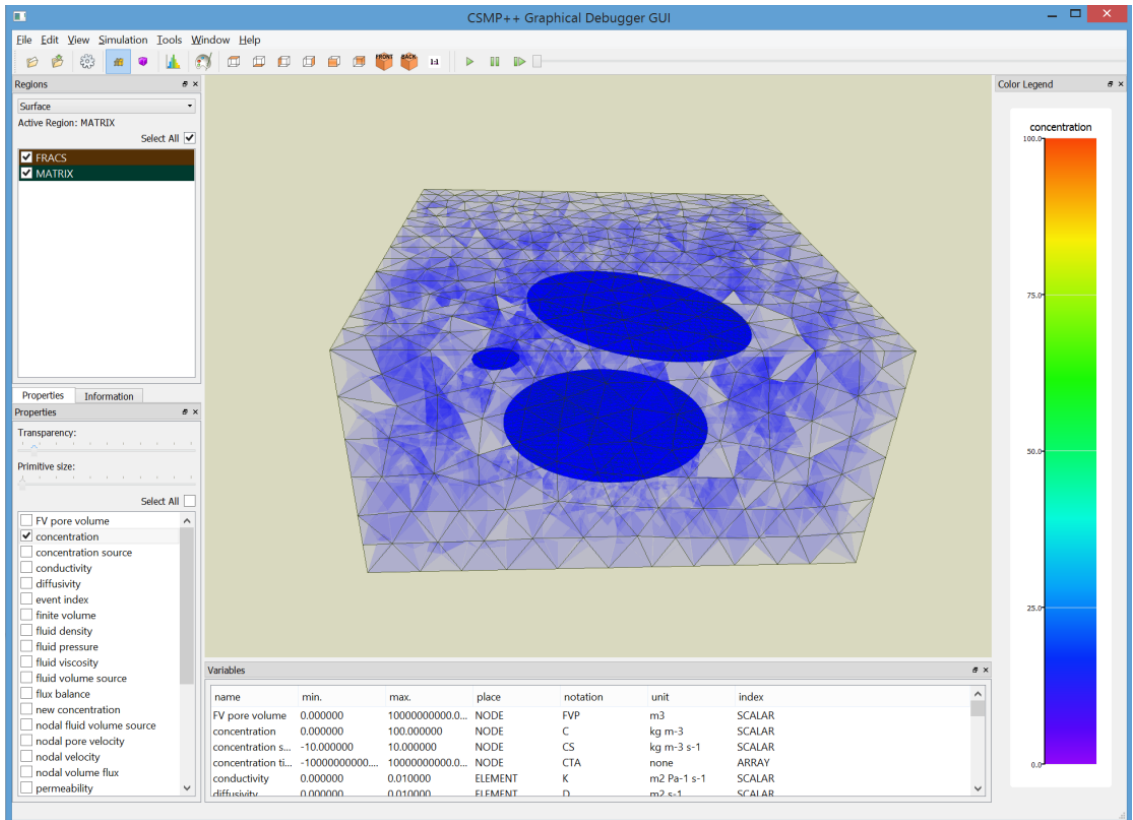


Figure 0.10 Example of the model transparency of the variable ‘concentration.’

Interactive model configuration

There are two options: 1) to create a new configuration or 2) to import an existing configuration from file. When configuring, there are defaults for variables. To create or import a configuration, the user needs to open the GUI dialog ‘Model Configurator’ by clicking the ‘Setup’ button in the toolbar. If the model is already open, click the ‘Next’ button to move on to the next step for variables setup.

The user can also select a model by clicking ‘Open File’ button.

Then, the user can follow the steps:

- Check the variables, and then click ‘Next’ button. You can edit by modifying any cell, or you can add/delete a new variable by clicking ‘Add Line’ or ‘Delete Line’ as shown in Figure 0.11.

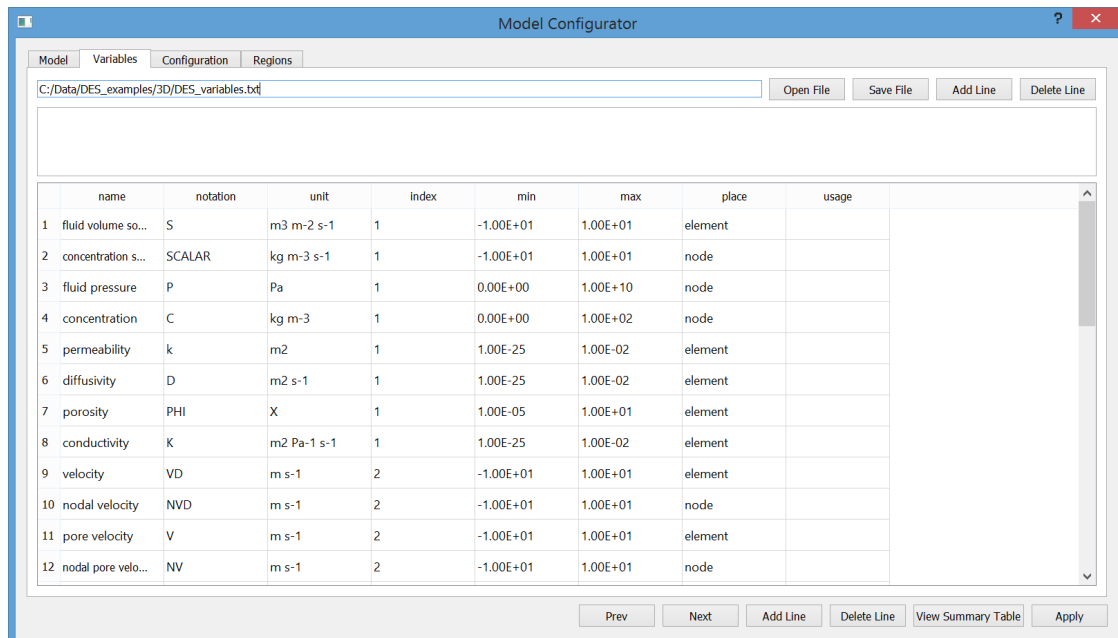


Figure 0.11 CSMP++ GUI – Variables dialog to create and edit a variables file as well as to import an existing file from file.

- Check proper data blocks and then select a configuration file. Or you can also edit comment lines and all inputs of the data blocks step by step.

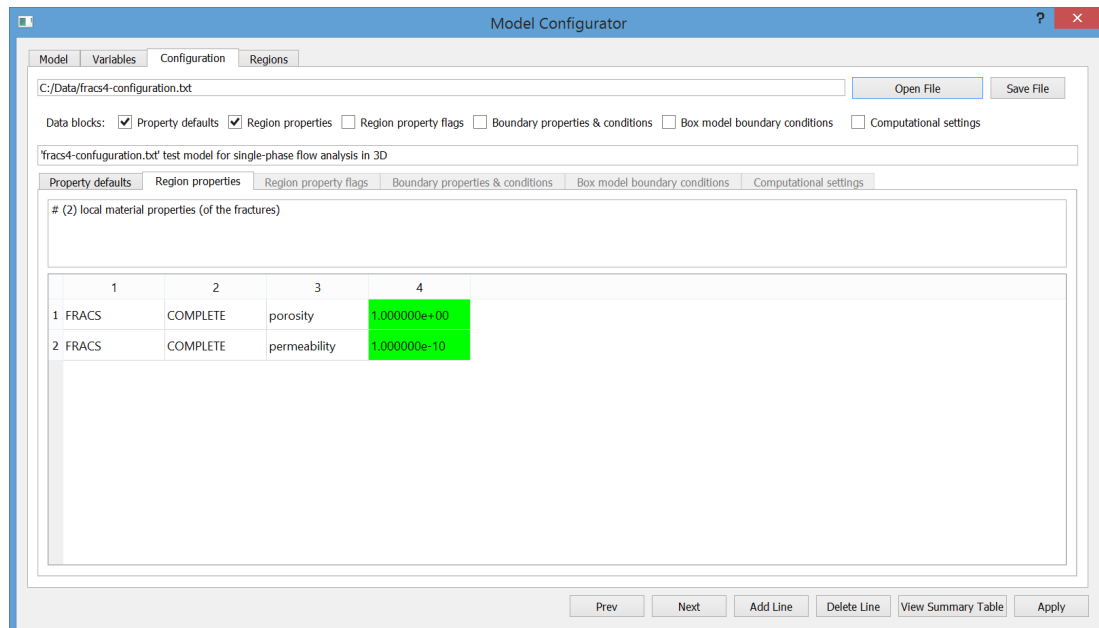


Figure 0.12 CSMP++ GUI – Model configuration dialog to create and edit a configuration as well as to import an existing configuration from file.

- Save the model as a configuration file by clicking ‘Save File’. The model configurator checks the inputs automatically whether they are valid (green) according to the defined variable ranges as shown in Figure 0.12. If there are red cells, they need to be corrected. Click ‘Next’ button to move on the next tab, and in the similar way you can check and modify, then move to the next tabs.
- In the last step as shown in Figure 0.13, the user can view the configuration as a summary table by clicking ‘View Summary Table’.

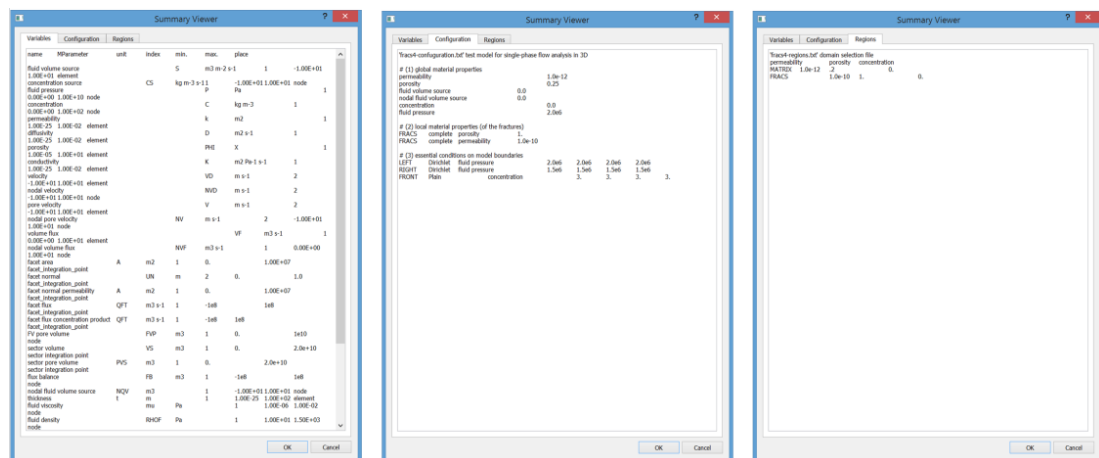


Figure 0.13 CSMP++ GUI – Dialog to show all of the configuration information.

Visualisation of Simulation Results

The GUI visualizes simulation results while running the CSMP++ core program. This is performed in two ways for:

1. Simulations based on conventional global time-stepping, simply called Time-Driven Simulation (TDS),
2. Simulation based on event queues, simply called Discrete Event Simulation (DES).

To manipulate the simulation, the user can start to play the simulation by clicking 'Play' button from the toolbar as shown in Figure 0.14. The user can also pause it or play one step forward.



Figure 0.14 Simulation control toolbar.

Time Driven Simulation (TDS) visualization

A simulation based on a time-stepping scheme is illustrated here with a 2D model shown in Figure 0.15. First, the user needs to open an ANSYS model by selecting 'Import' button in the 'File' menus. Then, select the input file 'box2d_fault.asc' with the option '2D Model', and select a variable file (*.txt). Also other options should be checked properly such as which parts of the mesh shall become model regions as is specified in the '*-regions.txt' file when no GUI is available. Rather than configuring the model in the GUI a '*-configuration.txt' can be imported. When the user starts the simulation, the GUI display will display evolving physical variables chosen by the user or just when the simulation reaches pre-specified time steps defined in the configuration file.

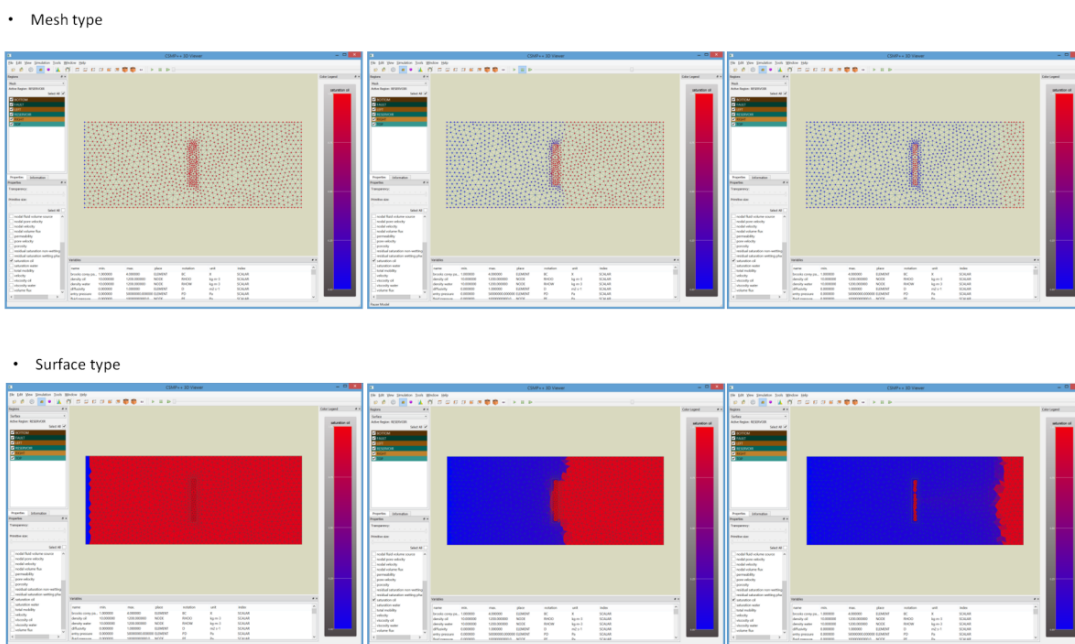
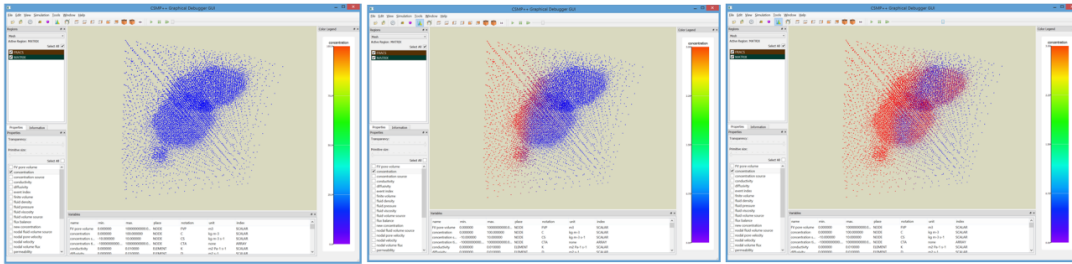


Figure 0.15 Visualization of results based from time-driven simulation (TDS).

Event (DES) visualization

The visualization of an asynchronous simulation with event queues is illustrated in the 3D ‘fracs4’ model shown in Figure 0.16. Again, the user has to first import a model, by selecting ‘Open’ (for CSMP native) or ‘Import’ (for third-party) models from the ‘File’ menu. The files shown will depend on whether 2D or 3D is selected. A suitable variable file must also be selected and the other options should be set according to the availability of input data such as ‘*-regions.txt’ and ‘*-configuration.txt’ files when pressing the ‘Open’ button. When the user starts the simulation, the GUI will display simulation results following the local changes in the model.

- Mesh type



- Surface type

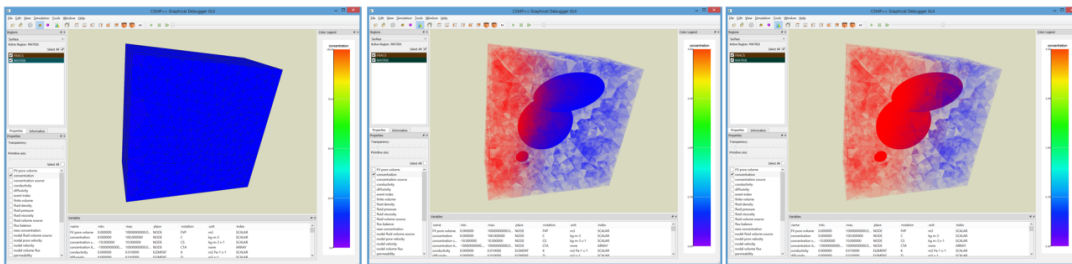


Figure 0.16: Visualization of the simulation results based on discrete event simulation (DES).

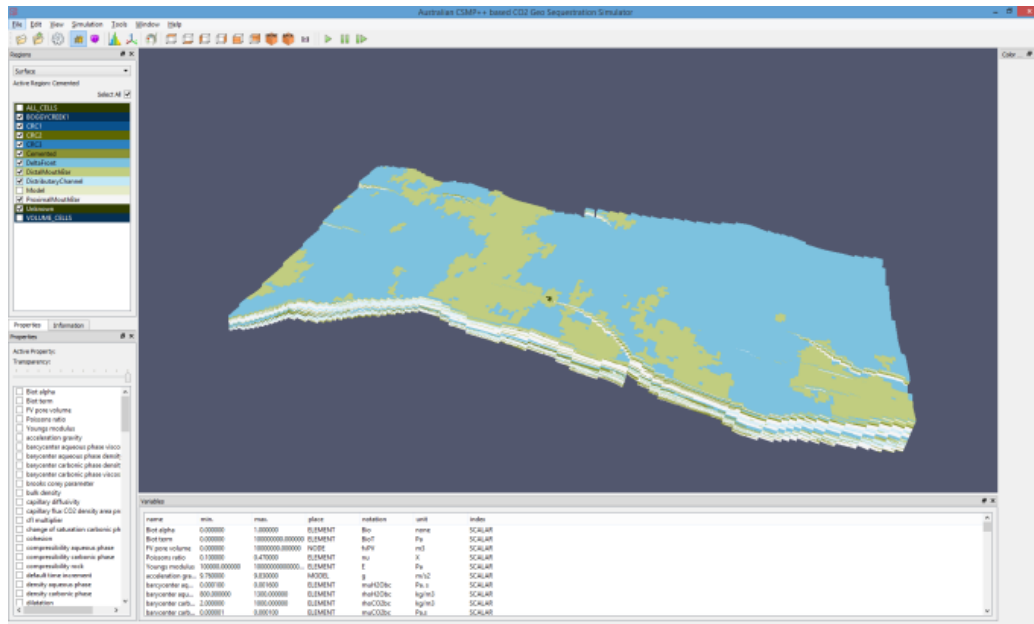
With regard to graphical display of event queues and their parallel event processing (PEP) counterparts, the GUI has parallel multi-processing capabilities used in the display of the dynamics of the discrete event queues and their interactions. This feature was implemented to debug FEM-FVM discretised asynchronous evolution processes linking the CSMP++ with a new CUDA kernel at runtime. Thus far, it has been used to monitor the coupled computation of pressure, PVT properties, multiphase flow, chemical reactions, and mechanics, helping to analyse event triggering and selective updating of the model during the forward simulation.

Specific visualisation examples

Figure 0.17 and Figure 0.18, show a Petrel/Eclipse (Schlumberger) corner-point model converted into a CSMP++ mesh using the EclipseInterface. The model domain comprises the Paaratte formation’s parasequence 1 at Otway, Victoria, Australia into which CO₂ was injected during the ‘Stage 2C’ plume spreading and stabilization experiment (e.g., Paterson et al., 2016). Displayed are the rocktype, porosity and permeability, populated using stochastic methods using well data and seismic attribute analysis. The GUI provides essential functionality which allows users and developers to configure such models and carry out computations with them. At the very least it can be used to QC and configure models and save them to a CSMP binary file so that they can be run from the command line or submitted for a run scheduling system on a HPC

cluster. When a model is read from a CSMP native binary, its variable distributions become visible directly, guiding the user in further property assignments and assisting with the model configuration process in a wizard like workflow, proceeding step by step: 1) default values, 2) region properties, 3) region flags, 4) boundary / essential conditions, and 5) time stepping parameters. Where values are missing, the GUI highlights these missing values. The configuration can also be output to configuration files again if so required. This way the user is able to check and modify them as well as insert comments and explanations for the documentation of simulation runs.

a)



b)

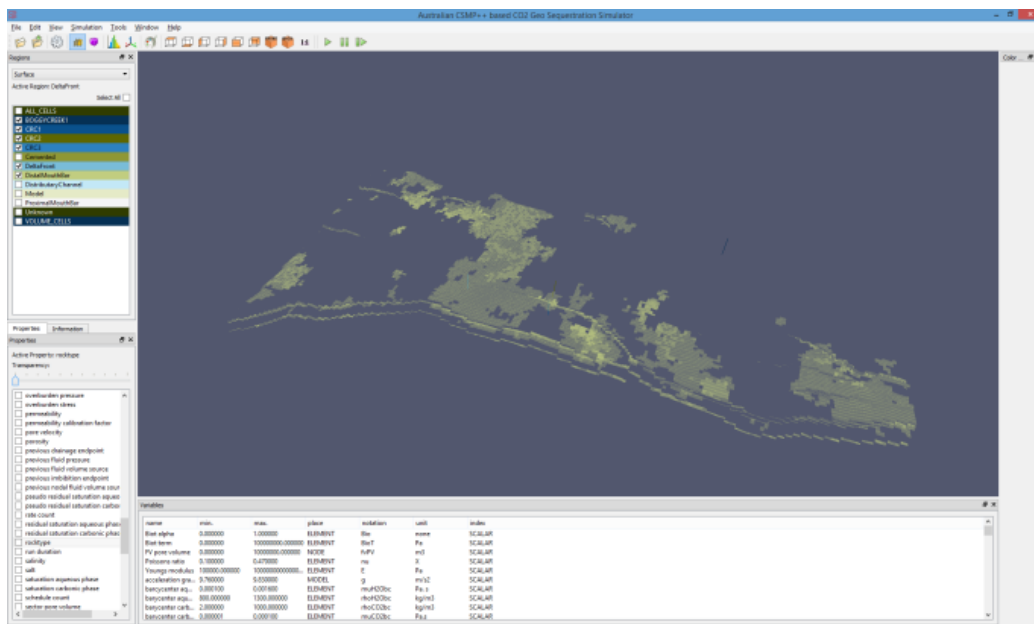


Figure 0.17: GUI display of ‘Otway’ Paaratte, parasequence 1: a) visualization of the mesh, regions, and properties. b) Demonstration of interactive selection and display of the ‘Otway’ model subregions.

a)

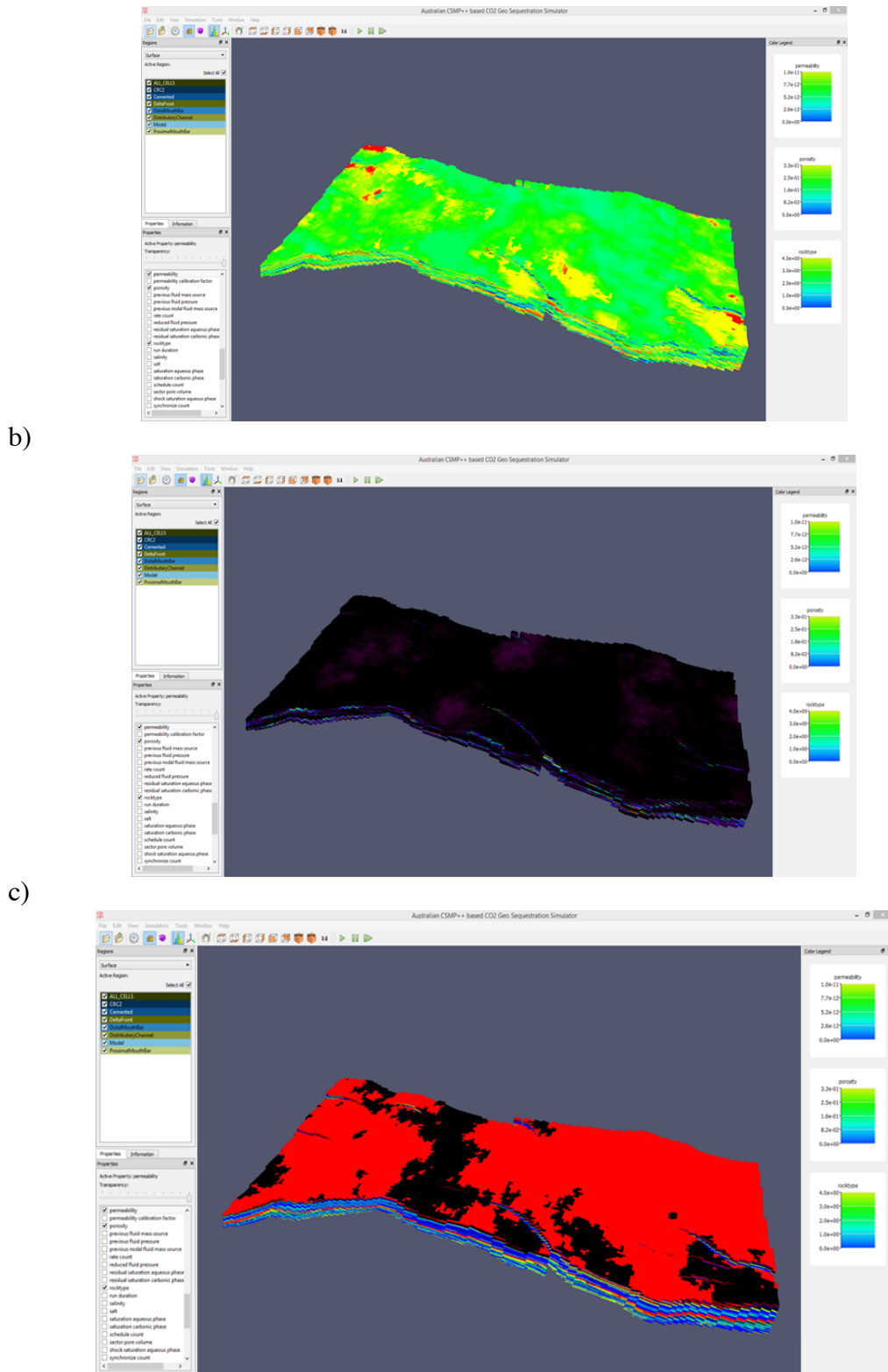


Figure 0.18 GUI property display: a) porosity, b) permeability, c) rock type of Otway model.

For example, Figure 0.19 shows discrete event queues-based asynchronous simulation (DES) of a 3D model. When the user runs the simulation, the graphical debugger displays cell activity and evolving events, i.e. the dynamics of the discrete event queues and their interactions, displaying relevant auxiliary parameters such as the event index, rate count, schedule count, synchronize count, and update count.

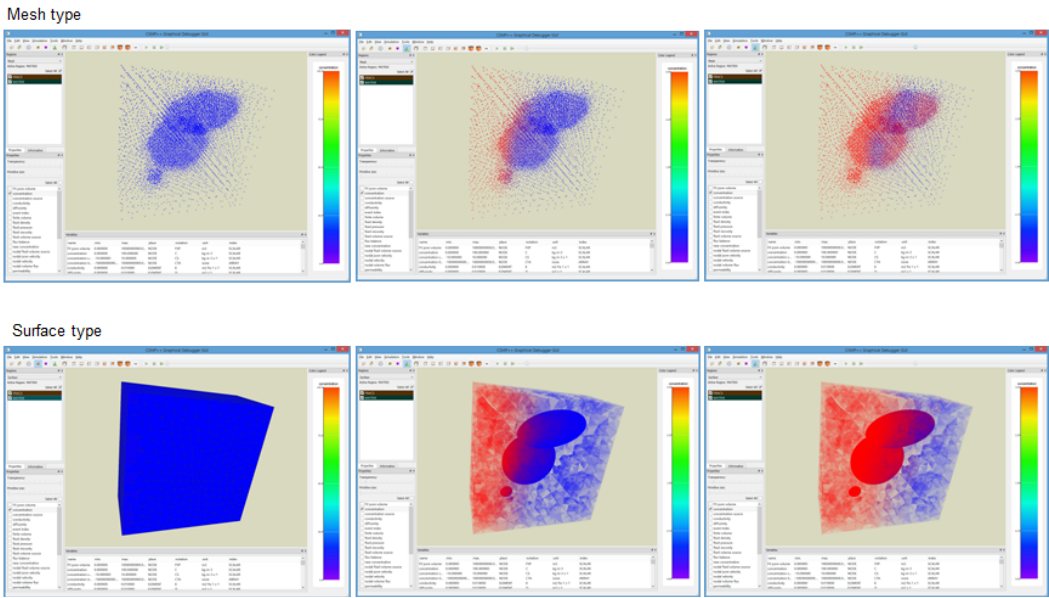


Figure 0.19 GUI display of the dynamics of discrete event queues and their interactions.

The CSMP++ GUI v1.0 is available for download from the git repository together with a guide-line for installing it provided in the ‘ReadMe.txt’ file. Git reference: git clone <https://csmp.eresearch.unimelb.edu.au/diffusion/7/csmp-viewer-vs>

Interfacing of CSMP++ with conventional reservoir simulation tools

The GUI has also been used for the prototyping of the interfacing of CSMP++ with other software tools commonly used in the G&G workflow, and the testing of import procedures for simulation grids created by third-party software tools, including Petrel/Eclipse and RINGmesh. For the former, a corner-point grid (Eclipse model) to finite-element mesh conversion algorithm has been implemented such that comparative simulations become possible. For corner-point grids such as the ones used by ECLIPSE, the grid conversion process considers 16 cases of degenerate hexahedral cells that can arise when converting inactive grid cells into finite elements, see Figure 0.20. The new CSMP - Eclipse interface also permits the creation of discrete line-element wells in the mesh generated from the Eclipse input model.**Error! Reference source not found.** Figure 0.21 shows the three wells CRC-1, CRC-2, and CRC-3, which were created from the ‘Otway 2012’ Eclipse model.

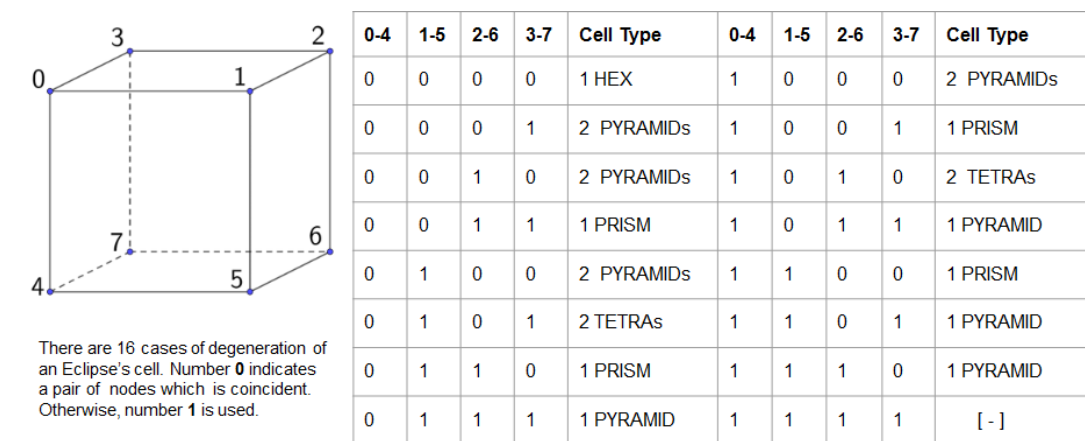


Figure 0.20: EclipseInterface - classification of degenerate corner-point cells and corresponding finite element types that they are converted into for building a CSMP++ finite element model.

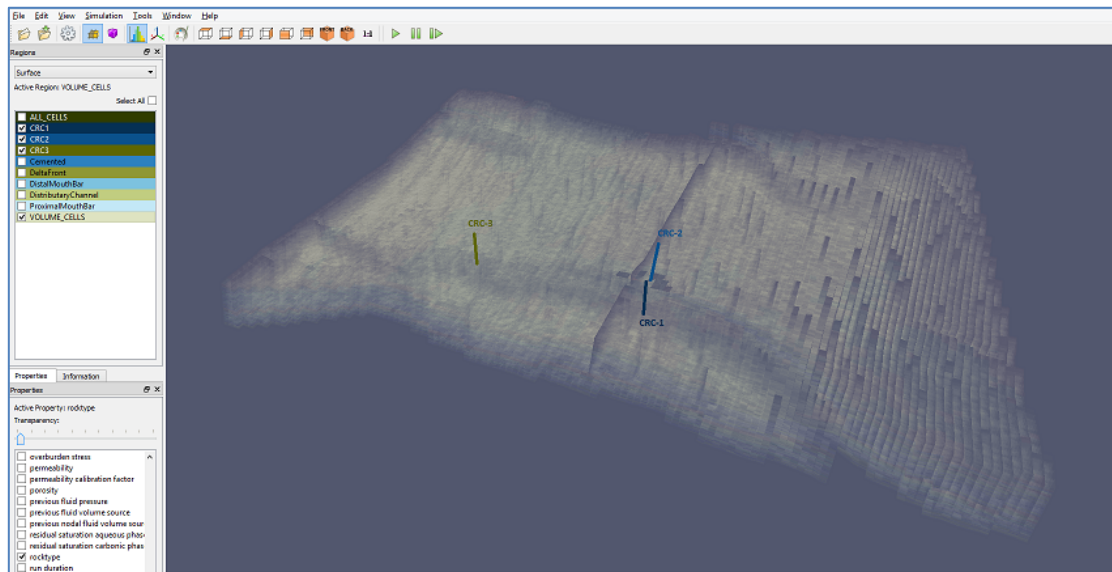


Figure 0.21: EclipseInterface - creation of 3 discrete wells (CRC1, CRC2, CRC3) in the mesh of the 'Otway' model.

The different types of grids and the problem of converting a regular corner-point grid, consisting of pillars of nodes into a finite element mesh is illustrated in Figure 0.22.

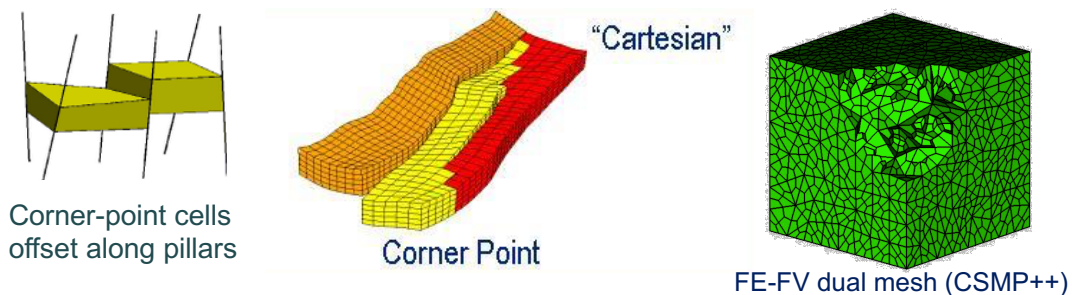


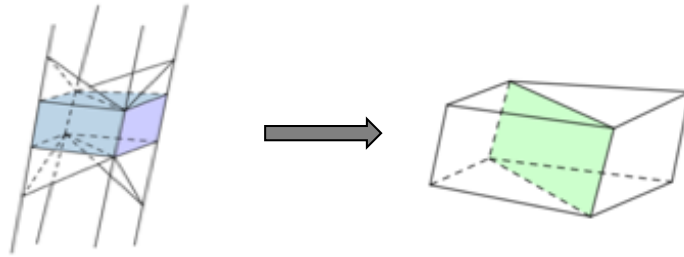
Figure 0.22 Corner point (regular) grids contrasted with unstructured FE-FV meshes into which they are converted by the EclipseInterface.

The most important grid-conversion issues that had to be overcome are:

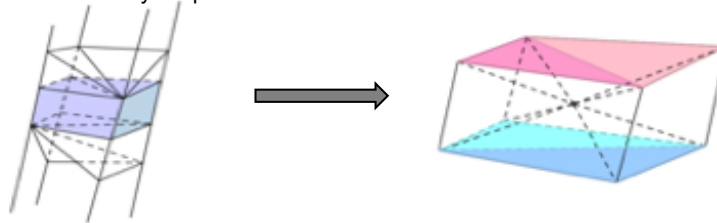
- Conversion of intact but degenerate cells into finite-element polytypes
- Elimination of twisted self-intersecting cells and geometry repair
- Representation of wells (CSMP uses a discrete representation)
- Property transfer

The first step, the cell conversion performed is illustrated in Figure 0.23.

Degenerate *corner-point* cells to *finite element* algorithm



Simple degenerate cell transformation process. From one degenerate hexahedron, two prisms are created; then connectivity is updated.



Complicated cell repair process of a cell. The top face of the colored hexahedron is split in '5-7' direction while the bottom face is split in '0-2' direction. The original hexahedron is divided into eight elements: four pyramids and four tetrahedrons. These new elements are formed around the barycenter of the original hexahedron and its surrounding faces.

Figure 0.23 Conversion of degenerate corner-point grid cells into finite elements.

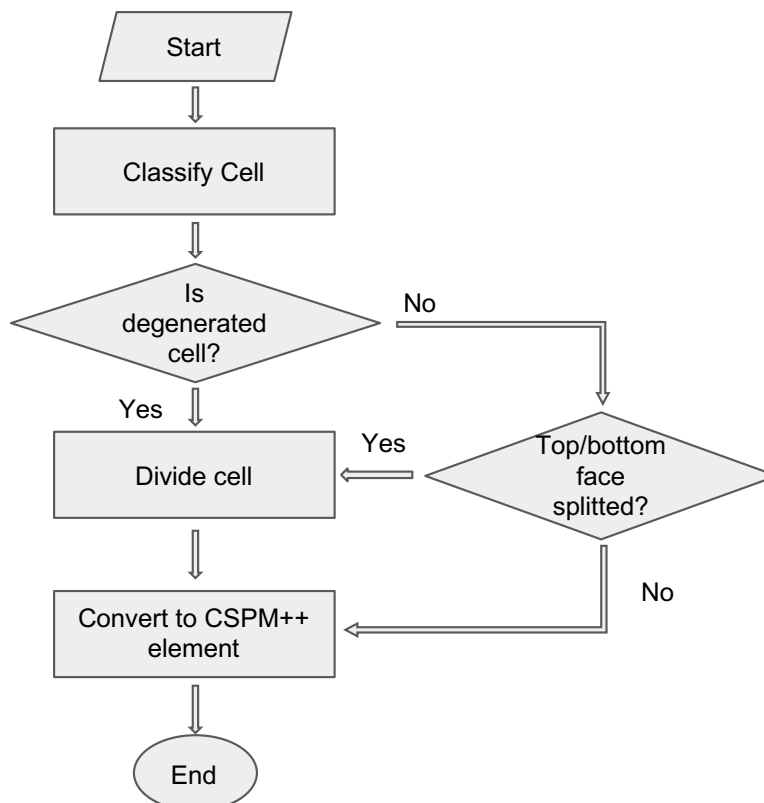


Figure 0.24 Flow chart of the conversion algorithm implemented in the EclipseInterface to transform degenerate corner-point cells into finite elements.

This algorithm and its decision points are illustrated by the flow chart (Figure 0.24). To convert an Eclipse model into a CSMP++ model, first, each cell is stored in a corner-

point-grid model that serves as an intermediate data contained in the EclipseInterface. Among other tasks, this grid serves to handle active/inactive cells and pillars in the Eclipse coordinate systems according to the Eclipse specifications. Then, the cells of the corner-point-grid are converted into finite elements.

The EclipseInterface and the conversion algorithm have been tested with a publically available Eclipse benchmark model provided online. This NPD5 model represents the Johansen field in the Norwegian sector of the North Sea and it has 5 layers and several faults (sintef.no/projectweb/matmora/downloads/johansen/). It contains 88,775 active cells and 100,476 unique points were detected on the pillars, respectively. A total of 100,486 elements were created successfully from the grid cells by applying the new conversion algorithm. In the mesh verification process, we detected 11 degenerate hexahedra and two tetrahedra with unfavourable aspect ratios. Such degenerate elements are removed automatically. Figure 0.25 shows the CSMP++ model generated from the NPD5 Eclipse model.

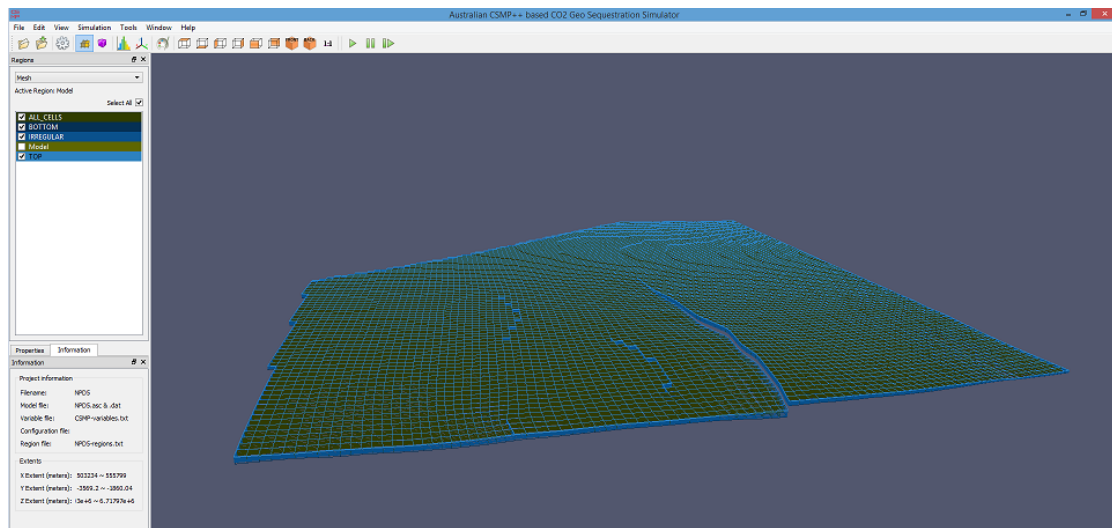


Figure 0.25 CSMP++ representation of the NPD5 model with 5 layers (100x100x11 cells).

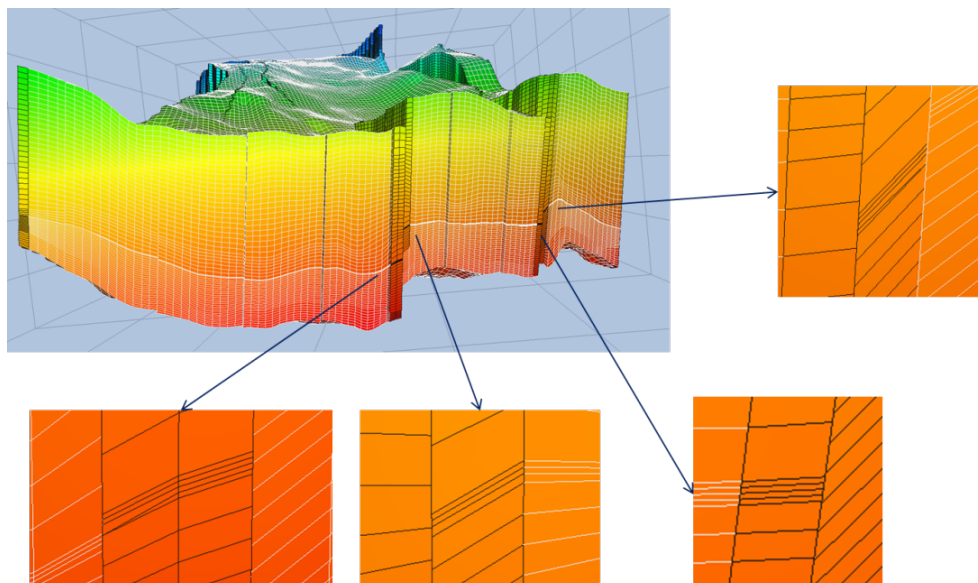


Figure 0.26 Non-standard connections illustrated with 'Otway' Eclipse model (CO2CRC).

There are limitations as to which type of Eclipse grids can be converted into CSMP mesh objects. For instance, when converting the 2008 ‘Otway’ Eclipse reference model of the CO2CRC company, the EclipseInterface detected non-standard connections (non-adjacent cells in the grid although there are no faults in these locations). The resulting non-neighbour connections (NNCs) as shown in Figure 0.26, in the CSMP++ GUI. The EclipseInterface cannot handle such grids and no attempts are planned to extend its functionality to accomplish this.

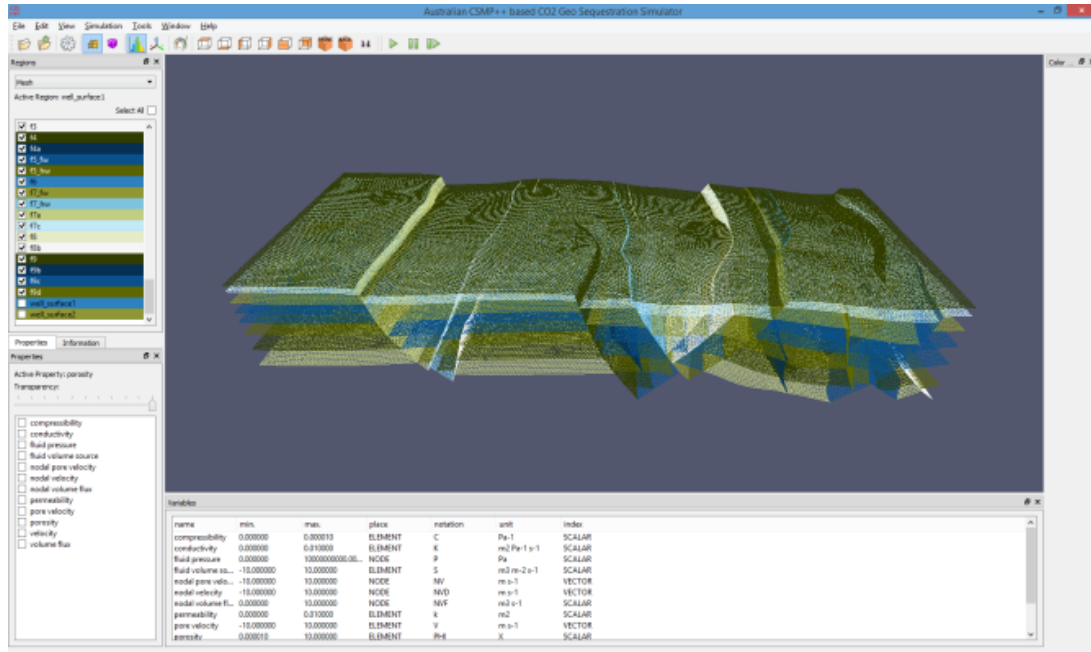


Figure 0.27 RING_Interface demonstration: SandBox model of a faulted layered sand-shale sequence created in SKUA and meshed using RINGmesh.

Interfaces to gOcad / SKUA. RINGmesh (<http://ringmesh.org>) and CSMP’s RING_Interface now facilitate meshing and transfer of GOCAD and SKUA models (<https://www.pdgm.com/products/skua-gocad/skua-modeling-platform/>) to CSMP++. The key obstacle that had to be overcome in order to create simulation models, was the creation of discrete well representations. CSMP++ now contains a work-around for this problem that involves the following steps:

1. To force the generation of finite-element edges along the well completion (the part of the well that connects with the formation), small isolated (fault) surfaces are inserted into the SKUA model, which intersect where the completion is located.
2. During finite-element mesh creation in RINGmesh, this fault intersection gets matched by element edges.
3. A well completion detection function in CSMP++ retrieves these edges using the meshed surfaces as input and creates line elements there.
4. The line elements are automatically converted into a lower-dimensional model region with the desired name of the well.
5. The auxiliary surface meshed needed to create the well are deleted.

Figure 0.27 is a visualization of a model of faulted sand-shale sequence generated in SKUA, meshed using RINGmesh, and endowed with a well completion using the work-around described above. Figure 0.28 shows the well completion which was created from the intersection curves between ‘well_surface1’ and ‘well_surface2’.

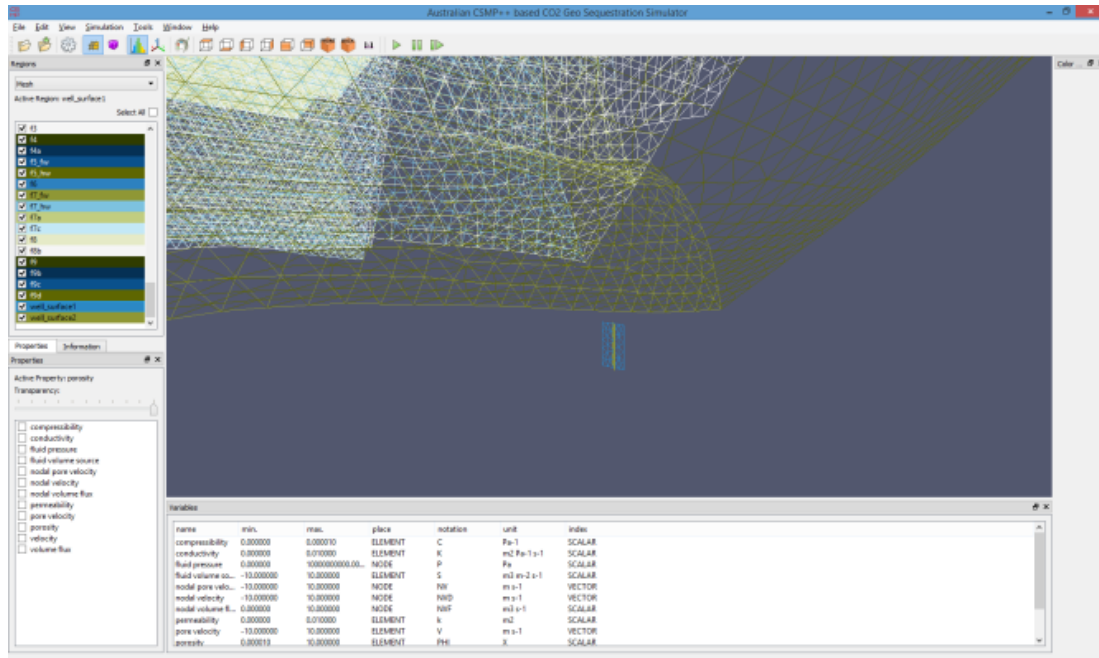


Figure 0.28 RING Interface example: Intersection curves between the regions ‘well_surface1’ and ‘well_surface2’ in the model were prepared to create a well intersection.

Visualisation of SplitBoundary objects. The CSMP++ GUI also supports the visualization of SplitBoundary objects and has been used to display and debug the new SplitBoundary formulation in CSMP++, see Figure 0.29.

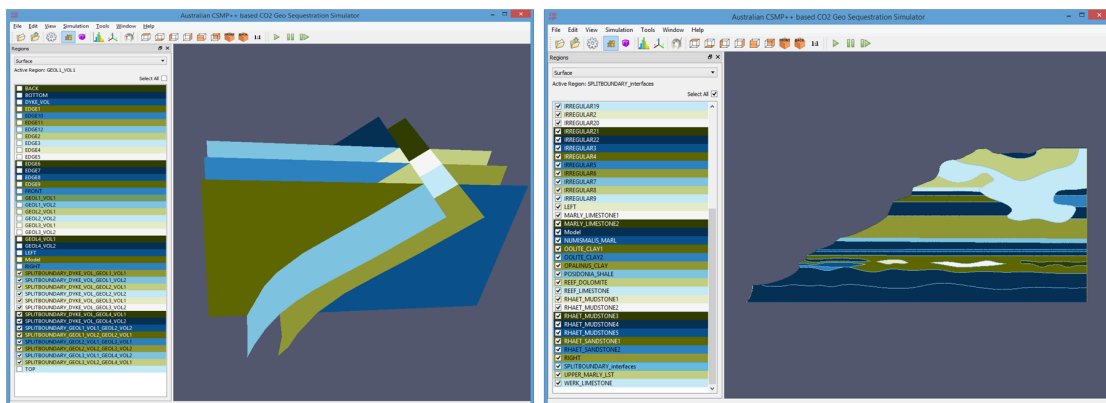


Figure 0.29 Visualization of 3D and 2D SplitBoundary objects. Note that although visualized properties are placed on the nodes, they vary discontinuously across the different geologic regions of the displayed models. This is the hallmark of split boundaries, i.e., initially node-matched mesh patches connected by interface elements.

Figure 0.30, Figure 0.31, and Figure 0.32 show SplitBoundary objects created from a 2D box shaped mode and a 3D box shaped model, respectively. The 2D box shaped model, contains two regions – ‘MATRIX_LEFT’ and ‘MATRIX_RIGHT’. To create the SplitBoundary object between these regions, a new non-unique map of

SplitBoundary objects is created at runtime into which objects are stored with unique names such as SPLITBOUNDARY_region1_region2, adding a number if necessary to make the name unique, and following the same conventions as for Boundary objects.

In the example shown in Figure 0.30(b), the boundary is named 'SPLITBOUNDARY_MATRIX_LEFT_MATRIX_RIGHT' because the SplitBoundary object sits between the regions 'MATRIX_LEFT' and 'MATRIX_RIGHT'. New InterFace objects (8 in total 8) replace the 8 Face objects of the previous boundary. Their Inside higher-dimensional neighbor Elements (blue) and Outside Elements (red) are correctly identified as shown in Figure 0.30(b).

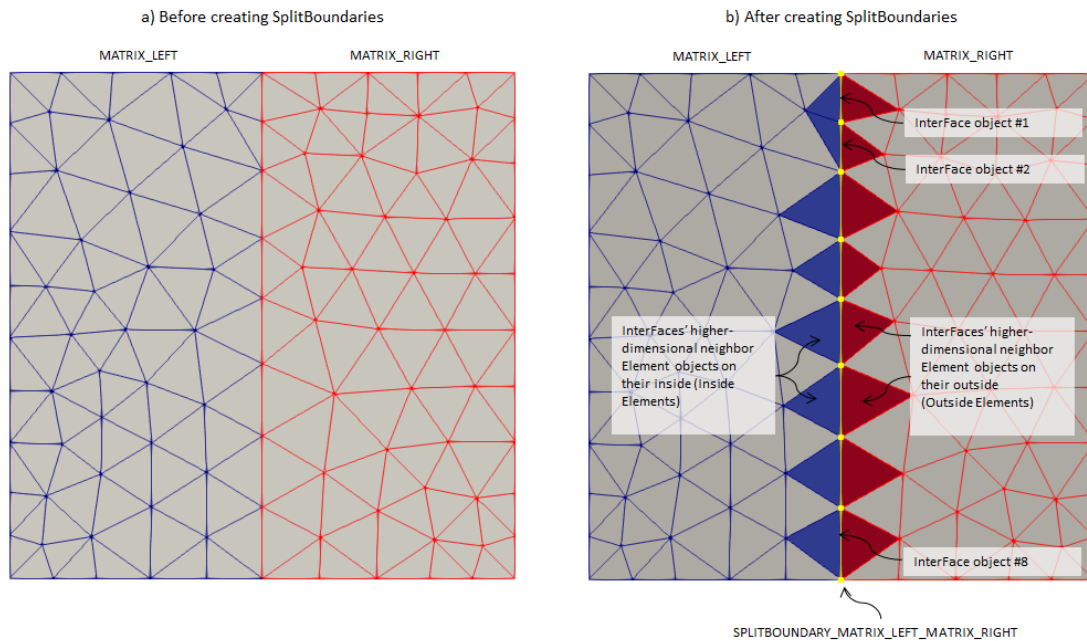


Figure 0.30 SplitBoundary inserted into rectangular test model 'BoxHalf2D': a) Original 2D Model including the regions - 'MATRIX_LEFT' and 'MATRIX_RIGHT' , b) 2D Model after creating the SplitBoundary and InterFace objects The blue represents 'Inside Elements' w.r.t. InterFace objects, and red 'Outside Elements' (situated where the InterFace normal point to).

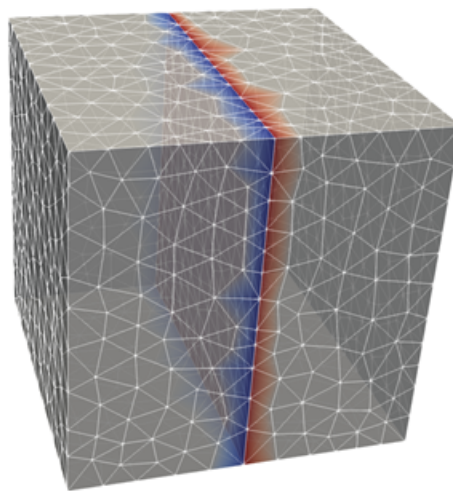


Figure 0.31 SplitBoundary in 3D box model 'BoxHalf3D': Model consists of tetrahedral elements (white); two sub-regions are divided by the SplitBoundary objects. Blue represents inside elements, and red outside elements.

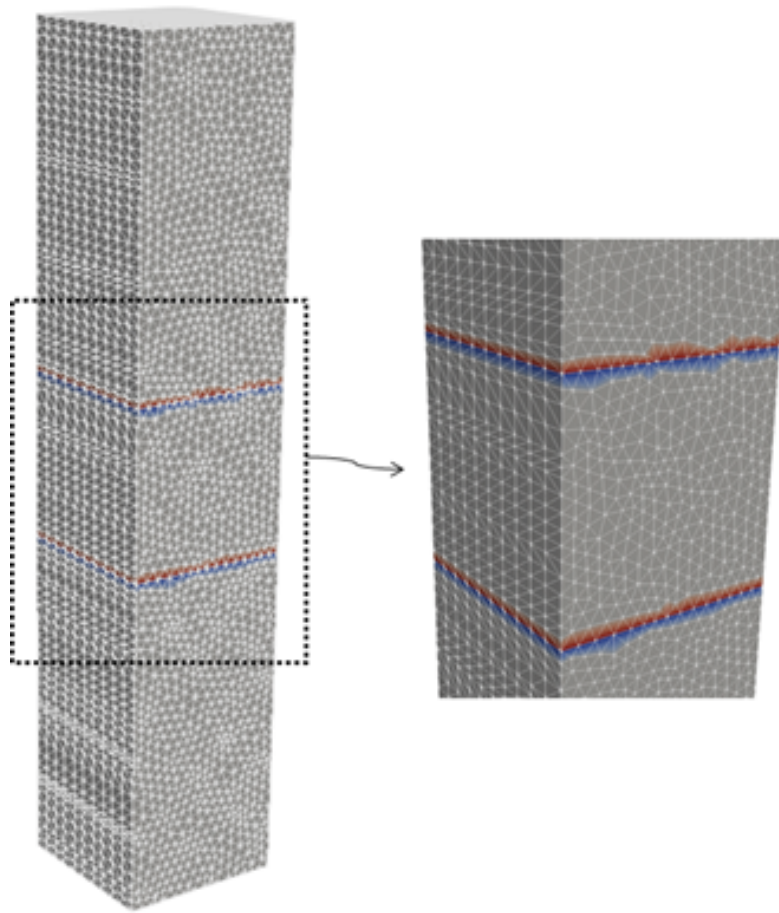


Figure 0.32 SplitBoundary objects among three different zones in a CSMP++ 3D model 'ThreeZonesD': The 3D model consists of tetrahedral elements (white), and its three sub-regions are distinguished from the generated SplitBoundary objects. The blue represents 'Inside Tetrahedral Elements' identified from the InterFace objects, and the red 'Outside Tetrahedral Elements.'

To create SplitBoundary objects within CSMP++ there is only a single method that generates them from Boundary objects: `CreateFrom( Boundary )`. This does however require remeshing and an update of all involved regions. And `CreateFrom()` offers considerable flexibility: thus, when a target region is inside of another region, a SplitBoundary can be created around it. Figure 0.33 shows three models with SplitBoundary objects created around line element regions.

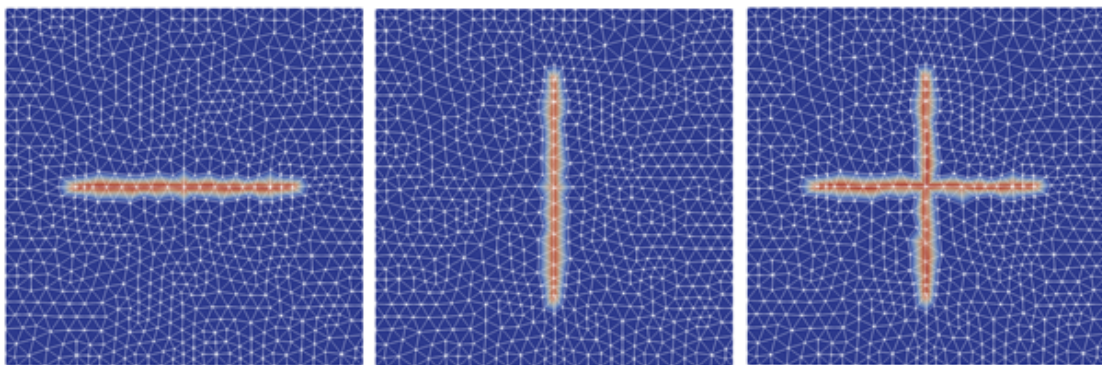


Figure 0.33 Examples of SplitBoundary objects which were created around specific regions: a) the inside region for the line elements parallel to x-axis, b) the inside region for the line elements parallel to y-axis, c) the inside region for two lines crossed.

It is important to note that SplitBoundary creation transforms an existing model such that it will have a range of new properties that are very different from the original model with Boundary objects only:

- New nodes with property storage will be created and the property values will be mapped to them from the original nodes
- Split boundaries decouple computational domains from one another; if nothing else is done, either side becomes a “natural boundary.”
- If split boundaries completely surround a part of a model, this part becomes a stand-alone mesh that needs extra essential conditions as integration constants for the PDE that will be solved.
- Lower-dimensional surface-type subregions can be stored as Boundary or SplitBoundary objects with special properties and connectivity. This also applies to line-element based subregions (such as wells).

7. Comprehensive Simulation Example: Carbonate Dissolution

This chapter uses idealized sequential non-iterative (SNIA) reactive transport calculation to illustrate how the Interrelation, Visitor and PDE_Integrator objects interact with the (data) Model. The computation focuses on the feedback between carbonate dissolution and permeability, which (after thousands of iterations) leads to wormholing, i.e. a positive feedback between flow and permeability increase. This chapter reviews CSMP++ core concepts introduced earlier and is written like a checklist for the implementation of the programme, which is left for the reader as an exercise.

0. INPUT GEOMETRY

Construct a geometrical model of a porous medium with heterogeneities that can serve as an input geometry for the calculation. A two-dimensional model on a small physical length scale (<1m) is recommended because it is easier to examine and faster to compute. Previously discussed options to build the model are:

1. Use NIH Image (current version >1.66) to create a rainbow color-coded representation of the model geometry and save it as TIFF file. Export an additional copy as text file from which a CSMP++ mesher will automatically generate a regular triangular mesh with dimensions 2*pixel rows x 2*pixel columns. For simple runs limit the image dimensions to <50 x 100. The color values will be translated into permeabilities using the factors defined in “utilities/convertColorToPermeability.h.” (see table in Input/Output chapter). Convert this image into a mesh using the **Triangulator** or **Quadrilaterater** meshers.
2. Build a PSLG (planar straight line graph) of the geometry. It will consist of closed polylines with a material point to which its properties are attached. The PSLG is output to a dxf file and converted by the **GeometryPreprocessor** to an input file for the **Triangle** mesher. This mesher produces a “*.poly” file with a constrained, conforming Delauney triangulation (see Model Building & Interface Guide for step by step instructions regarding this procedure).

1. PHYSICAL VARIABLES

The desired physical variables must be communicated to CSMP++. This is accomplished via the text file which defines the **Parameter** classes with will be stored in the **PropertyDatabase**. Name this file “CSMP++_carbonate_variables.txt”. Recall from Chapter 2 that the notation in the file’s headline should match the symbols used for writing the equations, units are SI units, and the index column tells CSMP++ what type of variable shall be constructed (scalar (index=1), vector (2), tensor (3), array(any higher number)). Min. max. define a physically meaningful range for each new variable. All variables used by the RTM must be defined in the physical-variables database.

2. MATERIAL PROPERTIES, INITIAL VALUES, AND BOUNDARY CONDITIONS

For high-level interactions with the Model, one most uses its interfaces (functions) of those of its Region, Boundary, SplitBoundary, and LocalVariableStorage policies. For instance, one may want to set a fluid-volume source term to zero on all elements and

assign Dirichlet boundary pressures on the left side of the geometry. This is done by the following code:

```
//      Model interface, condition, your variable, its desired value
model.InputPropertyValue("fluid volume source", makeScalar(ANY,0.) );
model.InputPropertyValue ("fluid pressure", makeScalar(DIRCH,1.0e+6) );
```

A more flexible solution than hardwiring this in the source code is to use a configuration file (extension: -configuration.txt), ready by the **InputDataManager**, see corresponding section of this guide.

For each PDE at least one Dirichlet-flagged value is required as integration constant, else there is no unique solution. Furthermore any of its coefficients and constitutive relationships have to be initialized, else there NaN (not-a-number) values will be propagated through the computations.

For the carbonate dissolution model, the **model geometry**, and its spatial variations in permeability can be regarded as initial condition and will ultimately determines where “wormholing” by calcite dissolution or “sealing” by calcite precipitation will occur, respectively.

3. INTERRELATIONS

To compute a fluid-pressure distribution by integrating the partial differential pressure equation derived by substituting the volume flux in the basic conservation law with Darcy's law, one must first calculate hydraulic conductivity values from permeability values. This calculation process is formalized via the Interrelation class hierarchy and a library of predefined Interrelation subclasses. To use one of these derived Interrelation objects one must:

1. Include its definition into the program file before main():

```
#include "ConductivityFromPerm.h"
```

2. Create an instance of the Interrelation object:

```
//                                     Pa.s
ConductivityFromPerm<2> relation( p_ref, 1.6 );
```

Here, the viscosity of the fluid is supplied as second argument to the interrelation subclass constructor.

3. Execute the Interrelation by passing it to the Model object:

```
model.Apply( relation );
```

After this step, the variable "*conductivity*" will have been assigned the desired value and pressure already has the correct initial condition. This can be verified with a call to:

```
printRangeOfVariable( model, "permeability" );
printRangeOfVariable( model, "fluid pressure" );
```

4. THE CALCULATION

How to compute a steady-state fluid pressure distribution in a model with spatial variations in the permeability and Dirichlet (=fixed pressure) boundary conditions was already discussed in Chapter 2.

In summary, one first constructs the Model representing the discretized version of the physical model. Secondly, one specifies initial and boundary conditions. Thirdly, one builds a PDE_Integrator with the correct PDE operators for conductance, capacitance and fluid volume sources and sinks. Finally, the computation is carried out and the results are output for display. The computed interstitial flow velocity field is used further by a **NodeCenteredFiniteVolumeTransport** object that advects the concentrations of the aqueous solutes across the finite volume mesh. After each advection step, speciation calculations are carried out with the AlterationVisitor to equilibrate the fluid with the rock.

5. THE REGIONS

In most geologic models one will have the need to treat specific model regions in a unique way to assign properties, restrict the application of Interrelations and/or PDE_Integrators to-, and-or monitor the results. For this purpose, Region objects can be defined in many different ways. Thus,

```
model.FormAndAddRegion( "high perm", "permeability", 1.0e-12, 1.0e-5 );
```

will include those elements of the model into a new region called “high perm”, whose permeability ranges between $1.0\text{e-}12$ and $1.0\text{e-}5 \text{ m}^2$. Note that region names such as “high perm” are allowed to contain white-space. The following statements assign new porosity values to the elements of this new region, distinguishing between the interior and its boundary.

```
Region<3>& hp(model.Region("high perm"));
hp.InputPropertyValue( "porosity", makeScalar(ANY,0.4), INTERIOR );
hp.InputPropertyValue( "porosity", makeScalar(ANY,0.3), PERIMETER );
```

6. THE VISITORS

Remember that the term visitor comes from the software design pattern Visitor. It facilitates a traversal of the Model hierarchy of Regions, Elements, Faces, Interfaces, and Nodes. A visitor can visit their public interfaces when traversing this object hierarchy. The calcite dissolution example, introduces the usage of the “TransportVisitor” and the “AlterationVisitor”.

7. FINITE DIFFERENCE GRIDS

Apart from unstructured-mesh finite-element functionality, CSMP++ also contains stencils for regular grids. These are used, for example, for the output of results as JPEG images. Data stored on the points of a regular grid data can be assigned to finite-elements and vice versa, permitting the transfer of geophysical data from arrays to meshes. Animations can be produced, when pixel-type regular-gridded data are output to JPEG files or pixmaps, using a value to RGB colour conversion after interpolating the values from the finite-element mesh to the grid points. Regular grids can be stored

in `FiniteDifferenceGrid` objects, which also support finite-difference calculations, kernel type operations (like smoothing), image processing etc. To build an instance of a finite-difference grid one writes

```
FiniteDifferenceGrid mygrid;
mygrid.Initialize( x0, x1, y0, y1, RESOLUTION_x, RESOLUTION_y,
                  framewidth_x, framewidth_y );
```

An advantage of such grids is that they require much less storage than a comparably refined finite-element mesh. For this reason the carbonate-dissolution example uses a `FiniteDifferenceGrid` for the advection of aqueous species.

8. SPECIATION CALCULATIONS

In the carbonate dissolution example, reactions are treated as rate laws using a forward rate and a backward rate defined in terms of the inverse of the equilibrium constant. Via the law of mass action, the typical equilibrium expressions are replaced by ordinary differential equations (ODE's) which are written in the form:

$$A + B = C \quad \text{with the ODE for A:} \quad dA/dt = -k_f [A][B] + k_b [C] / K$$

where A and B are the reactants, C is the reaction product, k_f is the forward (far from equilibrium) reaction rate and K is the equilibrium constant. This amounts to a first-order representation of reaction kinetics (Steefel et al., 2005, *EPSL Frontiers*).

Equilibrium is defined such that

$$k_f [A][B] = k_b [C] \quad \text{and} \quad K = [C] / [A][B].$$

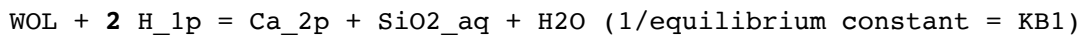
From this follows that

$$K = k_f / k_b.$$

at equilibrium. Hence, k_b is expressed in terms of K . This kinetic approach is outlined in detail in Carl Steefel's article in *Reviews in Mineralogy* 34, p. 93ff. For the carbonate-dissolution example, the system of ODE's is:

$$\begin{aligned} d\text{OH_1m}/dt &= -K_F \text{H_1p OH_1m} + K_{B1} \text{H}_2\text{O_aq} \\ d\text{CO}_2\text{aq}/dt &= -K_F \text{CO}_2\text{aq H}_2\text{O_aq} + K_{B2} \text{HCO}_3\text{1m H_1p} \\ d\text{CC}/dt &= -K_F \text{CC H_1p} + K_{B3} \text{HCO}_3\text{1m Ca_2p} \\ d\text{H_1p}/dt &= -K_F \text{H_1p OH_1m} + K_{B1} \text{H}_2\text{O_aq} + \\ &\quad K_F \text{CO}_2\text{aq H}_2\text{O_aq} + -K_{B2} \text{HCO}_3\text{1m H_1p} + \\ &\quad -K_F \text{CC H_1p} + K_{B3} \text{HCO}_3\text{1m Ca_2p} + \\ d\text{H}_2\text{O_aq}/dt &= K_F \text{H_1p OH_1m} + -K_{B1} \text{H}_2\text{O_aq} + -K_F \text{CO}_2\text{aq H}_2\text{O_aq} + \\ &\quad K_{B2} \text{HCO}_3\text{1m H_1p} \\ d\text{HCO}_3\text{1m}/dt &= K_F \text{CO}_2\text{aq H}_2\text{O_aq} + -K_{B2} \text{HCO}_3\text{1m H_1p} + \\ &\quad K_F \text{CC H_1p} + -K_{B3} \text{HCO}_3\text{1m Ca_2p} \\ d\text{Ca_2p}/dt &= K_F \text{CC H_1p} + -K_{B3} \text{HCO}_3\text{1m Ca_2p} \end{aligned}$$

where the rate of formation is equal to $-\text{forward reaction rate} + \text{backward reaction rate}$, and a simplified text representation is used to cations ($_ \#p$) and anions ($_ \#m$). KBx is the inverse of K . The products in the equations which produce the left-hand species always have a rate coefficient with a negative sign. Also, as follows from the law of mass action, stoichiometric factors in the conventional equations like $2A+B=C$ translate into dependent variable exponents ($2A \rightarrow A^2$). Stoichiometric factors must also be taken into account in the rate coefficients. Thus, if a reaction produces $2A$, this implies for the total production/consumption rate of A that this reaction contributes twice the amount of A . Consequently, both, forward and backward rates must be multiplied by 2 in any expression involving A . This is illustrated using a wollastonite producing reaction:



The contribution of this reaction to an ODE which describes the H^+ concentration is:

$$d \text{H_lp} / d t = 2 -K_F \text{WOL} \text{H_lp}^2 + 2 \text{KB1} \text{Ca_2p} \text{SiO2_aq} \text{H2O_aq} + \dots$$

since, during every reactive exchange, 2 moles of H^+ are produced or consumed. Stoichiometric factors must be multiplied into forward and backward rates, since the equation parser in CSMP's ODE solver cannot deal with 2 coefficients in a single product term. Alternatively, the number 2 is defined as an additional independent variable. This, however, is more costly than just generating

$$\text{KB1x2} = 2 * \text{KB1}$$

The reaction ODEs are integrated over the reaction time interval with a Burlisch-Stoer solution scheme for stiff ODE's (Bader and Deuflhard 1986). This scheme is implemented as a "ODE_StiffSolver" object which has a method that reads in the example text file "calciteOH_25.txt" with the definitions of the ODEs from above. The equilibrium constants and forward reaction rates for the corresponding reactions at 25°C are also defined in this file.

9. INITIAL CONDITIONS

Before the simulation of reactive transport is started, the fluid on the grid is equilibrated with the rock via a visitation of each finite element by the **AlterationVisitor**, which contains the **ODE_StiffSolver**. Before this visit, the concentrations on the transport grids are initialized to zero and the equilibration time given to the visitor is set to a very large value (in seconds) to ascertain that the fluid fully equilibrates with the rock. The speciation calculations are then carried out element by element when the **AlterationVisitor** visits them. The code which specifies how this is done is contained in the method,

```
virtual void AlterationVisitor::Visit( Element* e );
```

a purely virtual function of the base class `csmp::Visitor`.

This specific implementation of the `Visit()` method does the following:

1. Read the speciation of the fluid at the last timestep from the `Node` and assign it as initial condition to the ODE solver object (this requires a conversion of the concentrations from kg/m³ to mol/kg).

2. Obtain the new speciation in 1 kg of fluid (molality of each aqueous species).

```
// maxtime is just the time needed for full equilibration (no other meaning)
//          maxtime      init.t-step  save-steps  desired tolerance
solver.Solve( time_increment, 1.0e-17, 0, fmol*tolerance );
```

3. Calculate how much calcite was dissolved from the difference of the Ca⁺² molality between the old and the new speciation in 1 kg of fluid. Convert this amount to a unit volume of the fluid-rock assembly. Convert molalities back to concentrations and store the result back on the node.

```
// dissolved calcite is derived from change in Ca+2 concentration
// result is converted into kg
// information is stored in the basis species vectors
n->Store( pmem, cc_key.index,
          (sc=BASIS[2]+(BASIS[6]-solver(7))*100.08*1.0e-3*fmass) );
```

10. THE OVERALL MODEL

In the main program, reactive transport is simulated via a sequential iteration approach (SIA) as discussed by Yeh and Tripathi (1989, 1991) and summarized in Steefel (1996, see reference above). This technique means that transport and speciation steps are decoupled. An additional complexity arises as calcite-dissolution increases porosity which, in turn, increases the permeability. This is implemented via two Interrelation subclasses 'PorosityChange' and 'PermeabilityFromPorosity'. They are passed to the Model after each transport- and equilibration- pair of steps. Subsequently, the Darcy flow velocity distribution at $t+1$ is recomputed as was discussed in Chapter 2.

The carbonate dissolution alters the previous fluid-velocity distribution to the model state at $t = \text{previous time} + \text{time_increment}$.

To drive the chemical reactions, after each advection step, an acid but charge-balanced fluid composition is specified on the left margin of the grids. This acidic fluid is advected to the right into the cross-sectional model where it dissolves carbonate until it is neutralized by the rate-controlled reaction with calcite.

The model is run while the H⁺ concentration on the first of the grids and the “porosity” are output from time to time to DXF and VTK files, respectively, to monitor the progress of carbonate dissolution.

Exercises

- Run model with different inlet conditions,
- Remove the FD grid from the calculations and replace it with a finite volume scheme.

8. The CSMP++ Libraries

The API library is organised in layers of increasingly higher-level and specific functionality that it provides. The model/ directory contains the building blocks of the discretized geomodel and the base classes for most objects, which are tabulated in the *CSMP++ Distribution Section* of Appendix A. Specific applications (not listed here) build on top of the API library. In the github repository of the University of Melbourne (<https://csmp.ereseach.unimelb.edu.au>) the source code is organized in the following structure (find . -type d -maxdepth 3):

source/includes/

```
./includes/analysis
./includes/run_control
./includes/integration
./includes/integration/finite_elements
./includes/integration/pde_operators
./includes/integration/solver
./includes/integration/finite_volumes
./includes/integration/pde_integrators
./includes/property_manipulation
./includes/thmc
./includes/thmc/mechanics
./includes/thmc/two_phase_flow
./includes/thmc/single_phase_flow
./includes/thmc/fractures
./includes/thmc/geochemistry
./includes/discretization
./includes/math
./includes/regular_grids
./includes/utilities
./includes/model
./includes/eos
./includes/interfaces
./includes/interfaces/Rhinoceros
./includes/interfaces/Fracman
./includes/interfaces/Eclipse
./includes/interfaces/Maple
./includes/interfaces/ANSYS
./includes/interfaces/CGNS
./includes/interfaces/Tecplot
./includes/interfaces/VTK
./includes/interfaces/Matlab
./includes/interfaces/GeoModeller
./includes/interfaces/gOcad_SKUA
./includes/interfaces/JPEG
```

content examples

```
RegionMonitor, StatisticalAnalyser
ErrorHandler, ComputationalSettings

FiniteElementPolicy, FiniteElement
NumIntegral_dNT_op_dN_dV
SAMG_Solver
FiniteVolumePolicy
IterativeIntegrator, TrasientDiffusor
ModifyPropertyWhere, PropertyHandle

StressesAndStrains, BrittleFailure
TwoPhaseModel, TwoPhaseFlowFunctions
parallelPlatePermeabilityFromChannelWidth
HydroFractureVisitor, FourarLenormand
AlterationVisitor, QuartzSolubility
Quadrilaterater, mjl::Point
CubicSpline, RandomFieldGenerator
FiniteDifferenceGrid, CornerPointGrid
CSMP_highLevelUtilities, vsetMakers
Node, Element, Model
HaliteLiquidus, H2OLookup
TextFileIO, TRIANGLE_Interface
```

source/modules/

```
./modules/analysis
./modules/run_control
./modules/integration
./modules/integration/finite_elements
./modules/integration/pde_operators
./modules/integration/solver
./modules/integration/finite_volumes
./modules/integration/cvfe_hydrothermal
./modules/integration/pde_integrators
./modules/property_manipulation
./modules/property_manipulation/interrelations
./modules/thmc
./modules/thmc/mechanics
./modules/thmc/two_phase_flow
./modules/thmc/single_phase_flow
```

```

./modules/thmc/fractures
./modules/thmc/geochemistry
./modules/discretisation
./modules/math
./modules/regular_grids
./modules/utilities
./modules/model
./modules/eos
./modules/eos/h2o
./modules/eos/h2o_nacl_co2
./modules/eos/h2o_nacl_lookup
./modules/eos/h2o_nacl
./modules/interfaces
./modules/interfaces/Rhinoceros
./modules/interfaces/Fracman
./modules/interfaces/Eclipse
./modules/interfaces/Maple
./modules/interfaces/ANSYS
./modules/interfaces/CGNS
./modules/interfaces/Tecplot
./modules/interfaces/VTK
./modules/interfaces/Matlab
./modules/interfaces/GeoModeller
./modules/interfaces/g0cad_SKUA
./modules/interfaces/JPEG

```

The source code (source/) is partitioned into an include directory with the header files (declarations) and a module directory with the source files (definitions). Rather than defining template functionality in headers, it is also defined in the sources and explicit template instantiation is used to force compilation and object creation for the templates. Part of the motivation for this was the reduction of compile time, the exclusion of instantiation of templates with arguments that are not meaningful, and increased readability of the code.

The general directory structure of the Melbourne repository is:

directory	contents
./bin	binaries (OSX, windows, linux)
./development	design ideas, strategy papers etc.
./source	
./tests	
./tests/unit	
./tests/dataset	pool of test data
./tests/integration	tests of composite functionality
./tests/doc	
./tests/data	
./examples	source code and data for the examples
./examples/source	
./examples/data	
./doc	CSMP_UserGuide, DevelopersGuide etc.
./doc/figures	in the applications they were created in
./build	(OSX, windows, linux)
./thirdparty	
./thirdparty/iaps_h2o_eos	
./thirdparty/samg	
./thirdparty/meschach	
./thirdparty/catch	
./thirdparty/jpeg	
./reviews	
./deprecated	

The API library is gradually supplemented by additional objects from developers. These are shared with other CSMP++ developers and users such that they can tap this growing reservoir in order to build increasingly realistic simulations.

Code Examples

A library of CSMP++ examples have been created to demonstrate the capabilities of CSMP++. To run them, they must be compiled and linked with the CSMP++ API library. Their source code is contained in the `examples/` directory of the API and combines classes and functions from the library into a main program. Currently, there are the following examples

Averaging_Example.h	
ConcentrationFluidVolumeSource.h	
CuriouslyRecurringTemplate_Example.h	
DES2PhaseSlightlyCompressibleFlow2D_Example.h	
DES2PhaseSlightlyCompressibleFlow3D_Example.h	
DESAdvectionDiffusion2D_Example.h	
DESAdvectionDiffusion3D_Example.h	
DenseMatrix_Example.h	
EclipseMeshInterface_Example.h	
EffectiveStressDilatation2D_Example.h	
ErrorMetric_Example.h	
Example.h	example base class
ExampleSuite.h	functionality to run the examples
Examples.h	header in which new examples must be registered
Experimental_Example.h	empty example template to create your own
FluidDensityFromConcentration.h	
Geothermal_Example.h	
GroundwaterDarcyVelocity.h	
HydrostaticPressure.h	
LinearElasticity_Example.h	
LinearSolver_Example.h	
MeshRenumberVisitor.h	
ModelANSYS_Example.h	
PassiveAdvectionOfTracer_Example.h	
PermeabilityTensor_Example.h	
PolicyBased_Example.h	
PressureDiffusion_Example.h	
PressureSaturationInitializer.h	
QuadraticPressure_parallelPlatePermeability_Example.h	
ReadingBinaries_Example.h	
RegionProperties_Example.h	
Region_Example.h	
RhinoMesh_Example.h	
SlopeMechanics_Example.h	
StatisticalAnalyzer_Example.h	
SteadyStatePressureToVset_Example.h	
SteadyStatePressure_Example.h	
StokesDiscrepancyMeasureQuadratic_Example.h	
StokesDiscrepancyMeasure_Example.h	
StreamFunction_Example.h	
TDDUnitTest_Example.h	
TemperatureDensityPressure_Example.h	
TemplatizedIndex_Example.h	
TimeSteppingApproaches_Example.h	
TopographyDrivenFlow_Example.h	
TransientPressure_Example.h	
Tutorial1_Example.h	
Tutorial2_Example.h	
Tutorial3_Example.h	
Tutorial4_Example.h	
Tutorial4_Example_Revisited.h	
VariablePlacement_Example.h	
VariablesBasic_Example.h	
Variables_Example.h	
Visitor_Example.h	

The current examples are all documented in their Specification() methods. For the VisitorExample this reads:

```
void Visitor_Example::Specifications()
{
    SetTitle( "Visitor" );
    SetDifficulty( 2 );
    SetCategory( "Software Functionality" );
    AddAuthor( "Shaho" );
    AddDescription( "source in: Visitor_Example.cpp" );
    AddDescription( "application and implementation of csmp::Visitor" );
    AddRequirement( "LeftRight .asc,.dat,-regions.txt" );
    AddRequirement( "VisitorExample-var.txt" );
}
```

This source code shows that the examples are grouped into 3 categories:

1. C++ / software functionality
2. Simulation of physical processes
3. Numerical methods

And it also reveals that the examples are ranked by their difficulty ranging from simple (1) to advanced (3).

Like any CSMP++ class, each example consists of a header file which contains the declaration (*.h) and a source code file (*.cpp) that contains the definition of the class. Only developers have access to the definitions. They have the same name as the class contained in them.

In the following selected functionality of the API library is presented in more detail and combined with introductory material concerning the modelling process. This section is organized following the directory structure of the library.

/SOURCE/INCLUDES/INTEGRATION/PDE_INTEGRATORS LIBRARY

```
IterativeIntegrator.h
NewtonIntegrator.h
PDE_Integrator.h
PDE_IntegratorExperimental.h
PDE_Integrator_UoM.h
PicardIntegrator.h
SteadyStateDiffusor.h
TransientDiffusor.h
```

PDE_Integrators can be assembled for forward, central, and backward difference time stepping. In matrix notation these different procedures as illustrated for fluid pressure (p) diffusion (see Chapter 2) are:

Forward Difference (=explicit)

$$[C]\{p\}_{t+\Delta t} = ([C] - \Delta t[K])\{p\}_t + \Delta t \{Q\}_t$$

Central Difference (semi-implicit = Crank-Nicholson)

$$([C] + \Delta t/2[K])\{p\}_{t+\Delta t} = ([C] - \Delta t/2[K])\{p\}_t + \Delta t/2 (\{Q\}_t + \{Q\}_{t+\Delta t})$$

Backward Difference (=implicit = Backward-Euler)

$$([C] + \Delta t[K])\{p\}_{t+\Delta t} = [C]\{p\}_t + \Delta t \{Q\}_{t+\Delta t}$$

Here C , K , and Q refer to the capacitance, conductance and fluid source matrices.

These different finite-difference time-stepping schemes are illustrated in the:

TimeSteppingApproaches_Example.h

/SOURCE/INCLUDES/MATH/ LIBRARY

```
AP_algebraUtilities.h
CSMP_mathUtilities.h
CSMP_random.h
CompressedRowMatrix.h
CompressedRowMatrixParallel.h
ConsecutiveSequenceChecker.h
CubicSpline.h
DenseMatrix.h
FixedArray.h
LUdcmp_Solver.h
NewtonRaphson.h
ODE_StiffSolver.h
RandomFieldGenerator.h
SparseMatrix.h
SumOfProducts.h
SumOfProductsWithExponents.h
compareFloats.h
denseMatrixMethods.h
tensorOperations.h
vectorOperations.h
```

Importantly, this classes contain only a small subset of the numerical methods available in CSMP, see the other directories on integration etc.

/SOURCE/INCLUDE/DISCRETISATION/ LIBRARY

LineElementMesher.h
Quadrilateraterator.h
Triangulator.h

MJL_CSMP_glue.h
MJL_Edge.h
MJL_Edge3D.h
MJL_Node.h
MJL_Plane3D.h
MJL_Point.h
MJL_Point3D.h
MJL_Polygon.h
MJL_Triangle3D.h
MJL_Vertex.h
MJL_functions.h
MJL_geometry.h

Apart from the CSMP++ native yet primitive mesh generation classes, this directory includes an implementation of M. J. Lazlo's computational geometry library.

/SOURCE /INCLUDES/ANALYSIS/ LIBRARY

```
MeshDiagnostics.h  
NodeCenteredFiniteVolumeMonitor.h  
NodeMonitor.h  
RegionMonitor.h  
StatisticalAnalyzer.h
```

The monitors have an important role in collecting results from transient simulations and storing them in files.

/SOURCE/INCLUDES/EOS/ (EQUATION OF STATE) LIBRARY

Brine.h
ConvertConcentrationUnitsNaCl.h
CriticalCurve.h
CriticalCurveLookup.h
CriticalPointH2O.h
DynamicViscosity.h
EOS_CO2H2ONaCl_Spycher2004.h
Fluidproperties.h
GetLookupIndices.h
H2OFluidProperties.h
H2OLookup.h
H2ONaClFluidProperties.h
H2ONaClLookup.h
H2ONaClThermalEquilibrator.h
H2OPropertiesLookupTable.h
H2OThermalEquilibrator.h
Halite.h
HaliteLiquidus.h
HaliteLiquidusLookup.h
HaliteLookup.h
IAPWS_H2OPropertiesVisitor.h
IAPWS_H2O_Conductivity.h
IAPWS_H2O_Density.h
IAPWS_H2O_HeatTransferVelocity.h
IAPWS_H2O_Storativity.h
IAPWS_H2O_dVdT_Source.h
LH_HaliteLookup.h
LookupStorage.h
LookupPropertyIndex.h
NaClBoilingCurve.h
NaClMeltingCurve.h
NaClMeltingCurveHaliteLookup.h
NaClMeltingCurveLiquidLookup.h
NaClSaturatedVapor.h
NaClSaturatedVaporLookup.h
NaClSublimationCurve.h
PVTX_Calculator_H2O_CO2_NaCl.h
PhaseStateFinder_H2O_CO2_NaCl.h
Rock.h
States.h
ThreephaseHLV.h
TriplePointH2O.h
TriplePointNaCl.h
TwophaseLiquid.h
TwophaseLiquidLookup.h
TwophaseVapor.h
TwophaseVaporLookup.h
VH_HaliteLookup.h
VLH_HaliteLookup.h
VLH_LiquidLookup.h
VLH_VaporLookup.h
Water.h

Additional non-CSMP files are contained in the `thirdparty/` directory, such as the IAPS C code etc.

This library contains the implementations of the equation of state for pure H₂O and NaCl-H₂O, as well as interfaces to them, and Visitor and Interrelation classes for fluid property calculations, and a class to generate a lookup tables for pure H₂O fluid properties (Fig. 3). Using the eos/ library, fluid and transport properties for single-phase thermal and thermohaline convection simulations can be calculated.

The C-library PROST (Bauer, 2002), which has implemented the NBS/NRC Steam Tables (Haar et al., 1984), is included in the thirdparty/ library to compute fluid properties of pure H₂O. The H₂OPropertiesLookUpTable class employs PROST to generate a 105 MB lookup table for the pure H₂O properties density, enthalpy, viscosity, compressibility, isobaric heat capacity, and thermal expansivity. The lookup table is stored as a binary file generated from a FiniteDifferenceGrid object. It is valid from 5 to 800°C and 1 to 4000 bar pressure. Bi-linear interpolation is used to extract the fluid properties. The grid spacing is 1.0°C and 2.5 bar away from the critical point of H₂O and 0.1°C and 0.1 bar in its vicinity. In addition, the properties for the liquid and steam phase can be obtained along the saturation curve of pure H₂O for a given temperature or pressure value. The error of the interpolated fluid properties is below 0.01 % close to the critical point and below 0.0001 % away from it.

The IAPWS_H2O_Density class is an interface to PROST and may be used to obtain fluid properties as a function of temperature and pressure. The IAPWS_H2OPropertiesVisitor class employs PROST to compute nodal fluid properties as a function of nodal pressure and temperature values. The object terminates a simulation if non-physical, i.e. negative pressure or temperature values are encountered. It does not make a distinction between liquid and vapor. Hence it cannot be used to accurately model two-phase liquid-steam flow. The Interrelation classes IAPWS_H2O_Storativity and IAPWS_H2O_dVdT_Source compute the element variables 'storativity', respectively 'thermal expansion', as a function of the fluid and rock compressibilities, respectively expansivities. The Interrelation object IAPWS_H2O_Conductivity calculates the element variable 'conductivity' from the element permeability and the nodal fluid viscosities. The Interrelation object IAPWS_H2O_HeatTransferVelocity calculates the element variable 'heat transfer velocity' from the relation $(v c_p \rho) / ((1-\phi) c_{pr} \rho_r + \phi c_p \rho)$ using the element Darcy velocity, rock density, and rock heat capacity as well as the nodal fluid density and fluid viscosity.

The NaCl-H₂O equation of state can be accessed through the interface Fluid. The phase state of the fluid and the properties density, enthalpy, viscosity, saturation, heat capacity, mass fraction salt, and compressibility are returned for the brine, vapor, and halite phase as a function of pressure, temperature, and salinity. If a phase does not exist, its values are set to zero. The NaCl-H₂O equation of state employs a H₂OPropertiesLookUpTable object to retrieve the properties of pure H₂O, because pure H₂O properties must be computed repeatedly at small temperature, pressure, and salinity increments away from the values of interest. The object NaClH2O_PropertiesVisitor calculates nodal fluid properties as a function of nodal pressure, temperature, and salinity. It terminates a simulation if non-physical, i.e. negative, pressure or temperature values are encountered. A NaClH2O_PropertiesVisitor object does not return properties of a vapor phase. Instead, the simulation is terminated if vapor is present.

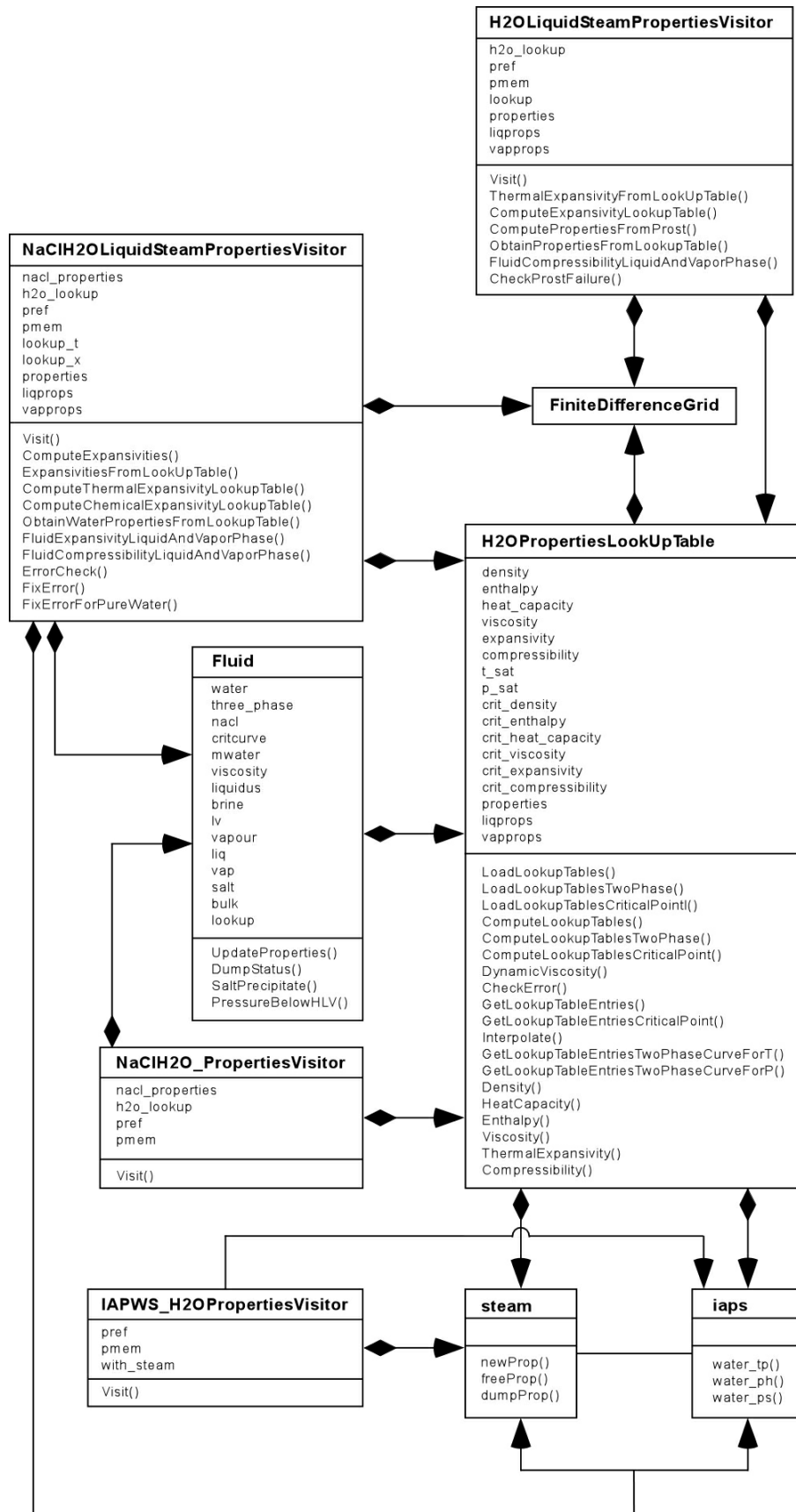


Fig. 9.1.: UML class diagram for the calculation of thermodynamic fluid properties. Associations between classes are denoted by a bold line. The arrow notes the direction of flow of the associations. The diamond denotes a composition.

The object `NaClH2O_TransportPropsVisitor` computes the element variables 'conductivity', 'storativity', and 'total source' as a function of nodal fluid and element rock properties. The Interrelation object `SalinityFromMassFractionSalt` converts the mass fraction salt into salinity, accounting for the possible presence of halite.

/SOURCE /INCLUDES/INTEGRATION/FINITE ELEMENTS/ LIBRARY

```

CSMP_ElementSpecifications.h
FiniteElement.h
FiniteElementPolicy.h
IsoparametricLinearHexahedron.h
IsoparametricLinearLineElement.h
IsoparametricLinearPrism.h
IsoparametricLinearPyramid.h
IsoparametricLinearQuadrilateral.h
IsoparametricLinearTetrahedron.h
IsoparametricLinearTriangle.h
IsoparametricQuadraticHexahedron.h
IsoparametricQuadraticLineElement.h
IsoparametricQuadraticPrism.h
IsoparametricQuadraticPyramid.h
IsoparametricQuadraticQuadrilateral.h
IsoparametricQuadraticTetrahedron.h
IsoparametricQuadraticTriangle.h
LinearCuboid.h
LinearLineElement.h
LinearRectangle.h
LinearTetrahedron.h
LinearTriangle.h
LinearTriangle3D.h
StabilizationParameterVisitor.h
VData.h
VSet.h
VSetConverter.h

```

This documentation only comprises the triangular-tetrahedral family of finite elements. Other types are documented in the developer's guide. Triangular-tetrahedral elements comprise the types, the `LinearTriangle`, `LinearTriangle2D` (3-nodes), the `IsoparametricQuadraticTriangle` (6-nodes), the `IsoparametricQuadraticBarycentricTriangle` (7-nodes), the `LinearTetrahedron` (4-nodes), and the `IsoparametricQuadraticTetrahedron` (10-nodes). Linear triangular and tetrahedral input meshes can be transformed for the quadratic element types with the `CSMP++_VSetConverter`. Hybride meshes with both triangular and tetrahedral elements have been used to model fluid flow in highly permeable but narrow fault zones.

The Linear Triangular Element: `LinearTriangle`

Linear interpolation functions, L , for the triangular element are equivalent to the *area*- or so-called *natural coordinates* of a straight-sided triangle

$$L_1 = (a_1 + b_1x + c_1y) / 2A$$

$$L_2 = (a_2 + b_2x + c_2y) / 2A$$

$$L_3 = (a_3 + b_3x + c_3y) / 2A$$

where a, b, c are linear coefficients and x, y are the Cartesian coordinates of the nodes of the triangle. The sums in L are divided by two-times-the-area, A of the triangle to achieve that L_1 to L_3 sum up to 1. The area coordinates of a point x, y in a triangle are linearly related to its Cartesian coordinates. Thus

$$\begin{Bmatrix} 1 \\ x \\ y \end{Bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{bmatrix} \begin{Bmatrix} L_1 \\ L_2 \\ L_3 \end{Bmatrix}$$

Here, the indexing of the Cartesian coordinates in the matrix refers to the node numbering in the triangle (Fig. 1).

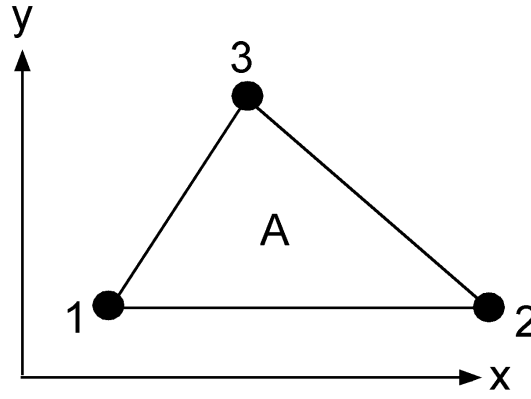


Fig. 9.2: Counterclockwise node numbering in linear triangular element.

The coefficients of the area-coordinate functions are calculated from the Cartesian coordinates of the nodes n_1 to n_3

$$a_1 = x_2 y_3 - x_3 y_2$$

$$a_2 = x_3 y_1 - x_1 y_3$$

$$a_3 = x_1 y_2 - x_2 y_1$$

$$b_1 = y_2 - y_3$$

$$b_2 = y_3 - y_1$$

$$b_3 = y_1 - y_2$$

$$c_1 = x_3 - x_2$$

$$c_2 = x_1 - x_3$$

$$c_3 = x_2 - x_1$$

The area of the triangle is

$$A = \frac{1}{2} (x_2 y_3 + x_1 y_2 + x_3 y_1 - x_2 y_1 - x_3 y_2 - x_1 y_3)$$

The spatial derivatives of the interpolation functions ($N=L$) in generic matrix form are

$$[dN] = \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_2}{\partial x} & \frac{\partial N_3}{\partial x} \\ \frac{\partial N_1}{\partial y} & \frac{\partial N_2}{\partial y} & \frac{\partial N_3}{\partial y} \end{bmatrix} = \begin{bmatrix} b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}$$

This matrix is expanded for linear elasticity problems, where each node has 2 degrees of freedom (x,y displacements). This expanded form is also known as the '**B**' matrix:

$$[B] = \begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix}$$

For the linear triangle, the interpolation functions can be integrated analytically over the area of the element, using the general formula

$$\int_A L_1^l L_2^j L_3^k dA = 2A \frac{l!j!k!}{(2+l+j+k)!}$$

where l, j, k represent exponents of the area-coordinate functions. These exponents are equivalent to the degree of the interpolation functions and their products as they arise during matrix multiplication. To illustrate the use of the integration formula for a single entry into the finite-element matrix integral consider

$$\int_A N_1 \frac{\partial N_1}{\partial x} dA = \frac{b_1}{2A} \int_A N_1 dA = \frac{b_1}{2A} 2A \frac{1!0!0!}{(2+1+0+0)!} = \frac{b_1}{6}$$

As the partial derivative of the interpolation function with respect to x is a constant, it can be pulled out of the integral, and, since the interpolation functions N_2 and N_3 do not appear in the equation, their exponents are zero.

For the conductance matrix K integration in this manner for all terms gives

$$[K]^{(e)} = \frac{K_x}{4A} \begin{bmatrix} b_1^2 & b_1 b_2 & b_1 b_3 \\ b_2 b_1 & b_2^2 & b_2 b_3 \\ b_3 b_1 & b_3 b_2 & b_3^2 \end{bmatrix} + \frac{K_y}{4A} \begin{bmatrix} c_1^2 & c_1 c_2 & c_1 c_3 \\ c_2 c_1 & c_2^2 & c_2 c_3 \\ c_3 c_1 & c_3 c_2 & c_3^2 \end{bmatrix}$$

The LinearTetrahedron

The interpolation functions for the LinearTetrahedron are implemented similar to those of the linear triangle and the isoparametric barycentric triangular element. The only difference is that a third spatial coordinate is incorporated. This turns the interpolation functions into volume coordinates instead of area coordinates

$$L_i = (a_i + b_i x + c_i y + d_i z) / 6V, \quad L_i(x_j, y_j, z_j) = \delta_{ij}$$

Where δ_{ij} (Kronecker-Delta) simply means that the interpolation functions are orthogonal to the weighting functions. Again, the normalization by $6V$ ascertains that the four orthonormal interpolation functions L sum up to 1 anywhere in the tetrahedron

$$\sum_{i=1}^4 L_i(x, y, z) = 1$$

Cartesian (x, y, z) and volume (L) coordinates have the relation

$$\begin{Bmatrix} 1 \\ x \\ y \\ z \end{Bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ x_1 & x_2 & x_3 & x_4 \\ y_1 & y_2 & y_3 & y_4 \\ z_1 & z_2 & z_3 & z_4 \end{bmatrix} \begin{Bmatrix} L_1 \\ L_2 \\ L_3 \\ L_4 \end{Bmatrix}$$

The Cartesian coordinate indices correspond to the four nodes of the tetrahedron. The determinant of the square (4×4) matrix is equivalent to $6V$, if the nodes are numbered so that the sequence 1-2-3 runs counterclockwise when viewed from node 4 (Fig. 19).

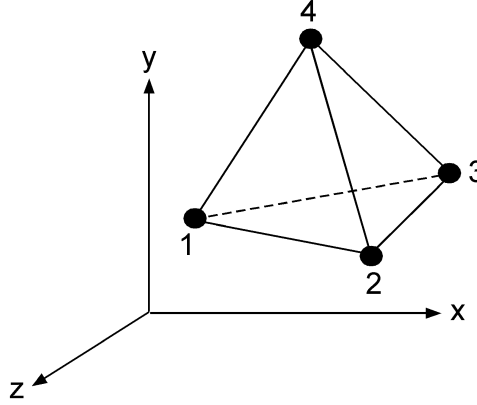


Fig. 9.3: Four-noded tetrahedral finite element in an right-handed coordinate system.

The coefficients for the basis functions are computed from the global node coordinates

$$a_1 = -x_2(y_4z_3 - z_4y_3) - x_3(y_2z_4 - z_2y_4) - x_4(y_3z_2 - z_3y_2)$$

$$b_1 = (y_4z_3 - z_4y_3) + (y_2z_4 - z_2y_4) + (y_3z_2 - z_3y_2)$$

$$c_1 = (z_4x_3 - x_4z_3) + (z_2x_4 - x_2z_4) + (z_3x_2 - x_3z_2)$$

$$d_1 = (x_4y_3 - y_4x_3) + (x_2y_4 - y_2x_4) + (x_3y_2 - y_3x_2)$$

The remaining coefficients are obtained by cyclic permutations of the indices with a sign change for each permutation.

The volume of the tetrahedron is

$$V = (a_1 + a_2 + a_3 + a_4) / 6$$

Differentiation of the interpolation functions gives

$$\frac{\partial L_i}{\partial x} = \frac{b_i}{6V} \quad \frac{\partial L_i}{\partial y} = \frac{c_i}{6V} \quad \frac{\partial L_i}{\partial z} = \frac{d_i}{6V}$$

The general analytical formula for the integration is

$$J(k, l, m, n) = \frac{1}{V} \int L_1^k L_2^l L_3^m L_4^n dx = 6V \frac{k!l!m!n!}{(3+k+l+m+n)!}$$

where k, l, m, n are equivalent to the exponents of the interpolation functions L_1 to L_4 . Each exponent is unity for linear, 2 for quadratic, and 3 for cubic interpolation functions. The factor 3 in the denominator, is equal to the space dimension of the element. When an interpolation function integral only contains first order terms, the integral reduces to

$$I(1,1,1,1) = 6V \frac{1}{5040} = 6V 1.1904762 \times 10^{-3}$$

($0! = 1$). Evaluating, the volume integral $\int_V r N dV$, one arrives at

$$\begin{aligned} [I] &= \int_V \begin{bmatrix} N_1 \\ N_2 \\ N_3 \\ N_4 \end{bmatrix} r \begin{bmatrix} N_1 & N_2 & N_3 & N_4 \end{bmatrix} dV = r \int_V \begin{bmatrix} N_1 \\ N_2 \\ N_3 \\ N_4 \end{bmatrix} \begin{bmatrix} N_1 & N_2 & N_3 & N_4 \end{bmatrix} dV \\ &= r \int_V \begin{bmatrix} N_1^2 & N_1 N_2 & N_1 N_3 & N_1 N_4 \\ N_2 N_1 & N_2^2 & N_2 N_3 & N_2 N_4 \\ N_3 N_1 & N_3 N_2 & N_3^2 & N_3 N_4 \\ N_4 N_1 & N_4 N_2 & N_4 N_3 & N_4^2 \end{bmatrix} dV \end{aligned}$$

Individual product terms are integrated as follows

$$\begin{aligned} \int_V N_1 N_1 dV &= \int_V N_1^2 dV = 6V \frac{2!0!0!0!}{(3+2+0+0+0)!V} = \frac{1}{10} V \\ \int_V N_1 N_2 dV &= 6V \frac{1!1!0!0!}{(3+1+1+0+0)!V} = \frac{1}{20} V \end{aligned}$$

and so forth, until one arrives at the usual integrals like that over the interpolation function products

$$[I] = r \begin{bmatrix} \frac{1}{10} & \frac{1}{20} & \frac{1}{20} & \frac{1}{20} \\ \frac{1}{20} & \frac{1}{10} & \frac{1}{20} & \frac{1}{20} \\ \frac{1}{20} & \frac{1}{20} & \frac{1}{10} & \frac{1}{20} \\ \frac{1}{20} & \frac{1}{20} & \frac{1}{20} & \frac{1}{10} \end{bmatrix} V$$

Integration exponents for interpolation function derivatives are 0 since these functions are linear for the LinearTetrahedron.

The Isoparametric Quadratic Triangle with 6 Nodes

Quadratic triangular elements (Fig. 19) are used very often for elasticity problems. The quadratic order of the interpolation functions of this element makes displacement derivatives continuous across element boundaries (not across corner nodes) and special ways of numerical integration can rule out zero-energy modes of deformation, which lead to unrealistic results. The element can also be constructed for three-dimensional computations in which case 3D coordinates are transformed into the plane of the 2D local reference element. In linear-elasticity applications, mesh convergence can usually be obtained in contrast with linear elements, which give appalling results. The quadratic interpolation implies that displacement derivatives are represented as bilinear fields inside each element.

In the following, the steps are discussed of how one solves a linear elasticity problem using quadratic triangular finite elements, including the post-processing calculation of strains and stresses at the elements integration points.

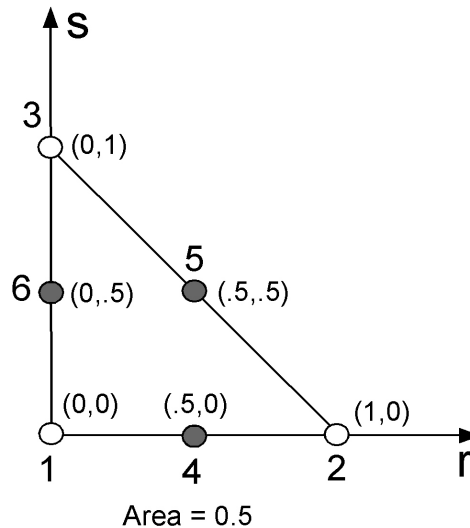


Fig. 9.4: Reference (unit) quadratic triangle showing node numbering and local coordinates. The black midside nodes are used as integration points.

Given the reference triangle shown in Figure 20, interpolation functions N_i (= shape functions since the isoparametric approach is used) in the local coordinate system, can be defined as:

$$\begin{aligned}
 N(3) &= 4. * r * (1. - r - s) \\
 N(4) &= 4. * r * s \\
 N(5) &= 4. * s * (1. - r - s) \\
 N(0) &= 1. - r - s - N(3)/2. - N(5)/2. \\
 N(1) &= r - N(3)/2. - N(4)/2. \\
 N(2) &= s - N(4)/2. - N(5)/2.
 \end{aligned}$$

and their spatial derivatives dN/dr , dN/ds , with regard to the local reference frame:

$$\begin{aligned}
 DNS(0) &= -3. + 4.*r + 4.*s \\
 DNS(1) &= 0. \\
 DNS(2) &= -1. + 4.*s
 \end{aligned}$$

```

DNS(3) = -4.*r
DNS(4) = 4.*r
DNS(5) = 4. - 4.*r - 8.*s
DNR(0) = -3. + 4.*r + 4.*s
DNR(1) = -1. + 4.*r
DNR(2) = 0.
DNR(3) = 4. - 8.*r - 4.*s
DNR(4) = 4.*s
DNR(5) = -4.*s

```

Jacobian Transformation. The use of a local coordinate system makes it necessary to transform element contributions, which are calculated on the reference element, into the global coordinate system. This is achieved via a Jacobian coordinate transformation. For this purpose one needs to form a Jacobian matrix, $\mathbf{J}$, with the size of spatial dimensions x spatial dimensions (2 x 2 in 2D):

$$\mathbf{J}(r,s) = \begin{bmatrix} \frac{\partial N_1}{\partial r}(r,s) & \dots & \frac{\partial N_6}{\partial r}(r,s) \\ \frac{\partial N_1}{\partial s}(r,s) & \dots & \frac{\partial N_6}{\partial s}(r,s) \end{bmatrix} \begin{bmatrix} x_1 & y_1 \\ \vdots & \vdots \\ x_6 & y_6 \end{bmatrix}$$

$\mathbf{J}$ can be calculated as the product of the shape-function derivative matrix, $\mathbf{DN}$, calculated at the point (r,s) in the local coordinate system, and the global node-coordinate matrix, $\mathbf{XY}$ (cf. Zienkiewicz, Vol I, (4th ed.) p. 162). With the inverse of $\mathbf{J}$ one can carry out the desired transformation of the local to the global shape-function derivative matrix, $\mathbf{DN}(x,y)$, but $\mathbf{J}^{-1}$ is only valid at the transformation point (x,y) of the local point (r,s) where the local shape-function derivatives were calculated. Therefore, $\mathbf{J}^{-1}$ must be calculated again at each point where derivatives shall be transformed into the global coordinate frame.

In summary, to calculate the global shape-function derivative matrix $\mathbf{DN}(x,y)$, at a point (x,y) representing the mapping of point (r,s), the following steps must be performed:

1. Form $\mathbf{DN}(r,s)$ and $\mathbf{XY}$,
2. Form $\mathbf{J} = \mathbf{DN}(r,s) * \mathbf{XY}$
3. Invert $\mathbf{J}$
4. Premultiply $\mathbf{J}^{-1}$ with $\mathbf{DN}(r,s)$: $\mathbf{DN}(x,y) = \mathbf{J}^{-1} * \mathbf{DN}(r,s)$

As a next step, depending on the degrees of freedom per node of the problem which shall be solved, $\mathbf{DN}(x,y)$ must be expanded into a correspondingly larger matrix, $\mathbf{B}$. Taking a linear elasticity problem in 2D, the nodal degree of freedom is 2, and the the often so-called $\mathbf{B}$ matrix is:

$$\mathbf{B}(x,y) = \begin{bmatrix} \frac{\partial N_1}{\partial x}(x,y) & 0 & \dots & \frac{\partial N_6}{\partial x}(x,y) & 0 \\ 0 & \frac{\partial N_1}{\partial y}(x,y) & \dots & 0 & \frac{\partial N_6}{\partial y}(x,y) \\ \frac{\partial N_1}{\partial y}(x,y) & \frac{\partial N_1}{\partial x}(x,y) & \dots & \frac{\partial N_6}{\partial y}(x,y) & \frac{\partial N_6}{\partial x}(x,y) \end{bmatrix}$$

This matrix is built by substitution of the elements of $DN(x,y)$ into the corresponding locations. Since there is no analytical integration formula for the curved (deformed) isoparametric triangular element, numerical integration has to be used. The numerical integration formula presented below, however, is only exact for an undeformed (straight-sided) triangle.

Element Stiffness Matrix. The element's contribution to the global stiffness matrix, $\mathbf{K}$, given by the spatial integral

$$K^{(e)} = \iint B^T D B dx y$$

over the matrix product of $\mathbf{B}$ -transposed $\mathbf{D} \mathbf{B}$, where $\mathbf{D}$ is the 3 x 3 material property matrix, is approximated numerically as the sum

$$K^{(e)}_{num.} = \sum_{i=1}^{nip} |J_i| w_i B_i^T D B_i$$

of such products computed at nip integration points in the triangle. In this example, the midside nodes are chosen as integration points (dark nodes in Fig. 1, $nip=3$). $|J_i|$ refers to the determinant of the Jacobian matrix which must be non-singular and w_i is a weight that is applied to the matrix contribution from each integration point. This weighting depends on the area of the reference element and the number of integration points chosen. Thus, with the reference element's area being $\frac{1}{2}$ and $nip=3$, it follows $w_i = 1/6$ such that the integral terms span the area of 0.5. Due to the location of the integration points, all weights can be treated equally.

For the case of *plane strain* (no strain normal to the plane of the two-dimensional model) which is considered here, the $\mathbf{D}$ matrix incorporates the properties Young's modulus, E , and Poisson's ratio, ν :

$$D = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix}$$

Matrix Inversion. A common linear-elasticity problem where the dependent variable is the nodal displacement, is defined by the global stiffness matrix, $\mathbf{K}$ on the lefthand side and a prescribed displacement boundary-condition vector $\mathbf{uv}$ on the righthand side

$$\mathbf{K} \mathbf{d} = \mathbf{uv}$$

The solution is returned into the nodal displacement vector $\mathbf{d}$, with dimensions nodes x degrees of freedom per node. Initial stresses and strains, boundary stresses and body forces are ignored.

Strains & Stresses. Among others Barlow (1976, Int. J. Num. Meth. Eng. 10, 243-251) has shown that the best locations for strain (change of length and shape) and stress computations in the quadratic (and cubic) triangular element are the midside nodes. He recommends to linearly interpolate or extrapolate these to the element center or to the nodes, respectively, for the later usual visualization (linear interpolation is recommended, because derivatives of quadratic- are of course linear functions). If

strains or stresses are interpolated to the nodes, an averaging of nodal values computed by the elements sharing the node is often applied since the stresses are not continuous from element to element when quadratic interpolation functions are used.

From the nodal displacements in the global coordinate system, dx_i, dy_i where i refers to node i , the strains at each integration point are calculated as

$$\begin{Bmatrix} \varepsilon_x \\ \varepsilon_y \\ \varepsilon_{xy} \end{Bmatrix}_j = \begin{bmatrix} \frac{\partial N_1}{\partial x} j & 0 & \cdots & \frac{\partial N_6}{\partial x} j & 0 \\ 0 & \frac{\partial N_1}{\partial y} j & \cdots & 0 & \frac{\partial N_6}{\partial y} j \\ \frac{\partial N_1}{\partial y} j & \frac{\partial N_1}{\partial x} j & \cdots & \frac{\partial N_6}{\partial y} j & \frac{\partial N_6}{\partial x} j \end{bmatrix} \begin{Bmatrix} dx_1 \\ dy_1 \\ \vdots \\ dx_6 \\ dy_6 \end{Bmatrix}$$

Where j refers to the integration point j . This multiplication is equivalent to multiplying B_i , the expanded shape function x,y-derivative matrix (see above) at the integration point i (a midside node) with the nodal displacement vector, $d^{(e)}$, for the element:

$$\varepsilon_i = B_i d^{(e)}$$

The B_i are computed *before* the nodal displacements are applied to change the elements shape in global coordinates. The stresses at the integration points are finally computed from the strains as

$$\sigma_i = D \varepsilon_i$$

Principal Strain- & Stresses Axes. The principal strain / stress axis are found as the Eigenvectors of the calculated strain and stress tensors

$$\Sigma^{2D} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}, \quad \Sigma^{3D} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{zx} \\ \sigma_{xy} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{zx} & \sigma_{yz} & \sigma_{zz} \end{bmatrix}$$

in which the off-diagonal elements are the shear strains / stresses. The Eigenvalues represent the principal strain /stress magnitudes (in the directions defined by the Eigenvectors). The Eigenvector matrix EIG contains the principal axis 1, 2, 3 in its corresponding rows.

Dilatation. The sum of the Eigenvalues of the strain tensor.

Mean Stress. The average of the principal stresses.

Testing. A series of tests can be performed to detect errors in the implementation of newly implemented elements like the discussed one. Thus, for any point inside the implemented element (including its boundary) and with reference to the global coordinate system:

1. The interpolation functions must sum up to 1 (down to numerical precision).
2. Their derivatives must sum up to zero.

3. The sum of the products of interpolation function derivatives with nodal scalar variables must represent the gradient of a scalar variable prescribed on the nodes anywhere inside the triangle.
4. The element volume as calculated by integrating the shape functions over all integration points in the element must be correct.
5. In an even-sided symmetric transformation of an element's shape, for instance an equal-sided triangle, the length of the vectors defined by the interpolation functions derivatives at the nodes and integration points, must be the same at opposite nodes. Thus, a transformation into the highest possible symmetry makes the testing easier.

Global to Local. Interpolation of property values at a point specified in global coordinates for mapped elements.

Property Value Gradients. If the gradient of a property value is desired at a global point (x,y) , the global interpolation-function derivatives at the element's nodes can be interpolated to this point using the element's interpolation functions. For this purpose, the global x,y-derivatives of shape function 1 at node 1, shape function 2 at node 2 and so forth are assembled into a derivative matrix $\mathbf{DN}$ (size spatial dimensions x nodes). These data are now interpolated with the element's interpolation functions to the point of interest, $\mathbf{DN}(x,y)$, just as one would interpolate any nodal property value. The desired property-gradient is found by summing the products of the interpolated derivatives with the corresponding nodal property values:

$$p_{,x}(x,y) = \sum_{i=1}^{i=\text{nodes}} \frac{\partial N_i}{\partial x}(x,y) p_i$$

where $p_{,x}(x,y)$ is the property gradient in the x-coordinate direction at the global point (x,y) .

Inter-Element Continuity. Once an element has been verified in isolation, further tests can be devised for element patches in order to see whether the nth order derivatives of the shape functions are continuous across the sides of the elements and their corner nodes. For the quadratic triangle they are not continuous across the corner nodes.

The IsoparametricBarycentricQuadraticTriangle with 7 Nodes

The barycentric triangular finite element has quadratic interpolation functions involving an extra node in the element's center. This particularity makes the element well suited for buoyancy-driven fluid flow and elastic-viscous deformation simulations. Since this element can be dramatically over integrated using a 7 or 13-point numerical integration scheme it is capable of very accurately representing integrated field variables like varying fluid density or body forces. The 13-point integration scheme is only recommended for calculations where buoyancy forces vary dramatically over short distances but the overall scale of the model dictates that the mesh has to be relatively coarse. An example of such a calculation would be a model of dike swarm intrusion. The node-numbering in a right-handed coordinate system and the splitting of the elemental area in areal coordinates is shown in Figure 9.5.

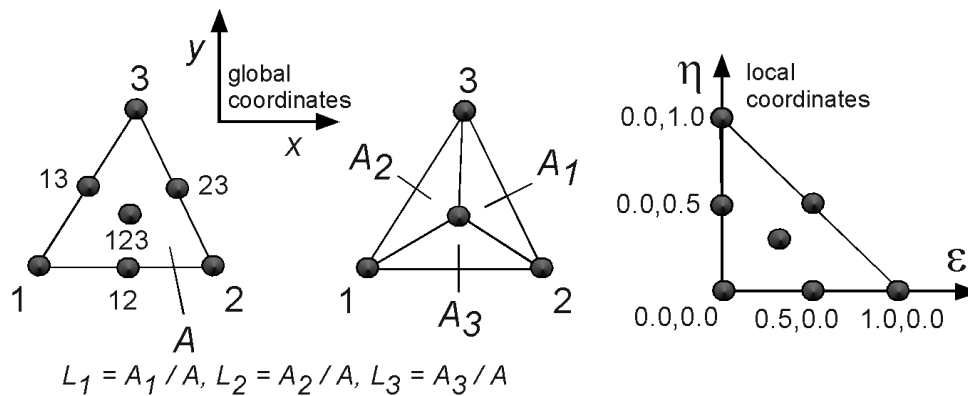


Fig. 9.5: The triangular barycentric element. (Left): Node-numbering scheme; (center): Natural (area) coordinate breakdown of the reference element; (right): Coordinate convention for the local coordinate system.

The origin of the local coordinate system is located in node 1 of the barycentric triangle and the limbs are stretched such that node 2 and 3 have the local two-dimensional coordinates $(1,0)$ and $(0,1)$, respectively.

Element Interpolation Functions in Global Coordinates

The `BarycentricQuadraticTriangle` is implemented as an isoparametric element using a local coordinate system and Jacobian transformations. However, element interpolation functions in global coordinates are still needed to interpolate property values at points whose coordinates are known only in the global coordinate system. Interpolation using this approach is correct only as long as the triangle remains straight sided.

As for the `LinearTriangle` element, interpolation functions in global coordinates are composed using area coordinates, L . The seven interpolation functions N_I to N_{123} are now expressed in terms of the area coordinate functions:

$$N_1 = L_1(2L_1 - 1) + 3L_1L_2L_3$$

$$N_2 = L_2(2L_2 - 1) + 3L_1L_2L_3$$

$$N_3 = L_3(2L_3 - 1) + 3L_1L_2L_3$$

$$N_{12} = 4L_1L_2 - 12L_1L_2L_3$$

$$N_{13} = 4L_1L_3 - 12L_1L_2L_3$$

$$N_{23} = 4L_2L_3 - 12L_1L_2L_3$$

$$N_{123} = 27L_1L_2L_3$$

N_{123} is sometimes called the bubble function and is unique in that the corresponding node is not shared among elements. The first partial derivatives of these functions with regard to the node coordinates x are:

$$\frac{\partial N_1}{\partial x} = (4L_1 - 1)\frac{\beta_1}{2A} + 3L_1L_2L_3$$

$$\frac{\partial N_2}{\partial x} = (4L_2 - 1)\frac{\beta_2}{2A} + 3L_1L_2L_3$$

$$\frac{\partial N_3}{\partial x} = (4L_3 - 1)\frac{\beta_3}{2A} + 3L_1L_2L_3$$

$$\frac{\partial N_{12}}{\partial x} = (L_2\beta_1 - L_1\beta_2)\frac{2}{A} - 12L_1L_2L_3$$

$$\frac{\partial N_{23}}{\partial x} = (L_3\beta_2 - L_2\beta_3)\frac{2}{A} - 12L_1L_2L_3$$

$$\frac{\partial N_{31}}{\partial x} = (L_1\beta_3 - L_3\beta_1)\frac{2}{A} - 12L_1L_2L_3$$

$$\frac{\partial N_{123}}{\partial x} = 27L_1L_2L_3$$

Again the analytical integration formula already presented for the triangular element could be used but this would become a fairly complicated expression:

$$\int_A N_1 N_2 N_3 N_{12} N_{13} N_{23} N_{123} dA = A \frac{j!k!l!m!n!o!p!}{(3+j+k+l+m+n+o+p)!}$$

Local Coordinates

Figure 9.5 illustrates a local coordinate reference frame for the barycentric element. Since local coordinates are used, local shape function derivatives required at the integration points are constants for the reference element. At the corresponding nodes the derivatives are:

node	1	2	3	12	23	31	123
ε_i	-3	3	0	0	5	-5	0
η_i	-3	0	3	-5	5	0	0

The use of a local reference element has the advantage that the sides of the triangle can deform into curvilinear shapes without compromising on the accuracy of the numerically evaluated integrals. The Jacobian matrix for each integration point is formed from the shape (=interpolation) function derivatives at this point and the elements global coordinates:

$$[J] = \underbrace{\begin{bmatrix} \frac{\partial N_1}{\partial \xi} & \dots & \frac{\partial N_n}{\partial \xi} \\ \frac{\partial N_1}{\partial \eta} & \dots & \frac{\partial N_n}{\partial \eta} \\ \frac{\partial N_1}{\partial \zeta} & \dots & \frac{\partial N_n}{\partial \zeta} \end{bmatrix}}_{3 \times n} \underbrace{\begin{bmatrix} x_1 & y_1 & z_1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix}}_{n \times 3}$$

Numerical Integration

The integration points and their weights are defined in terms of the local coordinates of the reference element. The location of the points (*IP7* or *IP13*, in *L* coordinates) and their weights (*W7* or *W13*) are computed as follows:

Matrix *IP7*(7,3), *IP13*(13,3);

```
double g1, g2, g3, g4;
g1 = (6.0+sqrt(15.0))/21.0;
g2 = (6.0-sqrt(15.0))/21.0;
g3 = (9.0-2.0*sqrt(15.0))/21.0;
g4 = (9.0+2.0*sqrt(15.0))/21.0;

IP7(0,0)=1./3.; IP7(0,1)=1./3.;
IP7(1,0)=g1; IP7(1,1)=g1;
IP7(2,0)=g3; IP7(2,1)=g1;
IP7(3,0)=g1; IP7(3,1)=g3;
IP7(4,0)=g2; IP7(4,1)=g2;
IP7(5,0)=g4; IP7(5,1)=g2;
IP7(6,0)=g2; IP7(6,1)=g4;
for ( i=0; i<7; i++ ) IP7(i,2) = 1.0 - IP7(i,1) - IP7(i,0);

IP13(0,0)=0.3333333; IP13(0,1)=0.3333333; IP13(0,2)=0.3333333;
IP13(1,0)=0.4793079; IP13(1,1)=0.2603459; IP13(1,2)=0.2603459;
IP13(2,0)=0.2603459; IP13(2,1)=0.4793079; IP13(2,2)=0.2603459;
IP13(3,0)=0.2603459; IP13(3,1)=0.2603459; IP13(3,2)=0.4793079;
IP13(4,0)=0.8697397; IP13(4,1)=0.0651301; IP13(4,2)=0.0651301;
IP13(5,0)=0.0651301; IP13(5,1)=0.0651301; IP13(5,2)=0.8697397;
IP13(6,0)=0.0651301; IP13(6,1)=0.8697397; IP13(6,2)=0.0651301;
IP13(7,0)=0.6384441; IP13(7,1)=0.3128654; IP13(7,2)=0.0486903;
IP13(8,0)=0.3128654; IP13(8,1)=0.6384441; IP13(8,2)=0.0486903;
IP13(9,0)=0.3128654; IP13(9,1)=0.0486903; IP13(9,2)=0.6384441;
IP13(10,0)=0.6384441; IP13(10,1)=0.0486903; IP13(10,2)=0.3128654;
IP13(11,0)=0.0486903; IP13(11,1)=0.6384441; IP13(11,2)=0.3128654;
IP13(12,0)=0.0486903; IP13(12,1)=0.3128654; IP13(12,2)=0.6384441;

Vector W7(7), W13(13);
W13(0) = -0.14957;
W13(1) = 0.1756152;
W13(2) = 0.1756152;
W13(3) = 0.1756152;
W13(4) = 0.0533472;
W13(5) = 0.0533472;
```

```

W13(6) = 0.0533472;
W13(7) = 0.0771136;
W13(8) = 0.0771136;
W13(9) = 0.0771136;
W13(10) = 0.0771136;
W13(11) = 0.0771136;
W13(12) = 0.0771136;

W7(0) = 0.225;
W7(1) = (155.0+sqrt(15.0))/1200.0;
W7(2) = (155.0+sqrt(15.0))/1200.0;
W7(3) = (155.0+sqrt(15.0))/1200.0;
W7(4) = (155.0-sqrt(15.0))/1200.0;
W7(5) = (155.0-sqrt(15.0))/1200.0;
W7(6) = (155.0-sqrt(15.0))/1200.0;

```

The Isoparametric Quadratic Tetrahedron with 10 Nodes

Fig 9.6 show the node-numbering for the QuadraticTetradron finite element. This element is not seen as such during visualization because it is partitioned into subtetrahedra when meshes are output to VTK or other OpenGL based visualization tools as OpenGL shading can only represent linear property variations.

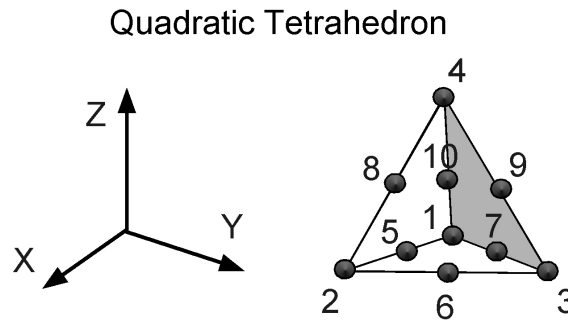


Fig. 9.6: The QuadraticTetrahedron with coordinate reference frame and node numbering scheme (NB: for consistency with the ICEM node numbering, nodes 8-10, now have a different placement, see HTML documentation of this element).

Like the other higher-order elements in CSMP++, the IsoparametricQuadraticTetrahedron is isoparametric and can therefore be used in models of progressive deformation where element boundaries become curved after the first displacement increment. The interpolation functions in local coordinates are

```

L1 = 1.0 - L2 - L3 - L4;
N(0) = L1 * ( 2.0 * L1 - 1.0 );
N(1) = L2 * ( 2.0 * L2 - 1.0 );
N(2) = L3 * ( 2.0 * L3 - 1.0 );
N(3) = L4 * ( 2.0 * L4 - 1.0 );
N(4) = 4.0 * L1 * L2;
N(5) = 4.0 * L2 * L3;
N(6) = 4.0 * L3 * L1;
N(7) = 4.0 * L2 * L4;
N(8) = 4.0 * L3 * L4;
N(9) = 4.0 * L1 * L4;

```

Note that $L1 = 1 - r - s - t$, $L2 = r$, $L3 = s$, $L4 = t$, see Bathe, p. 256 for midside nodes and Huyakorn & Pinder (1983) p. 91 for corner node interpolation function specifications, respectively.

The local partial derivatives of the interpolation functions at arbitrary points inside of the tetrahedron are with respect to the r local coordinate

```
DNR(0) = 1. - 4. * (1. - r - s - t);
DNR(1) = -1. + 4. * r;
DNR(2) = 0.;
DNR(3) = 0.;
DNR(4) = -4. * r + 4. * (1. - r - s - t);
DNR(5) = 4. * s;
DNR(6) = -4. * s;
DNR(7) = 4. * t;
DNR(8) = 0.;
DNR(9) = -4. * t;
```

and with respect to the s local coordinate

```
DNS(0) = 1. - 4. * (1. - r - s - t);
DNS(1) = 0.;
DNS(2) = -1. + 4. * s;
DNS(3) = 0.;
DNS(4) = -4. * r;
DNS(5) = 4. * r;
DNS(6) = -4. * s + 4. * (1. - r - s - t);
DNS(7) = 0.;
DNS(8) = 4. * t;
DNS(9) = -4. * t;
```

as well as with respect to the t local coordinate

```
DNT(0) = 1. - 4. * (1. - r - s - t);
DNT(1) = 0.;
DNT(2) = 0.;
DNT(3) = -1. + 4. * t;
DNT(4) = -4. * r;
DNT(5) = 0.;
DNT(6) = -4. * s;
DNT(7) = 4. * r;
DNT(8) = 4. * s;
DNT(9) = 4. * (1. - r - s - t) - 4. * t;
```

The local partial derivatives at the node points are

node	1	2	3	4	5	6	7	8	9	10
$r=\xi_i$	-3	3	0	0	0	2	-2	2	0	-2
$s=\eta_i$	-3	0	3	0	-2	2	0	0	2	-2
$t=\xi_i$	-3	0	0	3	-2	0	-2	2	2	0

All four integration points have a weight of $\frac{1}{4}$. Their locations in local coordinates are

```
IP(0,0) = 0.13819660, IP(0,1) = 0.13819660, IP(0,2) = 0.13819660;
IP(1,0) = 0.58541020, IP(1,1) = 0.13819660, IP(1,2) = 0.13819660;
IP(2,0) = 0.13819660, IP(2,1) = 0.58541020, IP(2,2) = 0.13819660;
```

```
IP(3,0) = 0.13819660, IP(3,1) = 0.13819660, IP(3,2) = 0.58541020;
```

The volume of the tetrahedron is found by numerically integrating the element interpolation functions over the tetrahedron.

The `IsoparametricQuadraticTetrahedron` is documented in more detail in the [HTML Reference Guide](#).

/SOURCE/INCLUDES/INTEGRATION/FINITE VOLUMES/ LIBRARY**DES_transport_scheme/**

FV_IntegrationPointsAndWeights.h
 FiniteVolumePolicy.h
 FiniteVolumeStencil.h
 FiniteVolumeStencilManager.h
 GenericNodePropertyGradient.h
 GenericNodePropertyGradientLimiter.h
 finiteVolumeAuxiliaryFunctions.h

generic_transport_scheme/**legacy_transport_scheme/**

The **DES transport scheme** implements asynchronous time stepping for single-phase, 2-phase and 2-phase compositional flow simulation. It is documented in Shao et al., (2019) and there are examples in the examples library.

DES2PhaseSlightlyCompressibleTransport.h
 DES2PhaseTransport.h
 DESAdvectionDiffusion.h
 Event.h

DESAdvectionDiffusion2D_Example.h
 DESAdvectionDiffusion3D_Example.h
 DES2PhaseSlightlyCompressibleFlow2D_Example.h
 DES2PhaseSlightlyCompressibleFlow3D_Example.h

The **legacy transport scheme** comprises a host of functionality that is documented in a wide range of publications (including Geiger et al., 2004, Geofluids; Paluszny et al., 2007; Matthai et al., 2009, TIPM) and illustrated examples listed below.

ExplicitMassBasedTransport.h
 ExplicitNodeCenteredFiniteVolumeTransport.h
 ExplicitStencilProcessor.h
 FV_Parameter.h
 MassBasedStencilProcessor.h
 NodeCenteredFiniteVolumeAlgorithm.h
 NodeCenteredFiniteVolumeTransport.h
 StencilProcessor.h
 TwoPhaseExplicitNodeCenteredFVTransport.h
 TwoPhaseImplicitNodeCenteredFVTransport.h

PassiveAdvectionOfTracer_Example.h
 Tutorial1_Example.h
 Tutorial2_Example.h
 Tutorial3_Example.h

The **generic transport scheme** represents the post 2015 implementation of the generic scheme discussed and sketched out at the Colleoli CSMP++ workshop in 2012 with further extensions to handle tensor permeability etc. The design of the scheme is illustrated in a class diagram (Fig. colleoli).

The top-level classes are:

GenericTransportScheme.h

ExplicitTransport.h

ImplicitTransport.h

The latter contains the PDE_operator-like finite-volume integral classes:

MatrixOperator.h

CompressiblePressureSourceLHS.h

DiffusionLHS.h

FluxLHS.h

PoreVolume.h

VectorOperator.h

PoreVolumeRHS.h

SourceTermRHS.h

Equation_TracerTransferImplicit.h

LinearSystemAccumulator.h

Both, the implicit and the explicit schemes use the:

FluxEvaluator.h

TimeStepEvaluator.h

And a range of variable sets that can be output a “*-variables.txt” files:

VariableSet_C02GeoSequestration-full_multiphysics_set.csv

VariableSet_C02GeoSequestration.csv

VariableSet_TracerTransfer.csv

VariableSet_TwoPhaseFlow.csv

While there are no examples yet, there are a series of well-documented unit tests which show the generic scheme in action. Such as:

GenericFiniteVolumeTransport_Test.cpp

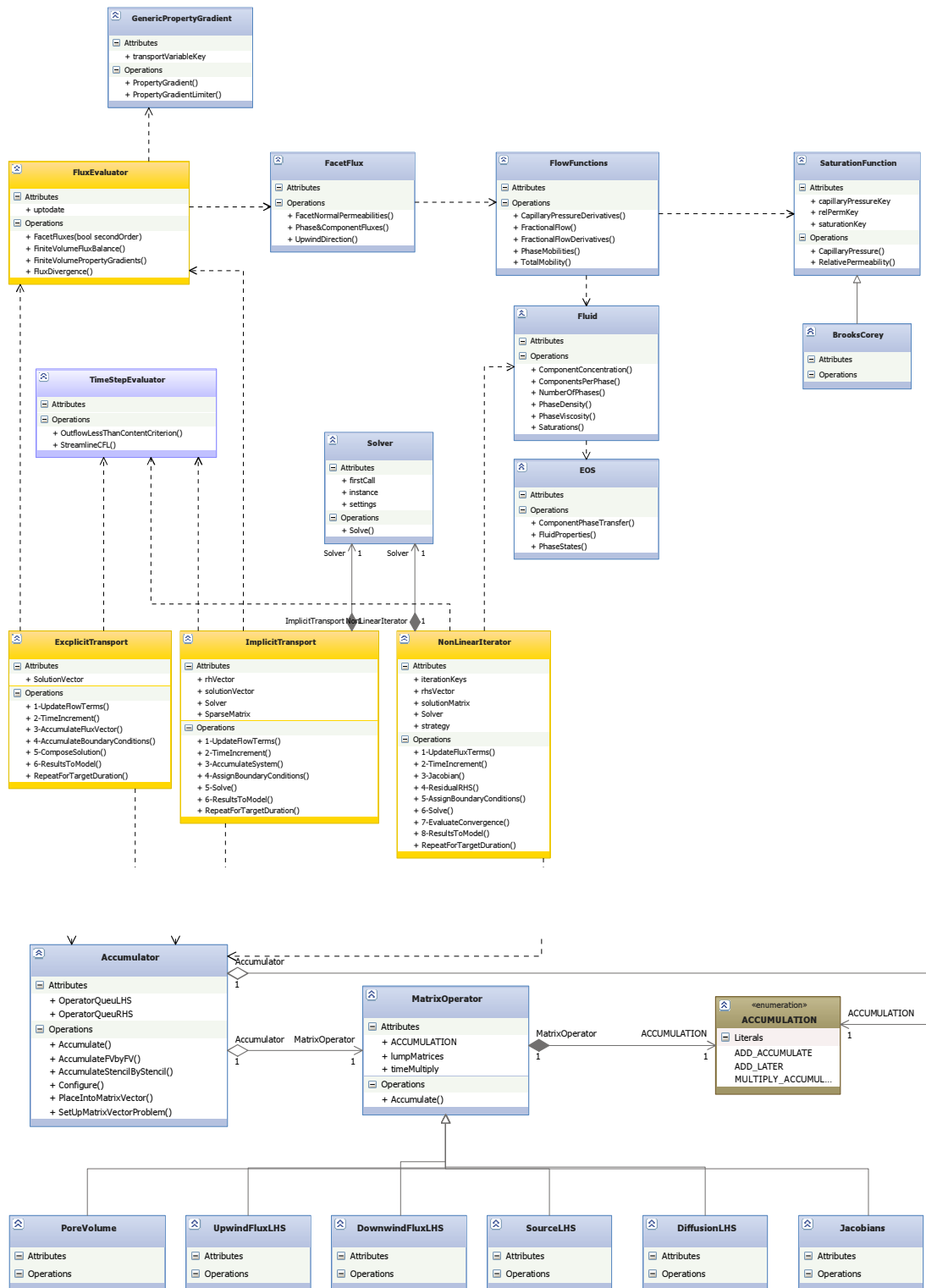


Figure colleoli: Original class design for generic multiphase multicomponent transport scheme as designed on the Colleoli CSMP workshop 2012 and as implemented in modified form in the new generic_transport scheme library.

/SOURCE_CODE/INCLUDES/PROPERTY MANIPULATION/ LIBRARY

Local interdependencies among physical variables in a model are expressed by subclasses inherited from the base class Interrelation. Their declarations are included in the main program file in standard C/C++ manner

This library retains some of the more useful Interrelation classes as well as the PropertyHandle. It clearly needs development or could perhaps be deprecated:

```
ArithmeticMean.h
Concatenate.h
ConstantFactor.h
ExtractTensorVariableComponent.h
ExtractTensorVariableEigenvalue.h
ExtractTensorVariableRow.h
ExtractVectorVariableComponent.h
ExtractVectorVariableLength.h
GroundwaterDarcyVelocity.h
MaximumDifference.h
ModifyPropertyWhere.h
PropertyHandle.h
PropertyDifference
```

Additional interrelations are contained int the THMC library:

```
CozenyCarman
Dilatancy
EnthalpySource MaximumDifference
FluidExpansionSource
HeatTransfer
HydraulicHead
ModifyPropertyWhere
MohrCoulombFailure
NodeToElementProperty
PermeabilityFromThickness
QuartzPrecipitation
QuartzSolubility
QuartzSolubilityNaCl
ScalarMassFlux
StreamingPotential
StokesEinsteinDiffusivity
ThermalDiffusivity
TransportStepSize
```

Interrelations are calculated when passed to the Model by calling its Pass () method:

```
model.Apply( inherited_interrelation );
```

Interrelation “Seed” library means that only a few interrelation subclasses are supplied with the CSMP++ API library to illustrate what interrelations can do and how to write them. These interrelations are briefly described below in an alphabetical order. All interrelations expect specific names and placements of physical variables (in italics) which they interrelate. These properties must be defined in the variable database input

file. The provided interrelations either completely *replace* or *add to* or *subtract from* the value of the target variable. This important behavioral difference is indicated in brackets after the subclass name.

Concatenate. Combine variables with mathematical operators, see examples 13-14.

Conductivity. (replace) Calculates the hydraulic conductivity “*conductivity*” [m s^{-1}] of a permeable medium from its “*permeability*” [m^2] and the physical variable “*fluid viscosity*” [Pa s^{-1}]. These variables are expected to be element variables. If the fluid viscosity is a node variable, the average value over the nodes of the element will be used in the calculation.

ConductivityFromPerm. (replace) Simplified version of previous interrelation. Calculates the hydraulic conductivity “*conductivity*” [m s^{-1}] of a permeable medium from its “*permeability*” [m^2] using a fixed value for the viscosity of the fluid [Pa s^{-1}].

ConstantFactor.

CozenyCarman.

Dilatancy. (replace) This 2D interrelation calculates the physical element variable “*dilatancy*” [$\text{m}^2 \text{ m}^{-2}$] not from the variable “*strain*”, but as the difference of the element “*area*” before and after displacements have been applied (“*new area*” [m^2]). “Dilatancy” can be applied to any 2D element type to obtain an independent measure how well a strain calculation performed (as is discussed in the description of the finite element library, strains tend to be most accurate at the integration points inside of elements and often erratic if they are calculated directly at the nodes).

EnthalpySource.

ExtractTensorVariableComponent. (replace) Permits to extract specific components of tensor variables as identified by their index pair (i,j), and place them into a scalar variable of choice. Thus, diagonal and off-diagonal components can be extracted if needed.

ExtractVectorVariableLength. (replace) Computes the length of vector variables assuming that the vectors originate in the coordinate origin. The computed length can be assigned to a scalar variable of choice.

ExtractVectorVariableComponent. (replace) As for tensor variables (see above), vector variable components identified by an index integer, can be mapped to user-specified scalar variables.

FluidExpansionSource. (add/subtract) Calculates a “*fluid volume source*” [$\text{m}^3 \text{ m}^{-3}$] source or sink term which arises due to the expansion or contraction of the pore fluid in the rock from one to the next timestep. The fluid-volume change is corrected for the thermal expansion or contraction of the country rock. If this value is not set by the user, a value for concrete at 20°C is applied ($12.5\text{e-}6 \text{ K}^{-1}$). A volume change usually accompanies a change in fluid pressure or temperature and is monitored by taking the difference between the element variables “*previous fluid density*” and “*fluid density*” [kg m^{-3}] multiplied by the “*porosity*” [X]. The temperature change is monitored by the variables “*previous temperature*” and “*temperature*” [$^{\circ}\text{C}$] and is used in conjunction with a user-specified thermal expansivity of the rock [$\text{m}^3 \text{ m}^{-3} \text{ K}^{-1}$], to calculate the thermal expansion of the pore space in the rock (“*porosity*”, [X]). Since a source term in a transient calculation is

always expressed as a rate [$\text{m}^3 \text{ m}^{-3} \text{ s}^{-1}$], the calculated “*fluid volume source*” is divided by the time increment over which the change occurred.

HeatTransfer. (replace) The variable “*heat transfer velocity*” [m s^{-1}], *vh*, is calculated from the Darcy flow velocity “*velocity*”, *v*, [m s^{-1}] the heat capacities of the fluid and the country rock, *hf*, *hc* (“*heat capacity fluid*”, “*heat capacity*”) [$\text{J kg}^{-1} \text{ K}^{-1}$], and their densities, *rf*, and *r* (“*fluid density*” and “*density*”) [kg m^{-3}], respectively, according to the formula:

$$vh = (v hf rf) / (hr r)$$

HydraulicHead. (replace) Calculates the nodal variable “*hydraulic head*” from the “*fluid pressure*” and the “*elevation*”, which refers to the height above the lowest topographical point in a model. The head, *h*, is calculated according to the formula

$$h = p_f / (rho0 g) + elevation$$

g is the acceleration of gravity (9.80665 m N) and *rho0* is the fluid reference density which is assumed to be 1000 kg m^{-3} .

ModifyPropertyWhere. (replace) For a user-specified physical variable, this generic interrelation searches for specific scalar values and replaces them with a fixed scalar value.

NodeToElementProperty. (replace) Takes the average value of a user-specified nodal physical variable of all the nodes of an element. The result is returned into a user-specified element variable.

QuartzSolubility. (replace) Calculates the solubility of quartz in pure water “*quartz solubility*” [kg m^{-3}] in an element from the nodal variables “*temperature*” [$^{\circ}\text{C}$] and “*nodal fluid density*” [kg m^{-3}]. The calculation is based on Craig Manning's fit of quartz-solubility data. The result is returned in grams per kg H_2O .

QuartzSolubilityNaCl. (replace) As above, but calculation of aqueous solubility according to the empirical formula of Fournier & Potter, GCA 46, p. 1969-73, with a correction for the presence sodium chloride according to Fournier 1983, GCA 47, p. 579-586. The fluid salinity is specified by the nodal variable “*mass fraction NaCl*”.

QuartzPrecipitation. (add/subtract) Integrates over a user-specified time-increment, how much quartz is precipitated or dissolved per unit volume of each finite element through which fluid flows at a Darcy velocity “*velocity*” [m s^{-1}]. This is achieved by projecting the “*quartz solubility gradient*” vector onto the flow velocity vector. In this calculation, the physical variable “*fluid density*” [kg m^{-3}] is used as element average density from which the time-integrated fluid mass flux [$\text{kg m}^{-2} \text{ s}^{-1}$] is derived. The result is added to or subtracted from the variable “*quartz*” [kg m^{-3}].

ScalarMassFlux. (replace) Calculates the length of the Darcy flow velocity vector [m s^{-1}] and multiplies it by the element variable “*fluid density*” [kg m^{-3}] in order to compute the physical variable “*scalar mass flux*” [$\text{kg m}^{-2} \text{ s}^{-1}$] in each finite element. If the flow model is three-dimensional, the scalar mass flux is calculated directly from the element variables “*fluid pressure gradient*” [Pa m^{-1}], “*conductivity*” [m s^{-1}], and “*fluid density*” [kg m^{-3}]. In this case, the variable fluid pressure gradient is computed conveniently, from the variable fluid pressure [Pa] via the Model method “`CopyGradientOfProperty_A_To_B()`”.

StokesEinsteinDiffusivity.

StreamingPotential.

ThermalDiffusivity.

TransportStepSize. (no variable is changed) Determines the maximum permitted advection-time-increment size for property advection on FiniteDifferenceGrid objects by the TransportVisitor or HeatTransportVisitor objects. This time increment is calculated on the basis of two vector variables that define nodal and elemental transport velocities, respectively, and a third variable which stores the radius of the inner circle of each triangular element. The stepsize is chosen such that the product of the velocity and the time increment is smaller than the diameter of the inner circle for all triangles. The physical variables are submitted to TransportStepSize as constructor arguments, and are named, in most cases “*nodal velocity*” [m s^{-1}], “*velocity*” [m s^{-1}], and “*inner radius*” [m]. When the transport-step-size interrelation is passed to the Model, it calculates the time-increment which can subsequently be retrieved using the method `AdvectionTimeIncrement()`.

Many more interrelations exist for a wide variety of purposes. Please contact the authors of this guide to find out more.

/SOURCE/INCLUDES/THMC/ LIBRARY

CSMP_physical_constants.h

geochemistry/
 single_phase_flow/
 two_phase_flow/
 mechanics/
 fractures/

The THMC library provides a collection of Visitor and Interrelation classes for more complex simulations of fluid flow, deformation, heat and mass transport under single-phase and two-phase conditions such as in magmatic hydrothermal systems. There are also constitutive relationships for fluid flow through rock fractures.

thmc/geochemistry/
 AlterationVisitor.h
 QuartzPrecipitation.h
 QuartzSolubility.h
 QuartzSolubilityNaCl.h
 Reactant.h

thmc/single_phase_flow/
 ComputeSinglePhaseGravityTermVisitor.h
 ConductivityVisitor.h
 DispersivityVisitor.h
 GravityProjectionVisitor.h
 GroundwaterDarcyVelocity.h
 HydraulicHead.h
 RegionBoundaryFluxVisitor.h
 SinglePhaseVelocityVisitor.h
 SourceVisitor.h
 StokesEinsteinDiffusivity.h
 StreamingPotential.h
 ThermalVisitor.h
 VelocityAndVolumeFlux.h
 parallelPlatePermeabilityFromChannelWidth.h

thmc/two_phase_flow/
 BrooksCorey.h
 BrooksCoreyCO2.h
 BrooksCoreyFrontVelocity.h
 BrooksCoreyFunc.h
 BrooksCoreySaturationFunctions.h
 BrooksCoreySaturationFunctionswithHysteresis.h
 BrooksCoreyWithHysteresis.h
 BrooksCoreyWithHysteresisCO2.h
 CapillaryNumberVisitor.h
 ComputeGravityTermVisitor.h
 ComputeMixturePropertyVisitor.h
 ComputeSecondPhaseSaturation.h
 Corey.h
 ExperimentalRT.h
 ExperimentalSaturationFunctions.h
 FlowFunctionsModule.h
 Fluid.h
 H2O_CO2_NaCl_FlowFunctions.h

```

HeterogeneityAndRateAwareModel.h
LayeredCompositeProcessor1.h
LinearTwoPhaseModel.h
OtwayCRC3_RockTypes.h
TwoPhaseFileBased.h
TwoPhaseFlowFunctions.h
TwoPhaseModel.h
TwoPhaseUndersaturatedFluidPropertyVisitor.h
TwoPhaseVelocityAndVolumeFlux.h
VanGenuchten.h

```

```

thmc/mechanics/
AndersonianBoundaryStressVisitor.h
BoundaryStressVisitor.h
BoundaryStressVisitor.h.old.h
BrittleFailure.h
BrittleFailureVisitor.h
EffectiveStressVisitor.h
FlatBoundaryStressVisitor.h
MechanicalProperties.h
MohrCoulombFailure.h
MohrCoulombFailure_Visitor.h
OpeningModeFracture.h
OverburdenPressureVisitor.h
StressInvariants.h
StressRegime.h
StressRotate.h
StressesAndStrains.h
StressesAndStrains2.h
StressesAndStrainsIP3D.h
mechanics.h

```

```

thmc/fractures/
ExponentialFractureMatrixTransfer.h
FourarLenormand.h
FractureBrooksCoreyParameterVisitor.h
FractureCapillaryPressureVisitor.h
FractureComputeCapillaryGradientVisitor.h
FractureComputeVelocityVisitor.h
FractureMatrixUpscaled.h
FracturePecletNumberVisitor.h
GenericTransferFunction.h
HydroFractureVisitor.h

```

Fluid Flow. The H2OLiquidSteamPropertiesVisitor object calculates nodal properties of pure H₂O liquid and steam as a function of nodal pressure and temperature values. If the saturation curve of pure H₂O is crossed at subcritical conditions, the energy balance between fluid and rock is calculated to compute the mass of liquid and steam present. This object can employ the PROST library or a H2OPropertiesLookUpTable object to calculate fluid properties. Calculations with PROST are computationally more costly than a direct retrieval from a H2OPropertiesLookUpTable object. On the other hand, a H2OpropertiesLookUpTable object requires an additional 105 MB memory. Sophisticated error checking is used in the H2OLiquidSteamPropertiesVisitor object, because PROST often fails at temperatures close to the critical point or pure H₂O saturation curve. If a failure of PROST is encountered, the object attempts to recompute fluid properties at some small temperature increment away from the current temperature. Only if this method fails repeatedly, the simulation is terminated.

The `NaClH2OLiquidSteamPropertiesVisitor` object calculates nodal brine, vapor, and halite properties for the NaCl-H₂O system as a function of nodal pressure, temperature, and salinity values (Fig. 3). It uses a `H2OPropertiesLookUpTable` object to calculate the pure H₂O properties and a `Fluid` object to obtain the NaCl-H₂O properties. If the salinity is below 10⁻¹⁵ Wt. % NaCl, the object treats fluid as pure H₂O. By setting the boolean in the `NaClH2OLiquidSteamPropertiesVisitor` constructor to true, thermal and chemical expansivities are retrieved from lookup tables as well. Each expansivity lookup table requires 65 MB of additional memory. Only one third of the computation time, however, is needed, because fluid properties do not have to be evaluated at temperature and salinity increments away from the values of interest. The `NaClH2OLiquidSteamPropertiesVisitor` object uses sophisticated error checking as well, attempting to recompute fluid properties at some small temperature increment away from the temperature at which property calculations failed.

Both fluid property Visitor object terminate if non-physical, i.e., negative pressure, temperature, or salinity values are encountered. If the pressure exceeds the maximum valid pressure of 4000 bar, a warning is issued, because properties can only be computed up to 4000 bar.

The `H2OLiquidSteamTransportPropsVisitor` object calculates the element transport properties 'storativity', 'expansivity', 'mobility liquid', 'mobility vapor', 'total mobility', 'mass mobility liquid', 'mass mobility vapor', and 'total mass mobility' for two-phase fluids from nodal fluid and element rock properties. The mobility variables refer to the property $k_{k_{ri}}/\mu_i$ and the mass mobility variables to $k\rho k_{ri}/\mu_i$. The expansivity includes thermal and chemical expansivity. If two-phase conditions are encountered, nodal fluid properties are averaged to the element considering only the nodes at which the phase is present.

The `H2OLiquidSteamVelocityVisitor` object solves Darcy's law for each element to compute fluid velocities for the liquid and vapor phase from nodal fluid pressures, element mobilities, and nodal relative fluid densities. The latter can be computed for each phase using a `LiquidSteamRelativeDensityVisitor` object. Relative fluid pressures, which result if buoyancy terms are evaluated using relative fluid densities, can be converted into the actual fluid pressure using a `RelativePressureToFluidPressure` object. This object adds the relative pressure to a reference pressure, which must be computed once at the beginning of a simulation, to obtain the actual fluid pressure.

Fluid phase velocities may be transformed into a 'heat transfer velocity' using a `HeatTransverVelocityVisitor` object. This object calculates the heat transfer velocity for two fluid phases on each element using nodal fluid and element rock properties. At two-phase conditions, fluid properties are averaged to the element only for the nodes where the phase is present.

The `NaClH2OMassFractionVisitor` object computes the mass of salt in the liquid and vapor phase such that the term $\text{div} \cdot (\mathbf{v}_i \rho X_i)$ can be solved. The `Interrelation` object `TotalMassFractionToSalinity` converts mass fraction back to salinity considering that brine, vapor, and halite may be present.

Advection of salt in brine and vapor can be modeled using the

NaClTwoPhaseFVAdvection object. This object employs a transport algorithm object and uses pre- and post processing steps to calculate fluid mass and salt transport in each phase. It automatically converts mass fractions back to salinity considering that brine, vapor, and halite may be present. If sub-stepping is used, i.e. if multiple advection steps are calculated during a single time-step, salt mass fractions are automatically updated using a NaClH2OMassFractionToFlowingMassVisitor object. Range checking at the end of each transport step returns error messages if salinity values exceed their prescribed ranges.

The Interrelation object TotalHeatCapacity and the Visitor object HeatCapacityVisitor both calculate the term $[(1-\phi) \rho_r c_{pr} + \phi (S_l \rho_l c_{pl} + S_v \rho_v c_{pv})]$ needed for the solution of the heat conduction equation. The Interrelation class TotalFluidVolumeSource can be used to add the source/sink term arising from the thermal and chemical expansivity to any additional fluid sources/sinks.

The aqueous diffusivity of NaCl in H₂O can be computed as a function of temperature, pressure, and salinity using an AqueousDiffusivityVisitor object. This object evaluates the Stokes-Einstein relation (e.g., Oelkers, 1996) at each node. The diffusivity of the dissolved particle within the pore-space can be computed from a PoreDiffusivityVisitor object. This object calculates an element pore diffusivity from the nodal aqueous diffusivity and the element porosity (e.g., Oelkers, 1996). For large scale convection models, however, it is not necessary to evaluate the diffusivities, because their changes are negligibly small in comparison to the numerical diffusion, even when using a higher order transport scheme. For simulations in which only diffusion of heat and salt is considered, it is important though to accurately calculate the diffusivities.

For single-phase fluid property calculations of pure H₂O or NaCl-H₂O, the objects provided by the CSMP++3D-eos library always should be used, since they require memory allocation for only a single fluid phase.

VISITORS INCLUDED IN THE THMC LIBRARY

Subclasses of Visitor base class facilitate the solute advection on regular grids, the mapping of variables to- and from FiniteDifferenceGrid objects to a two-dimensional mesh and a way to simulate hydraulic fracture in models where overpressure is generated. There are also two visitor subclasses for the monitoring of fluxes across model- and sub-region boundaries. Example source code is provided to illustrate the application of visitor objects in general. This code defines the CSMP++_ID_Visitor and illustrates how to traverse the mesh and collect ID numbers from finite elements, constraint points or nodes.

The ID is only intended to illustrate how visitors work and not for any practical application. The visitor has a private integer vector into which it collects the ID numbers. These may either be Node, Face or Element instances in the model. A key feature of the visitor pattern is an incomplete declaration in the base class Visitor, of the objects which shall be visited. This declaration allows to define access pointers to the visited objects as arguments of the pure virtual method visit(T* object). The interface of the objects pointed to by “T”, can later be accessed by the concrete visit(T* object) methods in the derived visitor classes. These classes provide definitions of the visited objects in corresponding header files.

Remember, to use one of the visitor subclasses, you must first include its declaration into your program file:

```
#include "MyVisitor.h"
```

Now instantiate the visitor object and submit it to the Model or Region:

```
MyVisitor<CSMP++_float,2>  guest(...);

Model.Accept( guest );
Model.Region("the garden").Accept( guest );

// or alternatively

guest.RestrictApplicationTo( "the garden" );
Model.Accept( guest );
```

That's it. This code example should persuade the user how easy it is to create and use original visitors. Feel free to contact CSMP++ authors for advice in writing new visitors to complement the library.

BoundaryTransferVisitor (2D linear triangles only)

The BoundaryTransferVisitor visits Groups determining fluxes across their boundaries. These fluxes are stored in STL maps separating the influx from the outflux and using the element ID numbers as keys. The maps are preserved after the visitation such that if the visitor is accepted a second time, only the flux values have to be changed. A specific call can be made to purge the maps before renewed application of the visitor. Boundary fluxes are always computed for the elements inside the target regions. The fluxes can also be integrated over time, if a time interval is supplied. A balance of the flux total (influx versus outflux) may also be obtained. Finally, the BoundaryTransferVisitor object can be used to assign NEUMANN (flux-type) boundary conditions.

InterfaceVisitor (2D linear triangles only)

The InterfaceVisitor visits the Model or Groups determining fluxes across model boundaries or interfaces between groups where groups terminate against each other. The group boundary can be output from the visitor to a VTK text file for visual verification. An UpdateGeometry() function permits to track changing group interfaces. The InterfaceVisitor can also monitor fluxes across a Model boundary where it coincides with a group boundary. The model boundaries are identified by values of the enumeration SG_BOUNDARY.

Like for the BoundaryTransferVisitor, fluxes are stored in STL maps separating the influx from the outflux and using element ID numbers as keys. The maps are preserved after the visitation such that if the visitor is accepted a second time, only the flux values have to be changed. A specific call can be made to purge the maps before renewed application of the visitor. Fluxes are averaged between the elements inside and outside

the interface. They can also be integrated over time, if a time interval is supplied. A balance of the flux total (influx versus outflux) may also be obtained.

AlterationVisitor

The AlterationVisitor computes the reactive exchange between the pore fluid and the rock in sequential node visitations. At the end of each visitation the new speciation is written back to the node property storage. The visitor can be used for reactive transport calculations where operator splitting is used to separate transport from reaction steps, see Steefel & MacQuarrie (1996) for a discussion of this and other approaches. In a typical application advection/dispersion steps alternate with the invocation of the visitor, but the user must ascertain that no more than the entire pore-volume is exchanged before the next reaction step occurs. From this restriction, it is evident that countless reaction steps are required to change the mineralogy of a low-porosity rock appreciably. To improve its performance, the AlterationVisitor monitors reaction rates and concentrations at each timestep and carries out speciation calculations only when the pore-fluid is out of equilibrium or equilibrium was not attained in the previous visitation of a node. Nodes where the compulsory property 'H2O_aq' is flagged DIRICH are ignored. The visitor incorporates the ODE_StiffSolver object, the functionality of which is demonstrated in the distributed 'example6.cpp' program.

The AlterationVisitor reads the chemical reactions it is supposed to solve from a text input file. Another text file with the same name but with the extra extension "-prop.txt" provides the charges and atomic weights of the involved aqueous and gaseous species as well as the solids:

```

reactant properties for carbonate dissolution model
OH_1m      AQUEOUS      -1      17.0073
CO2_aq     AQUEOUS       0      44.01
H_1p       AQUEOUS       1       1.0079
H2O_aq     AQUEOUS       0       18.02
HCO3_1m    AQUEOUS      -1     161.1043
Ca_2p      AQUEOUS       2      40.078
HCl_aq     AQUEOUS       0       36.46
Cl_1m      AQUEOUS      -1      35.4530
CC         SOLID        2710.0    100.09

```

This is discussed again in the carbonate-dissolution example code (Chapter 7). The AlterationVisitor is documented in more detail in the CSMP++ Reference Guide.

Steefel, C. I. & MacQuarrie, K.T.B, 1996, Approaches to modeling reactive transport in porous media. (In) Reactive Transport in Porous Media, Rev. in Mineralogy 34, 83-125, American Mineralogical Society.

H2OPropertiesVisitor

Calculates the PVT properties of pure H₂O at the Nodes of a mesh. This includes the physical variables "nodal fluid density" [kg m⁻³], "nodal relative fluid density" [kg m⁻³] for a reference pressure, "nodal heat capacity liquid" [J kg K⁻¹], "nodal enthalpy liquid" [J kg], "nodal fluid viscosity" [Pa s⁻¹] and, optionally, also the same properties

of the gas phase. These have the names "*nodal heat capacity steam*", "*nodal enthalpy steam*", as well as the "*nodal saturation*". The nodal variables are calculated from the "*absolute fluid pressure*" [Pa] and the "*temperature*" [°C]. The calculation is carried out using the IAPS equation of state for pure water which covers a temperature range from 0 to 1000°C and a pressure from atmospheric to 1 GPa.

The conditions for the reference density are set by a call to

```
void ReferenceDensity( double Tc, double Pa );
```

And the calculations of the properties of steam can be suppressed by a call to

```
void LiquidPhaseProperties();
```

Since this eliminates an iteration loop, it greatly speeds up the visitations. However, it should be applied only after verifying that the PT conditions in the model are always outside of the critical region. With the

```
void Verbose();
```

option water properties can be output to screen directly after their computation. This enables a first visual examination if a new simulation is setup.

HydroFractureVisitor

The HydroFractureVisitor enhances the hydraulic "*conductivity*" [m s⁻¹] within and beyond the boundary of regions in which the "*absolute fluid pressure*" [Pa] exceeds a fracture-threshold pressure given by the lithostatic load [Pa] augmented by the tensile strength [Pa] of the rock. Through this capability, the visitor permits just enough fluid to dissipate from the overpressured region to keep fluid pressure at the fracture threshold. This is achieved within the framework of an iteration loop. The visitation will terminate when the absolute fluid pressure in all the elements of the model is no higher than the confining pressure plus the tensile strength of the rock.

Conceptually, this treatment of hydraulic fracture is equivalent to creating an interconnected network of small fractures in the rock. This increases the effective hydraulic conductivity sufficiently, to bleed excess fluid out of the regions where fluid pressure would otherwise exceed the lithostatic pressure plus the tensile strength of the rock. When the hydro-fracture visitor is accepted by the Model it will perform six different operations on each element it visits:

1. Test whether at least one element node is overpressured. Only where an overpressure exists, conductivity will be enhanced and diffused.
3. Using the hydraulic conductivity, K which shall be adjusted, the visitor calculates a "trial" fluid pressure gradient [Pa m⁻¹] in overpressured elements.

$$\nabla P = \sqrt{\left(\frac{\partial P}{\partial x}\right)^2 + \left(\frac{\partial P}{\partial y}\right)^2}$$

The calculation employs the transport (Darcy) “velocity” [m s⁻¹].

4. The hydraulic conductivity, K , of elements with at least one overpressured node is enhanced proportional to the quotient $grad P$ over the “storativity”, β_c , [m³ m⁻³ Pa⁻¹] following the rationale that β_c is the proportionality constant, which relates the change of pressure in an element to the amount of fluid leaving or entering it. K is adjusted to the value which will just valve off enough fluid to bring the pressure down to the confining pressure plus the tensile strength of the rock.

5. Only in elements with an appreciable internal fluid-pressure gradient, an increased hydraulic conductivity fosters the dissipation of fluid. Therefore, if $grad P$ is negligible the conductivity is not enhanced.

6. The new hydraulic conductivity is compared with the previous hydraulic conductivity. Only if the new conductivity is greater than the old one it is assigned to the element.

In the code segment where the HydroFractureVisitor gets applied, a decision must also be made whether regions that are no longer overpressured shall be kept at an enhanced conductivity. Pseudo code of such a hydrofracture visitor application loop might look like this:

1. Calculate the hydrostatic pressure.
2. Calculate the hydro-fracture threshold pressure as the “lithostatic pressure” plus the tensile strength of the rock, minus the hydrostatic pressure.
3. Start the iteration procedure by computing the variable “effective pressure” ($P_f - P_{hydrostatic}$) and modifying the hydraulic “conductivity” with the hydro-fracture visitor until the overpressure is dissipated.
4. Recompute fluid properties and calculate fluid transport velocity.
5. Compute new “absolute fluid pressure” = “effective pressure” + “hydrostatic pressure”.

The HydroFractureVisitor gets applied in the iteration loop (3) as follows:

```
while ( dP > 0.0 && step <= MAX_PF_ITERATIONS )
{
    // 0. starting with fluid pressure before the erratic timestep
    // -----
    sg.CopyReplace( "previous effective pressure", "effective pressure" );

    // 1. diffusing new conductivity across border of hydrofracture
    //     regions (letting fracs grow)
    // -----
    sg.Accept( hydrofrac_visitor );

    // 2. re-compute fluid-pressure distribution (when hydrofractures
    //     are propagated one must not consider convection)
    // -----
    ...

    // 3. calculate new overpressures with an interration
    // -----
    sg.CopyReplace( "excess pressure", "previous excess pressure" );
    sg.Pass( excess_pressure );
}
```

```

// 4. test for loop exit condition
// -----
sg.MinMaxOf( "excess pressure", dP_min, dP_max );
if ( dP_max <= 0.0 ) break;
step++;

} // END iteration loop

```

Importantly, when a transient fluid pressure distribution is computed, a backup copy of the “*effective pressure*” from before the step must be kept and re-used as initial condition in each of the “tries” in the iteration loop.

TransportVisitor (2D only)

The TransportVisitor advects distributed variables on an array of regular grids using a two-dimensional velocity vector field defined by nodal and elemental velocities on the triangular element mesh. *The velocity field must not contain discontinuities across element boundaries.* These arise when the interstitial velocity is used and porosity changes occur from one element to the other. In this case, use a finite-volume approach to model transport (see code `example5.cpp` and following). To calculate incremental field displacements, the visitor uses a seven-point quadratic interpolation scheme (first order since no spatial derivatives of the advected property are considered). The grids are not managed by the visitor itself but have to be defined and initialized beforehand. This can be done by code similar to the following:

```

const CSMP++_float    RESOLUTION(0.1);
uint32                n_species = 5;
vector<CSMP++_float>   xy;
Model::Dimensions( xy );
frame_x = RESOLUTION * 10.0;
frame_y = RESOLUTION * 10.0;

vector<FiniteDifferenceGrid<CSMP++_float> >   aqueous_system(n_species);

for ( stl_index i=0; i<n_species; i++ )
    aqueous_system[i].Initialize( -frame_x, xy(1)+frame_x,
                                   -frame_y, xy(3)+frame_y,
                                   RESOLUTION, RESOLUTION );

```

Once the grids are initialized, values must be assigned to them in one of three ways: (1) by setting selected rows, columns, or the entire grid to specific values (see interface of FiniteDifferenceGrid class); (2) by mapping properties from the finite-element mesh onto the grids (`Model::WritePropertyToGrid()`); (3) by assigning property distributions specified as regular-gridded data onto the grids (methods like `In()` of FiniteDifferenceGrid).

When the TransportVisitor is constructed one must specify the node and element variables which define the velocity field:

```

TransportVisitor<CSMP++_float,2>
advect( Model_instance, "nodal velocity", "velocity",
        aqueous_system, time_increment );

TransportStepSize<CSMP++_float,2>
step_size("nodal velocity", "velocity", "inner radius");

```

```

Model.Pass( step_size );

advect.AdjustTimeIncrement( step_size.AdvectionTimeIncrement() );

while ( time <= total_time )
{
    ...
    advect.AdvectUntil( Model, time_interval );
    ...
}

```

In this code example, the advection `time_increment` is adjusted. This variable cannot assume an arbitrary value but geometrical arguments dictate that it must be less than the diameter of the inner circle of each triangular finite-element. The maximum increment which satisfies this conditions is calculated on an element-by-element basis from the velocity-vector field and the finite-element element size characteristics. This is done by the Interrelation `TransportStepSize` which uses the variables “*inner radius*”, “*nodal velocity*” and “*velocity*” to identify the maximum permissible advection time-increment for the current mesh and velocity distribution. The call to the method `AdvectUntil()` prompts the visitor to advect the property distributions on the grids in as many steps as it takes to span the time interval specified as method argument.

The `TransportVisitor` uses the velocity-vector field specified at the element centers as well as at the nodes to displace upstream solute concentrations on the transport grids. It also incorporates a scheme which interpolates concentrations advected into the model from outside the mesh. In order to achieve this, the `TransportVisitor` uses a five-point interpolation scheme which is fitted to solute concentrations along the flow path immediately adjacent to the grid boundary. This fit is used to extrapolate solute concentration outside of the grid. This scheme is capable of advecting linear gradients across boundaries. However, in some cases it fails to carry out such an extrapolation and the user receives warning messages. This may occur when (1) the transport grid resolution is very high, (2) the flow velocities are very high while the advection timespan is long, and (3) the concentration increases steeply behind an inflow boundary. Putting a frame around the transport grids with fixed concentrations or low permeabilities may help out of such a situation.

VelocityAndVolumeFluxVisitor

The `VelocityAndVolumeFluxVisitor` computes the flow velocities and the scalar volume flux from the fluid pressure distribution, the porosity, and the hydraulic conductivity of visited elements. The variable names “*conductivity*”, “*porosity*”, “*fluid pressure*”, “*velocity*”, “*pore velocity*”, and “*volume flux*” are hardwired into the `VelocityAndVolumeFluxVisitor`. The visitor is intended as a post-processor which is applied after a fluid pressure computation, but only if nodal velocities are desired for restricted model regions output to visualization tools. The application of this visitor restricted to surface representations of fractures or faults has the advantage that matrix element properties are not taken into account. The visitor will only use elements inside the target group, i.e. the surface, to compute nodal properties. In contrast, indiscriminate velocity post-processing of such models will lead to an averaging of volume (rock matrix) and surface (fracture plane) velocities.

/SOURCE/INCLUDES/INTERFACES/ LIBRARY

ANSYS/
Eclipse/
Fracman/
GeoModeller/
JPEG/
Maple/
Matlab/
Rhinoceros/
Tecplot/
VTK/
g0cad_SKUA/

BinaryFileHandler.h
BinaryFileInterface.h
StochasticPermeabilityFieldInterface.h
StraightLineMeshInterface.h
TextFileIO.h anything for the reading of text files
TextInterface.h output of models to text
TRIANGLE_Interface.h input of meshes from Setchuk's Triangle
XGRAPH_Interface.h
XML_Document.h

/SOURCE/INCLUDES/MODEL/ LIBRARY

ArrayVariable.h
Boundary.h
BoundaryConnector.h
BoundaryFluxVisitor.h
BoundaryInterface.h
Box.h
CSMP_definitions.h
CSMP_global_enumerations.h
CSMP_number_types.h
CopyReplaceVisitor.h
DataInputVisitor.h
Element.h
ElementToNodePropertyVisitor.h
Element_old.h
FEM_Data.h
FEM_DataInputVisitor.h
FEM_DataOutputVisitor.h
Face.h
FaceConstructionData.h
FiniteElementManager.h
FlaggedArrayVariable.h
GenericSingleton.h
GlobalVerbose.h
Index.h
IndexTracker.h
InputDataManager.h
IntegrationPointCount.h
IntegrationPointToNodePropertyVisitor.h
IntegrationPointVariables.h
InterFace.h
Interrelation.h
LocalVariableStorage.h
LocalVariableStorageIndexArithmetic.h
LocalVariables.h
MathOperatorLHS.h
MathOperatorRHS.h
Matrix.h
MeshManager.h
MeshRenumberVisitor.h
Model.h
Model1D.h
ModelSubDomain.h
ModelTime.h
ModelTopology.h
NimbleRegion.h
Node.h
Operand.h
Parameter.h
Point.h
PropertyAtPointVisitor.h
PropertyConstraints.h
PropertyData.h
PropertyDatabase.h
QuadrilateralFacet.h
Region.h
RegionInterface.h

ScalarVariable.h
SplitBoundary.h
SplitBoundaryInterface.h
TensorVariable.h
TensorVariable1.h
TensorVariable2.h
TriangularFacet.h
VariablePlacement.h
VariableTypeTraits.h
VectorVariable.h
VectorVariable1.h
VectorVariable2.h
Visitor.h
binaryReadWrite.h
variableOperations.h

/SOURCE/INCLUDES/PDE_OPERATORS/ LIBRARY

```

Integral_NT_M_dV_Triangle
Integral_NT_N_dV
Integral_NT_lhsop_N_dV
Integral_NT_op_N_dV
Integral_NT_op_dNi_dV
Integral_NT_op_dS
Integral_NT_rhsop_N_dV
Integral_dNT_op_dN_NT_v_dN_dV
Integral_dNT_op_dN_dV
Integral_dNT_rhsop2_dN_dV
Integral_dNT_rhsop_dN2_dV
Integral_dNT_rhsop_dN_dV
Integral_op_NT_dN_orthogonal_dV
Integral_var_NT_lhsop_N_dV
Integral_var_NT_rhsop_N_dV
Jacobian_Integral_dNT_op_dN_dV
Jacobian_Upwind_Integral_dNT_op_dN_dV
NT_op
NumIntegral_BT_D_B_dV
NumIntegral_BT_D_op_dV
NumIntegral_BT_op_dV
NumIntegral_PT_op_dV
NumIntegral_dNT_mixed_op_dN_NT_v_dN_dV
NumIntegral_dNT_mixed_op_dN_dV
NumIntegral_dNT_op_dN_NT_op_dop_dN_dV
NumIntegral_dNT_op_dN_dV
NumIntegral_dNT_op_dN_dV_NT_v_dN_dV.h
NumIntegral_DNT_rhsop_DN_dV
NumIntegral_op_NT_N_dV
NumIntegral_DNT_v_dV
NumIntegral_op_NT_dN_orthogonal_dV
NumIntegral_NT_lhsop_N_dV
NumIntegral_op_PT_P_dV
NumIntegral_NT_mixed_op_dNi_dV
NumIntegral_NT_op_N_dV
NumIntegral_NT_op_dS
NumIntegral_PT_lhsop_P_dV
NumIntegral_PT_op_P_dV
NumIntegral_PT_op_dS
NumIntegral_NT_op_dNi_dV
PointSource_rhsop
PT_op
StressesAndStrains
Upwind_Integral_dNT_op_dN_dV
Upwind_Integral_dNT_rhsop_dN_dV
Upwind_Integral_dNT_rhsop_g_dV
VelocityAndVolumeFlux

```

To solve PDEs with the finite-element method, they are converted into linear-algebraic matrix problems of the form

$$[A]\{x\} = \{b\}$$

where, on the lefthand side, the square matrix A is called the global solution matrix, the vector x , is called the solution vector, and the vector b , is called the righthand vector. The size n of A is equivalent to the number of unknowns squared. For “scalar” problems where the dependent variable is a scalar, like “fluid pressure”, n is equivalent to the number of non-Dirichlet nodes. For problems where the dependent variable is a vector property, like displacement for instance, there are several degrees of freedom (*DOF*) per node, and the size of A is $n \text{ DOF} \times n \text{ DOF}$. Since A holds the elements contributions to the global conductance or stiffness of the equation system it is sometimes also called the stiffness or conductance matrix. The volume or area integrals from the elements are accumulated into A one by one until one obtains a piecewise integration over the whole model. Integration constraints and constants are supplied via b . It will hold both, essential and initial conditions.

The PDE Operators in CSMP++ are grouped into those which compute (matrix) contributions to be assembled into the lefthand global solution matrix and are derived from the `MathOperatorLHS` base class, and into those which compute (vector) contributions to the righthand vector and are derived from the `MathOperatorRHS` base class. It was a design intent to completely decouple PDE operators from the type of finite elements used, and to allow for the integration of both element and node properties. Necessarily, there is a difference in the form of PDE operators which use analytical integration as is only possible for straight-sided elements, and PDE operators which employ numerical integration. This distinction is expressed in the PDE operator names:

`Integral...` analytical integration (`LinearTriangle` or `LinearTetrahedron`)
`NumIntegral...` numerical integration (all higher order elements)

Matrix contributions from the elements are called element “stencils”. These are again square matrices with a size equal to the number of nodes per element times the *DOF* per node. Further naming conventions used for the PDE operators are (with reference to the number of nodes per element).

Notation	Meaning
N, NT	interpolation function vector and its transposed with n =nodes entries
N2	duplicated interpolation function block matrix
dN, dNT	interpolation function derivatives matrix (dimensions x nodes)
P, PT	interpolation function matrix and its transposed for vector properties
B, BT	dN expanded for multiplication with 3 x 3 (2D) or 6 x 6 (3D) matrix
op	the material property(ies) which enter the MTRL matrix in integral
lhsop	lefthandside op if there arises an ambiguity in names when same operator is used in the left and the righthandside of the equation
dV	area- or volume integral
dS	surface integral (as for boundary stresses or Neumann fluxes)

Table 9.2: Naming conventions for the `MathOperatorRHS` and `MathOperatorLHS` subclasses.

When numerical integration is used or the properties are not constant on the element, interpolation functions and their derivatives must be computed at the integration points just like the material properties. This is how property variations are interpolated over the element and then integrated. In some cases the operator implementations are restricted such that only scalar properties or only element properties can be integrated by the specific operators. In these cases one will get a `FATAL_ERROR` message if one tries to use incompatible variables.

The matrix forms used further below are listed:

Element interpolation functions:

$$N = [N_1 \quad \cdots \quad N_n], \quad NT = \begin{bmatrix} N_1 \\ \vdots \\ N_n \end{bmatrix}, \quad N2 = \begin{bmatrix} N_1 & \cdots & N_n \\ N_1 & \cdots & N_n \end{bmatrix}$$

N must be computed at each point of interest; n is the number of nodes per element. If vector properties are interpolated, each vector component must be interpolated separately. This is expressed by writing $\mathbf{P}$ instead of N (this example is for the interpolation of a two-dimensional vector variable):

$$P = \begin{bmatrix} N_1 & 0 & \cdots & 0 & N_n & 0 \\ 0 & N_1 & 0 & \cdots & 0 & N_n \end{bmatrix}, \quad PT = \begin{bmatrix} N_1 & 0 \\ 0 & N_1 \\ \vdots & 0 \\ 0 & \vdots \\ N_n & 0 \\ 0 & N_n \end{bmatrix}$$

Element-interpolation function derivatives:

$$dN = \begin{bmatrix} \frac{\partial N_1}{\partial x} & \cdots & \frac{\partial N_n}{\partial x} \\ \frac{\partial N_1}{\partial y} & \cdots & \frac{\partial N_n}{\partial y} \end{bmatrix}, \quad dNT = \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_1}{\partial y} \\ \vdots & \vdots \\ \frac{\partial N_n}{\partial x} & \frac{\partial N_n}{\partial y} \end{bmatrix}$$

The expansion for multiplication with tensor variables (as is used in mechanics) is (for two-dimensional applications):

$$B = \begin{bmatrix} \frac{\partial N_1}{\partial x} & 0 & \cdots & \cdots & \frac{\partial N_n}{\partial x} & 0 \\ 0 & \frac{\partial N_1}{\partial y} & \cdots & \cdots & 0 & \frac{\partial N_n}{\partial y} \\ \frac{\partial N_1}{\partial y} & \frac{\partial N_1}{\partial x} & \cdots & \cdots & \frac{\partial N_n}{\partial y} & \frac{\partial N_n}{\partial x} \end{bmatrix}, \quad BT = \begin{bmatrix} \frac{\partial N_1}{\partial x} & 0 & \frac{\partial N_1}{\partial y} \\ 0 & \frac{\partial N_1}{\partial y} & \frac{\partial N_1}{\partial x} \\ \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \\ \frac{\partial N_n}{\partial x} & 0 & \frac{\partial N_n}{\partial y} \\ 0 & \frac{\partial N_n}{\partial y} & \frac{\partial N_n}{\partial x} \end{bmatrix}$$

For three dimensional applications, an entry for a single node (where the last three rows refer to the off-diagonal tensor components) is:

$$B_i = \begin{bmatrix} \frac{\partial N_i}{\partial x} & 0 & 0 \\ 0 & \frac{\partial N_i}{\partial y} & 0 \\ 0 & 0 & \frac{\partial N_i}{\partial z} \\ \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} & 0 \\ 0 & \frac{\partial N_i}{\partial z} & \frac{\partial N_i}{\partial y} \\ \frac{\partial N_i}{\partial z} & 0 & \frac{\partial N_i}{\partial x} \end{bmatrix}$$

In the following, analytically integrated operators are described first and numerically integrated ones second.

PDE Operators for Analytical Integration

The available operators for analytical integration are essentially all those which are also available (and addressed via enumeration strings) in conjunction with the CSMP++_PDE_Integrator:

Operator	Integration of PDE Term	Other Name
Integral_NT_lhsop_N_dV	$lhsop \nabla^2 N$	capacitance
Integral_dNT_op_dN_dV	$lhsop \nabla^2 N, \nabla^T lhsop \nabla N$	conductance
Integral_dNT_op_dN_NT_v_dNT_dV	$op \nabla^2 N + v \cdot \nabla N$	diffusion-advection
Integral_NT_op_N_dV	N	capacitance
Integral_N_op_dN_dV	e.g. $\rho g \nabla z$	buoyancy

Table 9.3: PDE operators for elements using analytical integration.

The actual form of these operators and the contributions arising from them are for 2D triangular elements:

Integral_dNT_lhsop_dN_dV (conductance matrix)

$$K^{(e)} = \iint_A dNT \begin{bmatrix} K_x & 0 \\ 0 & K_y \end{bmatrix} dN dx dy$$

Analytical integration of this integral for 2D linear triangles yields

$$\frac{K_x}{4A} \begin{bmatrix} b_i^2 & b_i b_j & b_i b_k \\ b_j b_i & b_j^2 & b_j b_k \\ b_k b_i & b_k b_j & b_k^2 \end{bmatrix} + \frac{K_y}{4A} \begin{bmatrix} c_i^2 & c_i c_j & c_i c_k \\ c_j c_i & c_j^2 & c_j c_k \\ c_k c_i & c_k c_j & c_k^2 \end{bmatrix}$$

and for 3D linear tetrahedral

$$\frac{K_x}{18V} \begin{bmatrix} b_i^2 & b_i b_j & b_i b_k \\ b_j b_i & b_j^2 & b_j b_k \\ b_k b_i & b_k b_j & b_k^2 \end{bmatrix} + \frac{K_y}{18V} \begin{bmatrix} c_i^2 & c_i c_j & c_i c_k \\ c_j c_i & c_j^2 & c_j c_k \\ c_k c_i & c_k c_j & c_k^2 \end{bmatrix} + \frac{K_z}{18V} \begin{bmatrix} d_i^2 & d_i d_j & d_i d_k \\ d_j d_i & d_j^2 & d_j d_k \\ d_k d_i & d_k d_j & d_k^2 \end{bmatrix}$$

where $\mathbf{B}$ for 3D linear tetrahedra is

$$\mathbf{B} = \frac{1}{6V} \begin{bmatrix} b_1 & b_2 & b_3 & b_4 \\ c_1 & c_2 & c_3 & c_4 \\ d_1 & d_2 & d_3 & d_4 \end{bmatrix}$$

Note that for linear finite elements, shape function derivatives are constants.

Integral_NT_op_N_dV (mass- or capacitance matrix)

consistent formulation

$$C^{(e)} = \iint_A NT S^{(e)} N dx dy = \frac{S^{(e)} A}{12} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

lumped formulation

$$C^{(e)} = \iint_A NT S^{(e)} N dx dy = \frac{S^{(e)} A}{3} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Integral_dNT_op_dN_NT_v_dNT_dV (diffusion-advection matrix)

$$D^{(e)} = \iint_A dNT \begin{bmatrix} D_{xx} & D_{xy} \\ D_{yx} & D_{yy} \end{bmatrix} dN N 2 \begin{bmatrix} v_x & 0 \\ 0 & v_y \end{bmatrix} dN dx dy$$

For 2D triangular elements, integration yields.

$$\begin{aligned}
& \frac{D_{xx}}{4A} \begin{bmatrix} b_i^2 & b_i b_j & b_i b_k \\ b_j b_i & b_j^2 & b_j b_k \\ b_k b_i & b_k b_j & b_k^2 \end{bmatrix} + \frac{D_{yy}}{4A} \begin{bmatrix} c_i^2 & c_i c_j & c_i c_k \\ c_j c_i & c_j^2 & c_j c_k \\ c_k c_i & c_k c_j & c_k^2 \end{bmatrix} \\
& + \frac{D_{xy}}{4A} \begin{bmatrix} b_i c_i & b_i c_j & b_i c_k \\ b_j c_i & b_j c_j & b_j c_k \\ b_k c_i & b_k c_j & b_k c_k \end{bmatrix} + \frac{D_{yx}}{4A} \begin{bmatrix} c_i b_i & c_i b_j & c_i b_k \\ c_j b_i & c_j b_j & c_j b_k \\ c_k b_i & c_k b_j & c_k b_k \end{bmatrix} \\
& - \frac{v_x}{6} \begin{bmatrix} b_i & b_j & b_k \\ b_i & b_j & b_k \\ b_i & b_j & b_k \end{bmatrix} - \frac{v_y}{6} \begin{bmatrix} c_i & c_j & c_k \\ c_i & c_j & c_k \\ c_i & c_j & c_k \end{bmatrix}
\end{aligned}$$

Additional decay and sorption terms also are based on “mass” matrices and therefore have the same consistent and lumped forms:

$$\begin{aligned}
RAD^{(e)} &= \iint_A NT[\lambda(\Phi + \rho_b K_d)]N dx dy \\
A^{(e)} &= \iint_A NT[\rho_b K_d + \Phi]N dx dy
\end{aligned}$$

PDE Operators for Numerical Integration

Operator	Integration of PDE Term	Other Name
<i>lefthand</i>		
NumIntegral_NT_lhsop_N_dV	$lhsop \nabla^2 N$	capacitance
NumIntegral_dNT_op_dN_dV	$lhsop \nabla^2 N$ see other form above	conductance
NumIntegral_NT_op_dN_dV	e.g. $\rho g \nabla z$	buoyancy
NumIntegral_BT_D_B_dV	lhs force balance $\frac{\partial \sigma_{ij}}{\partial x_i} = F_j^B$	stiffness
<i>righthand</i>		
NumIntegral_NT_op_dV		fluid body forces
NumIntegral_NT_op_N_dV	N	capacitance
NumIntegral_BT_op_dV		body forces, initial stresses
NumIntegral_BT_D_op_dV		volume strain
NumIntegral_PT_op_dS		surface stresses
PT_op		nodal forces

Table 9.6: PDE operators for elements using numerical integration. The operators in red have not been tested rigorously yet.

In order to carry out the numerical integration interpolation function values and their derivatives must be computed at all integration points. Derivates need to be computed

also in the assembly of mass matrices, since the Jacobian must be formed as its determinant post multiplies each integration point contribution.

Lumping of the mass matrix (if so desired for time-dependent calculations), is often implemented the crude way: The off-diagonal elements of the mass matrix are summed into its diagonal after forming the consistent form of the mass matrix:

$$b_{ii} = \sum_{j=1}^N b_{ij}$$

This adds computational effort rather than reducing it by a simpler form. An alternative approach circumventing this problem is discussed in Lapidus & Pinder (1999, p. 328-329). They advocate Simpson's rule-based integration to arrive naturally at a diagonal form of the mass matrix. For the isoparametric elements in CSMP++ lumping is implemented by multiplying the operand with the elements volume and dividing each of these nodal contributions by the number of nodes per element:

$$M_{lumped}^{(e)} = \frac{SV}{n} |J| \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

NumIntegral_dNT_op_dN_dV

This integral arises for elliptic partial differential equations, such a pressure diffusion or heat conduction. When the operand represents the hydraulic conductivity a conductance matrix results where K values are interpolated to the integration points using N . Numerical integration amounts to the following summation over the contributions from each integration point:

$$\underbrace{\underline{K}^{(e)}}_{n \times n} = \sum_{i=1}^{N_\varepsilon} \sum_{j=1}^{N_\eta} W_i(\varepsilon_i) W_j(\eta_j) \left\{ \underbrace{\underbrace{dNT(\varepsilon_i, \eta_j)}_{n \times 2} \underbrace{[J(\varepsilon_i, \eta_j)^{-1}]^T}_{2 \times 2}}_{2 \times n} \underbrace{\begin{bmatrix} K_x(\varepsilon_i, \eta_j) & 0 \\ 0 & K_y(\varepsilon_i, \eta_j) \end{bmatrix}}_{2 \times 2} \underbrace{[J(\varepsilon_i, \eta_j)^{-1}] \times}_{2 \times 2} \right\}$$

Integral_NT_op_N_dV

The capacitance or mass matrix computed by summing the contributions at the integration points has the form (3D):

$$C^{(e)} = \int_0^1 \int_0^1 \int_0^1 NT[S]N|J|d\varepsilon d\eta d\xi$$

The storativity is inserted as a scalar operand.

NumIntegral_dNT_D_dN_NT3_v_dN

NT3 is a matrix consisting whose 3 columns consist of interpolation function vectors. The diffusion-advection (dispersion) operator (not implemented yet) is based on the dispersivity tensor and the velocity vector. The accumulation of this term will lead to a non-symmetric solution matrix. If the matrix becomes strongly assymmetric the AMG solver can no longer be used and Gaussian Elimination may be the only method capable of inverting the solution matrix.

$$D^{(e)} = \int_0^1 \int_0^1 \int_0^1 dNT \left[J^{-1} \right]^T \begin{bmatrix} D_{xx} & D_{xy} & D_{xz} \\ D_{yx} & D_{yy} & D_{yz} \\ D_{zx} & D_{zy} & D_{zz} \end{bmatrix} J^{-1} dN |J| d\varepsilon d\eta d\xi \\ + \int_0^1 \int_0^1 \int_0^1 N3T \begin{bmatrix} v_x & 0 & 0 \\ 0 & v_y & 0 \\ 0 & 0 & v_z \end{bmatrix} \left[J^{-1} \right]^T dN |J| d\varepsilon d\eta d\xi$$

Radioactive decay can be incorporated into this element matrix by adding a *NumIntegral_NT_op_N_dV* term in which the scalar operand is composed from the decay constant, the porosity, fluid density and the KD value

$$R^{(e)} = \int_0^1 \int_0^1 \int_0^1 NT [\lambda(\Phi + \rho_b K_d)] N |J| d\varepsilon d\eta d\xi$$

and the (ad)sorption within the rock represented by the element is described by a similar *NumIntegral_NT_op_N_dV* term in which the operand contains the porosity, fluid density and the KD value

$$A^{(e)} = \int_0^1 \int_0^1 \int_0^1 NT [\rho_b K_d + \Phi] N |J| d\varepsilon d\eta d\xi$$

Volume integrals for mechanical calculations differ from the previous ones in that the dependent variable is no longer a scalar but a vector or tensor variable. These operators describe the elements stiffness, body forces, and volume strains. Surface integrals describe boundary tractions, stresses etc.

NumIntegral_BT_D_B_dV (stiffness)

$$K_s^{(e)} = \int_0^1 \int_0^1 \int_0^1 BT \left[J^{-1} \right]^T \underbrace{[D]}_{6 \times 6} \left[J^{-1} \right] B |J| d\varepsilon d\eta d\xi$$

NumIntegral_BT_op_dV (initial stresses)

$$IS_s^{(e)} = \int_0^1 \int_0^1 \int_0^1 BT \left[J^{-1} \right]^T \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{xz} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{zx} & \sigma_{zy} & \sigma_{zz} \end{bmatrix} |J| d\varepsilon d\eta d\xi$$

NumIntegral_PT_op_dV (body forces)

$$F^{(e)} = \int_0^1 \int_0^1 \int_0^1 NT \begin{bmatrix} F_x & F_y & F_z \end{bmatrix} J |d\varepsilon d\eta d\xi$$

NumIntegral_PT_op_dS (boundary tractions)

$$BS^{(e)} = \int_0^1 \int_0^1 NT [\sigma] J |dS$$

NumIntegral_BT_D_op_dV (volume strain)

$$E_s^{(e)} = \int_0^1 \int_0^1 \int_0^1 BT \begin{bmatrix} J^{-1} \end{bmatrix}^T \underbrace{[D]}_{6 \times 6} J |d\varepsilon d\eta d\xi$$

PT_op (nodal forces)

$$F^{(e)} = \begin{bmatrix} F_x^1 & F_y^1 & \dots & \dots & F_x^n & F_y^n \end{bmatrix}^T$$

/SOURCE_CODE/SOLVER/ LIBRARY

DuneISTL_Solver.h
GaussJordan_Solver.h
Gauss_Solver.h
LinearSolver.h
Meschach_Solver.h
PARDISO_Solver.h
SAMG_Configurator.h
SAMG_Exception.h
SAMG_Settings.h
SAMG_Solver.h
Solver.h
SolverSettings.h
meschach.h

/TEST/UNIT LIBRARY

catch.hpp

test_cases_main.cpp
unit_tests_catch.cpp
unit_tests_development.cpp
unit_tests_main.cpp

data/
dataset/
doc/
integration/
unit/

tests/unit/ANSYS_Model2D_Test.h
tests/unit/ANSYS_Model3D_Test.h
tests/unit/ANSYS_SplitBoundaryMatch_Test.h
tests/unit/ArrayVariable_Test.h
tests/unit/BinaryFileInterface_Test.h
tests/unit/BoundaryInterface_Test.h
tests/unit/Boundary_Test.h
tests/unit/Box_Test.h
tests/unit/CSMP_VariableBenchmarking_Test.h
tests/unit/ColorPalette_Test.h
tests/unit/CommandLineParser_Test.h
tests/unit/CompressedRowMatrix_Test.h
tests/unit/ConstraintPointToNodePropertyVisitor_Test.h
tests/unit/CopyReplaceVisitor_Test.h
tests/unit/CsmpIntro1_TestCase.h
tests/unit/CubicSpline_Test.h
tests/unit/DenseMatrix_Test.h
tests/unit/ElementPolicyIntegrity_Test.h
tests/unit/Element_Test.h
tests/unit/ExponentialTransferFunction_Test.h
tests/unit/FEM_Data_Test.h
tests/unit/FV_Parameter_Test.h
tests/unit/Face_Test.h
tests/unit/FibonacciHeap_Test.h
tests/unit/FiniteElement_Test.h
tests/unit/FiniteElement_TestData.h
tests/unit/FiniteVolumePolicy_Test.h
tests/unit/FiniteVolumeStencil_Test.h
tests/unit/FluxMismatch_Test.h
tests/unit/GenericFiniteVolumeTransport_Test.h
tests/unit/GenericSingleton_Test.h
tests/unit/Index_Test.h
tests/unit/InputDataManager_Test.h
tests/unit/Integral_var_NT_lhsop_N_dV_Test.h
tests/unit/Integral_var_NT_rhsop_N_dV_Test.h
tests/unit/IsnanIsinf_Test.h
tests/unit/IsoparametricLinearHexahedron_Test.h
tests/unit/IsoparametricQuadraticTetrahedron_Test.h
tests/unit/LinearCuboid_Test.h
tests/unit/LocalVariableStorage_Test.h
tests/unit/MathOperatorLHS_Test.h
tests/unit/MathOperatorRHS_Test.h

```
tests/unit/Matrix_Test.h
tests/unit/MeshManager_Test.h
tests/unit/ModelComparator_Test.h
tests/unit/ModelSubDomain_Test.h
tests/unit/ModelTopology_Test.h
tests/unit/Node_Test.h
tests/unit/Operand_Test.h
tests/unit/PDE_Integrator_Test.h
tests/unit/PDE_Integrator_UoM_Mock.h
tests/unit/PDE_Integrator_UoM_Test.h
tests/unit/Parallel_TestCase.h
tests/unit/Parameter_Test.h
tests/unit/Point_Test.h
tests/unit/PropertyAtPointVisitor_Test.h
tests/unit/PropertyData_Test.h
tests/unit/PropertyDatabase_Test.h
tests/unit/PropertyHandle_Test.h
tests/unit/PropertyStorageSpeed_Test.h
tests/unit/RegionMonitor_Test.h
tests/unit/Region_Test.h
tests/unit/RelativePermeabilityModel_Test.h
tests/unit/ScalarVar_Test.h
tests/unit/SparseMatrix_Test.h
tests/unit/SplitBoundary_Test.h
tests/unit/StatisticalAnalyzer_Test.h
tests/unit/TensorVar_Test.h
tests/unit/TensorVar_Test1.h
tests/unit/TensorVar_Test2.h
tests/unit/Test.h
tests/unit/TestSuite.h
tests/unit/Transient_Test.h
tests/unit/TwoPhaseModelCapillaryPressure_TestSuite.h
tests/unit/TwoPhaseModelWithCapillaryPressure_TestSuite.h
tests/unit/TwoPhaseModelWithHysteresis_Test.h
tests/unit/TwoPhaseModel_Test.h
tests/unit/TwoPhaseModel_TestSuite.h
tests/unit/VData_Test.h
tests/unit/VSet_Test.h
tests/unit/VTU_Interface_Test.h
tests/unit/VariableStorageSpeed_TestCase.h
tests/unit/Variables_Test.h
tests/unit/VectorVar_Test.h
tests/unit/VectorVar_Test1.h
tests/unit/VectorVar_Test2.h
tests/unit/Visitor_TestSuite.h
tests/unit/Vset_TestCase.h
tests/unit/geometricCalculations_Test.h
```

/SOURCE/INCLUDES/UTILITIES/ LIBRARY

AP_BoolVector.h
AP_Timer.h
AP_testUtilities.h
BE_Time.h
BE-TokenIterator.h
BE_require.h
CSMP_highLevelUtilities.h
ColorPalette.h
CoordinateTransformer.h
FaceData.h
FibonacciHeap.h
HashKey.h
Loki_static_check.h
LoopUnroller.h
ModelComparator.h
OS_Uutilities.h
PL_Uutilities.h
STL_utilities.h
SmallSet.h
UnionFind.h
convertColorToPermeability.h
vsetMakers.h

Appendix

A: THE DISTRIBUTION

CSMP++ comes as a set of header and binary library files for the specific platforms and compilers with which the simulations shall be carried out. The files are stored in subdirectories with names corresponding to the libraries.

B: INSTALLATION

The CSMP++ libraries interface with third-party libraries f2c (g2c), JPEG, Meschach, and SAMG. The complete distribution contains the additional components:

Library Name	ftp site
f2C	http://www.netlib.no/netlib/f2c/
Meschach	http://www.netlib.no/netlib/c/meschach/index.html
AMG (original)	http://www.netlib.no/netlib/AMG
SAMG	commercially marketed by SCAI, Fraunhofer Ges., DE
JPEG	
VTK	http://kitware.com
Blitz++	http://oonumerics.com/blitz

In order to adapt to the particularities of the various platform conditional compilation is used. The installer may have to uncomment a `#define` statement in the header file `"CSMP++_definitions.h"` to unveil the name of the operating system before compiling CSMP++-library based code on the platform of choice.

IMPORTANT: Before the compilation of the CSMP++ libraries on the target platform can start, the program `CSMP++_number_types_main.cpp` must be compiled and executed. It has no make dependencies but establishes the size and availability of C++ inbuilt number types. Once executed it writes the file `"CSMP++_number_types.h"` which is included into `"CSMP++_definitions.h"` automatically. This ascertains that results obtained on different platforms and compilers have the same numerical precision.

The user may also need to define which version of the STL library shall be used. There are different implementations which add multithread safety and specific exception handling capabilities to Stepanov's original STL resource. Different versions can be invoked on different platforms by un-/commenting `#define` statements in the `"CSMP++_definitions.h"` header file.

Windows

Put some doc on visual studio here.

Linux

The recommended compiler is Intel's C++ compiler 7.1 or greater. It generates excellent binary code which executes very reliably and fast. There are corresponding makefiles in the CSMP++ distribution with optimization flags pre-set to the best settings for the different processor types.

KDE offers a very nice integrated development environment under LINUX that operates by default in collaboration with the GNU C++ compiler. While the resulting code is significantly slower than that created with the Intel compiler this is still a very powerful combination.

CSMP++ Glossary

The Glossary below is a brief and by far not comprehensive alphabetical listing of terminology employed in the documentation of CSMP++. Please provide feedback and additions where items are missing and where explanations are too cryptic. There are also some explanations of C++ jargon. If the listed expression is C++ terminology, I have indicated this by putting (C++) behind it. Specific CSMP++ terminology like CSMP++ object names, is underlined.

aggregate. Association of objects that collaborate to achieve functionality but none of them manages the others in contrast to a composite.

AMG. Algebraic multigrid method (Ruge & Stüben 1987) used inside the CSMP++ solver. See Briggs, W.L.'s multigrid tutorial (SIAM 1987) for a description of the method.

Argument. Input or output value of a method or a function. This value is entered into or retrieved from the function by writing it into the braces behind the function name, e.g. `myFunction( argument1, argument2, argument3 )`. In the function declaration placeholders are used for the arguments. These defined the type of the expected input or output, e.g., `MyFunction( int argument1, House& argument2, Car& argument3 )`. The `&` means that a reference to the actual input/output object is passed. Otherwise the function will make a copy of the argument for internal use (logically, large objects should not be copied and therefore always be passed by references).

Base class. A class object which describes the commonalities of an object hierarchy. The base class defines an interface for inherited objects, thereby specifying to the program how to handle these even when these are not known at the time when the base class is implemented.

Composite. Object that consists of several other classes which it manages, example: Model.

const. C++ type qualifier which means that the expression which follows to the right is not modified after its definition or at least by the function to which it is passed, for example: `const int pi=3.1414;` or `MyFunction( const Region& g ) const;`

constructor. Default method with the same name as the class object, which it initializes when an instance of the class is created. Arguments to the constructor method provide initial data for the new object.

enum. C++ construct associating string variables with integer values such that these can be used as array indexing variables and for debugging.

Region. Collection of finite-elements, the global mesh ID numbers of which, are stored in an STL map. Use groups to manipulate geological entities of your model selectively.

Interface. List of operations supported by a piece of software or hardware; typically, the sum of the methods(=functions) of a class object which have been made accessible to the public.

Operand. A handle class (Operand) to a distributed CSMP++ physical variable described in the PropertyDatabase object and stored in the

CSMP++_MemoryManager (both objects inside the Model object). Operands allow you to access variables in computations. To do this you construct an Operand with the variable name as constructor argument.

Method. A function that is associated with a class so that it cannot be called by itself in contrast with normal functions like `double sqrt()`.

Polymorphism. The ability to treat different objects the same way, which is defined in a base class.

Subclass. A class inherited from a base class. The base class thereby captures the commonalities of all objects to which the subclass belongs, for example, automobiles when the subclass implements a VW beetle. A class A is a subclass of a baseclass B when the statement “A is a B” is true.